

머신러닝의 이해

Machine Learning Starter Pack Version 1.0

Version 1.0

2022.02.28.

한국공학대학교 전자공학부 김준수
junsukim@tukorea.ac.kr

무단 전재 및 재배포를 금함



목차

1. 선형회귀	1
2. 모델의 검증과 평가	21
3. 로지스틱 회귀	35
4. 인공 신경망	55
5. 의사결정 트리	81
6. 클러스터링	115

1. 선형회귀 (Linear Regression)

1. 선형회귀

회귀(regression)란 지도학습의 한 종류로, 훈련 데이터의 출력이 가질 수 있는 값이 일정한 범위 안에서 연속인 경우 최대한 정확하게 실제 값을 예측하는 방법이다. 선형회귀는 회귀 학습 중 선형모델을 사용하는 기법을 의미한다.

머신러닝에서는 이 기법을 회귀라고 하는데, 다른 여러 분야에서 피팅(fitting)이라는 용어를 사용하기도 한다. 예를 들어, 일반 물리학의 용수철 실험에서 추의 무게에 따라 늘어난 용수철의 길이를 측정하였다고 생각해보자. 실험을 통해 수집한 데이터의 입력은 추의 무게, 출력은 용수철의 길이가 된다. 이 데이터를 이용해 x 축에 추의 무게, y 축에 늘어난 용수철의 길이를 표시하면 2차원 그래프 상에 데이터 수만큼의 점이 표시될 것이다. 이때 추의 무게와 용수철 길이의 함수 관계를 구하기 위해 가장 쉽게 사용하는 기법은 점들이 늘어선 모양을 잘 묘사할 수 있는 직선을 구하는 것이다. 이렇게 만들어진 직선은 어떤 점은 관통하고 어떤 점은 관통하지 않을 수 있지만, 데이터가 갖는 전체적인 속성, 즉 추의 무게와 늘어난 용수철의 길이 사이의 함수 관계를 적절히 표현할 수 있을 것이다. 이렇게 데이터를 통해 함수 관계를 도출해내면, (이상적인 경우) 앞으로는 추가적인 실험 없이 임의의 무게에 대한 용수철의 길이를 정확하게 예측할 수 있다. 이 예에서, 데이터를 통해 직선을 도출하는 과정을 ‘선형회귀 모델의 학습’ 과정이라 할 수 있다.

2. 선형회귀 모델

회귀를 위해 선형모델을 사용한다는 것은 훈련 데이터의 출력을 입력들의 선형조합(linear combination)으로 표현하겠다는 것을 의미한다. 간단한 예를 통해 선형회귀 모델을 이해하고 이를 일반화하는 방식으로 설명을 진행한다.

앞서 제시한 탄성실험의 경우, 훈련 데이터는 하나의 입력(추의 무게)과 하나의 출력(늘어난 용수철의 길이)으로 구성된다. 실험 방법은 용수철에 무게추를 하나씩 늘려가며 용수철이 늘어난 길이를 측정하는 것이다. 이렇게 총 N 회의 실험을 실시하고 데이터를 수집한 결과는 표 1과 같이 표기할 수 있다.

[표 1] 용수철 실험의 측정 데이터

데이터 번호	무게 (g)	늘어난 길이 (cm)
0	x_0	y_0
1	x_1	y_1
$\vdots$	$\vdots$	$\vdots$
n	x_n	y_n
$\vdots$	$\vdots$	$\vdots$
$N-1$	x_{N-1}	y_{N-1}

표 1의 데이터를 이용해 학습한 선형회귀 모델의 출력을 $\hat{y}$ 으로 표기하면, $\hat{y}$ 은 주어진 입력으로 예측한 출력값이 된다. 학습이 잘 이루어진다면 $\hat{y}=y$ 가 될 것이다. 즉, 예측한 결과와 실제 결과가 같아진다는 것이다. 선형회귀 모델은 출력의 예측을 위해 입력들의 선형조합을 이용하는 것이지만, 이 예에서는 입력이 하나뿐이므로 다음과 같이 간단하게 표현할 수 있다.

$$\hat{y} = w_0x + w_1 \quad (1)$$

식 (1)은 잘 알려진 직선의 방정식이며 w_0 는 기울기 w_1 은 y 축 절편을 나타낸다. 이를 선형조합의 관점에서 생각해보면 w_0 는 변수들의 선형조합을 위한 가중치(weight)이며 w_1 은 바이어스(bias)라고 한다. 명칭이 무엇이 되었던, 우리의 목표는 식 (1)의 두 변수 w_0 와 w_1 을 적절히 조절하여 $\hat{y}=y$ 이 되도록 하는 것이다.

3. 비용함수(Cost function) 또는 손실함수(Loss function)

앞서 우리의 목표를 명확히 밝혔다. 즉, w_0 와 w_1 을 찾아 $\hat{y}=y$ 을 만족하는 식 (1)을 완성하는 것이다. 그러나 일반적으로 $\hat{y}=y$ 를 완벽하게 만족하는 식을 찾는 것은 매우 어렵다. 따라서 달성하기 어려운 조건을 조금 완화하여 $\hat{y} \approx y$ 라고 하자. 다시 말해, 식 (1)의 예측값이 최대한 실제값에 근접하는 w_0 와 w_1 을 찾는 것이다. 이를 위해서는 예측값과 실제값이 얼마나 가까운지 측정할 필요가 있다. 이를 위해 다음과 같은 함수를 정의한다.

$$\epsilon_{MSE} = \frac{1}{N} \sum_{n=0}^{N-1} (\hat{y}_n - y_n)^2 \quad (2)$$

식 (2)는 N 개의 데이터에 대한 각각의 예측값 $\hat{y}_n$ 과 실제값 y_n 의 차이를 제곱한 값들의 평균을 계산한 것이다. 즉, 오차의 제곱의 평균으로 이를 평균제곱오차(MSE, Mean Square Error)라 한다. (참고로 ϵ 은 입실론이라 읽는다.) 이 값은 오차가 클수록 커지고 오차가 작을수록 작아지는 함수이다. 따라서 우리의 목표인 $\hat{y} \approx y$ 를 달성하면 (2)의 값이 매우 작아질 것이다. 역으로 (2)를 최소화하는 w_0 와 w_1 을 찾으면 $\hat{y} \approx y$ 이 달성된다.

식 (2)와 같이, 개별 데이터의 오차에 비례하여 커지는 함수를 비용함수(cost function)라고 한다. 머신러닝에서는 다양한 비용함수를 사용하는데, 평균제곱오차는 가장 대표적인 비용함수이다. 비용함수는 종종 손실함수(loss function)라고 부르기도 한다.

4. 비용함수 최소화 문제

우리의 문제로 다시 돌아가, 우리의 목표를 생각해본다. 원래 목표는 다음과 같았다.

(목표) 최대한 $\hat{y} \approx y$ 을 만족하는 적절한 w_0 와 w_1 을 찾는다.

이 목표를 비용함수로 평균제곱오차를 도입해 수정하면 다음과 같다.

(수정된 목표) 평균제곱오차를 최소화하는 매개변수를 찾는다.

수정된 목표에서 매개변수란 우리가 구해야 할 변수 w_0 와 w_1 을 의미한다. 이를 수식으로 표

현하면 다음과 같다.

$$(w_0^*, w_1^*) = \arg \min_{(w_0, w_1)} \epsilon_{MSE} \quad (3)$$

식 (3)에서 (w_0^*, w_1^*) 은 평균제곱오차를 최소화하는 최적해를 의미하며, 우리가 찾아야 하는 값이다. 이 식의 의미는 최적해 (w_0^*, w_1^*) 는 다양한 w_0 와 w_1 중에서 ϵ_{MSE} 를 최소화하는 값이라는 것이다. 식 (3)을 직관적으로 풀어쓰면 다음과 같다.

$$(w_0^*, w_1^*) = \arg \min_{(w_0, w_1)} \frac{1}{N} \sum_{n=0}^{N-1} (w_0 x_n + w_1 - y_n)^2 \quad (4)$$

식 (3)과 (4)는 동일한 수식이지만, (4)에는 우리의 목표인 w_0 와 w_1 이 잘 보이므로 이해하기 쉽다.

5. 해석적인 방법으로 최적해 찾기

최적해(optimal solution)란 식 (4)의 해 w_0^*, w_1^* 을 의미하고 해석적인 방법이란 수학적 유도를 통해 해를 구하는 것을 의미한다. 비용함수가 복잡한 경우 해석적 접근이 불가능할 수 있으나 식 (4)와 같이 단순한 경우 해석적 방법으로 최적해를 구할 수 있다.

우리의 목표는 (4)의 해를 구하는 것인데, 이는 (2)에 정의된 비용함수를 최소화하는 w_0 과 w_1 을 찾는 것과 같다. 이를 위해 평균제곱오차를 다시 써보면 다음과 같다.

$$\epsilon_{MSE}(w_0, w_1) = \frac{1}{N} \sum_{n=0}^{N-1} (w_0 x_n + w_1 - y_n)^2 \quad (5)$$

평균제곱오차가 매개변수 w_0 와 w_1 의 함수임을 강조하기 위해 식 (5)에 (w_0, w_1) 을 표시하였다. 이제 식 (5)를 최소화하는 값이 최적해 w_0^* 과 w_1^* 이다. 이를 구하기 위해서는 다음의 두 식을 풀어야 한다.

$$\frac{\partial}{\partial w_0} \epsilon_{MSE}(w_0, w_1) = 0 \quad (6)$$

$$\frac{\partial}{\partial w_1} \epsilon_{MSE}(w_0, w_1) = 0 \quad (7)$$

식 (6)과 (7)을 통해 다음과 같은 최적해를 구할 수 있다. 구하는 과정은 그리 어렵지 않으니 각자 유도해보기 바란다.

$$w_0^* = \frac{\frac{1}{N} \sum_{n=0}^{N-1} y_n \left(x_n - \frac{1}{N} \sum_{i=0}^{N-1} x_i \right)}{\frac{1}{N} \sum_{n=0}^{N-1} x_n^2 - \left(\frac{1}{N} \sum_{n=0}^{N-1} x_n \right)^2} \quad (8)$$

$$w_1^* = \frac{1}{N} \sum_{n=0}^{N-1} (y_n - w_0^* x_n) \quad (9)$$

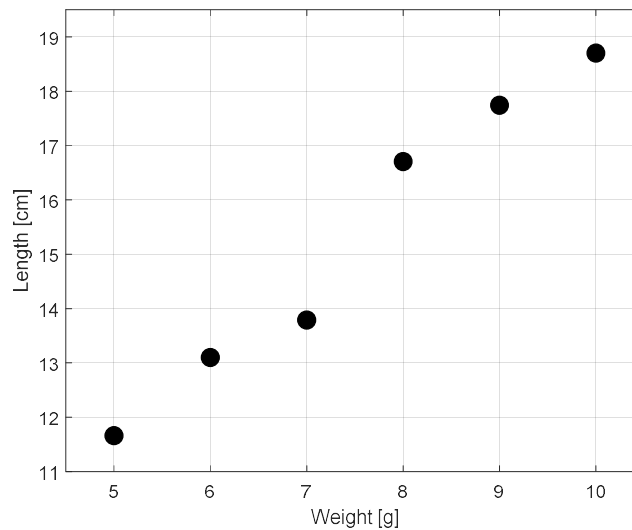
식 (8)과 (9)를 언뜻 보면 다소 복잡해 보이지만, 머신러닝에서 만날 수 있는 가장 간단한 솔루션이라 할 수 있다. 그리고 데이터만 있으면 언제든지 손쉽게 선형회귀 모델을 만들 수 있다는 장점이 있다.

6. 해석해를 이용한 선형회귀의 예

지금까지 공부한 내용을 구체적인 예와 함께 살펴보자. 표 2는 용수철 실험을 6번 실행하여 얻은 데이터이다. 입력은 무게, 출력은 용수철의 길이이다. 이를 그래프에 표시하면 그림 1과 같다.

[표 2] 6회 실험을 통해 얻은 데이터

데이터 번호	무게 (g)	늘어난 길이 (cm)
1	5	11.66
2	6	13.10
3	7	13.79
4	8	16.71
5	9	17.74
6	10	18.70



[그림 1] 데이터 그래프

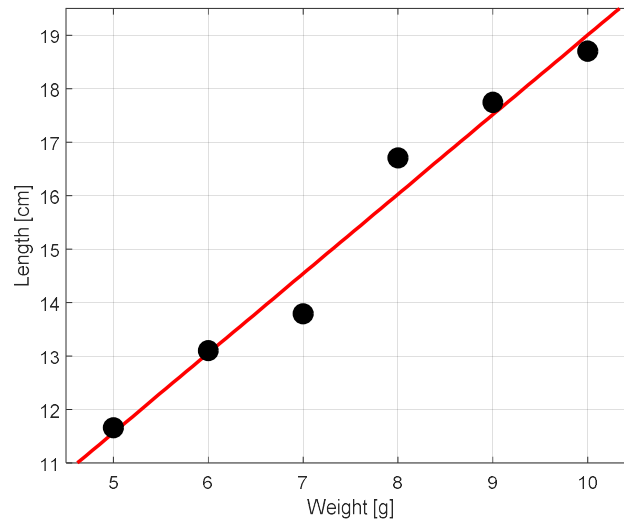
그림 1을 살펴보면 무게가 늘어날수록 용수철의 길이가 어느 정도 일정한 비율로 증가하는 것을 알 수 있다. 이제, 선형회귀 모델을 이용해 그림 1의 데이터에 담겨있는 무게와 용수철 길이 사이의 함수 관계를 찾아보자. 모델은 식 (1)과 같은 선형모델이고 비용함수는 식 (2)의 평균제곱오차를 사용한다. 여기에서, 우리는 이미 최적 매개변수가 식 (8)과 (9)에 의해 결정된다는 것을 알고 있으므로, 표 1의 데이터를 이 두 식에 대입하면 다음과 같은 값을 얻을 수 있다. 눈으로만 보지 말고 직접 계산하여 아래와 같은 값이 나오는지 확인해보자.

$$(w_0^*, w_1^*) = (1.4875, 4.1274)$$

따라서 다음과 같은 선형회귀 모델을 완성할 수 있다.

$$\hat{y} = 1.4875x + 4.1274 \quad (10)$$

이 모델을 그림 2에 표시하였다. 이때 평균제곱오차는 0.1962로 측정되었다. 이상적이라면 평균제곱오차가 0이 되겠지만, 그림 2에서 보는 바와 같이 1차 직선으로 모든 데이터를 관통하도록 만드는 것은 불가능하다. 따라서 표 2에 주어진 데이터로 만들 수 있는 최선의 선형회귀 모델은 식 (10)이라 할 수 있다.



[그림 2] 표 2의 데이터와 선형회귀 모델

7. 알고리즘을 이용한 최적해 탐색 - 경사하강법

다시 우리가 풀어야 하는 문제, 식 (3)을 다시 생각해보자. 다행히 5절과 6절에서 다루는 문제는 하나의 입력으로 이루어져 식 (4)와 같이 나타낼 수 있었고, 이는 수학적으로 풀 수 있었다. 그런데 안타깝게도 모든 머신러닝의 최적화 문제가 해석적으로 풀 수 있는 것은 아니다. 오히려 해석적으로 풀 수 없는 문제가 더 많다. 비용함수의 구조가 복잡하여 해석적으로 풀 수 없는 경우, 알고리즘을 이용해 최적해를 찾아간다. 이를 수치적인 접근이라 한다.

머신러닝 분야에서 다양한 수치적인 최적화 기법이 사용되는데, 여기에서는 가장 대표적인 경사하강법(Gradient Decent Method)을 소개한다. 경사하강법의 기본 아이디어는 다음과 같다. 먼저 우리가 찾으려는 최적 솔루션은 식 (6)과 (7)을 만족할 것이다. 즉, w_0 , w_1 , 평균제곱오차 $\epsilon_{MSE}(w_0, w_1)$ 을 x, y, z축으로 이루어진 3차원 공간의 그래프로 나타낸다면, 모든 w_0 와 w_1 에 대해 평균제곱오차는 향아리와 같이 오목한 구조물의 형태가 될 것이고, 그 구조물의 가장 낮은 지점이 바로 최적 솔루션이라는 것이다. 따라서 경사하강법은 향아리와 같은 구조물의 임의의 한 지점에 구슬을 놓고 그 구슬의 움직임을 따라가는 전략이다. 그리고 구슬이 정지하는 위치가 향아리에서 가장 낮은 위치, 즉 평균제곱오차가 가장 작은 최적 솔루션의 위치가 된다는 것이다.

경사하강법은 최적 매개변수 w_0^* 와 w_1^* 를 찾기 위해 단계별 접근 알고리즘을 사용한다. 다음은 현재 단계 t 에서의 매개변수 값을 바탕으로 다음 단계, 즉 $t+1$ 단계의 매개변수 값을 찾아가는 알고리즘을 나타낸다.

$$w_0[t+1] = w_0[t] - \alpha \frac{\partial}{\partial w_0} \epsilon_{MSE}(w_0, w_1) \quad (11)$$

$$w_1[t+1] = w_1[t] - \alpha \frac{\partial}{\partial w_1} \epsilon_{MSE}(w_0, w_1) \quad (12)$$

위 두 식에서 $w_0[t]$ 와 $w_1[t]$ 는 t 번째 단계에서의 매개변수 값이며 $w_0[0]$ 과 $w_1[0]$ 은 각 매개변수의 초기값(initial value)이다. 초기값은 일반적으로 무작위적으로 설정한다. 또한 $\frac{\partial}{\partial w_0} \epsilon_{MSE}(w_0, w_1)$ 과 $\frac{\partial}{\partial w_1} \epsilon_{MSE}(w_0, w_1)$ 은 3차원 공간에서 평균제곱오차가 갖는 w_0 와 w_1 방향의 기울기이다. 오목한 항아리 모양의 구조물 상의 임의의 위치에 놓인 구슬은 그 위치의 면의 가지는 기울기의 반대 방향으로 움직일 것이다. 따라서 $t+1$ 번째 단계에서의 구슬의 위치는 t 번째 단계의 위치의 기울기 반대방향(음의 방향)으로 이동한다. 또한, 한 단계 진행할 때 얼마만큼 이동하는지는 α 에 의해 결정된다. 즉, α 가 클수록 이전 단계의 위치로부터 멀리 이동한다. 따라서 α 는 매개변수가 얼마나 빨리 최적값으로 수렴하는지를 결정하는 값으로 학습률(learning rate)이라 한다.

식 (11)과 (12)는 현재 단계를 바탕으로 다음 단계의 매개변수 값을 찾아가는 식이므로 매개변수 업데이트 규칙 또는 매개변수 갱신 규칙이라 한다. 이 규칙에 따라 경사하강법의 단계를 진행해가면 매개변수의 값이 ①더 이상 변하지 않거나 ②변화량이 매우 작거나 ③같은 값이 반복적으로 나타나는 순간을 만나게 된다. 이는 최적 솔루션에 근접했거나 ①, ② 학습률이 너무 커 최적 솔루션으로 수렴하지 못하고 그 주변을 서성이는 경우(③)이다. 이 세 경우 모두 경사하강 알고리즘을 종료하는 조건에 해당하며 종료 단계의 w_0 와 w_1 의 값을 최적 솔루션으로 간주한다. 다만, ③의 경우 학습률을 조정하여 알고리즘을 반복 수행한 뒤 유사한 값으로 수렴하는지 확인해야 한다.

식 (3)의 문제에 경사하강법을 적용하기 위해서는 임의의 위치 (w_0, w_1) 에서의 평균제곱오차의 기울기를 알아야 한다. 이 경우 평균제곱오차의 형태가 단순하여 기울기도 다음과 같이 쉽게 구할 수 있다.

$$\frac{\partial}{\partial w_0} \epsilon_{MSE}(w_0, w_1) = \frac{2}{N} \sum_{n=0}^{N-1} x_n (w_0 x_n + w_1 - y_n) \quad (13)$$

$$\frac{\partial}{\partial w_1} \epsilon_{MSE}(w_0, w_1) = \frac{2}{N} \sum_{n=0}^{N-1} (w_0 x_n + w_1 - y_n) \quad (14)$$

따라서 식 (3)의 최적 솔루션을 구하기 위한 경사하강법의 업데이트 규칙은 다음과 같다.

$$w_0[t+1] = w_0[t] - \alpha \frac{2}{N} \sum_{n=0}^{N-1} x_n (w_0 x_n + w_1 - y_n) \quad (15)$$

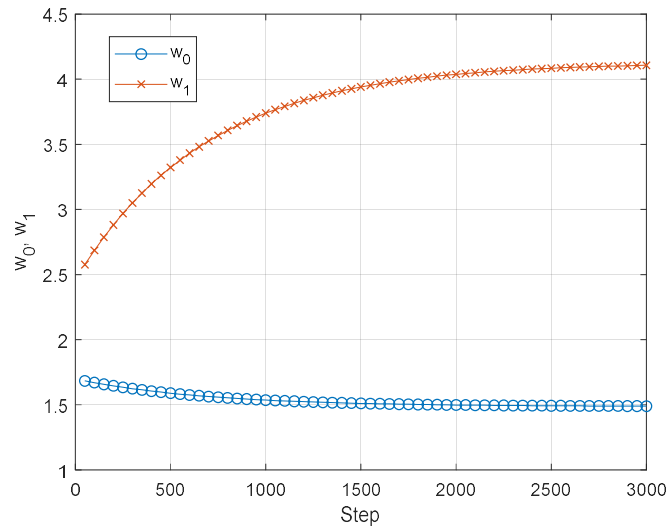
$$w_1[t+1] = w_1[t] - \alpha \frac{2}{N} \sum_{n=0}^{N-1} (w_0 x_n + w_1 - y_n) \quad (16)$$

8. 경사하강법을 이용한 선형회귀의 예

이번에는 6절에서 해석적인 방법으로 선형회귀 모델을 만들었던 표 2의 데이터를 이용해

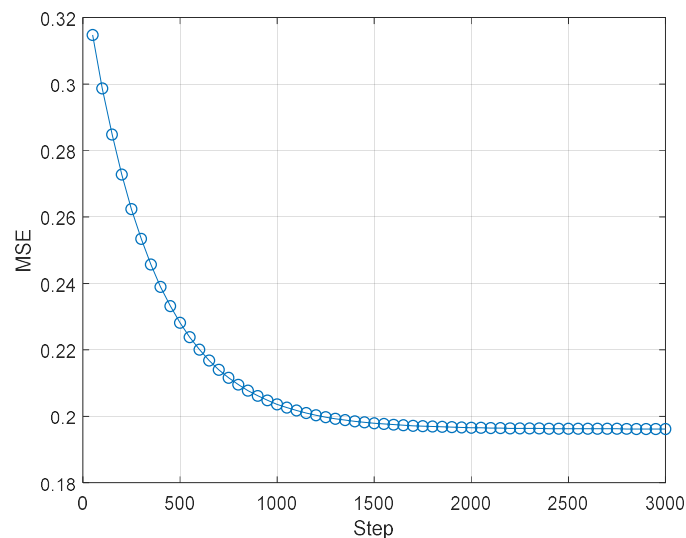
경사하강법을 적용한 선형회귀 모델을 만들고 해석해와 비교해보자.

그림 3은 경사하강법 적용을 위한 매개변수 초기값을 각각 $w_0[0] = 2$ 와 $w_1[0] = 2.5$ 로 설정하고 학습률을 $\alpha = 0.015$ 로 설정한 뒤 각 단계별 매개변수의 변화를 나타낸 것이다. 6절에서 해석적인 방법으로 찾은 최적 매개변수 (1.4875, 4.1274)와 비교하면 단계를 거듭할수록 최적 솔루션에 수렴하는 것을 확인할 수 있다.



[그림 3] 경사하강법의 단계별 매개변수 값의 변화, $\alpha = 0.015$.

그림 3에서 3000번의 업데이트를 진행한 후 얻은 매개변수는 $w_0 = 1.4902$, $w_1 = 4.1063$ 로 해석적으로 구한 최적 매개변수 값과 유사하다.

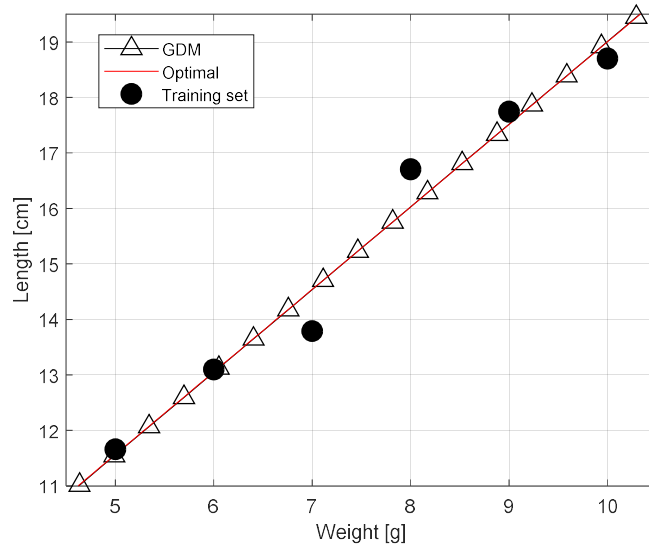


[그림 4] 단계별 평균제곱오차의 변화

그림 4는 경사하강법을 적용하였을 때 단계별 평균제곱오차의 변화를 그린 것이다. 그림 3

에서 확인하였듯이 단계를 거듭할수록 매개변수는 최적 솔루션으로 수렴하고 이에 따라 자연스럽게 평균제곱오차도 최소화된다. 그림 4의 최종 평균제곱오차는 0.1962로 최적해를 이용해 구한 평균제곱오차와 거의 일치한다.

그림 5는 해석적인 방법으로 찾은 최적 솔루션과 경사하강법으로 찾은 솔루션을 이용한 선형회귀 모델을 비교한 것이다. 그림에서 빨간색 실선은 최적 솔루션을, 삼각형 마커로 표시된 직선은 경사하강법을 적용한 모델이다. 두 모델 모두 거의 일치함을 확인할 수 있다.



[그림 5] 최적 솔루션과 경사하강법 솔루션 비교

이상에서 확인한 바와 같이 경사하강법은 매우 단순하면서도 강력한 성능을 보인다. 그러나 주의할 점도 있다. 가장 큰 문제는 경사하강법이 항상 수렴하는 것은 아니라는 점이다. 초기 값, 학습률, 비용함수의 구조에 매우 민감하며 최적 지점으로 수렴하지 않고 발산하는 경우가 자주 발생한다. 또 다른 문제는 로컬 미니멈(극소값)의 함정에 빠질 수 있다는 점이다. 이번 예와 같이 단순한 문제의 경우 비용함수가 하나의 극값을 갖는 항아리 모양이었다. 그러나 실제 문제에서는 비용함수가 이렇게 단순하지 않은 경우가 더 많다. 즉, 여러 높낮이의 항아리를 옆으로 이어 붙인 형태의 비용함수인 경우이다. 이 경우 경사하강의 결과물이 글로벌 미니멈(최소값)이라는 보장이 없다. 따라서 초기값을 다양하게 바꾸어가며 경사하강을 시도하고 결과를 검증하는 등의 추가적인 노력이 필요하다는 점을 기억해야 한다.

9. 다차원 데이터로의 확장

지금까지 살펴본 선형회귀 모델은 하나의 입력과 하나의 출력을 가지는 훈련 데이터를 위한 모델이었다. 이를 바탕으로 다수의 입력을 가지는 데이터에 대한 선형회귀 모델을 생각해본다. M 개의 입력과 한 개의 출력으로 이루어진 데이터 N 개로 구성된 훈련 데이터 집합을 표 3과 같이 나타낼 수 있다. 즉 하나의 출력 y 에 영향을 미치는 M 개의 입력 속성을 $x_0, x_1, \dots, x_{M-1}$ 으로 표시하며 이러한 데이터가 총 N 개가 모여 훈련 데이터 집합을 구성하는 것이다.

[표 3] 다중 입력 데이터 집합

데이터 번호	입력	출력
0	$x_{0,0}, x_{1,0}, \dots, x_{m,0}, \dots, x_{M-1,0}$	y_0
1	$x_{0,1}, x_{1,1}, \dots, x_{m,1}, \dots, x_{M-1,1}$	y_1
$\vdots$	$\vdots$	$\vdots$
n	$x_{0,n}, x_{1,n}, \dots, x_{m,n}, \dots, x_{M-1,n}$	y_n
$\vdots$	$\vdots$	$\vdots$
$N-1$	$x_{0,N-1}, x_{1,N-1}, \dots, x_{m,N-1}, \dots, x_{M-1,N-1}$	y_{N-1}

표 3의 데이터를 이용해 학습할 선형회귀 모델의 예측값 $\hat{y}$ 는 다음과 같다.

$$\hat{y} = w_0 x_0 + w_1 x_1 + \dots + w_{M-1} x_{M-1} + w_M \quad (17)$$

식 (17)은 식 (1)의 일반화 버전으로 매개변수 $w_0, w_1, \dots, w_{M-1}$ 은 M 개 입력의 선형조합을 위한 계수이고 w_M 은 절편을 나타낸다. 다음으로는 최적화를 위한 비용함수를 정의한다. 비용함수는 평균제곱오차를 사용한다.

$$\begin{aligned} \epsilon_{MSE} &= \frac{1}{N} \sum_{n=0}^{N-1} (\hat{y}_n - y_n)^2 \\ &= \frac{1}{N} \sum_{n=0}^{N-1} (w_0 x_{0,n} + w_1 x_{1,n} + \dots + w_{M-1} x_{M-1,n} + w_M - y_n)^2 \end{aligned} \quad (18)$$

식 (18)은 평균제곱오차를 이용한 비용함수이다. 이렇게 비용함수까지 정의하고 나면 최적 매개변수를 구하기 위해 해석적인 방법을 사용할지 아니면 경사하강을 이용해 최적에 근사한 값을 찾을지 선택해야 한다.

9.1 다차원 데이터의 해석해

식 (18)을 최소화는 매개변수를 손쉽게 구하기 위해서는 선형대수의 기법을 활용해야 한다. 또한 간단한 표현과 풀이를 위해 식 (17)에 간단한 트릭을 하나 적용한다. 다음 식을 보자.

$$\hat{y} = w_0 x_0 + w_1 x_1 + \dots + w_{M-1} x_{M-1} + w_M x_M \quad (19)$$

식 (19)는 식 (17)를 약간 변형한 것인데, (17)에는 없는 x_M 이라는 인자가 추가되었다. 이때 $x_M = 1$ 이라고 한다면 (19)는 (17)과 같다. 이렇게 표기하면 원래 식에 변화는 없으면서 다차원 선형회귀 모델을 다음과 같이 깔끔하게 정리할 수 있는 장점이 있다.

$$\hat{y} = [w_0 \ w_1 \ \dots \ w_{M-1} \ w_M] \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{M-1} \\ x_M \end{bmatrix} = \mathbf{w}^T \mathbf{x} \quad (20)$$

이때 벡터 $\mathbf{w}$ 와 $\mathbf{x}$ 는 다음과 같이 정의된다.

$$\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_{M-1} \\ w_M \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{M-1} \\ x_M \end{bmatrix} \quad (21)$$

식 (19)에 $x_M = 1$ 을 추가함으로써 (20)과 같이 간단하게 표현 가능한 것이다. 이렇게 표기의 편의를 위해 추가한 x_M 과 같은 변수를 더미(dummy) 변수라 한다. 이제 (20)을 이용해 평균 제곱오차를 정의하면 다음과 같다.

$$\epsilon_{MSE} = \frac{1}{N} \sum_{n=0}^{N-1} (\hat{y}_n - y_n)^2 = \frac{1}{N} \sum_{n=0}^{N-1} (\mathbf{w}^T \mathbf{x}_n - y_n)^2 \quad (22)$$

이때 벡터 $\mathbf{x}_n$ 은 n 번째 데이터의 입력값들로 다음과 같다.

$$\mathbf{x}_n = \begin{bmatrix} x_{0,n} \\ x_{1,n} \\ \vdots \\ x_{M-1,n} \\ x_{M,n} \end{bmatrix} \quad (23)$$

해석해를 찾기 위해서는 식 (22)를 각 매개변수에 대한 편미분 함수를 0으로 만드는 매개변수를 찾아야 한다. 따라서 임의의 매개변수 w_m 에 대한 편미분 함수를 다음과 같이 구한다.

$$\begin{aligned} \frac{\partial}{\partial w_m} \epsilon_{MSE} &= \frac{2}{N} \sum_{n=0}^{N-1} (\mathbf{w}^T \mathbf{x}_n - y_n) \frac{\partial}{\partial w_m} \mathbf{w}^T \mathbf{x}_n \\ &= \frac{2}{N} \sum_{n=0}^{N-1} (\mathbf{w}^T \mathbf{x}_n - y_n) x_{m,n} \end{aligned} \quad (24)$$

즉, $m = 0, 1, 2, \dots, M-1, M$ 의 $M+1$ 개의 매개변수의 해는 다음과 같은 연립방정식의 해와 같다.

$$\begin{aligned} m=0 \text{ 일 때, } & \sum_{n=0}^{N-1} (\mathbf{w}^T \mathbf{x}_n - y_n) x_{0,n} = 0 \\ m=1 \text{ 일 때, } & \sum_{n=0}^{N-1} (\mathbf{w}^T \mathbf{x}_n - y_n) x_{1,n} = 0 \\ & \vdots \\ m=M-1 \text{ 일 때, } & \sum_{n=0}^{N-1} (\mathbf{w}^T \mathbf{x}_n - y_n) x_{M-1,n} = 0 \\ m=M \text{ 일 때, } & \sum_{n=0}^{N-1} (\mathbf{w}^T \mathbf{x}_n - y_n) x_{M,n} = 0 \end{aligned}$$

위의 M 개의 연립방정식을 모아 다음과 같은 하나의 선형대수 표현으로 나타낼 수 있다.

$$\sum_{n=0}^{N-1} (\mathbf{w}^T \mathbf{x}_n - y_n) [x_{0,n} \ x_{1,n} \ \dots \ x_{M-1,n} \ x_{M,n}] = [0 \ 0 \ \dots \ 0 \ 0] \quad (25)$$

또한, 식 (23)을 이용하면 (25)는 다음과 같다.

$$\sum_{n=0}^{N-1} (w^T x_n - y_n) x_n^T = [0 \ 0 \ \cdots \ 0 \ 0] \quad (26)$$

여기에서 식 (26)의 좌변을 좀 더 정리하면 다음과 같다.

$$\sum_{n=0}^{N-1} (w^T x_n - y_n) x_n^T = \sum_{n=0}^{N-1} (w^T x_n x_n^T - y_n x_n^T) = w^T \sum_{n=0}^{N-1} x_n x_n^T - \sum_{n=0}^{N-1} y_n x_n^T$$

따라서 식 (26)은 다음과 같이 쓸 수 있다.

$$w^T \sum_{n=0}^{N-1} x_n x_n^T - \sum_{n=0}^{N-1} y_n x_n^T = [0 \ 0 \ \cdots \ 0 \ 0] \quad (27)$$

식 (27)의 좌변의 첫 번째 항에 있는 $\sum_{n=0}^{N-1} x_n x_n^T$ 을 먼저 살펴보자. 행렬의 2차원적인 구조를

수월하게 이해하기 위해 $M=2, N=3$ 인 경우를 생각해보자. 즉, 입력 속성이 2개이며 데이터가 총 3개인 경우이다.

$$\begin{aligned} \sum_{n=0}^2 x_n x_n^T &= \sum_{n=0}^2 \begin{bmatrix} x_{0,n} \\ x_{1,n} \\ x_{2,n} \end{bmatrix} [x_{0,n} \ x_{1,n} \ x_{2,n}] \\ &= \sum_{n=0}^2 \begin{bmatrix} x_{0,n}^2 & x_{0,n}x_{1,n} & x_{0,n}x_{2,n} \\ x_{1,n}x_{0,n} & x_{1,n}^2 & x_{1,n}x_{2,n} \\ x_{2,n}x_{0,n} & x_{2,n}x_{1,n} & x_{2,n}^2 \end{bmatrix} \\ &= \begin{bmatrix} x_{0,0}^2 & x_{0,0}x_{1,0} & x_{0,0}x_{2,0} \\ x_{1,0}x_{0,0} & x_{1,0}^2 & x_{1,0}x_{2,0} \\ x_{2,0}x_{0,0} & x_{2,0}x_{1,0} & x_{2,0}^2 \end{bmatrix} + \begin{bmatrix} x_{0,1}^2 & x_{0,1}x_{1,1} & x_{0,1}x_{2,1} \\ x_{1,1}x_{0,1} & x_{1,1}^2 & x_{1,1}x_{2,1} \\ x_{2,1}x_{0,1} & x_{2,1}x_{1,1} & x_{2,1}^2 \end{bmatrix} + \begin{bmatrix} x_{0,2}^2 & x_{0,2}x_{1,2} & x_{0,2}x_{2,2} \\ x_{1,2}x_{0,2} & x_{1,2}^2 & x_{1,2}x_{2,2} \\ x_{2,2}x_{0,2} & x_{2,2}x_{1,2} & x_{2,2}^2 \end{bmatrix} \\ &= \begin{bmatrix} x_{0,0} & x_{0,1} & x_{0,2} \\ x_{1,0} & x_{1,1} & x_{1,2} \\ x_{2,0} & x_{2,1} & x_{2,2} \end{bmatrix} \begin{bmatrix} x_{0,0} & x_{1,0} & x_{2,0} \\ x_{0,1} & x_{1,1} & x_{2,1} \\ x_{0,2} & x_{1,2} & x_{2,2} \end{bmatrix} \\ &= X^T X \end{aligned} \quad (28)$$

행렬 X 의 각 열은 M 개의 입력 속성을 의미하고 각 행은 N 개의 데이터 집합을 의미하는 데이터 행렬이라 할 수 있다. 즉, 표 3에 표현한 입력 데이터를 그대로 행렬로 나타낸 것이다. 이를 일반화하면 다음과 같다. 여기에서 행렬 X 의 마지막 열의 값은 모두 1이다. 이는 식 (19)에 적용한 트릭 때문임을 기억하기 바란다.

$$X = \begin{bmatrix} x_{0,0} & x_{1,0} & \cdots & x_{M-1,0} & x_{M,0} \\ x_{0,1} & x_{1,1} & \cdots & x_{M-1,1} & x_{M,1} \\ \vdots & \vdots & & \vdots & \vdots \\ x_{0,N-1} & x_{1,N-1} & \cdots & x_{M-1,N-1} & x_{M,N-1} \end{bmatrix} \quad (29)$$

다음으로, 식 (27)의 좌변의 두 번째 항 $\sum_{n=0}^{N-1} y_n x_n^T$ 을 살펴보자. 역시 같은 방법으로

$M=2, N=3$ 인 경우를 먼저 살펴보면 일반화하기 수월하다.

$$\begin{aligned}
 \sum_{n=0}^2 y_n \mathbf{x}_n^T &= \sum_{n=0}^2 y_n [x_{0,n} \ x_{1,n} \ x_{2,n}] \\
 &= \sum_{n=0}^2 [y_n x_{0,n} \ y_n x_{1,n} \ y_n x_{2,n}] \\
 &= [y_0 x_{0,0} \ y_0 x_{1,0} \ y_0 x_{2,0}] + [y_1 x_{0,1} \ y_1 x_{1,1} \ y_1 x_{2,1}] + [y_2 x_{0,2} \ y_2 x_{1,2} \ y_2 x_{2,2}] \\
 &= [y_0 \ y_1 \ y_2] \begin{bmatrix} x_{0,0} & x_{1,0} & x_{2,0} \\ x_{0,1} & x_{1,1} & x_{2,1} \\ x_{0,2} & x_{1,2} & x_{2,2} \end{bmatrix} \\
 &= \mathbf{y}^T \mathbf{X}
 \end{aligned} \tag{30}$$

이때 벡터 $\mathbf{y}$ 는 N 개의 데이터 출력을 나타내는 것으로 다음과 같이 일반화할 수 있다.

$$\mathbf{y} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{N-1} \end{bmatrix} \tag{31}$$

또한 행렬 $\mathbf{X}$ 는 데이터의 입력을 나타내는 것으로 식 (29)에 일반화하였다. 따라서 식 (28)과 (30)을 (27)에 적용하면 최적 솔루션을 구하기 위해 풀어야 할 방정식을 다음과 같이 간단히 표현할 수 있다.

$$\mathbf{w}^T \mathbf{X}^T \mathbf{X} - \mathbf{y}^T \mathbf{X} = [0 \ 0 \ \dots \ 0 \ 0] \tag{32}$$

이제 (32)의 솔루션 $\mathbf{w}$ 를 구하면 우리의 목표를 달성할 수 있다. 그런데 이 식에는 $\mathbf{w}^T$ 로 표현되어 있으므로 $\mathbf{w}$ 를 구하기 위해 양변에 트랜스포즈를 취하면 (32)의 좌변은 다음과 같다.

$$\begin{aligned}
 (\mathbf{w}^T \mathbf{X}^T \mathbf{X} - \mathbf{y}^T \mathbf{X})^T &= (\mathbf{w}^T \mathbf{X}^T \mathbf{X})^T - (\mathbf{y}^T \mathbf{X})^T \\
 &= (\mathbf{X}^T \mathbf{X})^T \mathbf{w} - \mathbf{X}^T \mathbf{y} \\
 &= \mathbf{X}^T \mathbf{X} \mathbf{w} - \mathbf{X}^T \mathbf{y}
 \end{aligned}$$

따라서 식 (32)의 양변에 트랜스포즈를 적용하면 다음과 같다.

$$\mathbf{X}^T \mathbf{X} \mathbf{w} - \mathbf{X}^T \mathbf{y} = [0 \ 0 \ \dots \ 0 \ 0]^T \tag{33}$$

이제 우리의 목표인 $\mathbf{w}$ 가 잘 보인다. 남은 단계는 다음과 같이 아주 간단하다.

$$\begin{aligned}
 \mathbf{X}^T \mathbf{X} \mathbf{w} &= \mathbf{X}^T \mathbf{y} \\
 \mathbf{w} &= (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}
 \end{aligned} \tag{34}$$

바로 식 (34)가 우리가 찾는 최적 솔루션이다. 이 식을 이용하면 훈련 데이터의 입력의 개수와 데이터의 개수에 상관없이 최적의 선형회귀 모델을 위한 매개변수를 구할 수 있다. 식 (8)과 (9)의 솔루션 역시 (34)의 특수한 형태이다. 따라서 식 (34)는 선형회귀 모델의 일반해이고, M 개의 입력 속성과 하나의 출력 속성을 갖는 어떠한 데이터에 대해서도 식 (34)를 통해 최적 매개변수를 구할 수 있다.

9.2 다차원 데이터를 위한 경사하강법

경사하강법을 적용하기 위해서는 모든 매개변수에 대한 비용함수 (18)의 기울기를 먼저 구해야 한다. 모든 매개변수에 대한 평균제곱오차의 기울기는 다음과 같다.

$$\begin{aligned}\frac{\partial}{\partial w_0} \epsilon_{MSE} &= \frac{2}{N} \sum_{n=0}^{N-1} x_{0,n} (w_0 x_{0,n} + w_1 x_{1,n} + \dots + w_{M-1} x_{M-1,n} + w_M - y_n) \\ \frac{\partial}{\partial w_1} \epsilon_{MSE} &= \frac{2}{N} \sum_{n=0}^{N-1} x_{1,n} (w_0 x_{0,n} + w_1 x_{1,n} + \dots + w_{M-1} x_{M-1,n} + w_M - y_n) \\ &\vdots \\ \frac{\partial}{\partial w_{M-1}} \epsilon_{MSE} &= \frac{2}{N} \sum_{n=0}^{N-1} x_{M-1,n} (w_0 x_{0,n} + w_1 x_{1,n} + \dots + w_{M-1} x_{M-1,n} + w_M - y_n) \\ \frac{\partial}{\partial w_M} \epsilon_{MSE} &= \frac{2}{N} \sum_{n=0}^{N-1} (w_0 x_{0,n} + w_1 x_{1,n} + \dots + w_{M-1} x_{M-1,n} + w_M - y_n)\end{aligned}$$

위 식에서 규칙성을 발견할 수 있다. 이를 이용한 매개변수 업데이트 규칙은 다음과 같다.

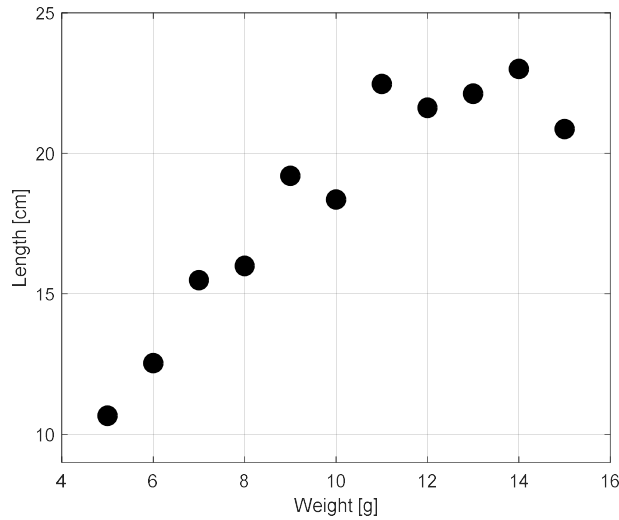
$$\begin{aligned}w_0[t+1] &= w_0[t] - \alpha \frac{\partial}{\partial w_0} \epsilon_{MSE} \\ w_1[t+1] &= w_1[t] - \alpha \frac{\partial}{\partial w_1} \epsilon_{MSE} \\ &\vdots \\ w_{M-1}[t+1] &= w_{M-1}[t] - \alpha \frac{\partial}{\partial w_{M-1}} \epsilon_{MSE} \\ w_M[t+1] &= w_M[t] - \alpha \frac{\partial}{\partial w_M} \epsilon_{MSE}\end{aligned}$$

각 매개변수의 업데이트 규칙을 이용하여 최적 매개변수에 근사한 값을 찾을 수 있다. 그러나 경사하강법을 통해 얻은 솔루션이 최적 솔루션임을 보장할 수 없다는 점을 기억해야 한다. 초기값, 학습률, 비용함수의 구조에 따라 로컬 미니멈에 수렴할 가능성이 늘 존재하기 때문이다.

10. 선형 기저함수 모델 (Linear basis function model)

선형회귀 모델은 입력 데이터의 선형조합을 통해 출력을 예측하기 때문에, 입력과 출력 사이에 선형적인 관계가 성립하는 데이터의 회귀예측에 적합하다. 입력과 출력이 선형적인 관계를 갖는다는 것은 그래프로 표현했을 때 직선으로 모델링 가능함을 의미한다. 앞에서 살펴본 그림 2의 예가 대표적인 선형관계라 할 수 있다.

그런데 실제 상황에서는 선형 데이터보다 비선형 데이터를 더 많이 만나게 된다. 예를 들어 용수철 실험을 다시 생각해보자. 용수철에 달린 추의 무게를 늘리면 용수철은 일정한 비율로 늘어난다. 그런데 용수철이 가지는 능력의 한계가 있으므로 무게를 무한정 늘린다고 길이가 무한정 늘어나지는 않을 것이다. 무게를 점점 늘리면 어느 순간 용수철이 늘어나지 않는 한계 구간을 만나게 된다. 이 지점에서 무게와 용수철의 길이 사이의 선형성이 깨지고 비선형 관계가 형성된다.



[그림 6] 추의 무게를 늘리면서 관찰한 용수철의 길이

그림 6은 추의 무게를 한계 구간 이상으로 늘리면서 측정한 길이를 표시한 것이다. 무게가 10g보다 작을 때에는 무게와 길이의 관계가 어느 정도 선형적이지만 10g을 넘어서면서 길이가 더 이상 같은 비율로 증가하지 않음을 알 수 있다. 이로 인해 데이터의 전체적인 형태가 일정한 비율로 증가하다가 어느 순간 포화되는 것처럼 나타난다. 이러한 특성을 비선형 특성이라 한다.

앞서 공부한 선형회귀 모델은 비선형성을 내포하는 데이터를 제대로 표현하지 못한다. 이를 극복하기 위해서는 선형회귀 모델에 비선형성을 추가해야 한다. 선형회귀 모델에 비선형성을 추가한 모델이 선형 기저함수 모델이다.

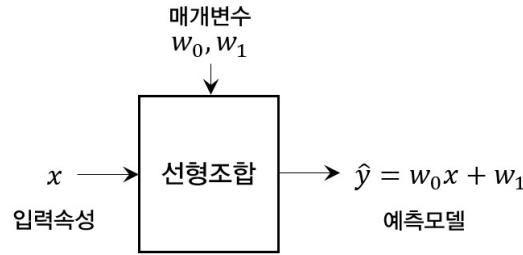
10.1 선형 기저함수 모델

선형회귀 모델은 출력을 예측하기 위해 입력 데이터의 선형조합을 사용하는 모델이다. 입력 속성이 하나인 경우 식 (1)과 같고 여러 개로 확장된 경우에는 식 (17)과 같다. 본 절에서는 설명의 편의를 위해 입력이 하나인 경우를 생각한다.

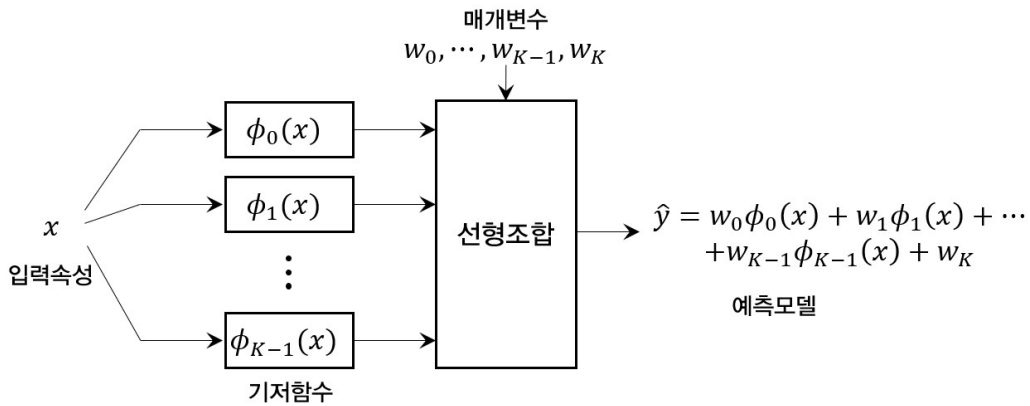
선형 기저함수 모델은 입력 속성을 그대로 선형조합하는 대신 기저함수(basis function)라는 함수에 입력을 넣어 얻은 출력값을 선형조합하는 모델이다. 이때 기저함수는 다양한 함수가 사용될 수 있으며, 비선형 함수여도 상관없다. 그림 7은 선형 기저함수 모델을 선형회귀 모델과 비교하여 그 차이를 명확하게 보여준다. 그림 7에서 (a)와 (b)의 가장 대표적인 차이는 선형조합을 위해 입력속성을 그대로 사용하느냐 아니면 가공하여 사용하느냐이다. 지금까지 공부한 선형회귀 모델은 선형조합을 위해 입력속성을 그대로 사용하는 반면 선형 기저함수 모델은 입력속성을 K 개의 기저함수로 가공한 뒤 이들을 선형조합한다.

그림 7(b)와 같이 입력속성 x 를 K 개의 기저함수에 인가하면 K 개의 새로운 값들을 얻을 수 있고 이들을 선형조합하면 다음과 같다.

$$\hat{y} = w_0\phi_0(x_0) + w_1\phi_1(x_1) + \cdots + w_{K-1}\phi_{K-1}(x_{K-1}) + w_K \quad (35)$$



(a) 선형회귀 모델



(b) 선형 기저함수 모델

[그림 7] 선형회귀 모델과 선형 기저함수 모델 비교

이때 바이어스(절편) w_K 를 포함한 $K+1$ 개의 매개변수 $w_0, w_1, \dots, w_{K-1}, w_K$ 는 기저함수 출력 값들의 선형조합을 위한 값들이다. 이제 목표는 $\hat{y} \approx y$ 가 되도록 최적의 매개변수를 구하는 것이다.

식 (35)를 위한 최적 매개변수를 쉽게 구할 수 있는 방법이 있다. 식 (35)를 (19)와 비교해 보자. 식 (19)는 입력속성이 M 개인 선형회귀 모델이다. 두 식을 비교해보면, 식 (19)에서 M 을 K 로 바꾸고, x_m 을 $\phi_k(x)$ 로 바꾸면 (35)가 된다는 것을 알 수 있다. 따라서 식 (19)를 풀었던 과정과 결과를 그대로 활용할 수 있다. 먼저, 식 (35)를 다음과 같이 벡터 표현으로 나타낼 수 있다.

$$\hat{y} = [w_0 \ w_1 \ \dots \ w_{K-1} \ w_K] \begin{bmatrix} \phi_0(x) \\ \phi_1(x) \\ \vdots \\ \phi_{K-1}(x) \\ \phi_K(x) \end{bmatrix} = \mathbf{w}^T \boldsymbol{\phi}(x) \quad (36)$$

이때 $\phi_K(x)$ 는 단순한 표기를 위해 추가한 것으로 항상 1이며, 각 벡터는 다음과 같이 정의된다.

$$\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_{K-1} \\ w_K \end{bmatrix}, \quad \boldsymbol{\phi}(x) = \begin{bmatrix} \phi_0(x) \\ \phi_1(x) \\ \vdots \\ \phi_{K-1}(x) \\ \phi_K(x) \end{bmatrix} \quad (37)$$

최적화를 위한 비용함수로서 평균제곱오차를 정의하면 다음과 같다.

$$\epsilon_{MSE} = \frac{1}{N} \sum_{n=0}^{N-1} (\hat{y}_n - y_n)^2 = \frac{1}{N} \sum_{n=0}^{N-1} (\boldsymbol{\phi}(x_n) - y_n)^2 \quad (38)$$

식 (38)에서 $\boldsymbol{\phi}(x_n)$ 은 n 번째 입력 데이터를 각 기저함수에 넣은 값들의 벡터로 다음과 같다.

$$\boldsymbol{\phi}(x_n) = \begin{bmatrix} \phi_0(x_n) \\ \phi_1(x_n) \\ \vdots \\ \phi_{K-1}(x_n) \\ \phi_K(x_n) \end{bmatrix} \quad (39)$$

다음 과정은 식 (38)을 최소화는 매개변수를 찾는 과정으로 식 (24) ~ (34)의 과정을 반복하게 된다. 따라서, 이곳에서는 앞의 결과를 최대한 활용한다. 식 (29)의 입력값들의 행렬은 다음과 같이 입력값을 기저함수에 넣은 값들의 행렬로 바꾸면 된다.

$$\boldsymbol{\Phi} = \begin{bmatrix} \phi_0(x_0) & \phi_1(x_0) & \cdots & \phi_{K-1}(x_0) & \phi_K(x_0) \\ \phi_0(x_1) & \phi_1(x_1) & \cdots & \phi_{K-1}(x_1) & \phi_K(x_1) \\ \vdots & \vdots & & \vdots & \vdots \\ \phi_0(x_{N-1}) & \phi_1(x_{N-1}) & \cdots & \phi_{K-1}(x_{N-1}) & \phi_K(x_{N-1}) \end{bmatrix} \quad (40)$$

식 (40)에서 각 행은 N 개의 입력 데이터 중 하나를 의미하고 각 열은 K 개의 기저함수 중 하나를 의미한다. 이때, 마지막 열의 값은 모두 1이라는 점을 기억해야 한다. 또한 N 개의 출력 데이터를 나타내는 벡터 $\mathbf{y}$ 는 다음과 같이 식 (31)과 동일하다.

$$\mathbf{y} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{N-1} \end{bmatrix} \quad (41)$$

이제 필요한 벡터와 행렬을 모두 정의했다. 복잡한 중간 과정은 생략하고 최종 결론인 (34)를 재활용한다. 이를 위해 식 (34)에서 $\mathbf{X}$ 를 $\boldsymbol{\Phi}$ 로 바꾸기만 하면 된다. 결론적으로 식 (35)의 선형 기저함수 모델의 최적 매개변수는 다음과 같이 구할 수 있다.

$$\mathbf{w} = (\boldsymbol{\Phi}^T \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^T \mathbf{y} \quad (42)$$

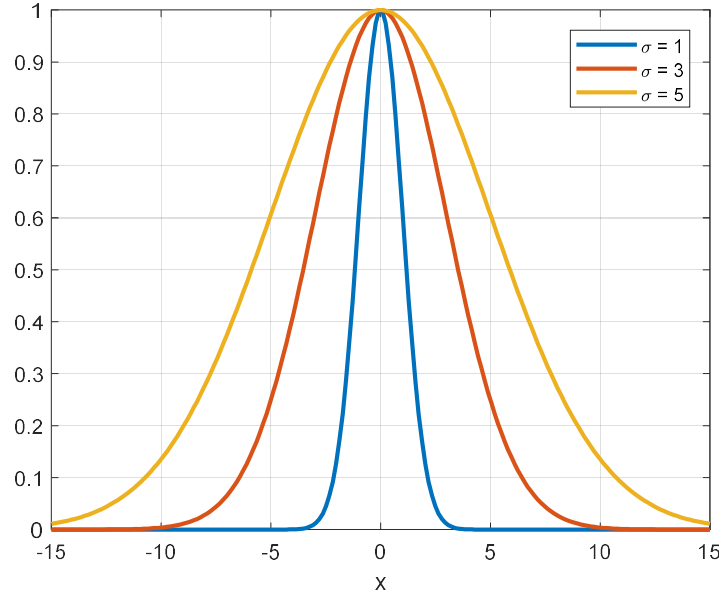
10.2 가우스 함수를 이용한 선형 기저함수 모델

위에서 살펴본 내용은 임의의 기저함수 $\phi_k(x)$ 를 사용하는 선형 기저함수 모델과 해석해에 관한 것이다. 본 절은 기저함수로서 가우스 함수를 사용하는 선형 기저함수 모델을 살펴본다. 가우스 함수는 통계학 등 다양한 분야에 널리 사용되는 함수로 다음과 같이 정의된다.

$$\phi_k(x) = e^{-\frac{1}{2} \left(\frac{x - \mu_k}{\sigma} \right)^2} \quad (43)$$

가우스 함수는 두 개의 매개변수 μ_k , σ 와 하나의 입력 x 로 구성된다. 정규분포의 확률밀도함

수로 사용될 때 μ_k , σ 는 평균과 표준편차로 사용된다. 가우스 함수는 $x = \mu_k$ 를 중심으로 좌우 대칭인 함수이며 σ 는 대칭을 중심을 기준으로 함수값이 얼마나 퍼져있는지를 결정한다. 그림 8은 $\mu = 0$ 이고, 서로 다른 σ 값을 갖는 가우스 함수들을 나타낸 것이다.



[그림 8] $\mu = 0$ 이고, 서로 다른 σ 값을 갖는 가우스 함수들

그림 6의 데이터틀 이용해 선형 기저함수 모델을 만들어보자. 먼저, 기저함수의 개수는 K 개로 한다. 다음으로 각 기저함수의 매개변수 μ_k 와 σ 값을 정해야 한다. 일반적으로 모든 기저함수가 데이터의 전 영역을 고루 표현할 수 있어야 하므로, 기저함수의 중심 위치를 결정하는 μ_k 값을 데이터의 영역의 가장 작은 값과 가장 큰 값 사이에 균일한 값을 갖도록 설정한다. 즉, 그림 6의 데이터에서는 추의 무게가 최소 5g, 최대 15g이므로 $x_{\min} = 5$, $x_{\max} = 15$ 으로 설정하고 각 기저함수의 μ_k 값을 다음과 같이 계산한다.

$$\mu_k = x_{\min} + \frac{x_{\max} - x_{\min}}{K-1}k, \quad k = 0, 1, \dots, K-1 \quad (44)$$

또한 각 기저함수가 μ_k 값으로부터 퍼져있는 정도를 나타내는 σ 값 역시 모든 기저함수가 모든 데이터 영역을 커버하면서 인근 기저함수에 미치는 영향을 줄이기 위해 다음과 같이 인근 기저함수 중심간의 거리의 평균으로 설정하도록 한다.

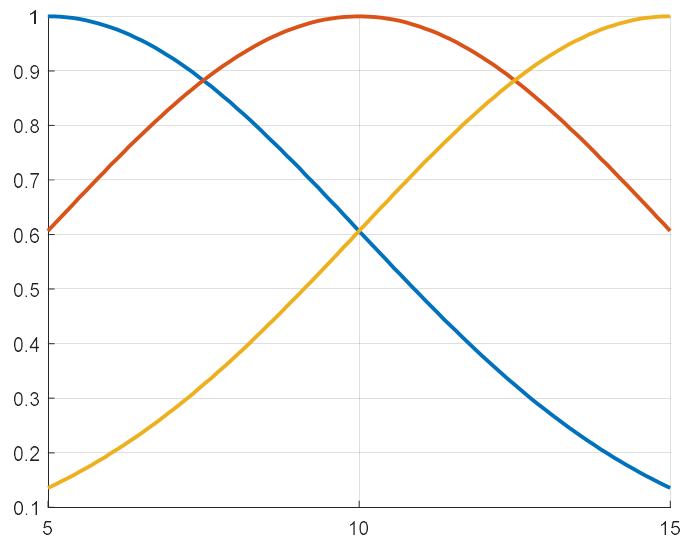
$$\sigma = \frac{x_{\max} - x_{\min}}{K-1} \quad (45)$$

기저함수가 세 개인 경우 식 (44)와 (45)에 따라 각 기저함수의 매개변수를 계산하면 표 4와 같다.

[표 4] $K=3$, $x_{\min}=5$, $x_{\max}=15$ 인 경우 각 기저함수의 매개변수 설정값

기저함수	매개변수	
	μ_k	σ
$\phi_0(x)$	5	5
$\phi_1(x)$	10	
$\phi_2(x)$	15	

표 4의 값으로 계산한 세 개의 가우스 함수를 그래프로 그리면 그림 9와 같다. 학습하려는 데이터의 범위 5~15 사이에 세 개의 가우스 함수가 균일하게 분포하는 것을 알 수 있다. 이제 이 세 개의 가우스 함수를 이용해 선형 기저함수 모델을 만들어보자.



[그림 9] 표 4의 매개변수로 그린 가우스 함수

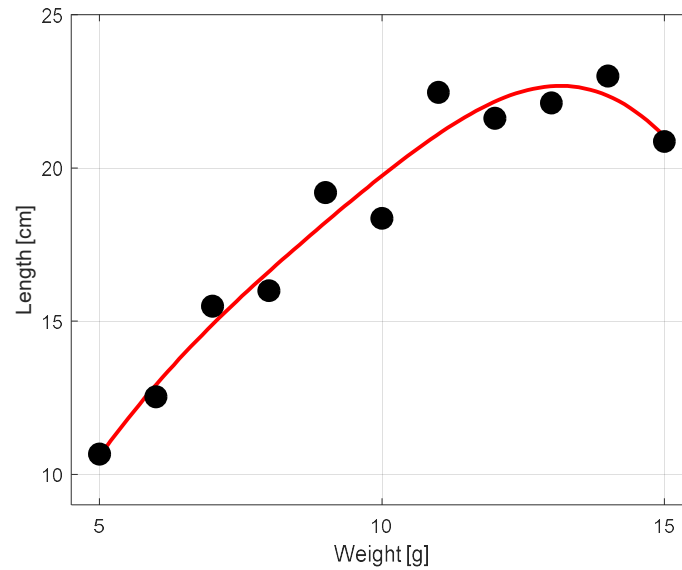
선형 기저함수 모델의 최적 매개변수 계산은 매우 간단하다. 먼저 행렬 (40)을 정의하고 이를 식 (42)에 넣으면 된다. 계산 과정은 생략하고 결과를 보면 다음과 같다. 컴퓨터나 계산기를 이용하면 쉽게 계산할 수 있으므로 반드시 계산해보기 바란다.

$$w = \begin{bmatrix} 27.02 \\ 3.46 \\ 39.08 \\ -23.82 \end{bmatrix}$$

따라서 선형 기저함수 모델은 다음과 같다.

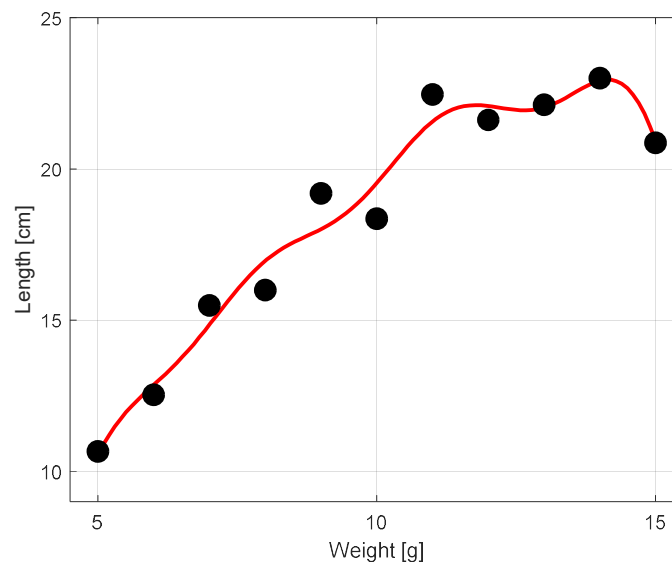
$$\hat{y} = 27.02 e^{-\frac{1}{2}\left(\frac{x-5}{5}\right)^2} + 3.46 e^{-\frac{1}{2}\left(\frac{x-10}{5}\right)^2} + 39.08 e^{-\frac{1}{2}\left(\frac{x-15}{5}\right)^2} - 23.82$$

그림 10은 위 결과를 이용해 나타낸 그래프이다. 가우스 함수를 이용해 비선형성을 추가한 결과 학습 모델이 데이터를 매우 잘 표현하고 있음을 알 수 있다. 이 경우 평균제곱오차는 0.6044이다.



[그림 10] 학습 데이터와 $K=3$ 인 선형 기저함수 모델과의 비교

그럼 K 값을 높이면 어떻게 될까? 즉, 기저함수의 수를 늘리면 더 좋은 성능의 모델을 만들 수 있을까? 당연히 K 가 커질수록 선형 기저함수 모델이 데이터의 속성을 표현할 수 있는 자유도가 증가하여 더욱 좋은 결과를 얻을 수 있다. 그림 11은 $K=8$, 즉 기저함수가 8개인 경우의 모델이다. 그림 10과 비교하면 그래프가 더욱 구불구불 데이터에 가까워지고 있다. 측정된 평균제곱오차는 0.4815로 $K=3$ 인 경우 대비 약 20%의 성능이 향상되었다.



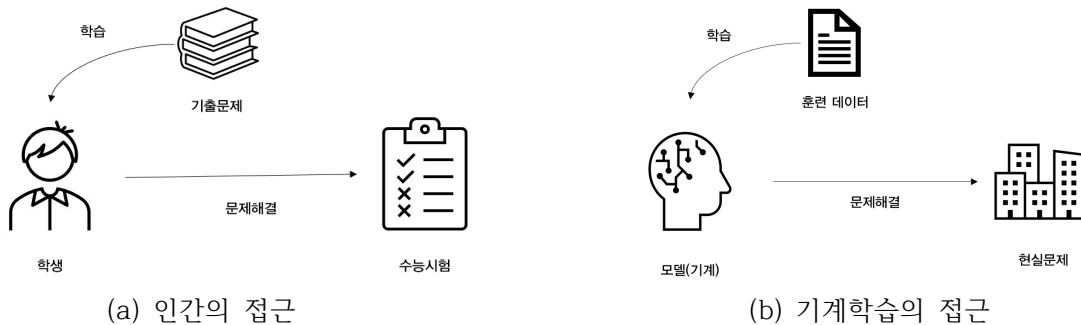
[그림 11] $K=8$ 인 선형 기저함수 모델의 성능

K 값을 높이면 항상 좋은 결과를 얻을 수 있는지에 대해서는 보다 깊은 논의가 필요하다. 그러나, 이 주제는 선형회귀의 범위를 넘어가므로 여기에서는 “ K 값이 커질수록 평균제곱오차를 줄일 수 있다”라는 결론을 내리는 선에서 마무리한다.

2. 모델의 검증과 평가 (Model Test and Evaluation)

1. 기계학습의 목표

기계학습의 목표는 컴퓨터(기계)를 학습시켜 실제 문제를 해결할 수 있도록 하는 것이다. 그림 1은 학습을 통한 문제해결 모델을 사람의 측면과 기계학습의 측면으로 간단히 나타낸 것이다. 그림 1의 (a)는 수능시험을 준비하는 어떤 학생의 접근 방법이다. 이 학생은 그동안의 기출 문제를 중심으로 학습하였다. 기출문제를 통해 문제의 패턴을 학습하고 자주 출제되는 유형을 파악할 수 있을 것이다. 학습을 마친 학생은 실제 수능시험에 도전해 문제를 해결하려고 한다.



[그림 1] 학습을 통한 문제해결 모델

그림 1(b)는 기계학습의 구조이다. 수능을 준비하는 학생과 같이 기계학습의 기계는 훈련 데이터를 학습하고 학습이 완료되면 현실문제에 적용해 문제를 해결한다. 따라서 기계학습의 궁극적 목표는 훈련 데이터를 학습해 현실문제를 잘 해결하는 것이다. 이렇게 기계학습을 통해 현실문제를 해결하는 능력을 일반화 능력(generalization capability)라고 한다. 즉, 기계학습은 훈련 데이터의 학습을 통해 일반화 능력을 극대화하는 과정이라 볼 수 있다.

기계학습의 일반화 능력을 최대화하기 위해서는 두 가지 조건이 선행되어야 한다. 첫째는 훈련 데이터가 현실문제의 특징을 제대로 반영해야 한다. 이는 좋은 교재로 공부해야 실제 시험에서 좋은 성적을 얻을 수 있다는 말과 같은 의미이다. 두 번째 조건은 모델의 학습 능력이 우수해야 한다. 즉, 훈련 데이터가 아무리 훌륭하더라도 훈련 모델의 능력이 부족하면 제대로 된 학습이 불가하기 때문이다.

어떤 모델의 성능을 평가하기 위해서는 객관적인 지표가 필요하다. 분류 시스템의 성능을 측정하기 위해 많이 사용하는 지표는 오차율(error rate)과 정확도(accuracy)이다. 분류 시스템에 N 개의 데이터를 적용했을 때 k 개의 데이터에 대해 예측에 실패했다면 오차율 p_{err} 과 정확도 p_{acc} 는 다음과 같이 정의된다.

$$p_{err} = \frac{k}{N}, \quad p_{acc} = 1 - p_{err} = \frac{N-k}{N} \quad (1)$$

이때 훈련 집합을 이용해 성능을 측정했다면, 측정 결과를 훈련 오차율과 훈련 정확도라고 부르고, 훈련 과정에 사용하지 않은 데이터를 이용해 성능을 측정했다면 그 결과를 일반화 오차율 및 일반화 정확도라 한다.

회귀 시스템의 경우 예측 성공과 실패를 정수로 표현할 수 없으므로, 평균제곱오차를 이용해 성능을 측정한다. 성능 측정을 위해 N 개의 데이터를 회귀 시스템에 적용해 얻은 예측치가 $\hat{y}$ 이고 실제 데이터 레이블(출력)이 y 라면 평균제곱오차 ϵ_{MSE} 는 다음과 같다.

$$\epsilon_{MSE} = \frac{1}{N} \sum_{n=0}^{N-1} (\hat{y}_n - y_n)^2 \quad (2)$$

평균제곱오차 역시 훈련 데이터에 대해 측정하면 훈련 성능, 훈련에 사용하지 않은 데이터에 대해 측정하면 일반화 성능이라 한다.

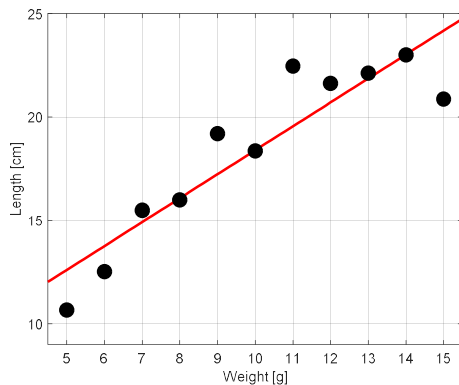
여기에 도전적인 문제가 있다. 머신러닝의 목표는 모델의 학습을 통해 일반화 성능을 극대화하는 것이다. 하지만, 훈련 과정에서는 실제 데이터를 경험할 수 없으므로, 주어진 훈련 데이터로 측정할 수 있는 성능은 훈련 성능뿐이다. 그렇다면, 어떻게 학습이 잘 이루어졌는지 알 수 있을까? 그리고 학습이 잘못되면 어떤 문제가 발생할까? 이 질문은 결코 단순하지 않은 문제이며 머신러닝의 중요한 과제이다. 이를 다른 말로 모델의 검증과 평가 문제라고 한다.

2. 과소적합과 과적합

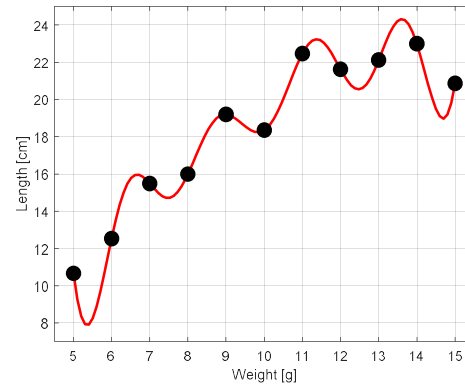
앞서 제기한 질문은 다음과 같은 두 가지이다.

- ① 학습이 잘 된 것을 어떻게 알 수 있을까?
- ② 학습이 잘못되었다면 어떤 문제가 발생할까?

위의 두 문제 중 두 번째 문제를 먼저 생각해보자. 머신러닝에서 학습이 잘못되는 경우는 크게 두 가지가 있다. 하나는 과소적합(underfitting)이고 다른 하나는 과적합(overfitting)이다. 과소적합은 훈련 데이터의 기본적인 특성을 제대로 학습하지 못해 발생하는 문제이고 과적합은 훈련 데이터에 대한 과도한 학습으로 발생하는 문제이다. 그림 2는 같은 훈련 데이터에 대한 다른 학습의 결과를 보여준다. 훈련 데이터는 어떤 곤충의 몸무게에 대한 신체 길이의 데이터이다. 그림 2의 (a)는 선형회귀 모델을 이용해 학습한 결과이다. 이 결과는 몸무게가 무거워질수록 키도 크다는 전반적인 경향을 잘 나타내고 있다. 그러나 훈련 데이터를 살펴보면 몸무게가 가벼울 때는 키가 선형적으로 증가하지만, 몸무게가 약 11g을 넘어서면 키의 성장 속도가 정체된다. 따라서 그림 2(a)의 선형회귀 모델은 훈련 데이터조차 제대로 예측하지 못한다. 이 모델은 실제 데이터에서도 좋은 성능을 기대하기 어렵다. 이는 과소적합의 대표적인 예라 할 수 있다. 그림 2의 (b)는 10개의 가우스 함수를 이용해 만든 선형 기저함수 회귀 모델로 (a)와 매우 상반된 결과를 보여준다. 회귀 결과가 모든 훈련 데이터를 정확히 통과하고 있다. 이는 훈련 데이터에 대한 예측이 완벽하여 평균제곱오차가 0이라는 것을 의미한다. 그렇다면 이 회귀 결과는 실제 상황에서도 우수한 예측 성능을 보일까? 그렇지 않을 것이다. 그림 2(b)는 과적합의 대표적인 예이다. 즉, 훈련 데이터의 학습에 지나치게 몰입한 나머지 훈련 데이터가 갖는 일반 데이터의 특성뿐만 아니라 훈련 데이터만 갖는 독특한 특성까지 과도하게 학습한 것이다. 그림 2(b)와 같이 학습하면 데이터 집합이 조금만 바뀌어도 매우 많은 예측 오류가 발생한다. 즉, 훈련 데이터에만 최적화된 모델이 된 것이다.



(a) 과소적합의 예



(b) 과적합의 예

[그림 2] 같은 훈련 데이터에 대한 다른 회귀 결과

앞서 살펴본 것처럼, 과소적합은 훈련 데이터를 제대로 학습하지 못해 발생하는 문제인 반면 과적합은 훈련 데이터를 과하게 학습해 발생한 문제이다. 과소적합은 모델의 학습능력이 부족하거나 학습량이 부족할 경우 발생한다. 그림 2(a)와 같이 훈련 데이터의 구조 자체가 비선형성을 가지고 있는 상황에서 선형 모델을 사용하면 아무리 열심히 학습하더라도 훈련 데이터의 비선형성을 학습할 방법이 없다. 또한, 모델의 능력이 충분하더라도 학습이 부족하면 과소적합이 일어날 수 있다. 예를 들어 경사하강법에서 비용함수가 충분히 작은 값으로 수렴할 때까지 반복 연산을 수행해야 하지만, 수렴 전에 반복을 종료하는 경우 과소적합이 발생할 수 있다. 다행스러운 점은 과소적합 문제는 비교적 간단하게 해결할 수 있다는 것이다. 즉, 더 많은 능력을 갖춘 모델을 사용하거나 더 많은 반복 학습을 수행하면 해결할 수 있다.

반면, 과적합은 과도한 학습 능력과 과도한 학습의 결과로 나타난다. 따라서 적절한 모델을 선택해 적절한 학습을 시키면 발생하지 않을 문제이다. 그런데, ‘과도’와 ‘적절’의 경계는 어디일까? 그리고 그 경계를 쉽게 알아낼 수 있을까? 여기에서 이번 장의 첫머리에 제기한 두 가지 질문 중 첫 번째 질문으로 이어진다. 즉, 학습이 잘 된 것을 어떻게 알 수 있을까?

과소적합과 과적합 중 해결하기 까다로운 문제는 과적합이다. 주어진 훈련 데이터에 최선을 다하다보면 자연스럽게 발생하는 문제이며, 어느 정도가 적정선인지를 알기는 쉽지 않다. 따라서 머신러닝의 대다수 학습 모델에 과적합의 위험은 늘 존재한다.

다시 처음의 원칙을 생각해보자. 기계학습의 목표는 모델의 일반화 성능을 높이는 것이었다. 이 원칙을 따른다면 과적합을 방지할 수 있을 것이다. 즉, 어떤 모델을 훈련할 때 일반화 성능이 개선되도록 학습을 진행하는 것이다. 학습 과정 중, 과적합이 발생하면 일반화 성능은 더 이상 개선되지 않고 오히려 악화될 것이므로 학습을 종료하면 되는 것이다.

그러나, 우리는 훈련 데이터만 가지고 있으므로 일반화 성능을 측정할 수 없다는 문제가 있다. 이를 해결하고 모델의 일반화 성능을 예측해 과적합을 방지하는 방법을 모델 검증(model test)이라 한다.

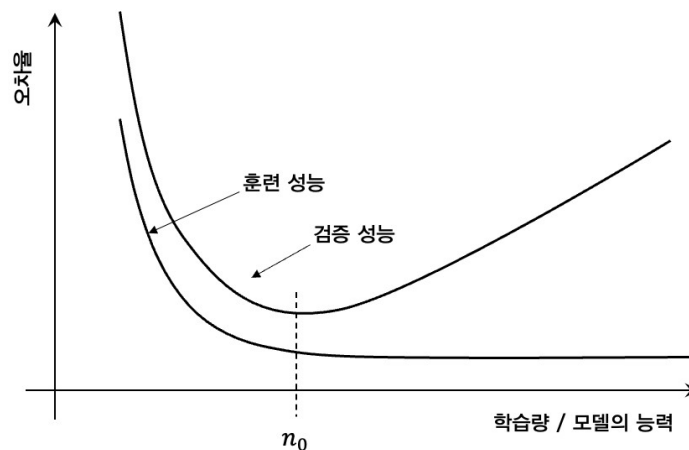
3. 모델 검증 (Model test)

모델을 검증하는 주목적은 과적합을 방지하는 것이다. 이를 위한 핵심 요소는 일반화 성능

의 예측이다. 즉, 학습 과정 중 일반화 성능을 지속적으로 예측하여 훈련 데이터 성능만 향상되고 일반화 성능은 열화되는 과적합을 방지하는 것이다. 그러나, 일반화 성능은 현실 세계에서나 측정 가능한 성능일 뿐 학습 과정에서는 얻을 수 없다.

이런 현실적인 제약을 극복하고 모델의 일반화 성능 예측을 위한 기본적인 접근 방식은 데이터 집합의 분리이다. 즉, 훈련 집합과 독립인 별도의 집합을 구성해 훈련 집합은 학습에만 사용하고 별도의 집합은 일반화 성능 예측에 사용한다. 이렇게 일반화 성능 예측에 사용하는 별도의 데이터 집합을 검증 집합(test set)이라 한다. 이때 훈련 집합과 검증 집합은 서로 독립이면서 동일한 통계적 특성을 가져야 한다. 즉 두 집합 모두 같은 현실 상황을 대표하는 데이터들의 집합이면서 공통된 데이터가 없거나 최소한으로 유지되어야 한다.

그림 3은 훈련 집합과 검증 집합을 이용한 모델 검증의 예를 보여준다. 그래프의 x축은 학습량 또는 모델의 능력을 나타내고 y축은 오차율을 의미한다. 훈련 성능은 학습 데이터를 이용해 측정한 성능을, 검증 성능은 학습에 사용되지 않은 검증 데이터를 이용해 측정한 성능이다. 학습 초기, 학습량을 늘리거나 모델의 능력을 높이면 훈련 성능과 검증 성능이 함께 향상된다. 예를 들어 생각해 보면, 학습량을 늘리는 것은 경사하강법의 반복 횟수를 늘리는 것과 같고 모델의 능력을 높이는 것은 선형기저함수회귀에서 기저함수의 수를 늘리는 것과 같다. 그러나, 그래프에서 보듯이 학습량 또는 모델의 능력이 어느 수준을 넘어서면 검증 성능은 더 이상 개선되지 않고 오히려 악화되기 시작한다. 그래프에서 n_0 지점이 이에 해당하며, 이 지점이 지나면 과적합이 발생하는 것이다. 따라서 모델의 학습은 n_0 에서 종료하는 것이 합리적이다.

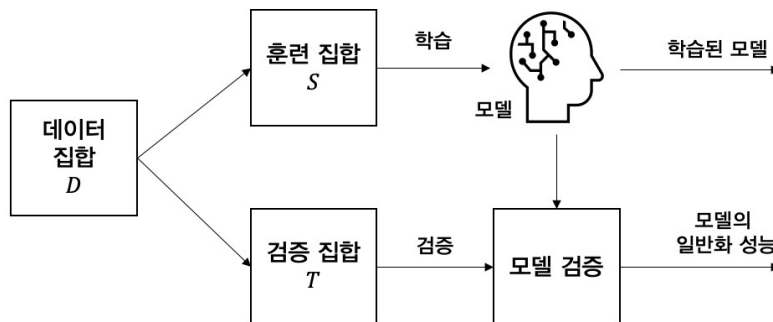


[그림 3] 훈련 성능과 검증 성능의 변화

일반적으로, 데이터 확보는 머신러닝에서 가장 큰 비용을 차지하는 부분 중 하나이므로 훈련 데이터와 별개로 검증 데이터를 추가로 확보하는 것은 현실적으로 어렵다. 따라서 주어진 훈련 데이터를 훈련 데이터와 검증 데이터로 분리하는 방안이 적합하다. 이렇게 데이터 집합을 분리하고 검증하는 세 가지 대표적인 방법으로 홀드아웃(hold-out), 교차검증(cross validation), 부트스트래핑(bootstrapping)을 들 수 있다. 본 장에서 이 세 가지 방식을 설명한다.

3.1. 홀드아웃 검증 (Hold-out validation)

가장 간단한 검증 방식으로 하나의 데이터 집합을 두 개의 겹치지 않는 집합으로 분리하고 하나의 집합으로 학습을, 다른 하나의 집합으로 검증을 진행하는 방식이다. 그림 4는 홀드아웃 검증의 전체적인 과정을 나타낸 것이다. 주어진 모든 데이터의 집합이 D 인 경우 이를 훈련 집합 S 와 테스트 집합 T 로 분리한 뒤 훈련 집합을 이용해 모델을 학습하고 테스트 집합을 이용해 학습된 모델의 정확도를 측정한다. 홀드아웃의 결과물은 훈련 집합으로 학습된 모델과 검증 집합으로 측정한 일반화 성능이다. 이때 훈련 집합과 테스트 집합은 서로소(disjoint) 관계여야 한다. 즉, 집합 S 와 T 사이에 공통된 원소가 없어야 한다.



[그림 4] 홀드아웃 검증

홀드아웃 검증에서 주의할 부분은 데이터 집합을 분리할 때 통계적 특성이 유지되어야 한다는 점이다. 즉 세 집합 D , S , T 가 동일한 통계적 특성을 가져야 한다. 이진 분류를 예로 들면, 집합 D 의 원소 중 양성 데이터가 70%, 음성 데이터가 30%인 경우 훈련 집합과 검증 집합도 양성 데이터와 음성 데이터를 각각 70%와 30%씩 가지고 있어야 한다.

홀드아웃의 성능은 데이터 집합을 어떻게 분리하느냐에 많은 영향을 받는다. 비록 데이터 집합 D 의 통계적 특성을 유지하면서 훈련 집합과 검증 집합으로 분리하더라도 다양한 방법을 통해 다양한 조합으로 분리할 수 있다. 그리고 모델은 항상 훈련 집합만으로 학습하기 때문에 다양한 조합 중 하나에 지나지 않는 훈련 집합에 영향을 받게 된다. 이는 모델의 학습 과정에 데이터 집합의 모든 데이터를 충분히 활용하지 못하는 결과를 초래한다. 즉, 데이터가 많을수록 기계학습의 일반화 성능이 좋아질 수 있고, 이 때문에 많은 데이터의 확보가 중요하다. 그런데 홀드아웃은 데이터 집합을 분리하고 분리된 일부 데이터만으로 학습하므로 의도적으로 훈련 데이터를 줄이는 결과를 초래한다. 만약 데이터 집합 D 가 충분히 크지 않다면, 홀드아웃을 통해 얻을 수 있는 효과보다 데이터 분리에 따른 역효과가 더 크게 나타날 수도 있다.

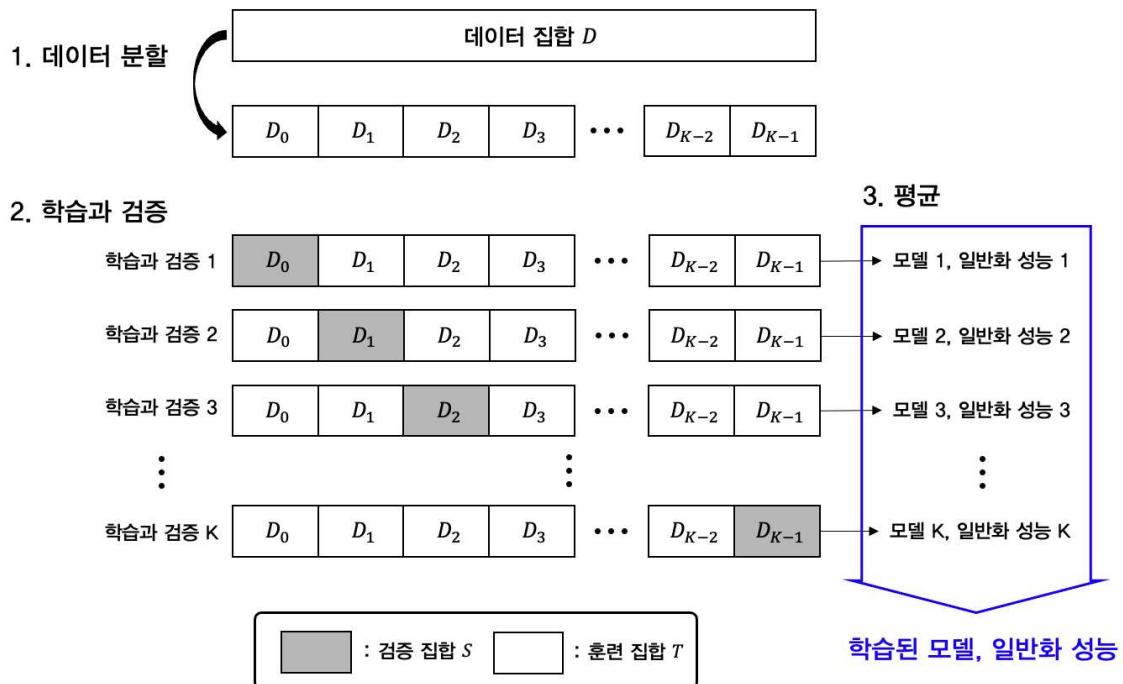
또 다른 이슈는 훈련 집합과 검증 집합의 비율을 어떻게 나누는 것이 좋은가이다. 이 문제 역시 데이터 집합이 충분히 크다면 어떤 비율로 나누어도 큰 영향이 없을 수 있지만, 현실 세계에서는 늘 데이터가 부족하다. 어떤 비율로 구성하는 것이 최적인지에 대한 정답은 알려지지 않았지만, 훈련 집합과 테스트 집합의 비율을 7:3과 8:2 사이에서 정하는 것이 일반적이다. 즉, 전체 데이터의 70%~80%를 훈련 집합으로 구성하고 나머지는 검증 집합으로 분리하는 것이다.

3.2. 교차검증 (Cross validation)

홀드아웃 검증의 단점은 일단 훈련 집합과 검증 집합을 구성하고 나면, 어떤 데이터는 학습

에만 사용되고 어떤 데이터는 검증에만 사용된다는 것이다. 따라서 우리가 가진 데이터를 충분히 활용하지 못하고, 특정 데이터에 대해서만 학습하게 된다. 이러한 단점을 극복하기 위한 방법이 교차검증이다.

그림 5는 교차검증의 원리를 나타낸다. 먼저 교차검증을 위해 데이터 집합을 K 개의 부분집합으로 분할한다. 그림에서 $D_0, D_1, \dots, D_{K-1}$ 은 데이터 집합 D 를 서로 겹치지 않게 나눈 K 개의 부분집합이다. 데이터 집합을 분할하고 나면, 학습과 검증 단계를 시작한다. 학습과 검증은 훈련 데이터와 검증 데이터를 달리하여 총 K 번 반복한다. 각각의 학습과 검증은 K 개의 부분집합 중 하나를 검증 집합 S 로 나머지를 훈련 집합 T 로 활용하는 홀드아웃과 같다. 그림에서 음영으로 표시한 집합은 검증 집합, 흰색으로 이렇게 K 번의 학습과 검증을 마치면 K 개의 학습된 모델과 각각의 일반화 성능을 얻을 수 있다. 교차검증의 최종 결과물을 얻기 위해 K 개의 학습된 모델과 각각의 일반화 성능을 모두 평균하여 최종 학습된 모델과 일반화 성능을 얻는다. 이 예에서 데이터 집합을 K 개로 나누었기 때문에 이를 K 겹 교차검증(K -fold cross validation)이라고 부른다.



[그림 5] K 겹 교차검증 (K -fold cross validation)

교차검증을 구성하는 K 개의 개별 테스트는 홀드아웃과 같은 방식으로 전체 데이터를 훈련 집합과 테스트 집합으로 분리하여 성능을 평가하는 방식이다. 따라서 홀드아웃에서와 마찬가지로 모든 집합의 통계적 특성이 같도록 구성해야 한다. 이를 위해서는 전체 데이터 집합 D 를 K 개의 부분 집합으로 분리할 때에도 각 부분 집합이 전체 집합과 같은 통계적 특성을 갖도록 나누어야 한다는 것을 의미한다. 이는 교차검증을 위해 전체 집합을 부분 집합으로 나눌 때 어떻게 나누는가에 따라 성능이 달라질 수 있음을 의미한다.

이러한 분리의 문제를 해결하는 방법은 부분 집합의 개수인 K 를 전체 데이터의 수와 같게

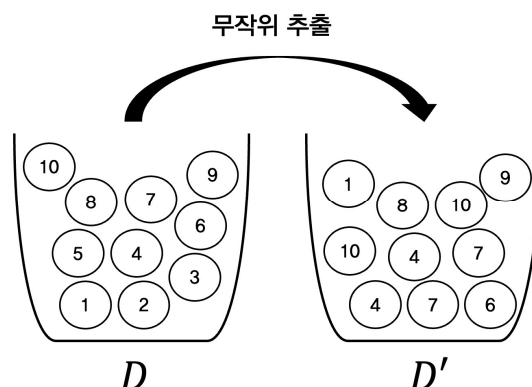
두는 것이다. 이 경우 부분 집합의 수와 모든 데이터의 수가 같으므로 각각의 부분 집합에는 하나씩의 데이터만 존재한다. 따라서 전체 데이터 중 한 개의 데이터를 제외한 나머지 데이터로 학습하고 선택된 한 개의 데이터로 성능을 측정하는 것이다. 이렇게 한 개의 데이터만 이용해 테스트하고 나머지 데이터로 학습하는 교차검증 방식을 LOOCV(Leave one out cross validation)라고 한다.

만약 전체 데이터 집합 D 의 원소의 개수를 N 이라 한다면, LOOCV 방식은 $K=N$ 이 되고 N 번의 테스트를 통해 N 개의 정확도를 구할 수 있으며 이를 모두 평균하여 최종 모델과 성능을 얻게 된다. LOOCV는 $N-1$ 개의 데이터로 훈련하고 1개의 데이터로 테스트하므로 거의 모든 데이터에 대해 훈련한 모델의 성능을 평가할 수 있어 상대적으로 객관적인 평가가 가능한 것으로 알려져 있다. 다만, N 번의 학습과 테스트가 이루어져야 하므로 연산 복잡도가 매우 높아진다는 점은 단점이라 할 수 있다.

3.3. 부트스트래핑 (Bootstrapping)

홀드아웃과 교차검증은 데이터 집합을 훈련 집합과 검증 집합으로 나누어 학습과 검증을 진행한다. 이 방식의 근본적인 문제점은 집합의 분할을 통해 훈련 집합의 크기가 작아진다는 것이다. 즉, 머신러닝 문제에서 가장 비싼 자원인 데이터를 학습에 충분히 활용하지 못하고 어쩔 수 없이 일부의 데이터를 검증을 위해 아껴둬야 한다는 것이다. 물론 LOOCV를 사용하면 단 한 개의 데이터를 제외한 모든 데이터를 학습에 활용할 수 있지만, 데이터 집합의 크기가 커지면 연산 복잡도가 급상승한다는 단점을 가지고 있다.

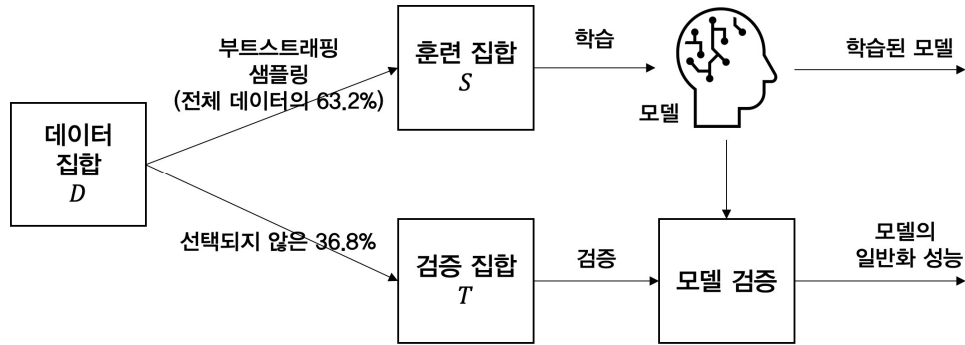
부트스트래핑은 부트스트래핑 샘플링이라는 기법을 활용해 홀드아웃과 교차검증이 갖는 훈련 데이터의 축소 문제를 최대한 해결하려는 검증 방식이다. 부트스트래핑 샘플링이란 어떤 집합에서 중복을 허용하며 무작위 추출하는 기법이다. 그림 6은 중복을 허용한 무작위 추출의 예를 보여준다. D 에는 1부터 10까지 적힌 10개의 공이 들어있고, D 에서 임의로 하나를 뽑아 똑같은 수가 적힌 공을 D' 에 넣고 D 에서 뽑은 공은 다시 D 에 넣는다. 이렇게 10번을 반복하면 D' 에도 10개의 공이 있다. 이렇게 무작위적으로 선택하는 과정에서 어떤 번호는 중복적으로 선택이 되고 어떤 번호는 한 번도 선택되지 않을 수 있다.



[그림 6] 중복을 허용한 무작위 추출

그림 6과 같이 중복을 허용하여 모집합 D 과 같은 크기의 집합 D' 을 만들면, 통계적으로 집합 D 에 들어있는 원소들의 36.8%는 선택되지 않고 나머지 원소들만 선택된다. 부트스트래핑

은 부트스트래핑 샘플링을 통해 만들어진 집합을 훈련 집합으로, 선택되지 않은 36.8%의 데이터를 검증 집합으로 하여 학습과 검증을 수행하는 기법이다. 그림 7은 부트스트래핑의 동작 구조를 나타낸다. 데이터 집합 D 로부터 부트스트래핑 샘플링을 통해 훈련 집합 S 를 구성하였으므로 구성하였으므로 집합 S 의 크기는 데이터 집합 D 의 크기와 동일하다.



[그림 7] 부트스트래핑의 동작

부트스트래핑의 장점은 훈련 집합의 크기가 줄어들지 않는다는 것이다. 그러나, 훈련 집합을 구성하는 동안 일부 데이터의 중복을 허용했기 때문에 훈련 데이터의 분포 또는 통계적 특성이 다소 왜곡될 수 있다는 단점도 있다.

4. 모델 평가 (Model evaluation)

지금까지 모델이 제대로 학습되었는지 검증하기 위한 몇 가지 방법을 살펴보았다. 모델의 검증은 일반화 성능을 측정하기 위한 집합의 분할에 초점이 맞춰져있다. 이제 살펴볼 주제는 성능 지표 자체에 관한 것으로 모델의 성능을 평가하기 위해 자주 사용되는 지표들과 활용 방법을 소개한다.

4.1. 평균제곱오차(Mean squared error)

회귀 모델의 출력값은 연속된 값을 가지므로 실제값과 예측값 사이의 차이가 클수록 예측 정확도가 낮은 것으로 판단할 수 있다. 따라서 각 데이터에 대한 실제값과 예측값의 차이의 제곱의 평균, 즉 평균제곱오차(MSE)는 회귀 모델의 대표적인 성능 지표로 사용된다. 이미 식 (2)에서 정의하였으며, 선형회귀에서 비용함수로 소개한 바 있다. 평균제곱오차가 회귀 모델의 성능 지표로 널리 사용되지만, 분류 모델에서도 사용 가능하다.

4.2. 오차율(Error rate)과 정확도(Accuracy)

회귀와 달리 분류에서는 출력 레이블이 몇 가지 클래스에 국한되므로, 예측 클래스와 실제 클래스가 같은 경우와 다른 경우를 명확하게 구분할 수 있다. 따라서 전체 데이터 중 클래스 예측에 실패한 데이터의 개수의 비율을 오차율 p_{err} 로 정의할 수 있으며 반대로 전체 데이터 중 클래스 예측에 성공한 데이터의 개수의 비율을 정확도 p_{acc} 로 정의할 수 있다. 당연히 오차

율과 정확도의 합은 1이다.

오차율과 정확도는 모델의 예측 성능을 직관적으로 표현하기에 좋은 성능 지표이다. 그러나, 클래스 구성이 불균형한 문제에 대한 분류 모델의 성능을 표현하기에는 적절하지 않을 수 있다. 예를 들어, 어떤 공장에서 생산되는 제품의 불량 여부를 판단하는 이진 분류 모델을 생각해보자. 이 공장에서 생산되는 제품의 95%는 정상이고 5%는 불량이라고 한다. 따라서, 훈련 데이터는 양성(정상) 데이터 95%, 음성(불량) 데이터 5%로 구성해야 한다. 이렇게 어느 한 클래스에 데이터가 집중되는 것을 클래스간 불균형(class imbalance)라고 한다. 이때 두 분류 모델 A와 B가 있다고 가정하자. 모델 A는 복잡한 학습 과정을 거쳐 설계하였는데, 정확도 측정 결과 80%였다. 그런데 모델 B는 학습 과정을 거치지 않고, 무조건 양성이라는 출력을 내 보내도록 설계하였다. 데이터의 95%가 양성이므로 모델 B의 정확도는 당연히 95%일 것이다. 여기에서, 복잡한 설계와 학습을 거쳐 80%의 정확도를 갖는 모델 A와 어떤 입력이 들어오건 상관없이 양성이라고 예측하여 95%의 정확도를 갖는 B 중 어떤 모델이 더 우수한가? 정확도를 기준으로 보면, 모델 B가 더 우수하다고 할 수 있다. 그러나 모델 B가 95%의 정확도를 갖게 된 것은 모델의 역량 때문이 아니라 전적으로 데이터의 구조 때문이다. 이 예에서 볼 수 있는 것과 같이 클래스 불균형이 존재하는 데이터에 대해서는 오차율과 정확도만으로 성능을 평가하는 것이 부적절할 수 있다.

4.3. 정밀도(Precision), 재현율(Recall), F1 스코어(F1 score)

4.3.1. 정밀도와 재현율

앞에서 살펴본 오차율, 정확도와 같이 정밀도, 재현율, F1 스코어도 분류 모델의 성능 측정을 위한 지표이다. 설명의 편의를 위해 클래스가 두 개인 이진 분류 모델을 가정한다. 따라서 데이터의 레이블은 양성(positive) 또는 음성(negative) 중 하나의 값을 갖는다.

그림 8은 이진 분류 모델의 혼동행렬(confusion matrix)을 나타낸다. 혼동행렬이란 예측값과 실제값의 조합을 나타내는 행렬을 의미한다. 문서에 따라 오류행렬(error matrix)라고 표현하는 경우도 있다. 이진 분류 모델이므로 데이터의 출력 레이블은 양성 또는 음성 중 하나이고 모델 역시 양성 또는 음성 중 하나의 값으로 예측 결과를 출력할 것이다. 따라서 그림과 같이 총 네 개의 조합이 가능하다.

		모델의 예측 결과	
		양성 (Positive)	음성 (Negative)
데이터의 실제 결과	양성 (Positive)	진짜 양성 (TP, True Positive)	가짜 음성 (FN, False Negative)
	음성 (Negative)	가짜 양성 (FP, False Positive)	진짜 음성 (TN, True Negative)

[그림 8] 이진 분류 모델의 혼동행렬(Confusion matrix)

그림 8의 네 가지 조합을 살펴보면 다음과 같다.

- 진짜 양성 (TP, True Positive)

실제로 양성인 데이터를 양성으로 예측한 경우이다. 공장의 불량품 선별 모델의 예에서

정상인 제품을 정상이라고 판단한 데이터들의 집합이다.

- 가짜 양성 (FP, False Positive)
실제로 음성인 데이터를 양성으로 예측한 경우이다. 이 경우는 불량품을 정상 제품으로 분류한 데이터들의 집합이다.
- 가짜 음성 (FN, False Negative)
실제로 양성인 데이터를 음성으로 예측한 경우이다. 이는 정상적인 제품을 불량품이라고 예측한 데이터들의 집합이다.
- 진짜 음성 (TN, True Negative)
실제로 음성인 데이터를 음성으로 예측한 경우이다. 이 경우는 정상적인 제품을 불량품으로 분류한 데이터들의 집합이다.

모든 데이터는 혼동행렬의 네 가지 경우 중 하나에 포함된다. 따라서 데이터의 개수가 모두 N 개라면 다음 관계가 성립된다.

$$TP + FP + FN + TN = N$$

또한 예측에 성공한 경우는 TP와 TN에 해당하고, 예측에 실패한 데이터는 FP와 FN이므로 앞에서 공부한 오차율과 정확도는 다음과 같이 표현할 수 있다.

$$p_{err} = \frac{FP + FN}{N}, p_{acc} = \frac{TP + TN}{N} \quad (3)$$

다음으로 살펴볼 성능 지표는 정밀도(precision)와 재현율(recall)이다. 정밀도(γ_P)의 정의는 **양성으로 예측한 데이터 중에서 실제로 양성인 데이터들의 비율**이다. 그리고 재현율(γ_R)은 **실제로 양성인 데이터 중에서 양성으로 예측한 데이터들의 비율**이다. 재현율을 민감도(sensitivity)라고 하기도 한다. 이를 수식으로 표현하면 다음과 같다.

$$\gamma_P = \frac{TP}{TP + FP} \quad (4)$$

$$\gamma_R = \frac{TP}{TP + FN} \quad (5)$$

정밀도는 “예측 결과가 얼마나 정확한가?” 또는 “예측 결과를 얼마나 신뢰할 수 있는가?”라는 질문에 답이 될 수 있는 지표이다. 즉, 양성으로 예측한 데이터가 모두 실제로 양성이었다면 식 (4)에서 가짜양성(FP), 즉 양성으로 예측했지만 실제로는 음성인 경우가 0이 될 것이고 정밀도 γ_P 는 1이 될 것이다. 따라서 정밀도가 높아질수록 예측의 신뢰도 또한 높아진다고 볼 수 있다.

재현율은 “어떤 데이터의 실제 클래스를 얼마나 잘 예측했는가?”에 대한 답이다. 공장에서 생산된 제품이 100개가 있고 이 중 정상 제품이 실제로 90개가 있을 때, 90개의 정상 제품을 모두 골라냈다면 식 (5)에서 정상인데 불량이라고 판단한 가짜음성(FN)이 0이므로 재현율은 1이 된다.

정밀도와 재현율은 다소 어려운 개념이다. 조금 더 쉬운 이해를 위해, 화재 감지기를 생각해 보자. 화재 감지기는 주변의 온도를 측정해 화재 발생 여부를 판단한다고 가정한다. 즉 입력은 온도이고 출력은 화재가 발생 여부이다. 화재가 발생한 경우를 양성, 그렇지 않은 경우를 음성이라 하자. 어느 날 화재 감지기에서 화재 경보가 울렸다. 이때 정말로 화재가 발생했을 가능성은 얼마나 될까? 이 질문에 대한 해답은 화재 감지기의 정밀도이다. 즉, 양성이라고 예측했을 때 실제로 양성일 경우의 비율이다. 그런데 관점을 조금 바꿔 이렇게 생각해 볼 수 있다. 또 다른 어느 날 건물에 실제로 화재가 발생했다. 이때 복도에 있는 화재 감지기가 경

보를 올릴 가능성은 어느 정도일까? 이 가능성을 나타내는 지표는 재현율이다. 즉 실제 양성 데이터 중 양성으로 예측한 데이터의 비율이다.

이진 분류의 문제는 뭐든 동일한 설명이 가능하다. 다른 예로 독감 진단 키트를 생각해보자. 독감이 걸렸으면 양성, 걸리지 않았으면 음성이 출력되는 키트이다. 진단 키트에 양성이라고 떼었을 때 진짜로 독감에 걸릴 가능성은 얼마나 될까? 바로 진단 키트의 정밀도가 이 가능성의 지표가 된다. 반대로 내가 독감에 걸렸는데, 진단 키트로 테스트 했을 때 양성이라고 뜰 가능성은 얼마일까? 이 가능성은 재현율로 나타낼 수 있다.

당연한 얘기이지만, 우수한 분류기(모델)일수록 정밀도와 재현율이 높다. 하지만, 정밀도와 재현율은 서로 영향을 주고받는 관계이고 정밀도를 높이면 재현율이 낮아지고 반대로 재현율을 높이면 정밀도가 낮아진다. 왜 그런지는 뒤에서 다시 설명하겠지만, 이 관계를 정밀도-재현율 트레이드오프(precision-recall tradeoff)라고 한다. 그래서 분류 모델 설계자는 정밀도와 재현율 중 하나를 선택해야 한다. 즉, 정밀도가 높은 모델을 설계할 것인지 아니면 재현율이 높은 분류 모델을 만들 것인지. 선택의 기준은 모델의 용도이다. 즉 분류 모델의 용도 중 정밀도가 중요한 모델이 있는가하면 재현율이 중요한 모델이 있다. 예를 들어 주식 시세 예측이나 학생들의 성적 처리 모델처럼 예측 정확도가 중요한 경우 정밀도를 높이는 것이 바람직한 반면, 적군의 침입 탐지나 질병 발생 유무와 같이 잘못된 예측(판단)이 치명적인 결과를 초래할 수 있는 경우에는 재현율이 중요하다.

4.3.2. 정밀도-재현율 트레이드오프

이제 정밀도-재현율 트레이드오프의 이유에 대해 생각해보자. 다시 화재 감지기를 생각해 보면 쉽게 설명이 가능하다. 화재 감지기는 주변의 온도를 측정하여 온도가 높으면 화재가 발생했다고 판단하고 그렇지 않으면 화재가 발생하지 않았다고 판단한다. 이를 위해 온도 문턱값(threshold)을 설정한다. 즉, 현재 온도가 문턱값보다 높으면 화재발생(양성), 그렇지 않으면 정상(음성)으로 결정하는 것이다. 따라서, 문턱값을 어떻게 설정하느냐에 따라 이 화재 감지기의 성능이 달라질 것을 쉽게 예상할 수 있을 것이다. 먼저 문턱값이 낮으면 어떻게 될까? 문턱값이 낮으면 주변 온도가 조금만 올라가도 화재가 발생한 것으로 판단할 것이다. 다양한 이유로 주변의 온도가 변할 수 있는데, 문턱값이 낮으면 아주 사소한 온도 변화에도 민감하게 반응하여 화재 경보를 울릴 것이다. 즉, 실제로는 음성인데 양성(화재발생)으로 판단하는 경우가 많아진다. 즉, **가짜양성(FP)이 많아진다**. 한편, 문턱값이 낮으면 아주 작은 열기도 화재 발생으로 판단하므로 거의 모든 화재 상황은 정확하게 예측할 수 있게된다. 이는 실제로 화재가 발생(양성)했는데 발생하지 않았다고(음성) 판단하는 일이 거의 발생하지 않게 된다는 것을 의미한다. 즉, **가짜음성(FN)이 줄어든다**. 따라서 가짜양성이 많아지는 것과 가짜음성이 줄어드는 현상은 같은 원인에 대한 결과이다. 여기에서 식 (4)와 (5)를 살펴보면, 가짜양성이 많아지면 정밀도는 낮아지고 가짜음성이 줄어들면 재현율은 높아진다.

반대로 정밀도를 높이려면 어떻게 해야할까? 정밀도가 높아지려면 가짜양성(FP)이 작아져야 한다. (식 (4) 참조) 가짜양성이란 실제로 음성인데 양성으로 판단된 데이터이다. 즉 불이 나지 않았는데 불이 난 것으로 판단한 경우이다. 즉, 진짜 불이 났을 때만 화재가 발생했다고 판단해야 하는 것이다. 이를 위해서는 화재 감지기의 문턱값을 높여야 한다. 애매한 온도는 화재로 판단하지 않고 아주 높은 온도인 경우에만 화재라고 판단하면 가짜양성은 줄어든다. 따라서 정밀도는 향상된다. 그런데, 문턱값이 높으면 실제 화재가 발생한 경우를 놓칠 가능성이 높아진다. 감지기로부터 거리가 떨어진 곳에 작은 화재가 발생한 경우 측정된 온도가 낮을 수 있

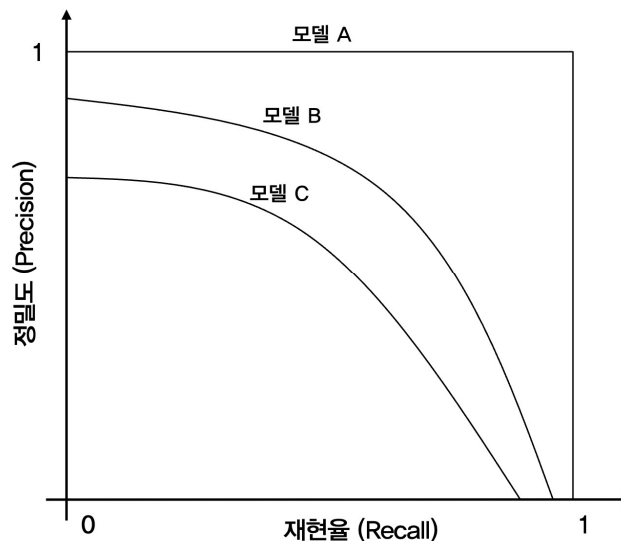
는데, 문턱값이 높기 때문에 화재가 아닌 것으로 판단될 가능성이 높기 때문이다. 이는 가짜 음성(FN)을 높이게 되고 재현율을 낮추는 결과로 이어진다.

화재 감지기의 예를 일반적인 이진 분류 모델에 적용해보자. 모든 이진 분류 모델은 내부 메커니즘을 통해 출력 레이블이 양성일 확률을 계산하고 이 확률을 미리 정해진 문턱값과 비교하여 문턱값보다 높으면 양성, 낮으면 음성으로 판단한다. 이 문턱값을 높이면 양성으로 판단하는 비율이 줄어들고 반대로 문턱값을 낮추면 음성으로 판단하는 비율이 줄어든다.

어떤 모델의 정밀도를 높이기 위해서는 가짜양성(FP)을 줄여야 한다. 즉, 양성이 되는걸 힘들게 만들면 가짜양성이 줄어든다. 따라서 문턱값을 높여야하고, 문턱값이 높아지면 실제 양성인데 음성으로 판단되는 비율이 함께 높아져 가짜음성(FN)이 많아진다. 이는 재현율을 낮추게 된다. 반대로 재현율을 높이기 위해서는 가짜음성(FN)을 줄이기 위해 문턱값을 낮춰야 하는데, 그러다보면 가짜양성(FP)이 많아진다. 따라서 정밀도는 낮아지는 것이다.

4.3.3. PR 곡선과 모델의 평가

그림 9는 서로 다른 분류 모델 A, B, C의 재현율과 정밀도를 그래프로 그린 것이다. 이렇게 재현율과 정밀도의 조합을 그린 그래프를 PR 곡선이라 한다.



[그림 9] 서로 다른 모델들의 PR 곡선

그림 9에서 각 모델이 가질 수 있는 예측 성능은 곡선을 포함한 아래 영역의 임의의 점의 좌표이다. 예를 들어, 2차원 평면의 한 좌표 (0.5, 0.8)이 모델 A와 B의 곡선 아래에 포함되지만 C의 영역에는 포함되지 않는다면 모델 A와 B는 재현율 0.5, 정밀도 0.8을 구현할 수 있지만 모델 C는 구현할 수 없음을 의미한다. 따라서 그래프의 곡선은 그 모델이 가질 수 있는 최대 성능의 한계를 나타내고, 곡선 아래 영역의 넓이가 넓을수록 모델의 역량이 크다고 판단한다. 이렇게 곡선 아래 영역의 넓이를 AUC(Area under curve)라고 한다. PR 곡선에서 AUC가 가장 큰 경우는 그림 9의 모델 A이다. 모델 A는 가장 완벽하고 이상적인 분류 모델이다. 따라서 PR 곡선에서 AUC가 클수록 또는 모델 A와 가까울수록 좋은 분류 모델이라 할 수 있다. 그림 9에서는 모델 B가 C보다 좋은 모델이다.

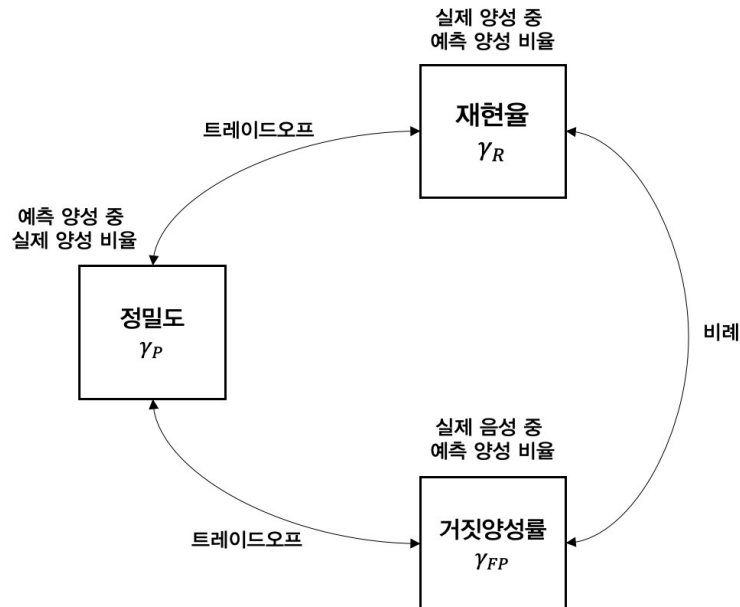
4.3.4. ROC 곡선과 모델의 평가

그림 8의 혼동행렬에서 정의할 수 있는 지표 중 거짓양성률(FPR, False positive rate)이라는 지표가 있다. 거짓양성률(γ_{FP})은 실제로 음성인 데이터 중에서 양성으로 판단한 데이터의 비율을 의미하고 다음과 같이 표현할 수 있다.

$$\gamma_{FP} = \frac{FP}{FP + TN} \quad (6)$$

화재 감지기의 예에서, 실제로 화재가 발생하지 않았는데 갑자기 화재 경보가 울릴 확률이 거짓양성률이다. 따라서, 거짓양성률을 가짜 알람 확률(False alarm probability)라고 부르기도 한다.

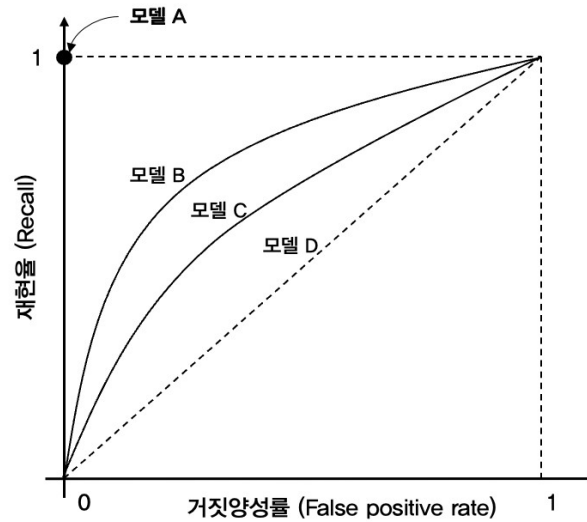
거짓양성률 역시 재현율과 밀접하게 관련되어있다. 재현율은 실제 양성일 때 양성이라고 판단할 확률이고, 거짓양성률은 실제 음성일 때 양성이라고 판단할 확률이다. 판단을 위한 문턱값을 낮추면 재현율이 높아지는데, 낮은 문턱값으로 인해 실제로는 화재가 발생하지 않았음에도 화재가 발생한 것으로 오인할 가능성도 높아진다. 따라서 거짓양성률도 같이 높아진다. 이를 종합하면 그림 10과 같이 정밀도, 재현율, 거짓양성률의 관계를 정리할 수 있다.



[그림 10] 정밀도, 재현율, 거짓양성률의 관계

앞에서 정밀도와 재현율을 2차원 평면에 그린 PR 곡선으로 모델의 성능을 평가하였는데, 재현율과 거짓양성률 그래프를 이용해 같은 평가를 할 수 있다. 재현율과 거짓양성률을 그린 그래프를 ROC(Receiver operating characteristic) 곡선이라 한다. 그림 11은 ROC 곡선의 예를 나타낸다. 좌표평면의 x축은 거짓양성률을 y축은 재현율을 나타낸다. 거짓양성률과 재현율은 비례하는 관계이므로 우상향하는 그래프가 만들어진다. 그래프에서 모델 A, 즉 (0,1)의 성능을 보이는 모델은 완벽한 분류가 가능한 이상적인 분류 모델이다. 반면에 기울기가 1인 직선으로 나타나는 모델 D는 무작위 분류 시스템, 즉 입력에 상관없이 무작위적으로 양성과 음성을 예측하는 가장 단순한 분류 모델이다. 따라서 실질적인 분류 모델의 성능은 모델 A와

모델 D 사이에 위치한다. 이때 ROC 곡선은 특정 분류 모델이 구현할 수 있는 거짓양성률과 재현율의 최대치를 표현한 것이므로, ROC 곡선 아래 영역의 넓이 즉 AUC가 모델의 성능이 된다. 따라서 AUC가 큰 모델일수록 좋은 분류 모델이라 할 수 있다. 그림에서는 모델 B가 C보다 우수한 성능을 갖는 모델이라는 것을 쉽게 파악할 수 있다.



[그림 11] ROC 곡선

4.3.5. F1 스코어

앞에서 PR 곡선과 ROC 곡선을 이용해 모델의 성능을 평가하는 것을 설명했다. 이와 함께 모델의 평가에 많이 사용되는 지표인 F1 스코어를 소개한다. F1 스코어는 다음과 같이 정밀도(γ_P)와 재현율(γ_R)의 조화평균으로 정의된다.

$$F_1 = \frac{2}{\frac{1}{\gamma_P} + \frac{1}{\gamma_R}} = \frac{2\gamma_P\gamma_R}{\gamma_P + \gamma_R} \quad (6)$$

F1 스코어는 정밀도와 재현율의 값이 모두 커질수록 커지는 값이며, 두 값 중 하나가 0이 되면 F1 스코어도 0으로 수렴한다. PR 곡선이나 ROC 곡선의 AUC와 같이 F1 스코어도 모델의 능력을 나타내는 지표이며 F1 스코어가 큰 모델이 좋은 분류 성능을 갖는다.

3. 로지스틱 회귀 (Logistic Regression)

1. 분류 (Classification)

분류(Classification)는 데이터의 출력이 이산적인 경우, 즉 출력 속성이 가질 수 있는 값의 종류가 유한한 데이터의 학습에 사용되는 지도학습 기법이다. 특히 출력의 값이 두 개인 분류 문제를 이진분류(Binary classification)라 한다.

회귀(Regression)와 비교하면 분류의 특징을 쉽게 이해할 수 있다. 회귀는 분류와 달리 출력 속성이 가질 수 있는 값의 종류가 무한한 데이터, 즉 출력 속성이 연속적인 값을 갖는 데이터의 학습에 사용된다. 예를 들어 우리나라 소아·청소년의 나이와 키를 측정한 데이터를 생각해보자. 이 데이터의 입력은 나이이고 출력 속성은 키이다. 키라는 속성은 실수값으로 어떤 값이라도 가질 수 있다. 이렇게 어떠한 값이라도 가질 수 있는 데이터를 연속적인 데이터라고 한다. 나이와 키의 데이터를 학습하여 나이에 따른 키의 함수 관계를 찾는 과정이 회귀이다. 반면, 소아·청소년의 나이, 키와 함께 비만도를 측정한 데이터를 생각해보자. 비만도는 ‘정상’ 또는 ‘비만’ 중 하나의 값을 가질 수 있다. 따라서 어떤 값이라도 가질 수 있는 키와 달리 비만도는 미리 정해진 두 값 중 하나의 값만 가질 수 있는 이산적인 데이터이다. 이 데이터를 학습하여 나이와 키에 따른 비만도의 상관관계를 구하는 작업을 분류라 할 수 있다. 특히 분류의 목표값인 비만도가 가질 수 있는 값이 ‘정상’ 또는 ‘비만’ 중 하나이므로 이진분류 문제가 된다.

데이터를 잘 분류하기 위해 다양한 머신러닝 기법이 사용될 수 있다. 본 장에서는 다양한 기법 중 로지스틱 회귀라는 기법을 다룬다. 로지스틱 회귀라는 용어 안에 ‘회귀’라는 단어가 들어있기는 하지만, 로지스틱 회귀는 분류에 해당한다는 점을 기억하자.

2. 이진분류를 위한 로지스틱 회귀 모델

2.1 로지스틱 함수

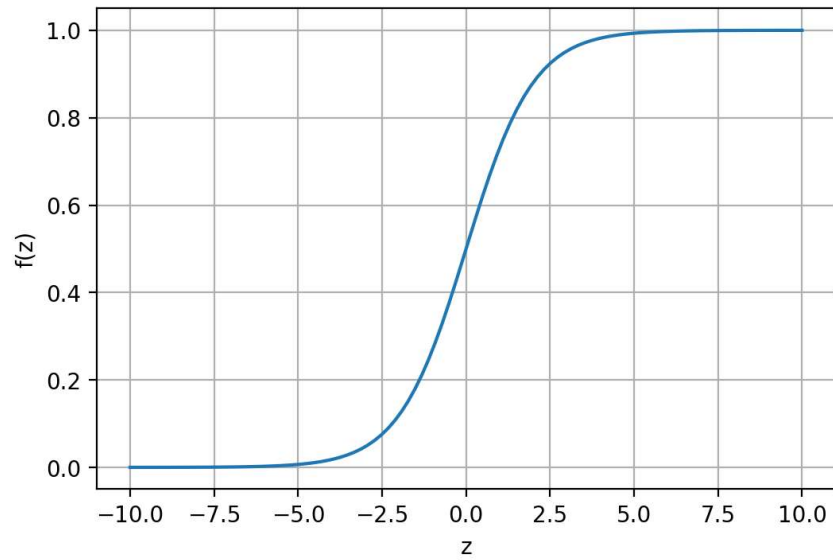
로지스틱 회귀 모델은 선형회귀의 결과를 로지스틱 함수에 적용하여 분류에 이용하는 방식이다. 따라서 로지스틱 함수를 먼저 알아보자. 로지스틱 함수(logistic function)의 정의는 다음과 같다.

$$f(z) = \frac{1}{1 + e^{-z}} \quad (1)$$

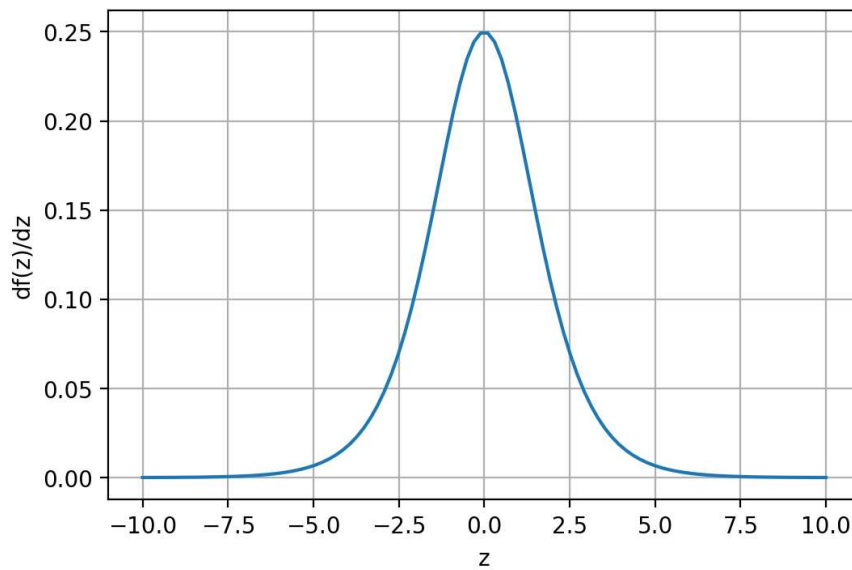
로지스틱 함수는 다음과 같은 성질을 가지고 있다.

$$f(z) = \begin{cases} 0, & z \rightarrow -\infty \\ 0.5, & z = 0 \\ 1, & z \rightarrow \infty \end{cases} \quad (2)$$

그림 1(a)는 로지스틱 함수 $f(z)$ 의 그래프이다. 그림에서 명확하게 알 수 있는 사실은 로지스틱 함수 $f(z)$ 는 임의의 입력 z 를 0과 1사이의 값으로 ‘변환’해준다는 점이다. 이 성질이 로지스틱 회귀의 핵심이라 할 수 있다.



(a) 로지스틱 함수



(b) 1차 도함수

[그림 1] 로지스틱 함수 $f(z)$ 와 1차 도함수

참고로 로지스틱 함수는 미분가능한 함수이며, 그 도함수는 다음과 같이 간단하다.

$$\frac{d}{dz}f(z) = f(z)(1 - f(z)) \quad (3)$$

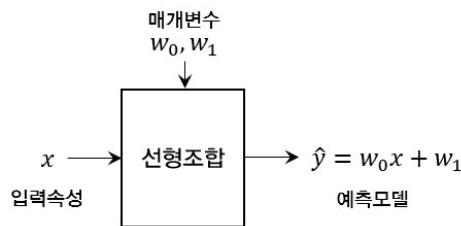
그림 1(b)는 1차 도함수의 형태를 나타낸다. 머신러닝에서는 최적화 문제를 풀어야 하는 경우가 많기 때문에 도함수의 형태가 간단한 것은 장점이 될 수 있다.

2.2 로지스틱 회귀 모델

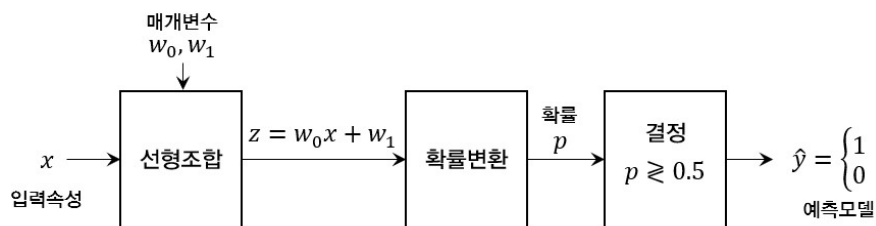
이제 로지스틱 함수를 이용해 회귀를 분류로 변환하는 로지스틱 회귀에 대해 알아보자. 일

반적인 선형회귀는 입력 속성들의 선형조합을 이용해 출력값을 예측한다. 이때 선형조합을 위한 최적 매개변수를 찾으면 선형회귀 모델을 통한 예측값이 훈련 데이터의 출력과 거의 같아진다. 그런데, 회귀와 달리 이진분류의 문제에서는 출력이 가질 수 있는 값이 두 개 뿐이다. 앞으로 설명의 편의를 위해 출력이 가질 수 있는 두 값을 각각 1(양성)과 0(음성)이라고 표시한다. 예를 들어, 비만도에서 '정상'을 양성(=1)이라고 하면 '비만'을 음성(=0)으로 표시할 수 있다.

여기에서 회귀와 분류의 근본적인 차이점을 설명한다. 매우 중요한 개념이므로 반드시 이해하고 넘어가야 한다. 회귀 모델의 결과는 데이터 출력에 대한 예측값 그 자체이다. 즉, 회귀 모델의 결과를 $\hat{y}$ 이라고 한다면, $\hat{y}$ 은 데이터의 실제 출력 y 에 대한 예측값이고 이상적인 경우 $\hat{y}=y$ 이다. 따라서 회귀 모델이 내어주는 결과는 출력에 대한 예측 그 자체이다. 반면, 분류모델의 출력은 데이터 출력이 양성 또는 음성일 **확률 또는 가능성**이다. 이때 출력이 가질 수 있는 값이 양성 또는 음성뿐이므로 양성일 확률이 p 라면 음성일 확률은 $1-p$ 이다. 결국, 분류모델은 데이터의 입력속성을 선형조합한 뒤 양성일 확률을 계산하고, 그 확률이 미리 정해진 문턱값(예를 들면 0.5)보다 크면 양성(1)으로 예측하고 문턱값보다 작으면 음성(0)으로 예측하는 모델이다.



(a) 선형회귀 모델



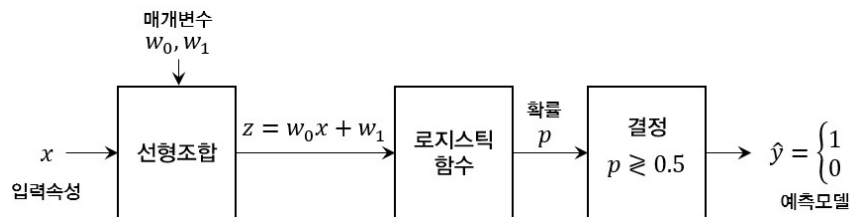
(b) 분류모델 (결정을 위한 문턱값이 0.5인 경우)

[그림 2] 선형회귀 모델과 분류모델의 비교

그림 2는 입력속성이 한 개인 훈련 데이터의 선형회귀 모델과 분류모델의 개념을 비교한 것이다. 그림 2의 (a)는 잘 알려진 선형회귀 모델이다. 즉, 입력속성 x 에 대한 선형조합을 예측값 그 자체로 사용한다. 따라서 $\hat{y}=w_0x+w_1$ 이 된다. 그러나 (b)의 분류모델은 입력속성에 대한 선형조합에 추가로 확률변환 및 결정(판단)의 과정이 더해진다. 단순히 생각해보면, 입력속성을 선형조합한 결과 z 는 연속인 값을 갖게 되는데 우리에게 필요한 것은 이 연속 값을 양성 또는 음성의 두 값 중 하나로 매핑할 필요가 있다. 이를 위해 가장 합리적인 방법은 z 값을 0과 1사이의 확률값으로 변환하고 이를 출력이 양성일 확률로 해석하는 것이다. 따라서 일

단 확률값 p 를 구하면, 확률이 가질 수 있는 가운데 값 0.5를 기준으로 p 가 기준보다 크면 최종 출력을 양성으로, 그렇지 않으면 음성으로 판단할 수 있다.

이제 남은 문제는 그림 2(b)의 분류모델에서 선형조합 결과 z 를 확률 p 로 변환하는 방법이다. 즉, $-\infty \sim \infty$ 의 값을 가질 수 있는 z 를 0~1의 확률값으로 매핑하는 방법이 필요하다. 우리는 이미 이 역할에 적합한 함수를 공부했다. 바로 식 (1)에 정의한 로지스틱 함수이다. 로지스틱 함수는 그림 1과 같이 임의의 z 값을 0과 1 사이의 한 값으로 변환한다. 이렇게 확률변환을 위해 로지스틱 함수를 사용하는 분류모델을 로지스틱 회귀라 한다.



[그림 3] 입력속성이 한 개인 경우의 로지스틱 회귀 모델

그림 3은 입력속성이 한 개인 경우의 로지스틱 회귀 모델을 나타낸다. 이를 수식으로 표현하면 다음과 같다.

$$p = f(w_0x + w_1) = \frac{1}{1 + e^{-(w_0x + w_1)}} \quad (4)$$

이때 확률 p 는 출력이 양성일 확률이라고 가정하자. 여기에서 한 가지 생각해야 할 점은 이 확률이 입력이 x 일 때 출력 y 가 양성일 확률, 즉 조건부 확률이라는 점이다. 따라서 식 (4)를 명확하게 표시하면 다음과 같다.

$$p = P(y=1|x) = \frac{1}{1 + e^{-(w_0x + w_1)}} \quad (5)$$

출력값은 양성 아니면 음성이므로, 양성일 확률이 (5)와 같으면 음성일 확률은 다음과 같다.

$$1-p = P(y=0|x) = \frac{e^{-(w_0x + w_1)}}{1 + e^{-(w_0x + w_1)}} \quad (6)$$

이때, 식 (5)와 (6)은 x 라는 입력이 주어졌을 때, 출력이 양성이거나 음성일 확률이다. 그런데 가만히 생각해보면, 입력 x 에 대한 출력은 이미 결정되어있다. 다만 그 출력이 양성일지 음성일지를 예측해보는 것일 뿐이다. 따라서, 우리는 모르지만 결과는 이미 정해져 있고 식 (5)와 (6)을 통해 뒤늦게나마 양성일지 음성일지를 가늠해보는 것이다. 이런 의미로 식 (5)와 (6)을 사후확률(posterior probability)이라 한다.

명칭이 무엇이건 중요한 것은 식 (5)와 (6)을 통해 입력 x 에 대한 출력이 양성인지 음성인지 확률을 계산할 수 있다는 점이다. 이 식은 입력속성이 한 개인 데이터에 대한 식이다. 이를 M 개의 입력속성으로 일반화하면 다음과 같다.

$$P(y=1 | x_0, x_1, \dots, x_{M-1}) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + \dots + w_{M-1}x_{M-1} + w_M)}} \quad (7)$$

$$P(y=0|x_0, x_1, \dots, x_{M-1}) = \frac{e^{-(w_0x_0 + w_1x_1 + \dots + w_{M-1}x_{M-1} + w_M)}}{1 + e^{-(w_0x_0 + w_1x_1 + \dots + w_{M-1}x_{M-1} + w_M)}} \quad (8)$$

식 (5)와 (6)이 M 개의 입력속성 $x_0, x_1, \dots, x_{M-1}$ 에 대해 동일한 규칙으로 확장된 것을 확인할 수 있다. 위 식에서 $w_0, w_1, \dots, w_{M-1}$ 은 M 개의 입력속성에 대한 매개변수이고 w_M 은 선형조합의 절편(바이어스)을 담당하는 매개변수이다. 식 (7)과 (8)의 지수함수의 지수부를 깔끔하게 정리하면 다음과 같다.

$$w_0x_0 + w_1x_1 + \dots + w_{M-1}x_{M-1} + w_Mx_M = [w_0 \ w_1 \ \dots \ w_{M-1} \ w_M] \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{M-1} \\ x_M=1 \end{bmatrix} = \mathbf{w}^T \mathbf{x} \quad (9)$$

이때 $\mathbf{w}$ 는 선형조합을 위한 $M+1$ 개 매개변수로 구성된 벡터이며 $\mathbf{x}$ 는 M 개의 입력속성 $x_0, x_1, \dots, x_{M-1}$ 과 절편을 위한 한 개의 1로 이루어진 벡터로 다음과 같이 정의된다.

$$\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_{M-1} \\ w_M \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{M-1} \\ x_M=1 \end{bmatrix} \quad (10)$$

따라서 일반화된 로지스틱 회귀의 사후확률은 다음과 같다.

$$P(y=1|\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}} \quad (11)$$

$$P(y=0|\mathbf{x}) = \frac{e^{-\mathbf{w}^T \mathbf{x}}}{1 + e^{-\mathbf{w}^T \mathbf{x}}} \quad (12)$$

아주 깔끔하고 보기 좋은 식이 완성되었다. 이를 바탕으로 일반적인 로지스틱 회귀 모델의 구조를 그림 4에 표현하였다. 그림 4에서 사후확률 p 는 출력이 양성일 확률로 $p = P(y=1|\mathbf{x})$ 이다.



[그림 4] 로지스틱 회귀 모델

남은 목표는 그림 4의 최종 결과물인 $\hat{y}$ 이 실제 출력 y 와 같아지도록 학습을 진행하는 것이다. 즉, 그림 4에서 선형조합을 위한 매개변수 $\mathbf{w}$ 를 적절한 값으로 설정하여 $\hat{y} \approx y$ 가 되도록 하는 것이다. 이를 위해서는 선형회귀에서와 유사하게 예측값 $\hat{y}$ 과 실제값 y 가 유사할수록 작아지는 비용함수를 설계하고 비용함수를 최소화하는 매개변수를 찾으면 된다. 따라서 다음 단

계는 분류모델에 적합한 비용함수를 설계하는 것이다.

3. 최대가능도법 및 교차엔트로피오차

3.1 최대가능도법 (Maximum likelihood method)

간편한 설명을 위해 입력속성이 하나인 데이터를 생각해보자. 즉 데이터의 입력은 x 이고 출력은 y 이다. 출력 y 는 양성(1) 또는 음성(0)의 값을 가질 수 있다. 그림 4의 로지스틱 회귀를 통해 얻은 사후확률 p 는 y 가 양성(1)일 확률이다. 그렇다면 당연히 $1-p$ 는 y 가 음성(0)일 확률이다. 즉, y 가 1일 확률은 p 이고 y 가 0일 확률은 $1-p$ 이다. 이 문장으로 하나의 식으로 표현하면 다음과 같다.

$$p^y(1-p)^{1-y} \quad (13)$$

너무나 당연한 표현인데, $y=1$ 을 대입하면 식 (13)은 p 가 되고 $y=0$ 을 대입하면 (13)은 $1-p$ 가 된다. 따라서 식 (13)은 출력 y 가 가질 수 있는 모든 값에 대한 확률을 나타내는 식으로 **상태확률**이라고 부른다. 이때 식 (13)의 y 는 데이터의 출력으로 이미 정해진 값이고 p 는 그림 4와 같이 로지스틱 회귀를 통해 구한 확률값이다. 즉, y 는 고정된 값인데 반해 p 는 입력 속성 x 와 매개변수 w 의 함수이다. 따라서 식 (13)을 **입력 x , 매개변수 w 가 주어졌을 때 출력이 y 일 확률**이라고 해석할 수 있으며 다음과 같이 표현할 수 있다.

$$P(y|x, w) = p^y(1-p)^{1-y} \quad (14)$$

이 식의 의미가 무엇인지 이해하는 것은 매우 중요하다. 먼저 식 (14)에서 y 와 p 의 역할을 이해해야 한다. y 는 데이터의 출력으로 1 또는 0의 값을 갖고, 우리가 예측해서 맞춰야 할 목표이다. p 는 y 를 예측하기 위해 계산한 확률이다. 그럼 식 (14)의 값은 언제 커질까? $y=1$ 인 경우를 생각해 보면 $P(y=1|x, w)=p$ 가 되기 때문에 p 가 1에 근접할수록 커진다. 반대로 $y=0$ 인 경우에는 $P(y=0|x, w)=1-p$ 이므로 p 가 0에 근접할수록 커진다. 즉, $y=1$ 이면 $p \rightarrow 1$ 일수록, $y=0$ 이면 $p \rightarrow 0$ 일수록 식 (14)의 값이 커진다. 다시 말해, p 가 y 에 근접할수록 식 (14)의 값은 커진다. 그런데, p 는 $y=1$ 일 확률이고 $1-p$ 는 $y=0$ 일 확률이다. 우리가 매개변수 설정을 매우 잘해서 y 값에 대한 예측을 아주 잘했다면 $p \approx y$ 가 될 것이다. 그렇다면 식 (14)도 아주 큰 값을 갖게 될 것이다! 따라서 예측을 잘할수록 식 (14)의 값은 커진다.

[표 1] N 개의 데이터에 대한 상태확률

데이터 번호	입력	출력	사후확률	상태확률, 식 (14)
0	x_0	y_0	p_0	$P(y_0 x_0, w) = p_0^{y_0}(1-p_0)^{1-y_0}$
1	x_1	y_1	p_1	$P(y_1 x_1, w) = p_1^{y_1}(1-p_1)^{1-y_1}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$N-1$	x_{N-1}	y_{N-1}	p_{N-1}	$P(y_{N-1} x_{N-1}, w) = p_{N-1}^{y_{N-1}}(1-p_{N-1})^{1-y_{N-1}}$

표 1은 N 개의 데이터가 있을 때 주어진 매개변수 w 를 이용해 각 데이터에 대해 계산한 사후확률과 상태확률을 나타낸 것이다. N 개의 출력은 각각 1 또는 0의 값을 갖는다. 앞에서 설명한 바와 같이 각 y 값이 무엇이든 상관없이 사후확률이 출력과 유사해지면 각각의 상태확률의 값은 커지고 반대로 사후확률이 출력과 달라질수록 상태확률의 값은 작아진다. 각 데이터에 대한 개별 상태확률을 모두 곱하면 다음과 같이 데이터 세트 전체에 대한 상태확률을 구할 수 있다.

$$\begin{aligned} P(y|X, w) &= \prod_{n=0}^{N-1} P(y_n | x_n, w) \\ &= \prod_{n=0}^{N-1} p_n^{y_n} (1-p_n)^{1-y_n} \end{aligned} \quad (15)$$

식 (15)는 N 개의 확률의 곱이므로 여전히 확률값이며 벡터 y 와 행렬 X 는 표 1의 N 개의 출력을 나타내는 벡터와 M 개의 입력속성을 갖는 N 개의 데이터 입력을 나타내는 행렬이다.

식 (15)의 의미를 잘 살펴보자. 식 (15)는 개별 확률의 곱이므로, 개별 확률이 최대값을 가질 때 식 (15)도 최대값을 갖게 될 것이다. 개별 확률이 최대가 되기 위해서는 각 데이터에 대해 계산한 사후확률이 최대한 실제 출력과 유사해야 한다. 다시 말해 식 (15)가 최대가 되었다면 우리가 계산한 각각의 사후확률들이 실제 출력값에 근접했음을 의미한다. 즉, 식 (15)는 N 개의 사후확률이 실제 출력 그 자체일 가능성의 정도를 나타내는 값이다. 따라서 식 (15)를 가능도 또는 우도라고 한다. 가능도 또는 우도는 ‘가능성의 정도’를 나타내는 한자어이고 영어 likelihood에 대한 번역 표현이다.

이쯤에서 우리의 목표를 일깨워줄 필요가 있을 듯하다. 최종 목표는 그림 4를 완성하는 것이다. 즉, 예측값 $\hat{y}$ 와 실제값 y 가 최대한 유사해지는 매개변수 w 를 찾는 것이다. 그럼 어떻게 w 를 찾아야 할까? 결론은 식 (15)의 가능도를 최대화하는 매개변수 w 가 우리의 솔루션이 된다. 가능도를 최대화한다는 것은 각 데이터의 상태확률을 최대화하는 것이고, 이는 각 데이터의 사후확률이 각 데이터의 출력과 같아질 가능성을 최대화하는 것이기 때문이다. 이렇게 가능도를 최대화하는 매개변수를 찾는 방식을 최대가능도법 또는 최대우도법(Maximum likelihood method)이라고 한다.

가능도를 나타내는 식 (15)는 N 개의 확률의 곱으로 이루어져 있으므로 구조가 복잡하고 계산이 까다롭다. 또한 1보다 작은 확률들이 연속적으로 곱해지기 때문에 N 이 커질수록 가능도의 값이 매우 작아지는 문제가 발생하기도 한다. 따라서 식 (15)의 양변에 로그를 적용하여 곱을 합으로 변환하면 이런 문제들을 해결할 수 있다. 또한 로그함수는 단조증가 함수이므로 로그를 적용해도 식 (15)를 최대화하는 매개변수의 값은 변하지 않는다. 식 (15)의 양변에 자연로그를 취하면 다음과 같다.

$$\begin{aligned} \ln P(y|X, w) &= \sum_{n=0}^{N-1} \ln P(y_n | x_n, w) \\ &= \sum_{n=0}^{N-1} \ln p_n^{y_n} (1-p_n)^{1-y_n} \\ &= \sum_{n=0}^{N-1} \left\{ \ln p_n^{y_n} + \ln (1-p_n)^{1-y_n} \right\} \end{aligned}$$

$$= \sum_{n=0}^{N-1} \{y_n \ln p_n + (1-y_n) \ln(1-p_n)\} \quad (16)$$

식 (16) 역시 가능도를 나타내는 식이다. 그런데, (15)와 구분하기 위해 (16)을 로그 가능도 또는 로그 우도(log likelihood)라고 부른다. 식 (15)는 확률이므로 최대값이 1이다. 따라서 (15)에 로그를 적용한 식 (16)은 항상 음수이다. 또한 식 (15)를 최대화하는 솔루션과 식 (16)을 최대화하는 솔루션은 동일하므로, 최대가능도법은 일반적으로 다루기 수월한 로그 가능도를 사용한다.

3.2 교차엔트로피오차 (Cross Entropy Error)

최적 매개변수를 찾기 위해서는 식 (16)과 같은 로그 가능도 식을 세우고, 이 식을 최대화하는 매개변수를 찾아야 한다. 그러나 식 (16)을 최대화하는 닫힌꼴(closed-form)의 해석하는 몇몇 특수한 경우를 제외하고는 알려진 바가 없다. 일반적인 해석해를 얻을 수 없다면 선형회귀에서 사용했던 경사하강법이라는 알고리즘을 이용해 솔루션을 탐색하는 방법을 생각해볼 수 있다.

경사하강법은 매개변수의 공간에 정의된 비용함수의 최소점을 찾아가는 알고리즘이다. 이 알고리즘을 적용하기 위해서는 모델의 예측 정확도가 높을수록 작은 값을 갖는 비용함수가 필요하다. 즉, 매개변수 공간에 정의된 비용함수가 항아리와 같이 오목한 형태이고 항아리의 가장 낮은 지점에서의 매개변수가 최적 솔루션이어야 하는 것이다. 그러나 최대가능도법에서는 식 (16)의 로그 가능도를 '최대화'하는 위치를 찾아야 한다. 로그 가능도는 음수이면서 볼록한 형태이므로 경사하강법을 곧바로 적용하는 것이 불가능하다.

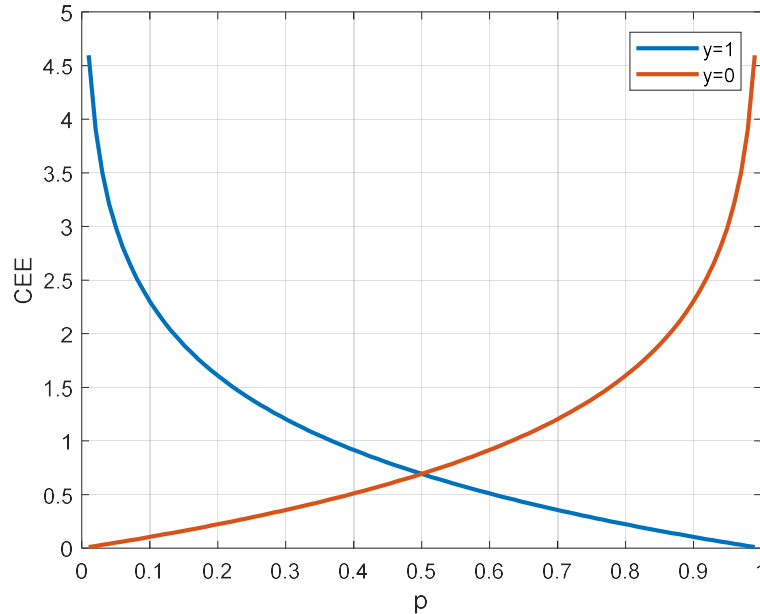
또한 로그 가능도를 비용함수로 사용하기에는 어색한 측면이 있다. 비용함수라는 용어가 내포하는 의미는 모델의 예측 정확도가 높을수록 시스템이 낮은 비용을 지불한다는 것으로 낮은 비용은 높은 성능을 의미한다. 그러나 로그 가능도는 이와 반대로 모델의 예측 정확도가 높을수록 큰 값을 갖게 된다. 로그 가능도를 비용함수로 활용하기 위한 가장 간단한 방법은 로그 가능도의 부호를 반전하는 것이다. 즉, 로그 가능도는 확률에 로그함수를 적용한 것이므로 항상 음의 값을 갖는 볼록한 함수이다. 따라서 로그 가능도에 -1 을 곱해 부호를 반전하면 양의 값을 갖는 오목한 함수가 되고, 이 함수의 가장 낮은 위치에서 모델의 예측 성능이 최적화된다. 즉, 부호 반전된 로그 가능도는 새로운 비용함수로 활용될 수 있다. 부호 반전된 로그 가능도를 수식으로 표현하면 다음과 같다.

$$\epsilon_{CEE} = -\frac{1}{N} \sum_{n=0}^{N-1} \{y_n \ln p_n + (1-y_n) \ln(1-p_n)\} \quad (17)$$

이 식의 정식 명칭은 평균 교차엔트로피오차(Cross Entropy Error)이며 식에 곱해진 $\frac{1}{N}$ 은 각각의 교차엔트로피오차에 대한 평균을 구하는 의미로 곱해진 것이다. 평균 교차엔트로피오차는 머신러닝 분야에서 평균제곱오차와 함께 가장 널리 사용되는 비용함수이다.

교차엔트로피오차는 로그 가능도의 반전된 형태라는 점을 기억하기 바란다. 로그 가능도는 모델의 예측값과 실제값이 유사할수록 큰 값을 가졌지만, 교차엔트로피오차는 반대로 예측값이 실제값과 유사할수록 작은 값을 갖는다. 이것이 식 (17)의 이름에 '오차'라는 단어를 붙인 이유이다. 그림 5는 식 (17)에서 $N=1$ 이라고 가정하고 p 값에 따른 교차엔트로피오차의 값을 그린 것이다. $y=1$ 일 때, 즉 실제값이 1일 때 예측한 값의 확률 p 가 1에 근접할수록 교차엔

트로피오차의 값은 줄어듦과 반대로 $y=0$ 일 때에는 p 가 0에 가까울수록 오차가 0이 되는 것을 확인할 수 있다.

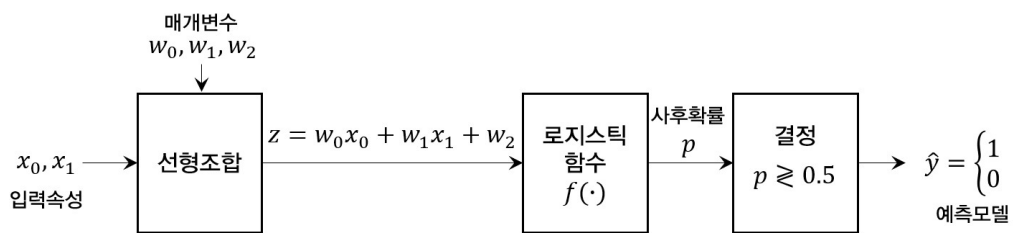


[그림 5] 교차엔트로피오차의 변화

4. 2입력 이진분류 모델

4.1 모델의 설계

지금까지 학습한 내용을 종합해 입력속성이 두 개인 이진분류 모델을 설계하고 경사하강법을 적용하여 최적 매개변수를 탐색하는 과정을 살펴보자. 그림 6은 2입력 이진분류 모델의 구조이고 표 2는 N 개의 데이터를 그림 6의 모델에 적용한 각종 변수의 표기법이다. 앞으로 표 2의 표기법을 사용해 설명을 진행한다.



[그림 6] 2입력 이진분류 모델

그림 6의 첫 번째 과정은 매개변수를 이용해 입력 데이터를 선형조합하는 것이다. n 번째 데이터에 대한 선형조합의 결과 z_n 은 다음과 같다.

$$z_n = w_0 x_{0,n} + w_1 x_{1,n} + w_2 \quad (18)$$

이 선형조합의 결과를 로지스틱 함수(식 (3))에 넣으면 사후확률을 구할 수 있다.

[표 2] N 개의 데이터에 대한 표기법

데이터 번호	입력		출력 y	선형조합 $z = w_0x_0 + w_1x_1 + w_2$	사후확률 $p = f(z)$	예측값 $\hat{y}$
	x_0	x_1				
0	$x_{0,0}$	$x_{1,0}$	y_0	z_0	p_0	$\hat{y}_0$
1	$x_{0,1}$	$x_{1,1}$	y_1	z_1	p_1	$\hat{y}_1$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
n	$x_{0,n}$	$x_{1,n}$	y_n	z_n	p_n	$\hat{y}_n$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$N-1$	$x_{0,N-1}$	$x_{1,N-1}$	y_{N-1}	z_{N-1}	p_{N-1}	$\hat{y}_{N-1}$

이때 사후확률은 n 번째 데이터의 출력 y_n 이 양성일 확률을 의미한다.

$$p_n = \frac{1}{1 + e^{-z_n}} = \frac{1}{1 + e^{-(w_0x_{0,n} + w_1x_{1,n} + w_2)}} \quad (19)$$

이를 이용해 비용함수인 교차엔트로피오차(식 (17))를 다음과 같이 정의할 수 있다.

$$\epsilon_{CEE}(w_0, w_1, w_2) = -\frac{1}{N} \sum_{n=0}^{N-1} \{y_n \ln p_n + (1 - y_n) \ln (1 - p_n)\} \quad (20)$$

식 (20)은 매개변수 w_0, w_1, w_2 의 함수이며, 경사하강법을 이용해 최적 매개변수를 찾으면 모델이 완성된다.

4.2 경사하강법 적용

경사하강법을 통해 식 (20)를 최소화하는 매개변수를 탐색한다. 이를 위해 매개변수의 업데이트 규칙을 다음과 같이 정의할 수 있다.

$$w_0[t+1] = w_0[t] - \alpha \frac{\partial}{\partial w_0} \epsilon_{CEE}(w_0, w_1, w_2) \quad (21)$$

$$w_1[t+1] = w_1[t] - \alpha \frac{\partial}{\partial w_1} \epsilon_{CEE}(w_0, w_1, w_2) \quad (22)$$

$$w_2[t+1] = w_2[t] - \alpha \frac{\partial}{\partial w_2} \epsilon_{CEE}(w_0, w_1, w_2) \quad (23)$$

t 는 업데이트 단계를 나타내는 변수로 정수값을 가질 수 있으며 $w_0[0], w_1[0], w_2[0]$ 은 각 매개변수의 초기값(initial value)이다. α 는 학습률을 나타낸다.

다음 절차는 교차엔트로피오차의 기울기를 계산해야 한다. 먼저 w_0 에 대한 기울기 $\frac{\partial}{\partial w_0} \epsilon_{CEE}(w_0, w_1, w_2)$ 를 구해보자. 식 (20)을 보면 교차엔트로피오차 ϵ_{CEE} 는 사후확률 p_n 의 함수이고 식 (19)에서 사후확률 p_n 은 선형조합의 결과 z_n 의 함수이며, 식 (18)에서 선형조합의 결과 z_n 은 매개변수 w_0 의 함수라는 사실을 알 수 있다. 따라서 다음과 같이 연쇄적으로 미분을 구할 수 있다.

$$\frac{\partial}{\partial w_0} \epsilon_{CEE}(w_0, w_1, w_2) = \frac{\partial \epsilon_{CEE}}{\partial p_n} \cdot \frac{\partial p_n}{\partial z_n} \cdot \frac{\partial z_n}{\partial w_0} \quad (24)$$

식 (24)의 우변을 차례로 구해 식 (24)에 대입하면 기울기를 얻을 수 있다. 먼저 첫 번째 항을 다음과 같이 구할 수 있다.

$$\begin{aligned} \frac{\partial \epsilon_{CEE}}{\partial p_n} &= -\frac{1}{N} \sum_{n=0}^{N-1} \left\{ y_n \frac{\partial}{\partial p_n} \ln p_n + (1-y_n) \frac{\partial}{\partial p_n} \ln(1-p_n) \right\} \\ &= -\frac{1}{N} \sum_{n=0}^{N-1} \left(\frac{y_n}{p_n} - \frac{1-y_n}{1-p_n} \right) \end{aligned} \quad (25)$$

다음으로 식 (24)의 두 번째 항 $\frac{\partial p_n}{\partial z_n}$ 은 로지스틱 함수의 1차 도함수이므로 식 (3)을 그대로 이용할 수 있다. 따라서 다음과 같다.

$$\frac{\partial p_n}{\partial z_n} = p_n(1-p_n) \quad (26)$$

마지막 항 $\frac{\partial z_n}{\partial w_0}$ 은 식 (18)로부터 다음과 같이 간단하게 구할 수 있다.

$$\frac{\partial z_n}{\partial w_0} = x_{0,n} \quad (27)$$

식 (25), (26), (27)의 결과를 식 (24)에 대입하면 다음과 같다.

$$\begin{aligned} \frac{\partial}{\partial w_0} \epsilon_{CEE}(w_0, w_1, w_2) &= \frac{\partial \epsilon_{CEE}}{\partial p_n} \cdot \frac{\partial p_n}{\partial z_n} \cdot \frac{\partial z_n}{\partial w_0} \\ &= -\frac{1}{N} \sum_{n=0}^{N-1} \left(\frac{y_n}{p_n} - \frac{1-y_n}{1-p_n} \right) (1-p_n) x_{0,n} \\ &= \frac{1}{N} \sum_{n=0}^{N-1} (p_n - y_n) x_{0,n} \end{aligned} \quad (28)$$

즉, 교차엔트로피오차의 w_0 에 대한 기울기는 사후확률과 데이터 출력의 차이를 입력 데이터와 곱한 값들의 평균이다. 여기에서 p_n 은 y_n 이 양성일 확률이므로 예측이 정확할수록 $p_n \approx y_n$ 이 된다. 따라서 $p_n - y_n$ 은 예측 오차를 나타내며, 식 (28)은 예측 오차가 클수록 기울기가 커진다는 것을 나타낸다.

식 (18)을 보면 w_0 와 w_1 의 구조적인 차이는 전혀 없다. 따라서 교차엔트로피오차의 w_1 에 대한 기울기는 식 (28)을 그대로 재 활용하여 다음과 같이 구할 수 있다.

$$\frac{\partial}{\partial w_1} \epsilon_{CEE}(w_0, w_1, w_2) = \frac{1}{N} \sum_{n=0}^{N-1} (p_n - y_n) x_{1,n} \quad (29)$$

다음으로 교차엔트로피오차의 w_2 에 대한 기울기는 앞의 방법과 동일하게 다음과 같이 구할 수 있다.

$$\frac{\partial}{\partial w_2} \epsilon_{CEE}(w_0, w_1, w_2) = \frac{\partial \epsilon_{CEE}}{\partial p_n} \cdot \frac{\partial p_n}{\partial z_n} \cdot \frac{\partial z_n}{\partial w_2} \quad (30)$$

그런데 식 (30)의 우변의 첫 두 항은 식 (25), (26)과 같고 세 번째 항은 다음과 같다.

$$\frac{\partial z_n}{\partial w_2} = 1 \quad (31)$$

따라서 식 (25), (26), (31)을 식 (30)에 대입하면 다음과 같은 기울기를 구할 수 있다.

$$\frac{\partial}{\partial w_2} \epsilon_{CEE}(w_0, w_1, w_2) = \frac{1}{N} \sum_{n=0}^{N-1} (p_n - y_n)$$

최종적으로 교차엔트로피오차를 비용함수로 사용할 경우 각 매개변수에 대한 기울기는 다음과 같이 정리할 수 있다.

$$\frac{\partial}{\partial w_0} \epsilon_{CEE}(w_0, w_1, w_2) = \frac{1}{N} \sum_{n=0}^{N-1} (p_n - y_n) x_{0,n} \quad (32)$$

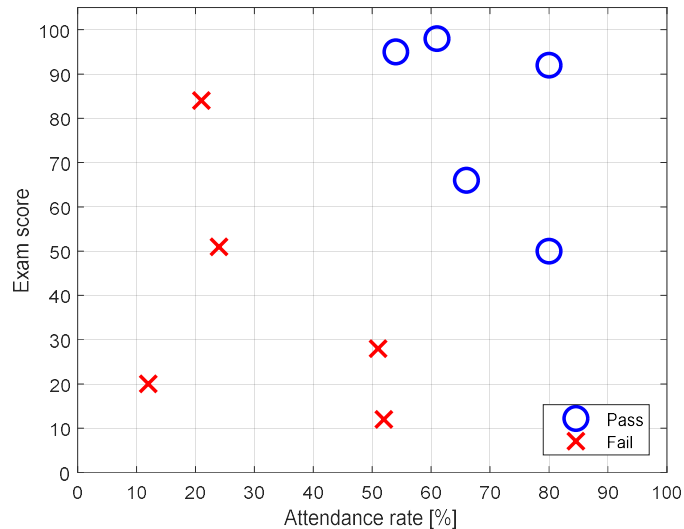
$$\frac{\partial}{\partial w_1} \epsilon_{CEE}(w_0, w_1, w_2) = \frac{1}{N} \sum_{n=0}^{N-1} (p_n - y_n) x_{1,n} \quad (33)$$

$$\frac{\partial}{\partial w_2} \epsilon_{CEE}(w_0, w_1, w_2) = \frac{1}{N} \sum_{n=0}^{N-1} (p_n - y_n) \quad (34)$$

따라서 식 (21), (22), (23)의 매개변수 업데이트 규칙에 식 (32), (33), (34)의 기울기 식을 적용하면 최적 매개변수 w_0, w_1, w_2 를 찾을 수 있다. 뿐만 아니라, 식 (32), (33), (34)로부터 반복적으로 나타나는 규칙성을 찾을 수 있는데 이 규칙성을 이용하면 입력속성이 M 개로 늘어나더라도 모든 $M+1$ 개의 매개변수에 대한 기울기를 쉽게 구할 수 있다.

4.3 실제 데이터를 이용한 설계

지금까지 이진분류를 위한 로지스틱 회귀 모델의 이론을 설명하였다. 이제 실제 데이터를 이용해 로지스틱 회귀 모델을 설계하고 어떻게 동작하는지 알아보자.



[그림 7] 출석률과 시험성적으로 이수(pass)와 낙제(fail)를 구분한 데이터의 예

그림 7은 어떤 과목을 수강한 학생 10명의 이수 성공 여부를 출석률과 시험성적으로 구분하여 표시한 데이터이다. 동그라미로 표시한 학생은 성공적으로 이수한 학생이고 x로 표시된

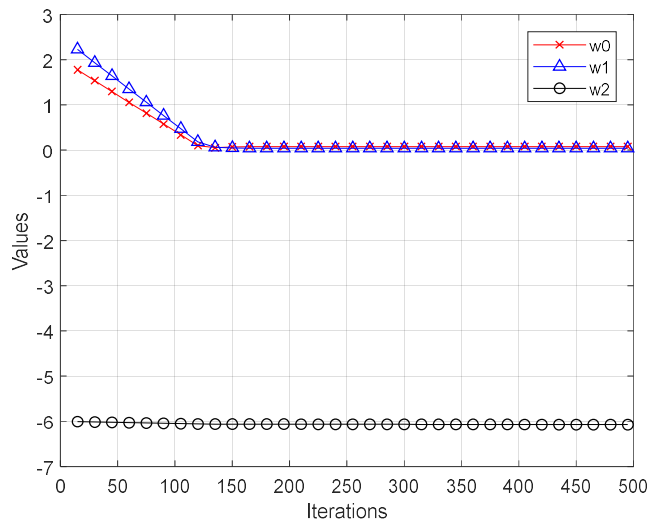
학생은 이수에 실패한 학생이다. 일반적으로 출석률과 시험성적이 높으면 성공할 확률이 높고 그렇지 않은 경우 낙제할 확률이 높다. 따라서 그래프에서 우상단에 위치할수록 성공할 확률이 높을 것이라 예상할 수 있다. 문제는 성공과 실패를 구분하는 경계의 위치이다. 이 경계를 결정경계(decision boundary)라 한다. 결국 분류의 문제는 가장 적절한 결정경계의 위치를 찾는 것과 같다. 그림 7의 원 데이터는 표 3과 같다.

[표 3] 그림 7의 원 데이터

데이터 번호	입력		출력 y (이수 여부, 1=pass, 0=fail)
	x_0 (출석률)	x_1 (시험성적)	
0	21	84	0
1	54	95	1
2	80	50	1
3	51	28	0
4	66	66	1
5	80	92	1
6	24	51	0
7	61	98	1
8	12	20	0
9	52	12	0

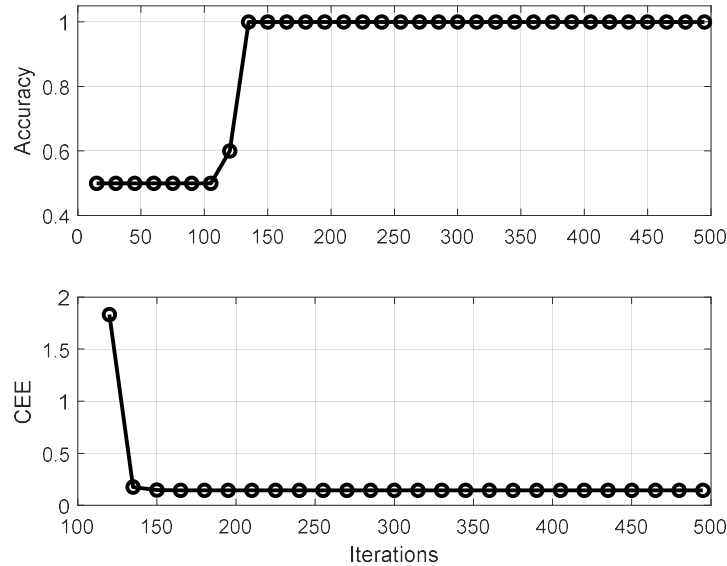
표 3의 데이터는 두 개의 입력을 가지고 있으므로 선형조합을 위한 매개변수는 w_0, w_1, w_2 가 될 것이다. 이를 구하기 위해 초기값을 $w_0[0] = 2, w_1[0] = 2.5, w_2[0] = -6$ 으로 할당하고 학습률 $\alpha = 0.001$ 로 설정한 뒤 식 (21), (22), (23)을 이용하여 경사하강법을 적용하였다.

그림 8은 경사하강법의 반복 횟수에 따른 매개변수의 변화 추이를 그린 것이다. 약 150회의 반복 이후 모든 매개변수가 최적값으로 수렴하는 것을 알 수 있다. 이를 좀 더 정확하게 확인하기 위해 교차엔트로피오차와 정확도를 확인해야 한다.



[그림 8] 경사하강법 알고리즘의 반복 횟수에 따른 매개변수의 변화

그림 9는 반복 횟수에 따른 정확도와 교차엔트로피오차의 변화를 그린 것이다. 정확도란 실제 데이터 y 와 예측한 값 $\hat{y}$ 중 일치하는 값의 비율이다. 그림 9에서 약 150회 반복 이후에 예측 정확도가 100%를 달성하는 것을 확인할 수 있다. 이와 함께 교차엔트로피오차도 매우 낮은 값으로 수렴하였다.

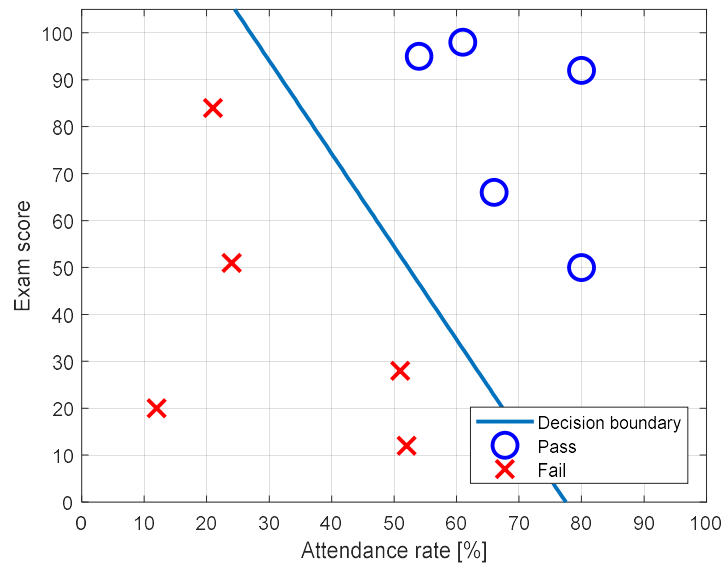


[그림 9] 정확도 및 교차엔트로피오차의 변화

그림 10은 원 데이터를 결정경계와 함께 그린 것이다. 우리가 설계한 분류 모델은 그림 6의 모델이다. 이 모델에서 사후확률 p 를 계산한 뒤 0.5를 기준으로 p 가 0.5보다 크면 $\hat{y}=1$ 로 결정하고 반대의 경우 $\hat{y}=0$ 으로 결정한다. 이 결정의 기준 0.5는 입력의 선형조합 z 에 대한 로지스틱 함수의 출력에 대한 기준이다. 본 장의 첫 그림인 그림 1에서 알 수 있듯이 로지스틱 함수에서 출력이 0.5가 되는 지점은 입력이 0이 되는 지점이다. 따라서 결정경계는 입력의 선형조합 z 가 0이 되는 지점과 같다. 즉, 입력의 선형조합은 $z = w_0x_0 + w_1x_1 + w_2$ 이므로 그림 10에서의 결정경계는 다음과 같다.

$$w_0x_0 + w_1x_1 + w_2 = 0 \rightarrow x_1 = -\frac{w_0}{w_1}x_0 - \frac{w_2}{w_1} \quad (35)$$

다시 말해서, 결정경계는 입력속성의 평면에서 기울기가 $-\frac{w_0}{w_1}$ 이고 절편이 $-\frac{w_2}{w_1}$ 인 직선이다. 이 직선이 입력속성들의 평면을 양성 영역과 음성 영역으로 분할한다.



[그림 10] 원 데이터와 결정경계

4.4 클래스 사이의 데이터 불균형 문제 - 리밸런싱

그림 4의 로지스틱 회귀 모델에서 별다른 의심 없이 받아들였던 부분은 마지막 결정의 단계에서 0.5를 기준으로 양성과 음성을 판정한다는 것이다. 사후확률 p 는 말 그대로 확률이기 때문에 확률의 가운데 값인 0.5를 결정의 임계값으로 삼는 것은 큰 문제가 없어 보인다. 그러나, 0.5를 기준으로 한다는 것은 전체 데이터 중 양성 데이터와 음성 데이터의 수가 동일하거나 완전히 동일하지는 않더라도 비슷한 수준으로 분포한다는 가정을 바탕에 두는 것과 같다. 예를 들어 100개의 데이터 중 양성 데이터가 95개이고 음성 데이터가 5개인 경우에는 굳이 애써 그림 4와 같은 모델을 만들 필요 없이 무조건 양성이라고 판정하기만 해도 무려 95%의 예측 성공률을 갖게 된다. 이러한 문제를 클래스 사이의 데이터 불균형 문제라고 한다. 따라서 훈련 데이터를 구성할 때에는 각 클래스에 속한 데이터의 수를 비슷하게 구성해야 한다.

그러나 근본적으로 클래스 간의 균형을 맞출 수 없는 경우도 존재한다. 머신러닝의 목적이 훈련 데이터로 학습한 모델을 실제 상황에서도 제대로 동작하는 모델을 개발하는 것, 즉 일반화 성능을 최대화하는 모델을 만드는 것이라는 점을 기억해야 한다. 이를 위해서는 훈련 데이터가 실제 상황의 특징을 잘 반영할수록 학습한 모델의 일반화 성능이 향상될 것이다. 이런 측면에서, 훈련 집합의 구성은 실제 상황과 유사해야 한다. 그런데 실제 상황이 클래스 사이에 불균형이 존재하는 경우에는 훈련 데이터의 클래스간 불균형 문제를 피할 수 없다. 예를 들어, 국내 전체 암 질환자 중 위암 환자는 13%인 반면 담낭암 환자는 3%에 불과하다. 따라서 암 질환자에 대한 훈련 데이터 역시 이 비율과 유사하게 13%는 위암, 3%는 담낭암으로 구성하는 것이 적절하다. 즉, 실제 상황이 클래스간 불균형 문제를 내포하고 있는 상황에서는 필연적으로 훈련 데이터 내에 클래스간 불균형 문제가 발생한다. 이와 함께 다양한 이유로 훈련 데이터의 클래스간 균형을 맞추는 것은 매우 어렵다.

따라서 모든 경우에 0.5라는 임계값을 기준으로 최종 결정을 내리는 것은 무리가 있다. 클래스간 불균형이 심할 때에는 반드시 임계값의 보정을 통해 불균형 문제를 완화해야 한다. 그림 4에서 사용했던 결정 규칙을 다시 써보면 다음과 같다.

$$\hat{y} = \begin{cases} 1, & p \geq \frac{1}{2} \\ 0, & \text{otherwise} \end{cases} \quad (36)$$

식 (36)의 결정 규칙은 사후확률 p 에 대해 정의한다. 종종 머신러닝의 다른 자료에서 p 대신 $\frac{p}{1-p}$ 에 대해 정의하기도 한다. p 는 입력 데이터가 양성일 확률을 의미하므로 $\frac{p}{1-p}$ 는 양성일 확률과 음성일 확률의 비율이다. 이 값을 odds라고 하는데, 영어사전을 찾아보면 확률 또는 가능성이라는 뜻이지만 여기에서는 그런 의미와는 조금 다른 의미로 사용되므로 우리말로 정확하게 번역하는 것이 어려워 그냥 발음대로 '오즈'라고 표현한다. 일종의 고유명사로 받아들이면 좋겠다. 그럼, 식 (36)의 결정 규칙을 오즈의 함수로 표현하면 다음과 같다.

$$\hat{y} = \begin{cases} 1, & \frac{p}{1-p} \geq 1 \\ 0, & \text{otherwise} \end{cases} \quad (37)$$

식 (36)과 (37)은 동일한 식으로, 데이터의 양성과 음성 비율이 동일하다는 가정 하에 사용할 수 있는 결정 규칙이다.

만약 전체 데이터의 수가 N 이고 이 중 양성 데이터의 수가 N^+ 인 경우 결정 규칙을 다음과 같이 조정하면 클래스 불균형 문제를 완화할 수 있다.

$$\hat{y} = \begin{cases} 1, & p \geq \frac{N^+}{N} \\ 0, & \text{otherwise} \end{cases} \quad (38)$$

만약 양성 데이터와 음성 데이터의 수가 동일하다면 식 (36)의 결정 임계값은 여전히 0.5가 될 것이다.

식 (38)은 클래스 불균형 문제에 대한 적절한 해법이지만 일반적으로 시스템의 임계값은 상수여야 한다. 즉 상황에 따라 임계값을 바꾸는 것은 시스템의 안정성을 해칠 위험이 있다. 따라서 식 (38)과 같이 임계값을 조정하는 대신 확률 p 값을 다음과 조정하는 것이 바람직하다.

$$p' = \frac{N}{2N^+} p \quad (39)$$

p' 은 클래스 불균형을 고려해 보정한 확률이다. 따라서 식 (39)와 같이 보정을 하면 결정 규칙은 다음과 같다.

$$\hat{y} = \begin{cases} 1, & p' \geq \frac{1}{2} \\ 0, & \text{otherwise} \end{cases} \quad (40)$$

또한 보정된 오즈를 이용한 결정 규칙은 다음과 같다.

$$\hat{y} = \begin{cases} 1, & \frac{p'}{1-p'} \geq 1 \\ 0, & \text{otherwise} \end{cases} \quad (41)$$

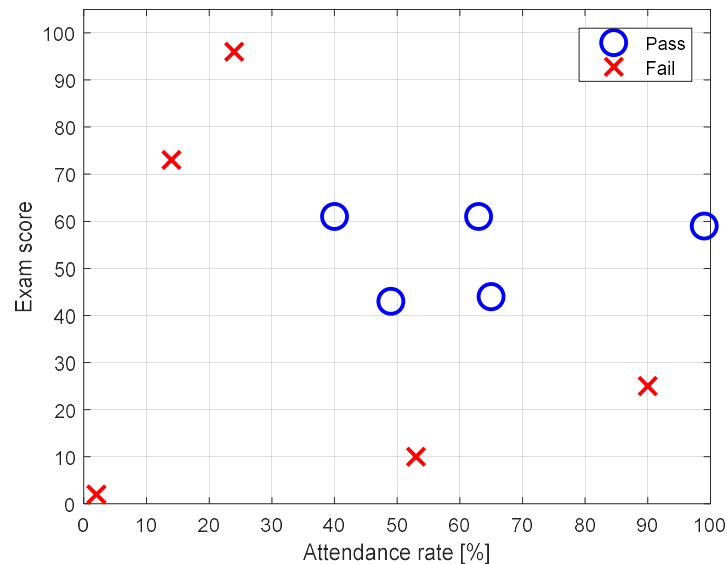
이와 같은 보정 기법을 리밸런싱(rebalancing)이라 한다.

5. 속성공간의 선형분할과 로지스틱 회귀의 한계

4.3절에서 보여준 그림 10의 결과는 매우 중요한 의미를 내포하고 있다. 이진분류의 목표는

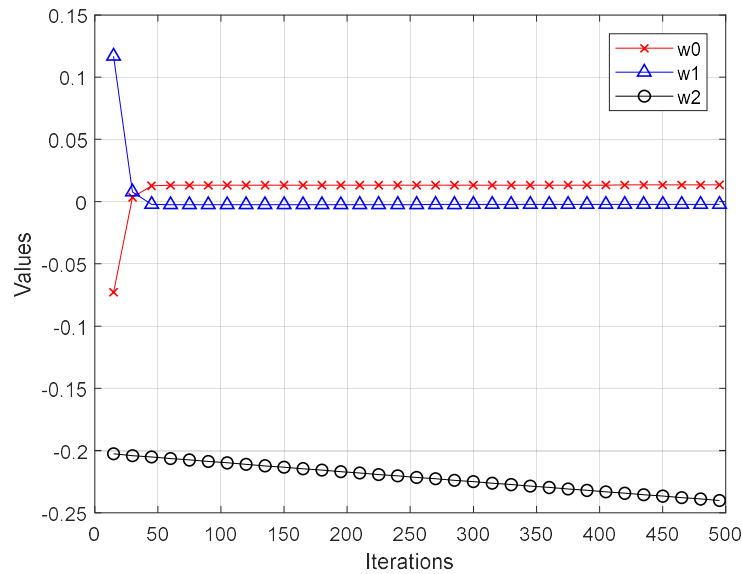
입력속성의 공간을 두 영역으로 구분하는 것이다. 두 영역을 구분하는 경계가 결정경계이다. 로지스틱 회귀는 근본적으로 입력속성의 선형조합을 이용해 분류 확률을 계산하기 때문에 그림 10에 나타난 바와 같이 결정경계는 항상 선형이다. 이를 다른 표현으로, 로지스틱 회귀는 입력속성의 공간을 선형분할한다고 말한다.

입력속성 공간의 선형분할은 로지스틱 회귀가 가지고 있는 특징이며 근본적인 한계이기도 하다. 예를 들어 그림 11의 데이터를 생각해보자. 그림 11 역시 출석률과 시험성적으로 이수 여부를 판단한 결과이다. 그런데, 이 데이터에는 낙제의 기준을 추가하였다. 시험성적이 지나치게 낮거나 출석률이 지나치게 낮으면 자동으로 낙제하는 방식이다. 따라서 이 과목을 이수하기 위해서는 시험성적과 출석률 모두 어느 수준 이상을 달성해야 한다. 이제 이 데이터를 로지스틱 회귀를 이용해 분류해보자. 본격적으로 모델을 만들기에 앞서 데이터를 살펴보자. 이 평면에 하나의 직선을 그려 이수한 학생과 낙제한 학생을 완벽하게 분류할 수 있을까? 이 데이터는 근본적으로 하나의 직선만으로 분류하는 것이 불가능하다.



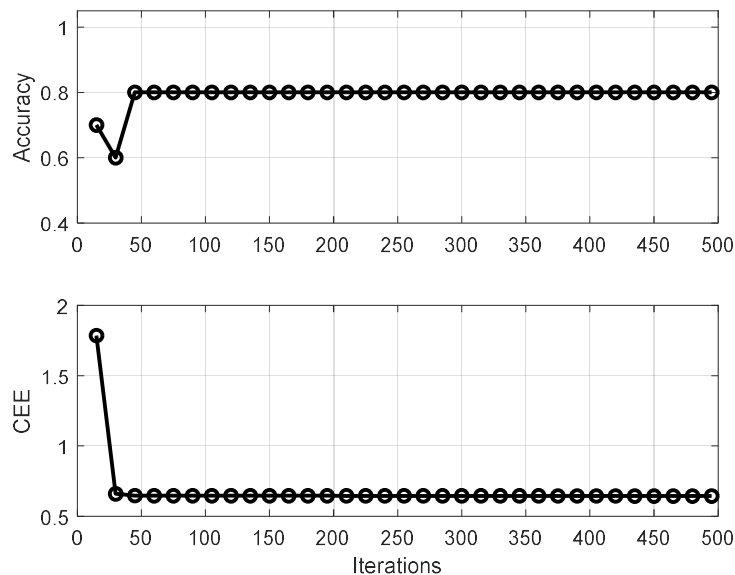
[그림 11] 비선형성이 내재된 데이터의 예

그림 12는 경사하강법을 이용해 매개변수를 탐색하는 과정을 보여준다. w_0 와 w_1 은 특정한 값으로 수렴하였고 w_2 는 계속 변하고는 있지만, 어느 정도 수렴의 과정에 있는 듯하다. 이렇게 수렴하면 우리가 원하는 분류 모델이 잘 만들어지고 있는 것일까?



[그림 12] 경사하강법을 이용한 매개변수의 탐색

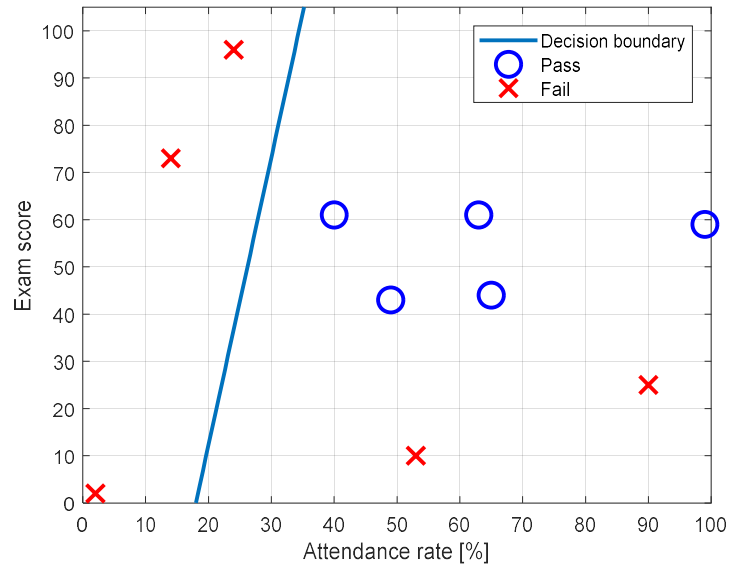
그림 13에 예측 정확도와 교차엔트로피오차의 변화를 나타내었다. 위에서 매개변수들이 어느 정도 수렴하는 것과 같이 정확도 및 교차엔트로피오차도 수렴하고 있다. 그런데, 정확도가 80%에서 더 이상 나아지지 않고 있다. 즉, 10개의 데이터 중 8개만 정확하게 예측하고 나머지는 예측에 실패하고 있다는 것이다. 더욱 비관적인 것은 경사하강을 아무리 많이 반복해도 변화가 없다는 점이다. 즉, 더 이상의 노력이 무의미한 상황이다.



[그림 13] 정확도와 교차엔트로피오차의 변화

이제 경사하강법을 마무리하고 최종적으로 수렴한 매개변수를 이용해 그림 14에 결정경계

를 표시하였다.



[그림 14] 원 데이터와 결정경계

결정경계의 좌측은 음성의 영역, 우측은 양성의 영역이다. 그런데 양성의 영역에 음성 데이터 두 개가 위치한다. 이 두 데이터 때문에 정확도가 80%인 것이다. 직선을 아무리 옮겨도 모든 데이터를 정확하게 구분할 수 있는 직선을 찾을 수 없다. 이것이 로지스틱 회귀의 속성공간 선형분할 특성과 그에 따른 근본적인 한계라 할 수 있다.

4. 인공 신경망 (Artificial Neural Network)

1. 인공 신경망 모델

인공 신경망(Artificial neural network, ANN)은 생물 신경계통의 상호작용을 모방하여 구성된 네트워크 형태의 인공지능 모델이다. 자연계의 모든 동물이 신경망을 가지고 있기 때문에, 이러한 천연 신경망과 구분하기 위해 인공 신경망이라 부른다.

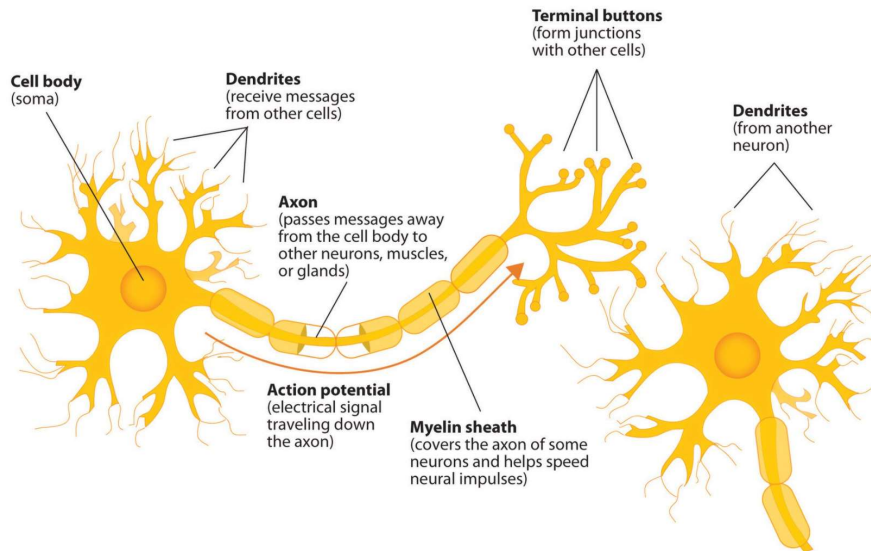
인공 신경망 모델은 현대 머신러닝 분야에서 가장 성공적인 모델로 평가받고 있다. 신경망을 통해 해결하고자 하는 문제는 궁극적으로 지도학습의 분류에 해당한다. 분류모델에서 미리 살펴본 로지스틱 회귀 모델 역시 인공 신경망의 한 특수한 형태라고 볼 수도 있다. 즉 로지스틱 회귀 모델은 인공 신경망 모델로 구현할 수 있는 분류모델 중 하나이다. 로지스틱 회귀 모델의 한계는 로지스틱 함수를 이용하는 경우 이진분류만 가능하고 속성공간의 선형 분리만이 가능하다는 점이다.

반면 인공 신경망 모델은 매우 유연한(flexible) 구조를 갖는다. 따라서 클래스의 수 또는 요구되는 속성공간의 분할 특성 등에 효율적으로 대응할 수 있다는 장점을 갖는 반면에 설계 및 연산 복잡도가 증가되는 단점을 함께 가지고 있다. 최근 인공지능이 활발하게 활용되고 있는 음성인식, 필기 인식, 영상 인식 등 많은 응용 분야가 다중 클래스 분류 문제에 해당하며 인공 신경망의 발전이 이러한 분야의 발전을 뒷받침하고 있다.

2. 뉴런 모델 (Neuron model)

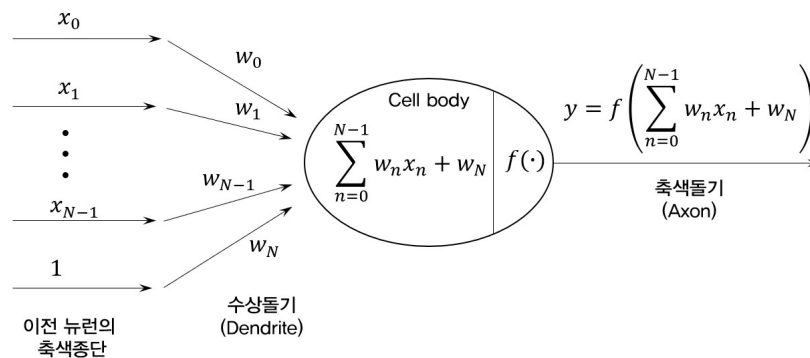
천연 신경망을 구성하는 기본 단위는 뉴런이라 부르는 세포이다. 그림 1은 천연 신경망에서의 뉴런의 구조를 나타낸다. 우리가 생물학을 공부하는 것이 목적이 아니므로 천연 뉴런에 대한 상세한 이해는 불필요하지만, 앞으로 공부할 신경망의 이해에 도움이 될 수 있는 수준에서 뉴런의 동작을 간단하게 설명한다.

그림 1은 두 개의 뉴런이 연결된 상황에서 감각 기관이 인지한 신호를 첫 번째 뉴런에서 두 번째 뉴런으로 전달하는 상황을 표현하고 있다. 첫 번째 뉴런의 좌측 끝에 cell body가 있다. 이 cell body에서 생성되거나 다른 뉴런으로부터 전달받은 신호는 axon, 우리말로 축색돌기라는 통로를 통해 axon terminal 또는 축색종단으로 전달된다. 이 축색종단은 두 번째 뉴런의 cell body에 흡수처럼 나와 있는 dendrite 또는 수상돌기와 인접해있다. 이 부분을 시냅스(synapse)라 한다. 첫 번째 뉴런의 축색종단에 도착한 신호의 강도에 따라 두 번째 뉴런으로 전달될 수도 있고 그렇지 않을 수도 있다. 만약 신호의 강도가 특정 임계치를 넘게되면 축색종단이 활성화되어 도파민이라는 화학 물질이 분비되고 이를 두 번째 뉴런의 수상돌기가 감지하여 다시 신호를 다음 뉴런으로 전달하는 과정을 반복한다. 여기에서 하나의 뉴런에 집중하면 신호를 받아들이는 수상돌기, 신호를 처리하여 전달하는 축색돌기, 신호를 다음 뉴런으로 전달하는 축색종단 그리고 신호를 다음 뉴런으로 전달할지 말지 결정하는 활성화 임계치가 주요 기능임을 알 수 있다.



[그림 1] 뉴런의 구조 (출처:위키피디아, <https://en.wikipedia.org>)

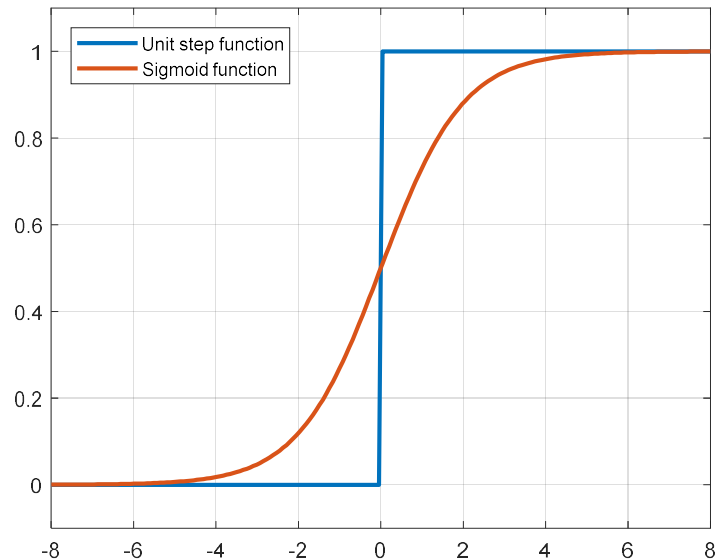
천연 뉴런의 동작을 단순히 요약하자면, 하나의 뉴런은 신호의 입력과 처리를 담당하는 연산 기능과 신호의 전달을 위한 활성화 기능으로 이루어진다고 볼 수 있다. 그림 2는 그림 1의 천연 뉴런의 구조를 모방하여 만든 인공 뉴런의 구조이다. 1943년 이 구조를 처음 제안한 Warren McCulloch와 Walter Pitts의 이름을 따 MP 뉴런이라 부르기도 한다.



[그림 2] 인공 뉴런의 구조

이 뉴런 구조는 입력과 출력, 연산, 활성화라는 천연 뉴런의 기능을 모두 포함하고 있다. 입력은 $x_0 \sim x_{N-1}$ 까지 총 N 개이며 이전의 뉴런으로부터 전달된 신호를 의미한다. N 개의 입력 밑에 있는 1은 연산을 위해 추가한 상수이다. 이 값들은 수상돌기를 거쳐 뉴런으로 입력되는데 각 돌기마다 매개변수에 의해 선형조합된다. 따라서 뉴런의 cell body에 입력된 신호는 $\sum_{n=0}^{N-1} w_n x_n + w_N$ 이 된다. 이때 $w_0 \sim w_{N-1}$ 은 입력의 선형조합을 위한 매개변수이며 w_N 은 선형조합의 절편 또는 바이어스라 부른다. 만약 N 개의 입력 밑에 있는 상수 1을 x_N 이라 정의한

다면 선형조합은 $\sum_{n=0}^N w_n x_n$ 과 같이 간단하게 표기할 수 있다. 선형조합에 의해 뉴런의 연산 기능이 결정된다. 다음으로 천연 뉴런의 활성화 기능을 모방하기 위해 함수 $f(\cdot)$ 를 선형조합의 결과에 적용한다. 이 함수 $f(\cdot)$ 를 활성화함수(activation function)라 부른다. 대표적인 활성화함수는 단위계단 함수와 로지스틱 함수를 들 수 있다. 그림 3은 단위계단 함수와 로지스틱 함수를 나타낸다. 그림에서 로지스틱 함수를 시그모이드(sigmoid) 함수로 표시하였는데, 시그모이드란 로지스틱 함수와 같이 s자 형태의 함수를 총칭하는 용어로 로지스틱 함수는 시그모이드 함수의 일종이다. 특히 인공 신경망 분야에서는 이 함수를 시그모이드 함수라고 칭하기 때문에 본 장에서도 시그모이드 함수라고 표현한다. 그림 3을 보면 왜 활성화함수라는 용어를 사용하는지 짐작할 수 있다. 즉, 입력이 어떤 임계치(여기에서는 0)를 넘으면 1, 그렇지 않으면 0을 갖는다. 따라서 뉴런의 축색종단의 활성화 여부를 나타내기에 적절한 것이다. 뒤에서 다시 다루겠지만, 그림 3의 두 함수 중 일반적으로 미분 가능한 시그모이드 함수가 널리 사용된다.



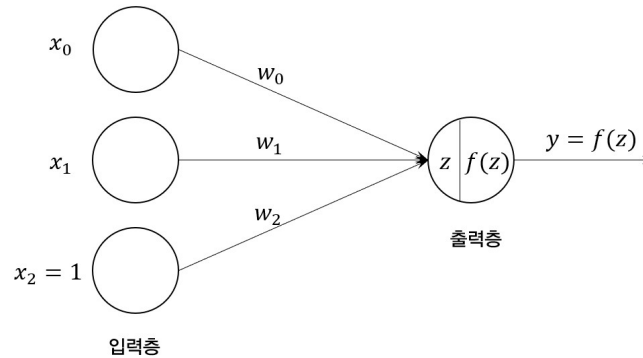
[그림 3] 단위계단 함수와 로지스틱 함수

3. 퍼셉트론 (Perceptron)

인공 신경망은 그림 2에 소개한 뉴런을 응용하여 설계한다. 특히 가장 단순한 형태의 인공 신경망을 퍼셉트론(perceptron)이라 하는데, 퍼셉트론의 일반적 정의는 단층 신경망, 즉 single layer neural network이다.

그림 4는 두 개의 입력 x_0 와 x_1 을 갖는 퍼셉트론을 나타낸다. 인공 신경망은 층(layer)으로 구성되며 각 층은 노드(node)로 이루어져 있다. 입력층은 세 개의 노드로 구성되는데, 위의 두 노드는 입력 x_0 와 x_1 을 나타내는 노드이고 마지막 노드는 더미 노드(dummy node)로 x_2 라는 더미 입력을 나타낸다. 더미 입력 x_2 는 입력들의 선형조합을 위한 절편 혹은 바이어스를

담당하는 가상의 입력이다. 더미 입력 x_2 의 값은 항상 1이다.



[그림 4] 두 개의 입력을 가지는 퍼셉트론

출력층 노드에는 z 라는 입력이 인가되고 y 라는 출력이 생성되며 f 라는 활성화함수가 z 와 y 를 연결해준다. 출력층 노드의 입력 z 는 각 입력의 선형조합으로 다음과 같다.

$$z = w_0x_0 + w_1x_1 + w_2x_2 = \sum_{n=0}^2 w_nx_n \quad (1)$$

w_0, w_1, w_2 는 선형조합을 위한 매개변수이다. 퍼셉트론의 출력 y 는 활성화함수에 z 를 입력하여 얻은 값이다. 즉, 다음과 같다.

$$y = f(z) = f\left(\sum_{n=0}^2 w_nx_n\right) \quad (2)$$

여기에서는 시그모이드 함수를 활성화함수로 사용한다. 다른 형태의 활성화함수에 대해서는 뒤에 서 다루도록 한다. 시그모이드 함수의 정의는 다음과 같다.

$$f(z) = \frac{1}{1 + e^{-z}} \quad (3)$$

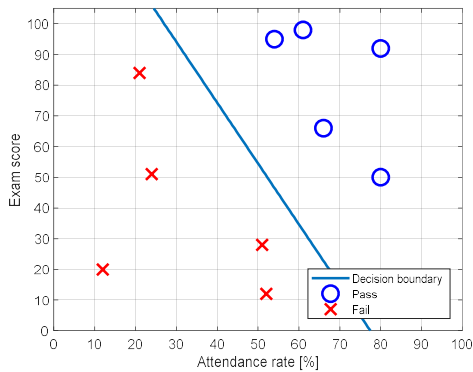
그림 4에서 입력층 노드와 출력층 노드의 차이는 입력층 노드는 입력을 받아들이는 기능만 갖지만 출력층 노드는 입력을 받아 활성화함수 연산을 통해 출력을 결정하는 연산기능을 가지고 있다는 것이다. 신경망의 중요 요소 중 하나는 신경망을 구성하는 층의 개수인데, 일반적으로 연산기능이 없는 계층은 층으로 세지 않는다. 따라서 그림 4의 퍼셉트론은 입력층과 출력층으로 구성되기는 했지만 연산기능이 없는 입력층을 제외하면 계층이 하나인 단층 신경망이라 볼 수 있다.

퍼셉트론 학습의 최종 목표는 매개변수 w_0, w_1, w_2 의 최적값을 구하는 것이다. 여기에서 한 가지 더 짚고 넘어갈 부분은, 퍼셉트론의 구조가 로지스틱 회귀와 같다는 점이다. 즉, 로지스틱 회귀는 입력 속성을 선형조합한 뒤 시그모이드 함수를 통과시켜 데이터가 양성일 확률과 음성일 확률을 계산하고 결정 임계치와 비교하여 최종적으로 양성 또는 음성을 판정하였다. 그림 2에 표현된 퍼셉트론 역시 이와 다르지 않다. 따라서 두 가지 결론을 얻을 수 있다.

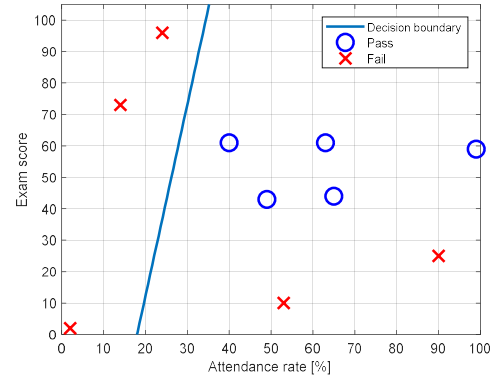
- ① 퍼셉트론은 단층 신경망으로 이진분류 모델이다.
- ② 퍼셉트론의 학습은 로지스틱 회귀 모델의 학습과 동일하다.

퍼셉트론이 로지스틱 회귀 모델과 같으므로 퍼셉트론은 로지스틱 회귀 모델이 갖는 한계를 그대로 가지고 있다. 즉, 퍼셉트론은 로지스틱 회귀 모델과 같이 속성공간을 선형분할한다. 그림 2와 같이 두 개의 입력 속성을 갖는 데이터의 경우, 퍼셉트론은 x_0 와 x_1 으로 구성되는 속

성공공간을 하나의 직선으로 양성 영역과 음성 영역을 구분할 수 있다. 따라서 더욱 복잡한 공간 구조를 갖는 데이터의 분류 문제에는 퍼셉트론이 적절히 대응할 수 없다. 그림 5는 로지스틱 회귀에서 다루었던 예제로, 선형분할 가능한 데이터와 선형분할 불가능한 데이터의 예이다. (b)와 같이 음성 데이터가 양성 데이터를 감싸고 있는 데이터는 하나의 직선으로 두 영역을 완전히 구분할 수 없다. 따라서 퍼셉트론도 이 문제는 제대로 해결할 수 없다.



(a) 선형 분할 가능한 데이터



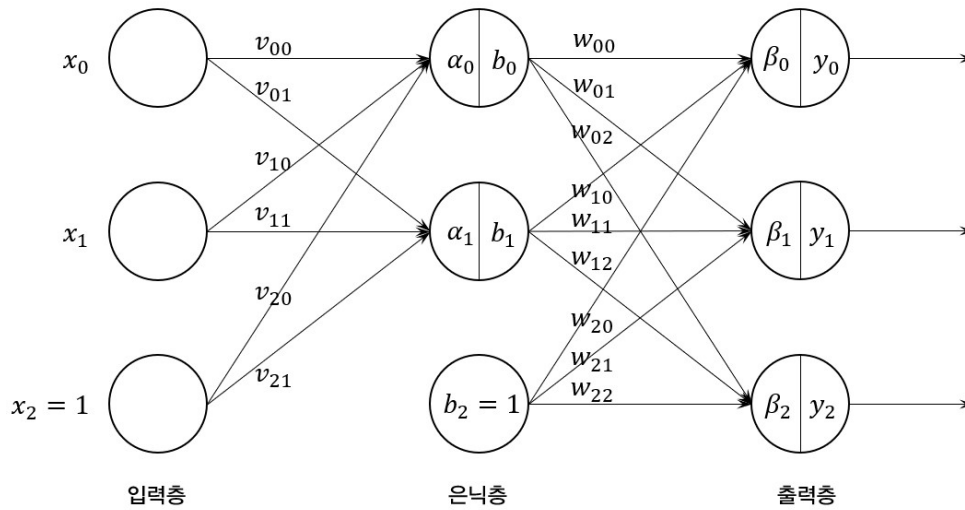
(b) 선형 분할 불가능한 데이터

[그림 5] 선형 분할 가능한 데이터와 불가능한 데이터

4. 다계층 신경망 (Multi-layer Neural Network)

인공 신경망 중 가장 기본적인 단위인 퍼셉트론은 선형분할 불가능한 데이터에 적용할 수 없다는 한계를 가지고 있다. 이 한계의 근본적 원인은 퍼셉트론이 갖는 연산 능력의 부족이라 할 수 있는데 다행히 신경망은 다른 모델과 달리 유연하게 연산 능력을 추가할 수 있다. 신경망에 연산 능력을 추가하는 방법은 입력층과 출력층 사이에 계층을 추가하는 것이다. 사용자 관점에서는 입력층과 출력층만 보이고 그 사이에 있는 계층은 보이지 않기 때문에, 입력층과 출력층 사이에 있는 계층을 은닉층(hidden layer)이라 한다. 은닉층의 수와 각 은닉층에 있는 노드의 수는 완전히 열린 문제(open problem)이다. 즉, 몇 개의 은닉층과 각 은닉층에 몇 개의 노드를 두어야 최적인지에 대한 일반 규칙은 알려지지 않고 있다. 따라서 은닉층은 신경망의 설계 및 학습 복잡도를 높이지만, 신경망의 유연성을 극대화하고 시스템의 성능을 향상하는 핵심 요소이다.

그림 6은 입력층과 출력층 사이에 하나의 은닉층을 추가한 다계층 인공 신경망의 예를 보여준다. 시스템의 입력은 x_0 와 x_1 이고 출력은 y_0 , y_1 , y_2 이다. 은닉층은 얼마든지 추가할 수 있으며 각 은닉층의 노드 수도 자유롭게 설정할 수 있다. 다만, 그림에서 알 수 있듯이 은닉층이 많아질수록 그리고 노드의 수가 많아질수록 노드와 노드 사이의 연결이 많아지고, 각 연결에 할당된 매개변수의 수도 함께 늘어난다. 매개변수의 값들은 학습을 통해 찾아야 한다.



[그림 6] 다계층 인공 신경망의 예

그림 6의 예에서 출력층 노드가 세 개인 것을 생각해보면, 이 모델은 3 클래스 분류모델임을 짐작할 수 있다. 즉, 출력 y_0 , y_1 , y_2 는 현재 주어진 입력 x_0 와 x_1 이 클래스 0, 클래스 1, 클래스 1에 해당할 확률에 해당한다. 클래스가 두 개뿐인 이진분류 모델에서는 그림 4와 같이 양성일 확률 하나만 계산하면 되지만, 세 개 이상이면 분류하고자 하는 클래스의 수와 출력층 노드의 수가 같아진다.

그림 6에서 보는 바와 같이 다계층 인공 신경망의 모든 화살표는 항상 입력층에서 출발하여 출력층 방향을 향한다. 이러한 속성을 강조하기 위해 다계층 순방향(feed forward) 인공 신경망이라고 부르기도 한다. 한 계층의 출력은 다음 계층의 입력이 되고, 매개변수들은 인접한 두 계층을 연결한다. 또한, 출력층을 제외한 모든 계층에 더미 노드가 있다는 점도 눈여겨 봐야 한다. 더미 노드의 값은 항상 1이며 다음 계층에서 이전 계층의 출력을 선형조합할 때 바이어스로 사용되는 노드이다.

그림 6에 표시한 시스템의 동작을 구체적으로 표현해보자. 은닉층의 각 노드에 인가되는 입력은 다음과 같다.

$$\alpha_0 = v_{00}x_0 + v_{10}x_1 + v_{20}x_2$$

$$\alpha_1 = v_{01}x_0 + v_{11}x_1 + v_{21}x_2$$

또한 각 노드의 출력은 이 값의 활성화함수 출력과 같다.

$$b_0 = f(\alpha_0)$$

$$b_1 = f(\alpha_1)$$

다음으로 은닉층의 출력은 출력층의 입력으로 제공되므로, 출력층의 각 노드에 인가되는 입력은 다음과 같이 표현할 수 있다.

$$\beta_0 = w_{00}b_0 + w_{10}b_1 + w_{20}b_2$$

$$\beta_1 = w_{01}b_0 + w_{11}b_1 + w_{21}b_2$$

$$\beta_2 = w_{02}b_0 + w_{12}b_1 + w_{22}b_2$$

그리고 최종 출력은 활성화함수를 통해 다음과 같이 구할 수 있다.

$$\hat{y}_0 = f(\beta_0), \hat{y}_1 = f(\beta_1), \hat{y}_2 = f(\beta_2)$$

여기에서 출력을 $\hat{y}_0, \hat{y}_1, \hat{y}_2$ 로 표시한 이유는 인공 신경망의 출력이 실제 값 y_0, y_1, y_2 에 대한 예측값이기 때문이다. 그림 6의 매개변수에 따라 예측값이 달라질 것이다. 따라서 우리의 목표는 다음을 최대한 만족하는 매개변수를 찾는 것이다.

$$\hat{y}_0 \approx y_0 \quad (4)$$

$$\hat{y}_1 \approx y_1 \quad (5)$$

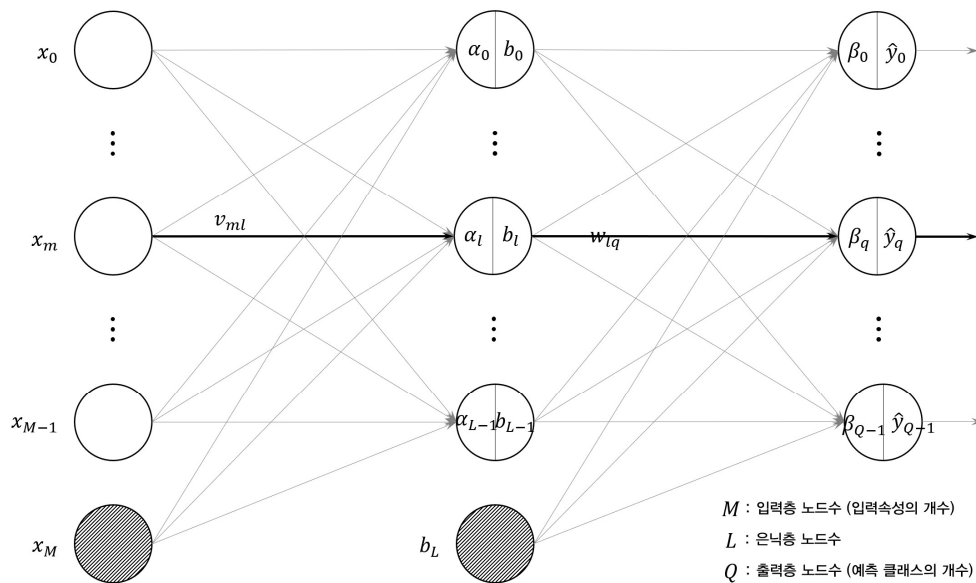
$$\hat{y}_2 \approx y_2 \quad (6)$$

5. 오차 역전파 알고리즘 (Error Backpropagation Algorithm)

5.1 다층 신경망의 입출력 관계

다층 인공 신경망 역시 퍼셉트론의 확장된 버전이므로, 결과적으로는 경사하강법을 이용해 최적화할 수 있다. 퍼셉트론과의 차이는 계층의 수와 이에 따른 매개변수의 개수이다. 따라서 퍼셉트론보다 훨씬 복잡한 구조를 갖는다. 다층 인공 신경망의 학습에 사용되는 알고리즘으로 오차 역전파 알고리즘이 있다. 이는 경사하강법에 기반을 둔 매개변수 업데이트 알고리즘으로 신경망 학습에 널리 활용되고 있다.

그림 7은 오차 역전파 알고리즘의 설명을 위한 인공 신경망 모델이다. 한 개의 은닉층을 가지고 있으며 입력층, 은닉층, 출력층의 노드의 수는 각각 M 개, L 개, Q 개다. 그리고 입력층과 은닉층에 있는 M 번 노드와 L 번 노드는 더미 노드이다.



[그림 7] 한 개의 은닉층을 갖는 인공 신경망 모델

은닉층의 임의의 노드에 인가되는 입력은 다음과 같다.

$$\alpha_l = v_{0l}x_0 + \dots + v_{ml}x_m + \dots + v_{Ll}x_L = \sum_{m=0}^M v_{ml}x_m, \quad l = 0, 1, \dots, L-1 \quad (7)$$

또한, 활성화함수가 $f(\cdot)$ 일 때, 각 노드의 출력은 다음과 같다.

$$b_l = f(\alpha_l) = f\left(\sum_{m=0}^M v_{ml}x_m\right) \quad (8)$$

다음으로 출력층의 임의의 노드에 들어오는 입력은 다음과 같다.

$$\beta_q = w_{0q}b_0 + \dots + w_{lq}b_l + \dots + w_{Lq}b_L = \sum_{l=0}^L w_{lq}b_l, \quad q = 0, 1, \dots, Q-1 \quad (9)$$

따라서 최종 출력, 즉 클래스 예측값은 다음과 같다.

$$\hat{y}_q = f(\beta_q) = f\left(\sum_{l=0}^L w_{lq}b_l\right), \quad q = 0, 1, \dots, Q-1 \quad (10)$$

5.3 인공 신경망 입출력 관계식의 행렬 표현

그림 7의 인공 신경망 구조를 보면 각 층의 매개변수와 출력을 행렬과 벡터로 표현할 수 있다. 또한 행렬 표현을 이용하면 식 (7), (8), (9), (10)을 간단하게 나타낼 수 있을 뿐만 아니라 프로그래밍이나 수치연산 프로그램을 이용할 때 유용하므로 다음과 같이 정리해둔다.

① 입력 벡터 $\mathbf{x}$, 크기 $(M+1) \times 1$

먼저 입력을 다음과 같이 벡터 $\mathbf{x}$ 로 표시한다.

$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{M-1} \\ x_M \end{bmatrix} = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{M-1} \\ 1 \end{bmatrix}$$

입력 벡터 크기는 $(M+1) \times 1$ 이고 M 은 입력 속성의 개수이다. 벡터 $\mathbf{x}$ 의 $M+1$ 번째 원소 x_M 은 입력이 아니라 바이어스를 담당하는 원소이며 값은 항상 1이다. 즉, $x_M = 1$ 이다.

② 은닉층 매개변수 행렬 V , 크기 $(M+1) \times L$

은닉층의 매개변수는 입력층과 은닉층의 노드를 연결하는 값들로 다음과 같이 행렬 V 로 나타낼 수 있다.

$$V = \begin{bmatrix} v_{00} & v_{01} & \dots & v_{0(L-1)} \\ v_{10} & v_{11} & \dots & v_{1(L-1)} \\ \vdots & \vdots & & \vdots \\ v_{(M-1)0} & v_{(M-1)1} & \dots & v_{(M-1)(L-1)} \\ v_{M0} & v_{M1} & \dots & v_{M(L-1)} \end{bmatrix}$$

행렬 V 의 크기는 $(M+1) \times L$ 이고, M 은 입력 속성의 개수, L 은 은닉층 노드의 개수이다. V 의 마지막 행 $[v_{M0} \ v_{M1} \ \dots \ v_{M(L-1)}]$ 은 바이어스 노드와 L 개의 은닉층 노드 사이의 매개변수를 의미한다.

③ 은닉층 노드의 입력 벡터 α , 크기 $L \times 1$

입력 속성들을 선형조합한 값이 은닉층의 각 노드에 입력되므로, 은닉층 노드의 입력 벡터는 다음과 같다.

$$\alpha = \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \vdots \\ \alpha_{L-1} \end{bmatrix} = \begin{bmatrix} v_{00} & v_{10} & \cdots & v_{M0} \\ v_{01} & v_{11} & \cdots & v_{M1} \\ \vdots & \vdots & & \vdots \\ v_{0(M-1)} & v_{1(M-1)} & \cdots & v_{(L-1)(M-1)} \\ v_{0M} & v_{1M} & \cdots & v_{(L-1)M} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{M-1} \\ x_M \end{bmatrix} = V^T \mathbf{x}$$

④ 은닉층 노드의 출력 벡터 $\mathbf{b}$, 크기 $(L+1) \times 1$

은닉층에 입력된 값들을 활성화함수 $f(\cdot)$ 에 인가하여 얻은 출력이 은닉층의 출력이 된다. 은닉층 노드의 개수는 L 개이고 다음 계층인 출력층을 위한 바이어스 노드가 하나 추가되므로 $L+1$ 개의 값이 은닉층의 출력이라 할 수 있다.

$$\mathbf{b} = \begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_{L-1} \\ b_L \end{bmatrix} = \begin{bmatrix} f(\alpha_0) \\ f(\alpha_1) \\ \vdots \\ f(\alpha_{L-1}) \\ 1 \end{bmatrix} = \begin{bmatrix} f(\alpha) \\ 1 \end{bmatrix}$$

⑤ 출력층 매개변수 행렬 W , 크기 $(L+1) \times Q$

출력층의 매개변수는 은닉층의 L 개의 노드와 한 개의 바이어스 노드, 출력층의 Q 개의 노드를 연결하므로 $(L+1) \times Q$ 의 크기를 갖는 행렬이 되며 다음과 같이 정의할 수 있다.

$$W = \begin{bmatrix} w_{00} & w_{01} & \cdots & w_{0(Q-1)} \\ w_{10} & w_{11} & \cdots & w_{1(Q-1)} \\ \vdots & \vdots & & \vdots \\ w_{(L-1)0} & w_{(L-1)1} & \cdots & w_{(L-1)(Q-1)} \\ w_{L0} & w_{L1} & \cdots & w_{L(Q-1)} \end{bmatrix}$$

⑥ 출력층 입력 벡터 β , 크기 $Q \times 1$

출력층 매개변수를 이용해 은닉층의 출력을 선형조합하면 출력층의 입력이 생성된다. 출력층의 노드는 Q 개이므로 $Q \times 1$ 의 벡터로 나타나며 다음과 같은 관계를 갖는다.

$$\beta = \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \vdots \\ \alpha_{L-1} \end{bmatrix} = \begin{bmatrix} w_{00} & w_{01} & \cdots & w_{0(Q-1)} \\ w_{10} & w_{11} & \cdots & w_{1(Q-1)} \\ \vdots & \vdots & & \vdots \\ w_{(L-1)0} & w_{(L-1)1} & \cdots & w_{(L-1)(Q-1)} \\ w_{L0} & w_{L1} & \cdots & w_{L(Q-1)} \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_{L-1} \\ b_L \end{bmatrix} = W^T \mathbf{b}$$

⑦ 출력층 출력 벡터 $\hat{\mathbf{y}}$, 크기 $Q \times 1$

출력층의 출력이자 시스템의 예측값은 출력층의 입력을 활성화함수에 인가하여 얻은 값이므로 다음과 같다.

$$\hat{\mathbf{y}} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{Q-1} \end{bmatrix} = \begin{bmatrix} f(\beta_0) \\ f(\beta_1) \\ \vdots \\ f(\beta_{Q-1}) \end{bmatrix} = f(\boldsymbol{\beta})$$

5.3 훈련 데이터를 적용한 입출력 관계

위에서 살펴본 식 (7), (8), (9), (10)은 일반적인 입력 변수 $x_0, x_1, \dots, x_{M-1}$ 에 대한 예측 결과 $\hat{y}_q$ 의 결정 과정을 나타낸 것이다. 이제 N 개의 훈련 데이터가 있다고 가정하고, n 번째 데이터에 대한 표현으로 확장해보자. 우선 n 번째 데이터의 표기는 표 1과 같이 정의한다. 즉, 아래 첨자의 두 번째 숫자를 데이터 번호 n 으로 할당한다.

[표 1] 훈련 데이터 집합

데이터 번호	입력	출력
	$x_0, \dots, x_m, \dots, x_{M-1}$	$y_0, \dots, y_q, \dots, y_{Q-1}$
0	$x_{00}, \dots, x_{m0}, \dots, x_{(M-1)0}$	$y_{00}, \dots, y_{q0}, \dots, y_{(Q-1)0}$
$\vdots$	$\vdots$	$\vdots$
n	$x_{0n}, \dots, x_{mn}, \dots, x_{(M-1)n}$	$y_{0n}, \dots, y_{qn}, \dots, y_{(Q-1)n}$
$\vdots$	$\vdots$	$\vdots$
$N-1$	$x_{0(N-1)}, \dots, x_{m(N-1)}, \dots, x_{(M-1)(N-1)}$	$y_{0(N-1)}, \dots, y_{q(N-1)}, \dots, y_{(Q-1)(N-1)}$

표 1의 표기법에 따라 n 번째 데이터에 대한 식 (7), (8), (9), (10)의 표현은 다음과 같다.

$$\alpha_{ln} = v_{0l}x_{01} + \dots + v_{ml}x_{mn} + \dots + v_{Ll}x_{Ln} = \sum_{m=0}^M v_{ml}x_{mn}, \quad l = 0, 1, \dots, L-1 \quad (11)$$

$$b_{ln} = f(\alpha_{ln}) = f\left(\sum_{m=0}^M v_{ml}x_{mn}\right) \quad (12)$$

$$\beta_{qn} = w_{0q}b_{0n} + \dots + w_{lq}b_{ln} + \dots + w_{Lq}b_{Ln} = \sum_{l=0}^L w_{lq}b_{ln}, \quad q = 0, 1, \dots, Q-1 \quad (13)$$

$$\hat{y}_{qn} = f(\beta_{qn}) = f\left(\sum_{l=0}^L w_{lq}b_{ln}\right) \quad (14)$$

참고로 다양한 기호들이 헷갈릴 수 있어 표 2에 정리하였으니 필요할 때 참고하기 바란다.

[표 2] 각 기호의 의미

기호	의미	범위
N	훈련 데이터의 개수	$0, 1, \dots, N-1$
M	입력 속성의 개수	$0, 1, \dots, M$ (더미 노드 포함)
L	은닉층 노드의 개수	$0, 1, \dots, L$ (더미 노드 포함)
Q	출력 클래스(노드)의 개수	$0, 1, \dots, Q-1$

5.3 오차 역전파 알고리즘의 기본 구조

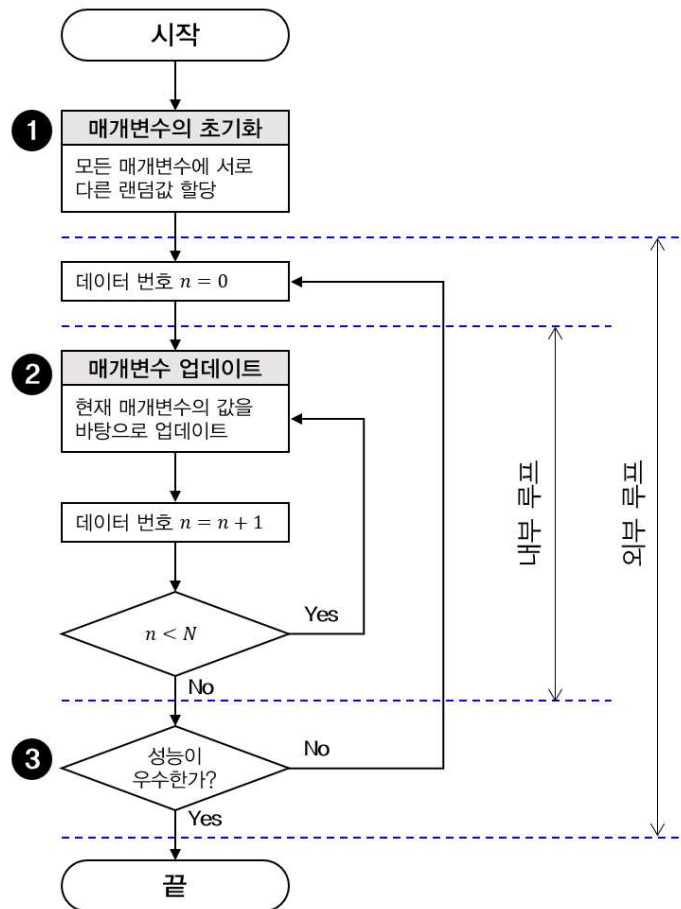
왜 알고리즘의 이름이 오차 역전파인지는 이 단계에서 신경 쓸 필요는 없다. 내용을 따라가면 자연스럽게 그 이유를 알게 된다. 중요한 것은 최적 매개변수를 찾기 위해 어떤 접근 방식을 사용할 것인가, 즉 알고리즘의 철학을 먼저 알고 상세한 내용으로 들어가는 것이다.

그림 7의 다층 신경망 모델은 비교적 간단한 모델이지만 은닉층에 $(M+1) \times L$ 개, 출력층에 $(L+1) \times Q$ 개의 매개변수가 있다. 이는 결코 적은 수가 아니다. 예를 들어, 입력 속성이 두 개, 은닉층 노드의 수가 세 개, 클래스가 세 개인 모델의 경우 $M=2$, $L=3$, $Q=3$ 이 되고 매개변수는 무려 21개가 된다. 어떤 시스템이건 이렇게 많은 변수의 솔루션을 단번에 구하는 것은 매우 어렵다. 여기에서 할 수 있는 일은 단계를 나누고 단계별로 차근차근 값을 찾아 나가는 것이다. 이런 방법의 대표 주자가 경사하강법이다. 즉, 목표를 설정하고 그 목표를 향해 한 걸음씩 다가가는 전략이다.

다층 신경망에서도 이 전략을 사용하기로 한다. 이때 ‘단계’와 ‘목표’를 정해야 한다.

단계는 각각의 훈련 데이터로 한다. 즉 N 개의 훈련 데이터가 있을 때, 첫 데이터를 가지고 매개변수를 정한 뒤 두 번째 데이터로 그 매개변수를 업데이트한다. 이렇게 N 개의 데이터로 N 번 업데이트된 매개변수를 얻을 수 있다. 이 매개변수가 최적일 수도 있고 그렇지 않을 수도 있다. 만약 이 매개변수로 얻은 성능이 만족스럽다면 학습 과정을 끝내도 좋다. 그러나 성능이 좋지 않은 경우 이 매개변수를 시작점으로 다시 첫 번째 데이터부터 업데이트 과정을 반복한다.

그림 8은 이 과정을 순서도로 나타낸 것이다. 알고리즘이 시작되면 은닉층과 출력층의 모든 매개변수에 랜덤한 값을 할당한다(순서도 1번). 이 값들은 랜덤한 값들이므로 좋은 예측 결과를 만들지 못할 것이다. 다음 차례는 N 개의 데이터에 대해 차례로 매개변수를 업데이트하는 것이다(순서도 2번). 이렇게 모든 데이터에 대해 차례로 업데이트를 마친 후 얻은 매개변수의 성능을 평가한다(순서도 3번). 만약 성능이 만족스럽지 않다면 다시 N 개의 데이터에 대해 차례로 매개변수를 업데이트하는 과정을 반복한다. 이렇게 반복함으로써 랜덤값에서 시작한 매개변수가 점차 최적값으로 수렴하게 된다.



[그림 8] 오차 역전파 알고리즘의 구조 (개요)

그림 8을 보면 알고리즘이 이중 루프로 구성된 것을 알 수 있다. 안쪽 루프는 N 개의 개별 데이터에 대해 차례로 매개변수를 업데이트하는 과정이고 바깥쪽 루프는 데이터 세트 전체에 대한 학습을 반복적으로 수행하는 과정이다.

N 개의 개별 데이터에 대해 학습을 진행하는 내부 루프를 epoch라 부른다. 영어 사전에서는 시대 또는 기간이라는 의미이지만, 머신러닝에서는 하나의 고유명사처럼 사용된다. 따라서 국문 표기도 에포크로 한다. 만약 어떤 알고리즘을 100 에포크 수행했다고 한다면, 이는 훈련 데이터 세트에 대한 학습을 100회 반복했다는 것을 의미한다.

지금 우리는 오차 역전파 알고리즘의 전략을 공부하고 있으며, 구체적으로 ‘단계’와 ‘목표’ 중 단계에 대한 설명을 마쳤다. 다음으로는 목표를 정해야 한다. 목표는 그림 8의 순서도에서 2번을 어떻게 설계할 것인가에 관련된 것이다. 당연히 시스템의 예측 성능이 좋아지는 방향으로 매개변수의 업데이트가 일어나야 할 것이다. 따라서 비용함수를 줄이는 것을 목표로 삼는 것이 매우 자연스럽다. 비용함수로는 평균제곱오차(MSE) 또는 교차엔트로피오차(CEE) 중 하나를 사용하면 된다.

5.4 오차 역전파 알고리즘

오차 역전파 알고리즘의 핵심은 그림 8의 순서도에서 2번 블록을 설계하는 것이다. 오차 역전파 알고리즘은 매개변수 업데이트 규칙을 설계하기 위해 경사하강법을 활용한다. 즉, 주어

진 매개변수에서 비용함수가 갖는 기울기의 반대편으로 매개변수를 이동하는 것이다.

이를 위해 비용함수를 정의한다. 오차 역전파 알고리즘은 그림 8과 같이 n 번째 데이터에 대해 매개변수 업데이트를 수행하므로 n 번째 데이터에 대한 비용함수를 정의해야 한다. 비용함수는 평균제곱오차를 사용하도록 한다.

$$\epsilon_{MSE}(n) = \sum_{q=0}^{Q-1} (\hat{y}_{qn} - y_{qn})^2 \quad (15)$$

식 (15)는 n 번째 데이터에 대한 Q 개의 출력의 평균제곱오차이다. 참고로 전체 N 개의 훈련 데이터에 대한 평균제곱오차는 다음과 같다.

$$\epsilon_{MSE} = \frac{1}{N} \sum_{n=0}^{N-1} \sum_{q=0}^{Q-1} (\hat{y}_{qn} - y_{qn})^2 = \frac{1}{N} \sum_{n=0}^{N-1} \epsilon_{MSE}(n) \quad (16)$$

그러나 오차 역전파 알고리즘의 내부 루프는 n 번째 데이터에 대한 평균제곱오차를 다루므로 식 (15)에 집중한다. 이제 경사하강법을 이용해 매개변수 v_{ml} 과 w_{lq} 의 업데이트 규칙을 정의하면 다음과 같다.

$$v_{ml} \leftarrow v_{ml} - \eta \frac{\partial}{\partial v_{ml}} \epsilon_{MSE}(n) \quad (17)$$

$$w_{lq} \leftarrow w_{lq} - \eta \frac{\partial}{\partial w_{lq}} \epsilon_{MSE}(n) \quad (18)$$

즉, 현재 주어진 매개변수 v_{ml} 과 w_{lq} 에서 식 (15)의 기울기 반대 방향으로 η 만큼 업데이트한다는 의미이다. 따라서 η 는 학습률(learning rate)로 매개변수의 업데이트 속도를 조절하는 변수이다. 경사하강법의 학습률과 같은 의미라 볼 수 있다. 이제 남은 일은 식 (17), (18)의 기울기를 구하는 것이다.

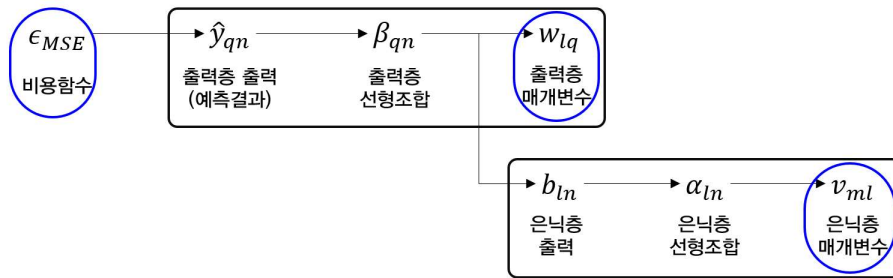
① 비용함수의 구조

무작정 미분을 하기에 앞서 비용함수의 구조를 이해하면 체계적인 접근이 가능하다. 지금 우리가 구하려고 하는 것은 식 (17)과 (18)에 있는 기울기 $\frac{\partial}{\partial v_{ml}} \epsilon_{MSE}(n)$ 과 $\frac{\partial}{\partial w_{lq}} \epsilon_{MSE}(n)$ 이다. 즉, 비용함수의 v_{ml} 에 대한 편미분과 w_{lq} 에 대한 편미분을 구하려고 한다. 이를 식 (11), (12), (13), (14)를 이용해 비용함수와 다양한 변수 사이의 상관관계를 정리해보면 표 3과 같다.

[표 3] 비용함수의 함수관계

단계	상관관계	관련식
1	비용함수 $\epsilon_{MSE}(n)$ 는 $\hat{y}_{qn}$ 의 함수이다.	(15)
2	$\hat{y}_{qn}$ 은 β_{qn} 의 함수이다.	(14)
3	β_{qn} 은 w_{lq} 와 b_{ln} 의 함수이다.	(13)
4	b_{ln} 은 α_{ln} 의 함수이다.	(12)
5	α_{ln} 은 v_{ml} 의 함수이다.	(11)

표 3의 관계를 그림 9와 같이 나타내면 더욱 이해하기 쉽다. 그림에 우리의 목표인 비용함수, 출력층 매개변수, 은닉층 매개변수의 위치를 파란색 원으로 표시하였다. 구하고자 하는 것은 비용함수의 각 매개변수에 대한 기울기이다. 그림으로부터 비용함수와 각 매개변수 사이의 관계를 명확하게 확인할 수 있다. 즉, 비용함수는 출력층 결과 $\hat{y}_{qn}$ 과 출력층 선형조합 β_{qn} 을 통해 출력층 매개변수 w_{lq} 과 관계를 맺고 있으며 은닉층 매개변수와는 은닉층 출력과 은닉층 선형조합을 통해 관계를 맺고 있다. 이 관계를 이용하면 의외로 간단하게 기울기를 구할 수 있다. 구하는 순서는 비용함수로부터 거리가 가까운 출력층 매개변수에 대한 기울기를 먼저 구한 뒤 은닉층 매개변수에 대한 기울기를 구한다.



[그림 9] 비용함수와 각 매개변수와의 관계

② $\frac{\partial}{\partial w_{lq}} \epsilon_{MSE}(n)$ 구하기

그림 9를 보면 비용함수는 $\hat{y}_{qn}$ 의 함수이고 $\hat{y}_{qn}$ 은 β_{qn} 의 함수이며 β_{qn} 은 w_{lq} 의 함수이다. 따라서 비용함수의 w_{lq} 에 대한 편미분은 다음과 같은 연쇄 미분을 이용해 구할 수 있다.

$$\frac{\partial}{\partial w_{lq}} \epsilon_{MSE}(n) = \frac{\partial \epsilon_{MSE}(n)}{\partial \hat{y}_{qn}} \cdot \frac{\partial \hat{y}_{qn}}{\partial \beta_{qn}} \cdot \frac{\partial \beta_{qn}}{\partial w_{lq}} \quad (19)$$

식 (19)의 우변의 첫 번째 항은 다음과 같다.

$$\frac{\partial \epsilon_{MSE}(n)}{\partial \hat{y}_{qn}} = 2(\hat{y}_{qn} - y_{qn}) \quad (20)$$

식 (19)의 두 번째 항은 시그모이드 함수 $f(z)$ 의 도함수가 $f'(z) = f(z)(1 - f(z))$ 라는 관계를 이용하면 다음과 같이 쉽게 구할 수 있다.

$$\frac{\partial \hat{y}_{qn}}{\partial \beta_{qn}} = f(\beta_{qn})(1 - f(\beta_{qn})) = \hat{y}_{qn}(1 - \hat{y}_{qn}) \quad (21)$$

식 (19)의 세 번째 항은 다음과 같다.

$$\frac{\partial \beta_{qn}}{\partial w_{lq}} = b_{ln} \quad (22)$$

따라서 식 (20), (21), (22)를 (19)에 적용하면 다음과 같다.

$$\frac{\partial}{\partial w_{lq}} \epsilon_{MSE}(n) = 2(\hat{y}_{qn} - y_{qn}) \hat{y}_{qn} (1 - \hat{y}_{qn}) b_{ln} \quad (23)$$

식 (23)에서 비용함수의 출력층 매개변수 w_{lq} 에 대한 기울기에 영향을 미치는 요소를 살펴보자. 첫 번째 요소는 예측값과 실제값의 오차인 $(\hat{y}_{qn} - y_{qn})$ 이다. 오차가 클수록 식 (23)의 기울기가 가팔라진다는 것을 의미한다. 즉 오차가 0으로 수렴할수록 기울기도 0으로 수렴하게 된다. 두 번째 요소는 $\hat{y}_{qn}(1 - \hat{y}_{qn})$ 으로, $\hat{y}_{qn}$ 는 예측 결과 클래스가 q 일 확률을 의미하고 $1 - \hat{y}_{qn}$ 는 클래스 q 가 아닐 확률이다. 이 두 값의 곱은 $\hat{y}_{qn} = 0.5$ 일 때 최대가 되고 0 또는 1일 때 최소가 되는 값이다. 다시 말해 예측 확률이 확실히 1이나 0이면, 즉 의심의 여지가 없는 상황에서는 기울기가 0으로 수렴하고 확률이 0.5 주변이면, 즉 맞는지 틀리는지 확신이 없는 상황에서는 기울기가 커진다는 것을 의미한다. 이 두 요소는 경사하강법에서 매개변수를 업데이트할 때 예측값의 오차는 작아지는 방향으로, 클래스 예측 확률은 0 또는 1로 확실한 값을 갖는 방향으로 움직이도록 한다. 이 두 요소를 큰 의미에서의 예측 오차로 간주하고 다음과 같이 쓸 수 있다.

$$e_{qn} = 2(\hat{y}_{qn} - y_{qn}) \hat{y}_{qn} (1 - \hat{y}_{qn}) \quad (24)$$

식 (24)의 예측 오차를 이용하면 식 (23)의 기울기는 다음과 같이 간단하게 표현된다.

$$\frac{\partial}{\partial w_{lq}} \epsilon_{MSE}(n) = e_{qn} b_{ln} \quad (25)$$

③ $\frac{\partial}{\partial v_{ml}} \epsilon_{MSE}(n)$ 구하기

앞의 과정과 같이 그림 8을 참고하여 $\frac{\partial}{\partial v_{ml}} \epsilon_{MSE}(n)$ 을 구하기 위한 연쇄 미분을 다음과 같이 표현할 수 있다.

$$\frac{\partial}{\partial v_{ml}} \epsilon_{MSE}(n) = \frac{\partial \epsilon_{MSE}(n)}{\partial b_{ln}} \cdot \frac{\partial b_{ln}}{\partial \alpha_{ln}} \cdot \frac{\partial \alpha_{ln}}{\partial v_{ml}} \quad (26)$$

식 (26)의 첫 번째 항은 식 (15)에서 다음과 같이 구할 수 있다.

$$\begin{aligned} \frac{\partial \epsilon_{MSE}(n)}{\partial b_{ln}} &= 2 \sum_{q=0}^{Q-1} (\hat{y}_{qn} - y_{qn}) \frac{\partial \hat{y}_{qn}}{\partial b_{ln}} \\ &= 2 \sum_{q=0}^{Q-1} (\hat{y}_{qn} - y_{qn}) \hat{y}_{qn} (1 - \hat{y}_{qn}) w_{lq} \end{aligned} \quad (27)$$

식 (26)의 두 번째 항은 식 (12)와 시그모이드 함수의 도함수를 이용하면 다음을 얻을 수 있다.

$$\frac{\partial b_{ln}}{\partial \alpha_{ln}} = f(\alpha_{ln})(1 - f(\alpha_{ln})) = b_{ln}(1 - b_{ln}) \quad (28)$$

끝으로, 식 (26)의 세 번째 항은 식 (11)에서 다음과 같이 구할 수 있다.

$$\frac{\partial \alpha_{ln}}{\partial v_{ml}} = x_{mn} \quad (29)$$

따라서 식 (27), (28), (29)를 식 (26)에 대입하면 다음과 같다.

$$\frac{\partial}{\partial v_{ml}} \epsilon_{MSE}(n) = 2 \sum_{q=0}^{Q-1} (\hat{y}_{qn} - y_{qn}) \hat{y}_{qn} (1 - \hat{y}_{qn}) w_{lq} b_{ln} (1 - b_{ln}) x_{mn} \quad (30)$$

여기에 식 (24)에 정의한 예측 오차를 이용하면 식 (30)의 기울기 역시 다음과 같이 단순화할 수 있다.

$$\frac{\partial}{\partial v_{ml}} \epsilon_{MSE}(n) = b_{ln} (1 - b_{ln}) x_{mn} \sum_{q=0}^{Q-1} e_{qn} w_{lq} \quad (31)$$

④ 수식 정리

지금까지 조금 복잡한 과정을 통해 수식을 유도했다. 이제 꼭 필요한 식을 다시 모아 살펴보자. 오차 역전파 알고리즘에서 주어진 n 번째 훈련 데이터에 대한 매개변수 업데이트 규칙은 다음과 같다.

$$w_{lq} \leftarrow w_{lq} - \eta \frac{\partial}{\partial w_{lq}} \epsilon_{MSE}(n) \quad (32)$$

$$v_{ml} \leftarrow v_{ml} - \eta \frac{\partial}{\partial v_{ml}} \epsilon_{MSE}(n) \quad (33)$$

이때, 기울기는 다음과 같다.

$$\frac{\partial}{\partial w_{lq}} \epsilon_{MSE}(n) = e_{qn} b_{ln} \quad (34)$$

$$\frac{\partial}{\partial v_{ml}} \epsilon_{MSE}(n) = b_{ln} (1 - b_{ln}) x_{mn} \sum_{q=0}^{Q-1} e_{qn} w_{lq} \quad (35)$$

위 식에서 e_{qn} 은 예측 오차로 다음과 같다.

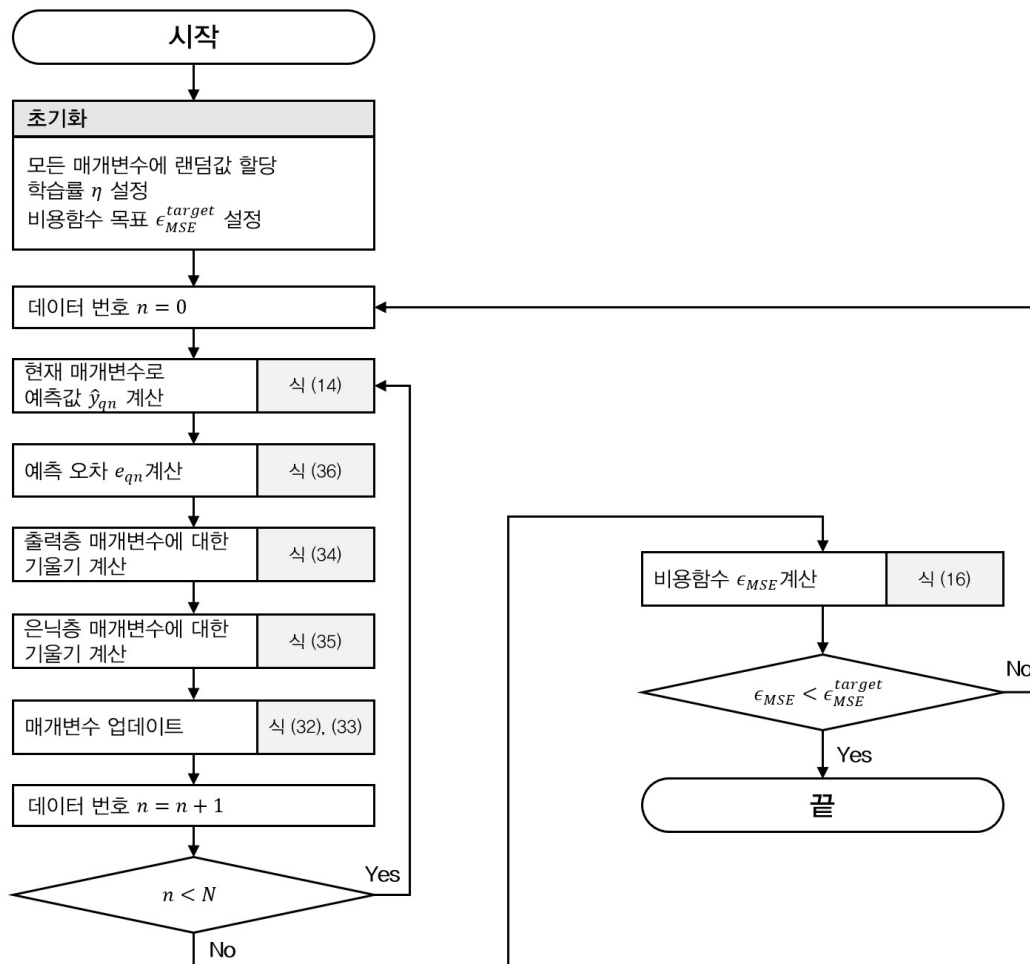
$$e_{qn} = 2(\hat{y}_{qn} - y_{qn}) \hat{y}_{qn} (1 - \hat{y}_{qn}) \quad (36)$$

식 (32), (33)는 그림 8의 알고리즘 순서도에서 2번 매개변수 업데이트 규칙에 해당한다. 그림 8과 함께 이 식을 살펴보면 이 알고리즘의 의미를 알 수 있다. 즉, n 번째 데이터와 현재까지 업데이트된 매개변수로 각 클래스의 예측값을 구하고 실제 데이터의 출력과의 차이, 즉 오차를 계산하여 각 출력 노드 q 에서의 e_{qn} 값을 계산할 수 있다. 이 계산값과 은닉층의 각 노드의 출력을 곱하면 식 (34)와 같이 출력층 매개변수 w_{lq} 의 업데이트를 위한 기울기를 구할 수 있고 식 (32)의 업데이트 규칙을 통해 출력층 매개변수를 업데이트한다. 다음으로 식 (35)를 이용해 은닉층 매개변수의 업데이트를 위한 기울기를 구할 수 있다. 이 과정에서도 예측 오차 e_{qn} 이 사용된다. 이 결과를 식 (33)에 대입하여 은닉층 매개변수 v_{ml} 을 업데이트하게 된다.

이 과정을 보면, 주어진 데이터에 대한 매개변수 업데이트는 출력 노드에서의 예측 오차를 계산하는 것으로부터 시작되고 이 예측 오차를 이용해 출력층 매개변수를 업데이트한 뒤 은닉층 매개변수를 업데이트한다. 즉, 예측 오차가 출력층 $\rightarrow$ 은닉층 $\rightarrow$ 입력층 방향으로 전파되며 매개변수가 업데이트된다. 이 특징 때문에 이 알고리즘을 오차 역전파 알고리즘이라 한다.

⑤ 종료 조건

그림 8의 개략적인 알고리즘의 구조에서 N 개의 데이터에 대한 매개변수 업데이트 과정, 즉 내부 루프가 종료되면 3번 “성능이 우수한가?”라는 질문을 통해 매개변수 업데이트를 추가로 수행할지를 판단한다. 이 질문을 종료 조건이라 할 수 있다. 종료 조건으로 가장 적합한 지표는 전체 데이터에 대한 평균제곱오차이다. 내부 루프에서 개별 데이터에 대한 평균제곱오차를 줄이는 방향으로 매개변수를 업데이트했으므로 종료 조건은 모든 데이터에 대한 평균제곱오차가 될 것이다. 모든 데이터에 대한 평균제곱오차는 식 (16)으로 개별 제곱오차의 평균값이다. 따라서 모든 데이터에 대한 평균제곱오차가 목표치보다 적을 때 알고리즘을 종료할 수 있다. 일반적으로는 내부 루프가 끝날 때마다, 즉 1 에포크마다 종료 조건을 확인하는 대신 수행할 에포크 수를 미리 정해두고 정해진 에포크만큼 내부 루프를 수행한 뒤 종료 조건을 확인하는 것이 일반적이다. 그림 10은 지금까지 논의된 내용을 바탕으로 종합적으로 정리한 오차 역전파 알고리즘의 순서도이다.



[그림 10] 오차 역전파 알고리즘 (종합)

6. 다층 인공 신경망을 이용한 분류 시스템 설계

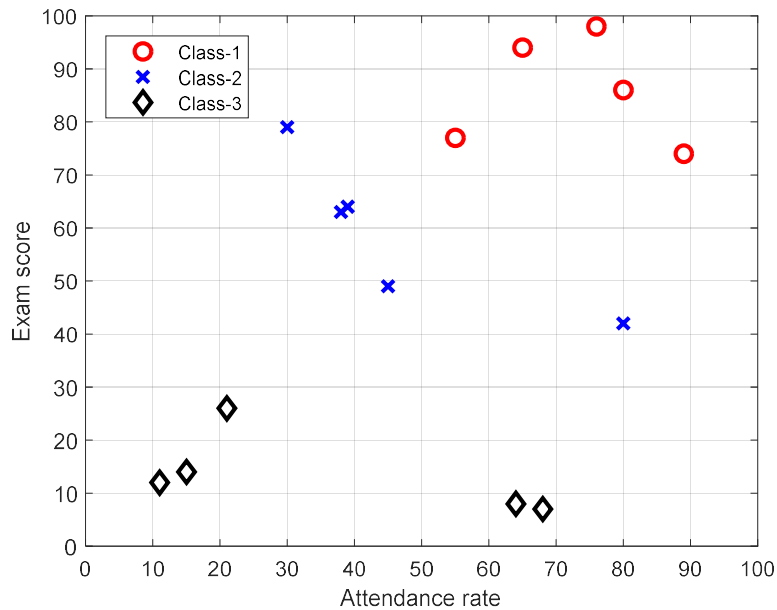
6.1 데이터

이제 위에서 공부한 내용을 바탕으로 다층 인공 신경망을 설계해보자. 실제 데이터를 이용해 시스템을 설계하는 과정에서 이론 설명에서 충분히 다루지 못한 부분을 함께 다루게 된다. 설계에 사용할 훈련 데이터는 학생들의 출석률과 시험성적으로 학점(A, B, C등급)을 부여한 데이터이다. 표 4는 훈련 데이터를 나타낸다.

[표 4] 분류를 위한 훈련 데이터

데이터 번호	입력		출력
	출석률	시험성적	학점
0	76	98	A
1	65	94	A
2	80	86	A
3	89	74	A
4	55	77	A
5	30	79	B
6	39	64	B
7	38	63	B
8	45	49	B
9	80	42	B
10	68	7	C
11	64	8	C
12	21	26	C
13	15	14	C
14	11	12	C

표 4와 같이 총 15명의 학생을 평가하여 A, B 또는 C의 학점을 부여하였다. 출석률과 시험 성적은 훈련 데이터의 입력이라 할 수 있고, 출력은 학점이다. 학점은 세 값 중 하나를 가질 수 있으므로 3 클래스 분류 시스템이라 할 수 있다. 그림 11은 표 4의 데이터를 그래프에 표시한 것이다. 전반적으로 출석률과 시험점수가 동시에 우수할수록, 즉 그래프의 오른쪽 위에 위치할수록 높은 학점을 받을 확률이 높다.



[그림 11] 3 클래스 분류를 위한 훈련 데이터

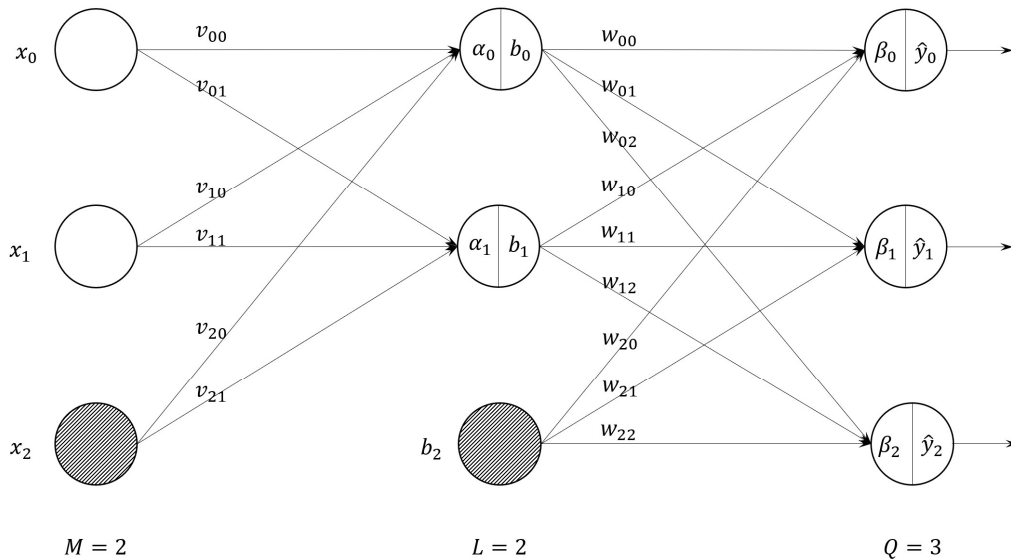
표 4의 데이터를 이용해 다층 인공 신경망을 학습하기 위해서는 출력을 인공 신경망에 맞게 수치화해야 한다. 출력의 클래스가 세 개이므로 출력층의 노드는 세 개로 구성한다. 각 노드의 출력은 각 클래스에 해당할 확률 또는 가능성으로 해석하면 된다. 이를 적용하면 훈련 데이터를 표 5와 같이 가공할 수 있다.

[표 5] One-hot 인코딩을 적용한 훈련 데이터

데이터 번호	입력		출력		
	출석률	시험성적	y_0	y_1	y_2
0	76	98	1	0	0
1	65	94	1	0	0
2	80	86	1	0	0
3	89	74	1	0	0
4	55	77	1	0	0
5	30	79	0	1	0
6	39	64	0	1	0
7	38	63	0	1	0
8	45	49	0	1	0
9	80	42	0	1	0
10	68	7	0	0	1
11	64	8	0	0	1
12	21	26	0	0	1
13	15	14	0	0	1
14	11	12	0	0	1

표 5를 표 4와 비교해보면 출력이 달라진 것을 알 수 있는데, 출력이 가질 수 있는 클래스를 y_0, y_1, y_2 와 같이 세 개의 출력값으로 변환하고 학점이 A인 학생은 $(y_0, y_1, y_2) = (1, 0, 0)$, B인 학생은 $(y_0, y_1, y_2) = (0, 1, 0)$, C인 학생은 $(y_0, y_1, y_2) = (0, 0, 1)$ 을 갖도록 가공하였다. 즉 세 클래스 중 해당하는 클래스의 출력만 1을 갖도록 구성하는 것이다. 이처럼 데이터를 표현하는 방식을 **원-핫 (one-hot) 인코딩** 방식이라고 한다.

다음으로 결정해야 할 것은 은닉층의 개수와 은닉층에 있는 노드의 개수이다. 본 예에서는 은닉층의 개수를 1개, 은닉층의 노드를 2개로 가정한다¹⁾. 따라서 입력 2개, 은닉층 노드 2개, 출력층 노드 3개, 즉 $M=2, L=2, Q=3$ 인 다층 인공 신경망을 구성하게 되며, 입력층과 은닉층에 추가로 하나씩의 더미 노드가 있다는 점을 기억해야 한다. 그림 12는 다층 인공 신경망의 구조이며 은닉층이 하나이므로 2계층 인공 신경망이다. 입력층과 은닉층의 회색 노드는 바이어스를 위한 더미 노드이다. 이제 목표는 그림 12의 출력 예측치 $\hat{y}_0, \hat{y}_1, \hat{y}_2$ 가 표 5의 실제 출력 y_0, y_1, y_2 와 최대한 같아지는 매개변수 V 와 W 의 값들을 찾는 것이다.



[그림 12] 설계한 2계층 인공 신경망

최적 매개변수를 찾기 위해서는 비용함수를 정의하고 오차 역전파 알고리즘을 적용한다. 비용함수는 평균제곱오차를 사용하기로 한다. 교차엔트로피오차를 사용해도 상관없으나, 본 장에서 평균제곱오차를 기준으로 오차 역전파 알고리즘을 유도하였으므로 평균제곱오차를 사용하는 것이다. n 번째 훈련 데이터에 대한 평균제곱오차 식 (15)로부터 다음과 같이 정의할 수 있다.

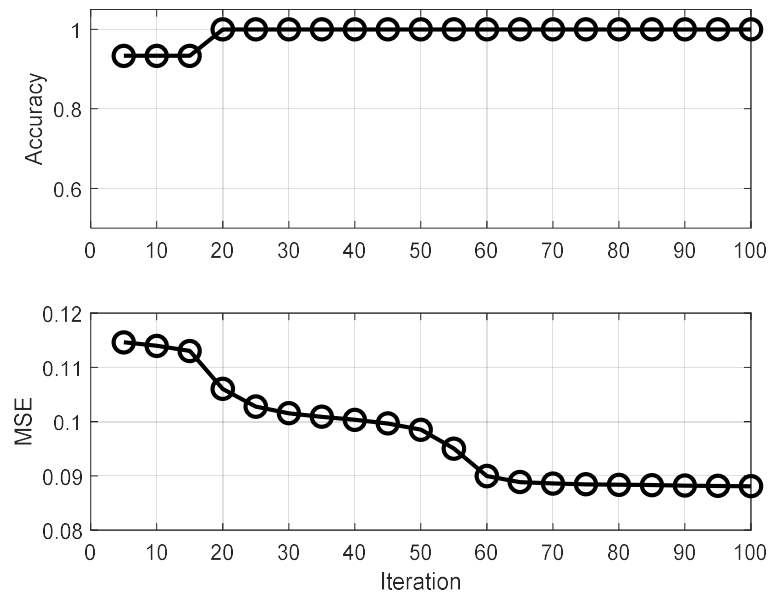
$$\epsilon_{MSE}(n) = \sum_{q=0}^2 (\hat{y}_{qn} - y_{qn})^2$$

다음으로 각 매개변수를 임의의 수로 설정한 뒤 그림 8의 알고리즘을 절차대로 수행한다.

그림 13은 학습률 η 를 0.001로 설정하고 오차 역전파 알고리즘을 100회 수행하는 동안 예

1) 은닉층의 수와 은닉층의 노드의 수는 설계자가 자유롭게 정할 수 있다.

측 정확도와 평균제곱오차의 변화를 나타낸 것이다. 알고리즘을 약 20회 반복한 이후, 즉 20 에포크 수행한 뒤부터 정확도는 100%를 달성하고 있다. 이는 훈련 데이터에 대한 클래스 예측을 정확하게 한다는 것을 의미한다. 이와 함께 평균제곱오차도 함께 낮아지는 것을 확인할 수 있다.

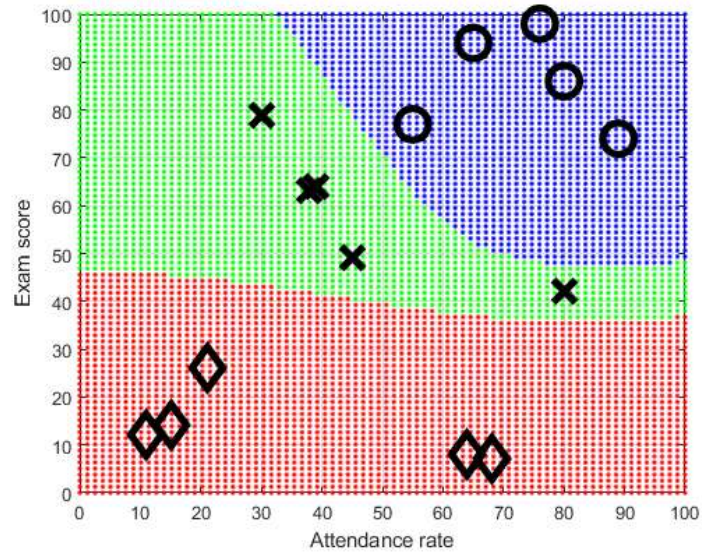


[그림 13] 오차 역전과 알고리즘의 반복 횟수와 예측 정확도 및 평균제곱오차의 변화

그림 13과 같이 100회의 알고리즘 반복 후 얻은 매개변수 값은 다음과 같다.

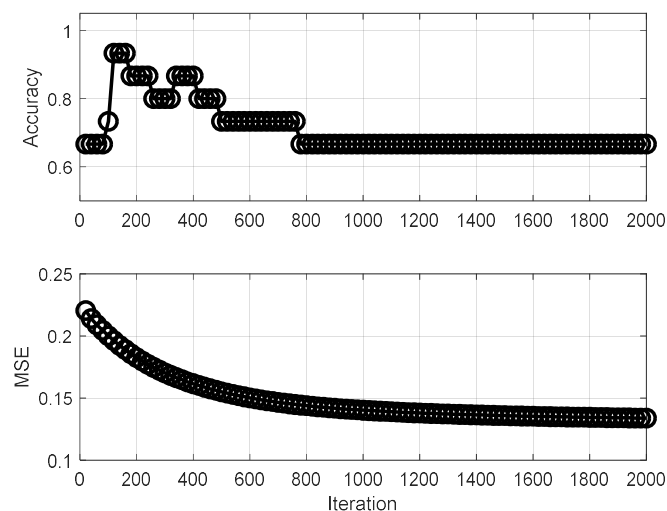
$$V = \begin{bmatrix} -0.0837 & -0.0259 \\ -0.0496 & 0.1757 \\ 6.3349 & -5.1625 \end{bmatrix}, \quad W = \begin{bmatrix} -5.0855 & -0.6823 & 3.9907 \\ 3.5843 & 0.9311 & -5.2163 \\ -2.7844 & -1.0409 & 0.2020 \end{bmatrix}$$

이 매개변수를 적용하면 그림 12의 2계층 인공 신경망은 출석률과 시험성적으로 구성된 입력 속성공간을 세 개의 영역으로 나눈다. 그림 14는 인공 신경망에 의해 분할된 속성공간을 나타낸다. 훈련 데이터의 위치도 함께 표시하였다. 그림에서 파란색으로 표시된 영역은 A 학점에 해당하는 영역이고 초록색 영역은 B 학점, 그리고 빨간색 영역은 C 학점으로 분류된 영역이다. 퍼셉트론이 속성 영역을 선형분할하는 한계를 가지고 있었던 것과 달리 은닉층을 보유한 다층 인공 신경망의 경우 속성공간을 비선형의 형태로 분할 할 수 있음을 확인할 수 있다. 이는 다층 인공 신경망이 갖는 연산 능력과 구조적 유연성에 의한 결과이다.



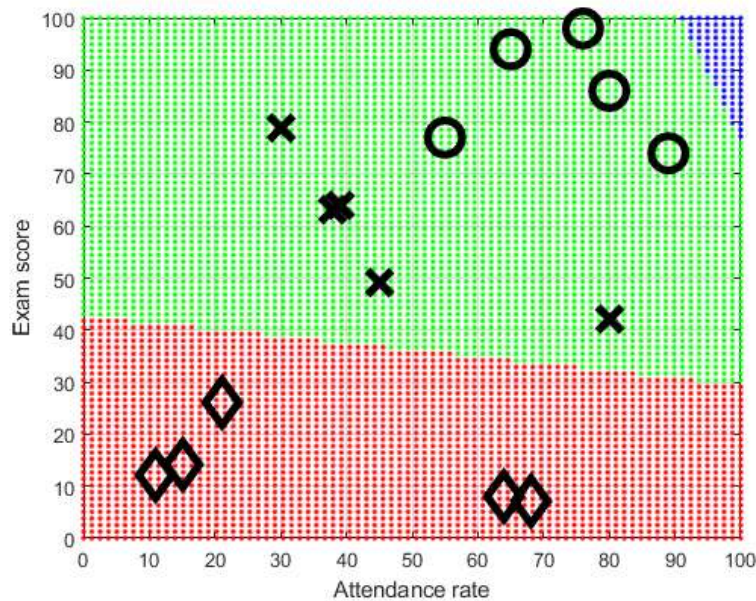
[그림 14] 인공 신경망이 구분한 클래스의 영역

여기에서 한 가지 지적할만한 점은, 다층 인공 신경망이 많은 매개변수를 가지고 있으므로 최적값을 찾는 것이 언제나 쉽게 가능한 일은 아니라는 점이다. 이는 비용함수가 속성공간에서 매우 많은 로컬 미니멈(극소값)을 가질 수 있기 때문이다. 만약 비용함수의 형태가 항아리처럼 오목한 부분이 하나라면, 어디서 출발하더라도 항상 하나의 글로벌 미니멈(극소값)으로 수렴할 것이다. 그러나 비용함수가 냉장고에 사용하는 얼음 틀처럼 매우 많은 로컬 미니멈이 존재한다면, 출발점에 따라 글로벌 미니멈(최소값)이 아닌 로컬 미니멈에 빠질 위험이 커진다. 그림 15는 오차 역전파 알고리즘의 출발점을 잘못 설정했을 때 반복 횟수와 성능의 관계를 나타낸다. 이 예에서 알고리즘을 2000회 반복하였으나 평균제곱오차는 줄어들지 않고 정확도 역시 약 70%에서 변화가 없는 것을 확인할 수 있다.



[그림 15] 잘못된 출발점에서 시작한 오류 역전파 알고리즘의 결과

그림 16은 이 상황에서 다층 신경망의 속성공간 분할 성능을 보여준다. 세 영역으로 나누기는 하지만, 성공적인 분할이라 볼 수 없다. 이러한 문제는 인공 신경망 분야에서 잘 알려진 문제로, 이런 상황에서도 안정적으로 글로벌 미니멈을 찾으려는 방법들이 연구되고 있다. 현재 가장 성공적인 접근으로 알려진 방식은 통계적 경사하강법(stochastic gradient descent method, SGD)을 사용하는 것으로, 매개변수 업데이트를 위한 기울기(그래디언트)를 구할 때 무작위성(randomness)을 가미하는 방법으로 로컬 미니멈에 빠질 가능성을 줄일 수 있다. 현재 다양한 통계적 경사하강법 기반의 최적화 기법이 개발되어 인공 신경망 학습에 활용되고 있다.



[그림 16] 잘못 설계된 다층 신경망의 분할 성능

7. 활성화함수 (Activation function)

하나의 뉴런은 입력과 활성화함수, 출력으로 구성된다. 활성화함수는 뉴런의 입력을 출력으로 변환하는 역할을 하며 지금까지 시그모이드 함수를 사용했다. 하지만, 시그모이드 함수 외에 다양한 함수들이 활성화함수로 사용될 수 있다. 본 절에서는 많이 사용되는 활성화함수를 소개한다.

7.1 시그모이드(로지스틱) 함수

대표적인 활성화함수로 다음과 같이 정의된다.

$$f(z) = \frac{1}{1 + e^{-z}} \quad (37)$$

임의의 실수 z 를 0과 1사이의 값으로 변화하며 1차 도함수가 매우 간단한 형태이므로 경사하강법의 적용이 수월한 특징을 가지고 있다. 시그모이드 함수의 1차 도함수는 다음과 같다.

$$\frac{d}{dz}f(z) = f(z)(1-f(z)) \quad (38)$$

시그모이드 함수는 은닉층과 출력층의 어떤 뉴런에서도 사용될 수 있다.

7.2 소프트맥스 함수 (Softmax function)

시그모이드 함수가 하나의 변수에 대해 정의된 함수라면, 소프트맥스 함수는 여러 변수에 대해 정의된 함수이다. n 개의 변수 $z_0, z_1, \dots, z_{n-1}$ 에 대해 총 n 개의 소프트맥스 함수가 정의되며 각 변수에 대한 소프트맥스 함수의 정의는 다음과 같다.

$$f(z_k) = \frac{e^{z_k}}{\sum_{i=0}^{n-1} e^{z_i}}, \quad k = 0, 1, \dots, n-1 \quad (39)$$

예를 들어 세 변수 z_0, z_1, z_2 가 있다면 각 변수에 대한 소프트맥스 함수는 다음과 같이 쓸 수 있다.

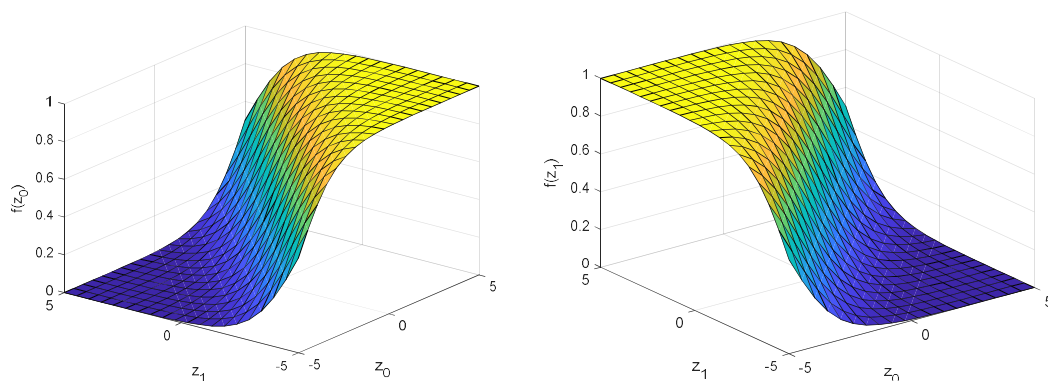
$$f(z_0) = \frac{e^{z_0}}{e^{z_0} + e^{z_1} + e^{z_2}},$$

$$f(z_1) = \frac{e^{z_1}}{e^{z_0} + e^{z_1} + e^{z_2}},$$

$$f(z_2) = \frac{e^{z_2}}{e^{z_0} + e^{z_1} + e^{z_2}}.$$

위 식을 보면, 분모는 모두 같고 분자만 다른 세 개의 식이 된다. 소프트맥스 함수는 두 가지의 중요한 특징을 갖는다. 첫 번째 특징은 각 소프트맥스 함수의 값이 0과 1 사이의 값을 갖는다는 것이다. 이는 시그모이드 함수와 같은 특징이다. 두 번째 특징은 모든 소프트맥스 함수의 합은 항상 1이라는 것이다. 이 두 성질을 함께 고려하면, 어떤 소프트맥스 함수의 값이 커지면 나머지 소프트맥스 함수의 값은 자연스럽게 작아지게 된다.

소프트맥스 함수는 시그모이드 함수를 다변수 함수로 확장한 것으로 해석할 수 있다. 예를 들어 두 개의 변수 z_0, z_1 에 대한 소프트맥스 함수를 그래프로 그려보면 그림 17과 같다. 앞서 살펴본 바와 같이 0과 1 사이의 값을 가지고 있으며 두 함수의 합은 항상 1이 된다. 또한, 한 변수에 대한 함수의 모양은 시그모이드 함수의 형태를 가지고 있다.



[그림 17] 2변수 소프트맥스 함수

소프트맥스 함수는 다변수 시그모이드 함수이고 소프트맥스 함수의 합이 1이라는 성질로 인해 인공 신경망의 출력층에 자주 사용된다. 다중 클래스 분류를 위한 인공 신경망을 생각해 보면, 출력값은 각 클래스로 분류될 확률 또는 가능성을 나타낸다. 예를 들어 그림 12의 출력값 $\hat{y}_0, \hat{y}_1, \hat{y}_2$ 는 주어진 입력이 각 클래스에 해당할 가능성의 정도를 나타내는데, 1에 가까울수록 해당 클래스에 속할 가능성이 큰 것으로 판단한다. 그런데 여기에서 $\hat{y}_0, \hat{y}_1, \hat{y}_2$ 를 확률로 해석하지 못하는 이유는 출력층의 활성화함수로 시그모이드를 사용하여 $\hat{y}_0 + \hat{y}_1 + \hat{y}_2 \neq 1$ 이기 때문이다. 만약 출력층의 활성화함수로 소프트맥스 함수를 사용해 예측값 $\hat{y}_0, \hat{y}_1, \hat{y}_2$ 를 생성했다면 $\hat{y}_0 + \hat{y}_1 + \hat{y}_2 = 1$ 을 항상 만족하게 되고, 이 예측값을 각 클래스에 대한 확률로 해석할 수 있다. 이러한 장점 때문에 출력층 활성화함수로 소프트맥스 함수가 자주 사용된다.

7.3 ReLU 함수 (ReLU function)

인공 신경망에서 많이 사용되는 전통적인 활성화함수는 시그모이드 함수와 소프트맥스 함수가 있다. 이 함수들은 $-\infty \sim \infty$ 의 임의의 수를 0과 1 사이의 값으로 변환하는 함수로 입력이 아무리 커져도 출력은 1 이상의 값을 가질 수 없다. 이러한 함수를 포화함수(saturation function)라 한다. 포화함수는 로지스틱 회귀 모델 등에서 선형조합의 결과를 확률이나 가능성으로 변환하는 유용한 성질을 가지고 있다.

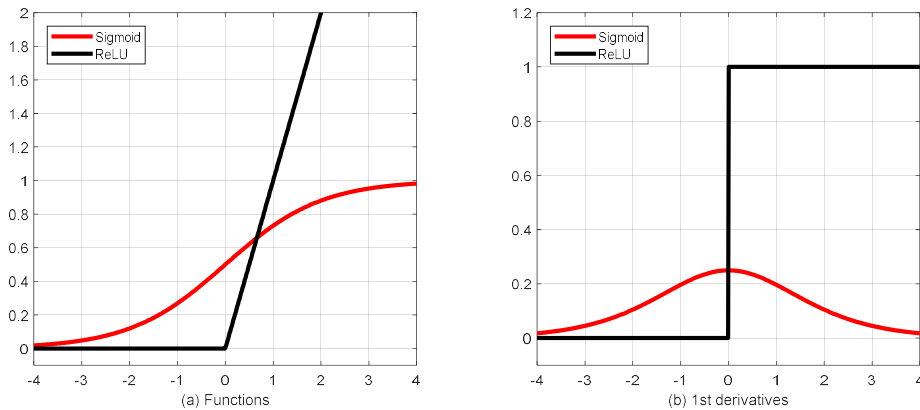
그러나 다계층 인공 신경망 분야에서 포화함수가 시스템의 복잡도를 높여 더 이상의 발전을 저해하는 요인으로 지적되기도 한다. 더욱이 최근 인공 신경망의 은닉층과 노드의 수가 극단적으로 많아지는 상황에서 시스템의 연산 복잡도는 매우 중요한 문제이다. 또한 이전 계층에서 다음 계층으로 신호를 전달할 때 어떤 노드에서 생성된 신호가 다른 노드의 신호보다 중요할 수 있다. 신호의 중요도는 신호의 크기로 나타나는데, 시그모이드 함수는 입력 신호의 크기에 상관없이 출력이 최대 1로 제한하므로 각 노드의 중요도가 최종 결과에 반영되지 못하는 문제도 있다.

다른 문제로 그래디언트 소멸이라는 문제가 있다. 인공 신경망 학습의 기본은 경사하강법이며 경사하강법 적용을 위해서는 각 노드에서의 그래디언트, 즉 기울기를 측정해야 한다. 그런데 시그모이드 함수의 경우 입력이 커지면 그 출력이 항상 1이 되므로 기울기가 0으로 소멸되는 문제를 그래디언트 소멸 문제라 한다. 이는 경사하강법의 수렴을 더디게 하거나 적절한 솔루션으로 수렴하지 못하게하는 문제의 원인이 되기도 한다.

시그모이드 함수가 갖는 다양한 장점에도 불구하고, 인공 신경망의 대형화에 따라 제기된 문제를 해결하기 위한 새로운 활성화함수가 필요한 상황에서 대안으로 등장한 활성화함수는 ReLU 함수이다. ReLU는 Rectified Linear Unit의 약자로 전자회로의 정류기, 즉 교류를 직류로 바꿔주는 장치의 수학적 모델이다. ReLU 함수의 정의는 다음과 같다.

$$f(z) = \max(0, z) \quad (40)$$

즉, 입력 z 가 음수일 때는 0을 출력하고 양수일 때는 입력을 그대로 출력으로 내보내는 함수이다. 그림 18은 ReLU 함수를 시그모이드 함수와 비교한 것이다. (a)는 함수값 자체의 비교이고 (b)는 두 함수의 1차 도함수를 비교한 것이다.



[그림 18] 시그모이드와 ReLU의 비교: 함수 및 1차 도함수

그림 18(a)와 같이 ReLU 함수는 양의 입력을 그대로 통과시키는 특징을 가지고 있으며 18(b)에서 시그모이드 함수의 1차 도함수가 0으로 소멸하는 대신 ReLU의 도함수는 항상 1을 유지하는 것을 알 수 있다. 이 두 특성으로 다층 인공 신경망이 복잡해질 경우 시그모이드 함수가 갖는 연산 복잡도 상승과 그래디언트 소멸 문제를 해결할 수 있다.

식 (40)과 그림 18에서 보듯이, ReLU 함수는 매우 단순한 함수이며 이 함수가 딥러닝 모델의 성능을 크게 향상시킨다는 사실이 흥미롭다. 오늘날의 다층 인공 신경망 및 딥러닝 분야에서는 ReLU 함수를 활성화함수의 실질적 표준으로 여기고 있다.

ReLU 함수와 관련해 중요한 논문 두 편을 소개하고 마무리한다. 두 논문 모두 같은 연구 그룹에서 제출한 논문인데, 첫 번째 논문은 2010년 ICML에서 발표된 “Rectified Linear Units Improve Restricted Boltzmann Machines”라는 논문으로 ReLU 함수의 소개와 함께 영상 인식 분야에서 우수한 성능을 보인다는 내용이고 두 번째 논문은 “ImageNet Classification with Deep Convolutional Neural Networks”라는 논문으로 2012년 NeurIPS에서 발표한 것으로 ReLU 함수를 이용한 경우의 학습 속도를 포화함수를 이용한 경우의 학습 속도와 실험적으로 비교하며 “We would not have been able to experiment with such large neural networks for this work if we had used traditional saturating neuron models.”라고 평가했다.

5. 의사결정 트리 (Decision Tree)

1. 정답을 찾아가는 연속된 질문 - 의사결정 트리

분류 시스템은 지도학습 중에서 데이터의 출력 변수 또는 레이블이 가질 수 있는 값의 종류가 유한한 시스템을 뜻하며 출력 변수의 값을 클래스라 한다. 분류 시스템을 구현하는 대표적인 모델로 로지스틱 회귀 또는 인공 신경망이 있다. 이 모델들의 기본 원리는 데이터의 입력 속성들을 선형 조합한 뒤, 이를 어떤 함수를 통해 각 클래스에 속할 가능성(또는 확률) 값으로 변환하여 가장 높은 가능성을 갖는 클래스를 선택하는 방식이다. 이를 다른 측면으로 생각해 보면, 로지스틱 회귀 또는 인공 신경망은 클래스 예측을 위해 모든 입력 속성값을 동시에 고려하는 방식이라 할 수 있다.

그러나 이와 달리, 클래스 예측을 위해 입력 속성을 순차적으로 고려하는 접근도 가능하다. 한 가지 예를 가지고 생각해 보자. 최종 성적으로 통과(P=pass) 또는 실패(F=fail)로 부여하는 어떤 교과목이 있다고 가정한다. 이 과목에서 평가하는 세부 사항은 출석, 과제, 중간고사, 기말고사이다. 이 데이터는 입력 속성이 출석, 과제, 중간고사, 기말고사이고 출력 클래스가 P 또는 F인 전형적인 이진 분류(binary classification) 모델을 위한 데이터이다. 표 1은 각 학생들의 세부 성적과 최종 성적을 나타낸 표이다. 이 표에서 각 입력 속성의 값은 상/중/하 중 하나의 값을 갖도록 사전에 처리되었다.

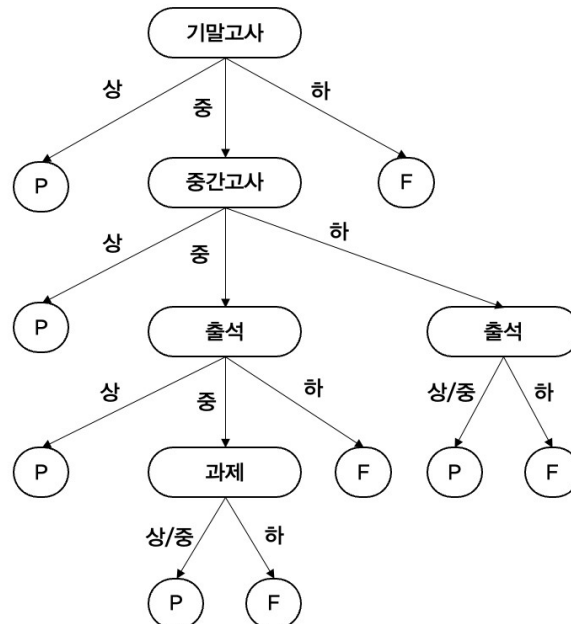
[표 1] 예제 데이터

학생 번호	출석	과제	중간고사	기말고사	최종 성적
1	중	상	상	상	P
2	하	중	중	하	F
3	상	상	상	상	P
4	하	중	중	중	F
⋮	⋮	⋮	⋮	⋮	⋮

이 표에서 한 눈에 파악하기는 어렵지만, 각각의 입력 속성이 최종 성적에 미치는 영향은 서로 다르다. 또한 어떤 경우에는 한 두 가지 속성만으로 최종 성적을 판단할 수도 있다. 예를 들어, 데이터 분석을 통해 기말고사 성적이 '상'에 속하면 다른 속성값에 상관없이 모두 P를 받았다거나 기말고사 성적은 '중'에 속하고 중간고사 성적이 '상'에 속하면 출석이나 과제 성적에 상관없이 P를 받는다는 등의 규칙성을 찾게 된다면 일부 속성만을 빠르게 판단하여 최종 성적을 효과적으로 예측할 수 있을 것이다. 따라서 네 가지 입력 속성을 동시에 고려할 필요 없이 중요한 속성을 먼저 살펴보고, 필요에 따라 나머지 속성을 순차적으로 고려하여 최종 클래스를 결정하는 것이 효율적일 수 있다.

그림 1은 입력 속성을 순차적으로 판단하여 최종 성적을 예측하는 절차의 한 예를 그림으로 표현한 것이다. 그림의 절차에서는 네 가지 입력 속성 중 기말고사 성적을 가장 먼저 확인한다. 기말고사 성적이 '상'이거나 '하'면 다른 속성을 볼 필요 없이 P 또는 F로 결정하고 더 이상의 판단을 진행하지 않는다. 다만, 기말고사 성적인 '중'인 경우 중간고사 성적을 추가적으로 살펴봐야 한다. 중간고사 성적이 '상'인 경우에는 곧바로 P를 부여하고 종료하면 되지만

‘중’ 또는 ‘하’인 경우에는 출석을 추가적으로 살펴봐야 한다. 이런 방식으로 최종 성적인 P 또는 F에 도달할 때까지 순차적으로 판단을 진행한다.



[그림 1] 입력 속성의 순차적 판단을 통한 분류의 예

그림 1과 같이 출력 클래스의 결정(decision)을 내리기 위해 입력 속성을 순차적으로 판단하는 기법을 의사결정 트리(decision tree) 라고 한다. ‘트리’라는 이름의 이유는 그림에서와 같이 의사결정 과정이 마치 나무의 가지가 뻗어나가는 모양과 유사하기 때문이다.

결론적으로 의사결정 트리는 ‘정답을 찾아가기 위한 연속된 질문’의 집합이라 할 수 있다. 여기에서 ‘정답’은 출력 클래스를 ‘질문’은 입력 속성을 나타낸다. 이제 남은 과제는 어떤 질문을 먼저 던지는 것이 좋은가를 찾는 것이다. 던져야할 질문을 차례로 찾는다는 것은 의사결정 트리를 만드는 것과 동일한 작업이다.

2. 트리를 구성하는 요소들

본격적으로 의사결정 트리를 만드는 방법에 대해 학습하기 전에 트리를 구성하는 기본 요소들에 대해 소개한다. 트리는 노드(node)와 가지(branch)로 구성된다. 노드는 ‘질문’을 담당하고 그 질문에 대한 답은 ‘가지’로 나타난다. 또한 노드는 역할과 위치에 따라 뿌리노드(root node), 내부노드(internal node), 단말노드(leaf node)로 구분되며 각각의 역할과 기능은 다음과 같다.

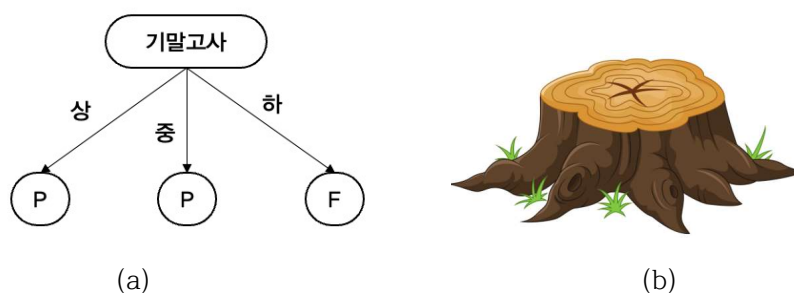
- **뿌리노드(root node):** 의사결정 트리의 최상단에 위치하는 노드로서 시스템의 첫 번째 질문을 담당한다. 그림 1의 예에서 뿌리노드는 ‘기말고사’이며 이 노드로부터 트리 구조가 퍼져 나가므로 이를 뿌리노드라 부른다.
- **단말노드(leaf node):** 의사결정 트리의 최종 결과에 해당하는 마지막 노드이다. 그림 1의 예에서 ‘P’ 또는 ‘F’를 나타내는 노드들이 단말노드이다. 단말노드는 분류 시스템

의 출력, 즉 라벨을 담당한다.

- **내부노드(internal node):** 트리에서 뿌리노드와 단말노드를 제외한 모든 노드를 내부노드라 한다. 내부노드들은 출력에 이르기 위한 연속적인 질문들을 담당한다.

모든 의사결정 트리의 판단 과정은 뿌리노드에서 시작해 내부노드들을 거쳐 단말노드에서 끝난다. 뿌리노드로부터 단말노드를 연결하는 다양한 경로가 존재하는데, 이 경로들을 분류 규칙(classification rule)이라 한다. 그림 1의 예는 총 9개의 단말노드로 구성되므로 뿌리노드와 단말노드를 연결하는 9개의 경로, 즉 9개의 분류 규칙이 내재된 의사결정 트리이다. 가령 기말고사가 '중'이고 중간고사가 '상'이면 분류 결과는 'P'라는 것은 9개 중 하나의 분류 규칙이다.

의사결정 트리 중 뿌리노드와 단말노드만으로 구성된 트리를 결정 그루터기(decision stump)라 부른다. 뿌리와 단말노드만으로 구성된 트리에는 내부노드가 없으므로 하나의 질문과 곧바로 이어지는 단말노드로 구성된다.



[그림 2] 결정 그루터기 예와 그루터기의 이미지

그림 2의 (a)는 결정 그루터기의 예를 나타낸다. 기말고사라는 뿌리노드와 세 개의 단말노드로 구성된 것을 확인할 수 있다. 왜 이런 이름이 붙었을까? 그림 2의 (b)는 그루터기의 이미지이다. 그루터기란 나무의 밑동을 베어내고 남은 부분을 말한다. 결정 그루터기는 그림 1과 같은 큰 트리를 잘라내어 만든 것과 같다는 의미에서 붙여진 이름이다.

3. 트리 구성 전략 - 선택과 분할

이제 의사결정 트리를 어떻게 만들 것인가를 고민해야할 시점이다. 앞에서 의사결정 트리는 정답을 찾아가기 위한 연속된 질문들의 집합이라고 표현했던 사실을 기억하자. 그렇다면, 의사결정 트리를 만드는 문제는 결국 여러 입력 속성 중에서 어떤 속성을 먼저 선택해 질문할 것인가의 문제가 된다.

질문할 속성이 선택되면 그 질문의 결과에 따라 데이터 집합이 분할된다. 예를 들어 그림 1의 경우, 첫 질문으로 선택한 속성은 기말고사 성적이다. 이 질문에 대한 결과로 전체 데이터 집합은 기말고사 성적이 '상'인 집합, '중'인 집합, '하'인 집합으로 분할될 것이다. 그리고 '중'으로 분할된 집합에 대해서는 다시 중간고사 속성의 질문을 던지고 그 결과에 따라 분할된다. 따라서 트리의 경로를 따라 아래로 내려갈수록 데이터 집합은 더 작은 부분집합으로 분할된다. 이렇게 의사결정 트리를 만드는 과정은 질문할 속성의 '선택'과 그 선택에 따른 데이터 집합의 '분할' 과정의 연속이라 할 수 있다.

그러면 어떤 속성을 먼저 선택하는 것이 좋을까? 이 질문에는 여러 가지 답이 가능하지만, 본 교재에서는 분할을 통해 얻을 수 있는 정보의 양이 더 많은 속성을 우선적으로 선택해 분할을 진행하는 전략을 설명한다. 이 전략을 달리 표현하자면 분할을 통해 데이터의 복잡도를 더 많이 낮출 수 있는 속성을 우선적으로 선택해 분할을 진행한다는 것과 같다.

4. 데이터가 가진 정보의 양 - 정보 엔트로피(Information entropy)

당연한 표현이지만 데이터에는 정보가 들어있다. 그럼 어떤 데이터에 정보가 더 많이 들어 있을까? 또는 어떤 데이터가 더 좋은 데이터일까? 이 질문에 답하기 위해서는 정보라는 추상적인 개념을 정보량이라는 객관적인 개념으로 수치화해야 한다. 이를 위한 지표가 바로 정보 엔트로피(information entropy)이다.

정보 엔트로피는 어떤 데이터 집합을 구성하는 서로 다른 종류의 데이터들의 비율에 의해 결정된다. 보다 구체적으로, 데이터 집합 D 의 원소들을 같은 것끼리 분류했을 때 그 종류의 개수가 K 개이고, 전체 원소 중 각 종류의 원소들이 차지하는 비율이 각각 $p_0, p_1, \dots, p_{K-1}$ 이라고 한다면, 집합 D 의 정보 엔트로피 I_D 는 다음과 같다.

$$I_D = - \sum_{k=0}^{K-1} p_k \log_2 p_k \quad (1)$$

식 (1)에서 밑이 2인 로그함수 대신 자연로그를 사용해도 상관없다. 다만, 밑이 2인 로그함수를 사용할 경우 정보 엔트로피의 단위는 비트(bit) 또는 새넨(Sh)이 되고 자연로그를 사용하는 경우에는 네트(nat)가 된다. 본 교재에서는 밑이 2인 로그함수로 통일하여 사용한다.

[표 2] 예제 데이터 집합 D

번호	학점	번호	학점
1	A	6	C
2	B	7	B
3	A	8	C
4	B	9	A
5	C	10	A

표 2의 예제 데이터 집합 D 를 이용해 정보 엔트로피를 계산해보자. 집합 D 는 10명의 학생이 받은 학점을 수집한 데이터 집합이다. 따라서 이 집합의 원소는 10명의 학점이고, 학점은 A/B/C 중 하나이므로 $K=3$ 이다. 편의상 $A=0, B=1, C=2$ 라고 표기하자. 그러면, 전체 10개의 원소 중 A학점이 4명, B학점이 3명, C학점이 3명이므로 각 학점이 차지하는 비율은 다음과 같이 계산할 수 있다.

$$p_0 = \frac{4}{10}, p_1 = \frac{3}{10}, p_2 = \frac{3}{10}$$

따라서 식 (1)에 의해 집합 D 의 정보 엔트로피는 다음과 같다.

$$I_D = - \frac{4}{10} \log_2 \frac{4}{10} - \frac{3}{10} \log_2 \frac{3}{10} - \frac{3}{10} \log_2 \frac{3}{10} = 1.571 \quad (2)$$

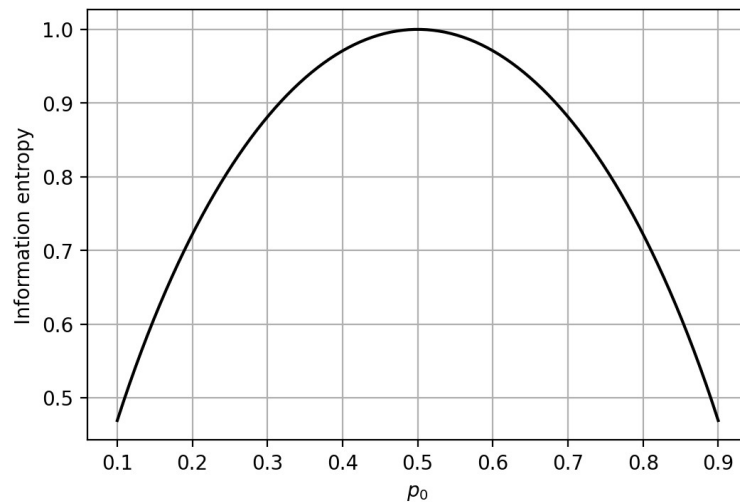
이 결과에 따르면 집합 D 이 가지고 있는 정보량은 1.571이다.

위의 예에서 본 것처럼, 식 (1)을 이용해 어떤 데이터 집합의 정보량을 계산하는 것은 어려

운 일이 아니다. 그런데, 여기에서 정보량을 계산하는데 그치지 말고 정보 엔트로피의 물리적 의미를 확인해보자. 이를 위해, 원소의 종류가 2개인 데이터 집합 D 를 생각해보자. 이런 집합을 이진 데이터 집합이라 한다. 데이터의 종류가 2개이므로 한 종류를 0(음성), 다른 종류를 1(양성)이라고 표시하면, 이 집합의 정보 엔트로피는 다음과 같다.

$$I_D = -p_0 \log_2 p_0 - p_1 \log_2 p_1$$

여기에서 $p_0 + p_1 = 1$ 이다. 이때 p_0 의 값을 0.1부터 0.9까지 변화시키면서 정보 엔트로피의 변화를 관찰해보자. 그림 3은 p_0 의 변화에 대한 집합 D 의 정보 엔트로피 변화를 보여준다. p_0 가 0.1부터 0.9까지 변하는 동안 p_1 은 0.9에서 0.1까지 변한다. 그림 3에서 정보 엔트로피가 증가하다가 감소하는 것을 관찰할 수 있다. 특히 p_0 가 0.5가 되는 지점에서 최대값을 갖는다. p_0 가 0.5인 경우는 전체 데이터 중 절반이 음성 데이터이고 나머지 절반이 양성 데이터인 경우이다. 반면 p_0 가 0.1 또는 0.9일 때 정보 엔트로피는 매우 낮은 값을 갖는다. 이 경우는 데이터의 대부분이 양성이거나 대부분이 음성인 경우에 해당한다.



[그림 3] 이진 데이터 집합의 p_0 에 따른 정보 엔트로피의 변화 ($p_1 = 1 - p_0$)

이 결과를 정리하자면, 데이터 집합이 서로 다른 종류의 데이터가 골고루 섞여있는 경우 정보 엔트로피가 높고 어느 한 종류가 대다수인 경우 정보 엔트로피가 낮아진다. 여기에서 정보 엔트로피가 정보의 양을 나타내는 이유를 알 수 있다. 우리는 어떤 데이터에서 많은 정보를 얻을 수 있을까? 쉬운 예로, 어떤 실험을 10회 반복하여 수행하여 데이터를 수집하는 과학 실험을 생각해보자. 10회 반복하는 동안 거의 동일한 데이터가 나온다면, 굳이 10회 반복할 이유가 없다. 즉, 1회 실험하나 10회 실험하나 얻을 수 있는 정보는 같다. 그런데 10회 반복할 동안 다양한 결과가 관찰된다면 10회 모두 의미 있는 실험이었으며 다양한 해석과 분석을 도출할 수 있을 것이다. 즉, 후자의 데이터에 더 많은 양의 정보가 숨어있는 것이다. 이처럼 다양한 종류의 데이터가 혼재될수록 정보의 양이 많아지고, 소수 종류의 데이터가 대부분을 차지하게 될수록 정보의 양은 적다. 이러한 물리적 의미를 정확하게 표현할 수 있는 지표가 바로 정보 엔트로피이다.

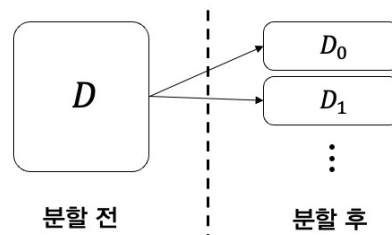
다른 자료에서 엔트로피를 종종 ‘복잡도’라고 표현하는 경우가 있다. 이 역시 앞의 예를 통해 쉽게 이해할 수 있다. 다른 종류의 데이터가 혼재될수록 정보의 양이 많아지는데, 이렇게

다른 종류의 데이터가 혼재된 데이터는 복잡하다. 따라서 어떤 데이터의 엔트로피가 높다는 것은 정보량이 많다는 사실과 복잡도가 높다는 사실을 동시에 의미하는 것이다. 반면 데이터가 대부분 어느 한 종류에 속한다면, 그 데이터 집합은 매우 가지런하다. 즉 복잡하지 않은 것이다. 따라서 가지런한 데이터는 복잡도가 낮고 정보량도 적으며 정보 엔트로피도 낮다.

5. 분할을 통해 얻을 수 있는 정보량의 이득 - 정보이득 (Information gain)

다시 의사결정 트리에서 어떤 속성을 먼저 선택하여 데이터 집합을 분할해야 하는가의 문제로 돌아가자. 속성 선택 전략이 **분할을 통해 얻을 수 있는 정보량이 더 많은 속성을 우선 선택**한다는 것을 기억하자. 그리고 앞 절에서 어떤 데이터 집합의 정보량을 측정하는 방법을 학습했다. 그렇다면, 분할을 통해 얻을 수 있는 정보량은 어떻게 측정하면 될까?

그림 4는 데이터 집합 D 를 두 개의 부분 집합 D_0, D_1 으로 분할한 예를 나타낸 것이다. 분할을 통해 데이터 집합의 정보량이 줄어들었다면 그 차이가 바로 분할을 통해 얻는 정보이득 (information gain)이 된다. 따라서 **정보이득은 분할 전 정보 엔트로피와 분할 후 정보 엔트로피의 차이**이다.



[그림 4] 데이터 집합 분할의 예

그러나 정보 엔트로피는 개별 데이터 집합에 대해 계산되는 값이고 분할 후에는 두 개 이상의 집합이 만들어지므로 분할 후 생성되는 여러 집합을 대표하는 정보 엔트로피의 정의가 필요하다. 이때, 분할 후 전체 엔트로피는 각 집합의 정보 엔트로피의 가중평균을 사용한다. 가중평균이란 각 집합이 전체에서 차지하는 비율로 가중치를 조정하는 평균이다.

데이터 집합 D 를 속성 a 를 기준으로 V 개의 부분집합으로 분할하여 $D_0, D_1, \dots, D_{V-1}$ 을 만들었을 때, 분할을 통해 얻을 수 있는 정보이득 $G(D, a)$ 는 다음과 같다.

$$G(D, a) = I_D - \sum_{v=0}^{V-1} \frac{|D_v|}{|D|} I_{D_v} \quad (3)$$

이때, $|\cdot|$ 은 집합의 원소의 개수를 나타낸다. 따라서 $\frac{|D_v|}{|D|}$ 은 전체 데이터 중에서 부분집합 D_v 의 비율을 나타내는 가중치가 되고 $\sum_{v=0}^{V-1} \frac{|D_v|}{|D|} I_{D_v}$ 은 분할된 V 개 부분집합의 평균 정보 엔트로피이다.

이제 예제를 이용해 정보이득을 계산해보자. 예를 들어 표 2의 데이터 집합을 표 3과 같이 무작위적으로 두 개의 그룹으로 분할한 경우를 생각해보자.

[표 3] 분할된 데이터 집합

집합 D_0		집합 D_1	
번호	학점	번호	학점
1	A	2	B
3	A	6	C
4	B	8	C
5	C	10	A
7	B		
9	A		

분할 전의 집합 D 는 표 2와 같고 D 의 정보 엔트로피는 식 (2)에서 계산한 것과 같이 1.571이다. 다음으로 분할 후의 평균 정보 엔트로피와 분할을 통해 얻을 수 있는 정보이득을 다음과 같이 계산할 수 있다.

- 집합 D_0 정보 엔트로피:

$|D_0| = 6$ 이고 A/B/C의 비율은 각각 $\frac{3}{6}$, $\frac{2}{6}$, $\frac{1}{6}$ 이므로,

$$I_{D_0} = -\frac{3}{6}\log_2 \frac{3}{6} - \frac{2}{6}\log_2 \frac{2}{6} - \frac{1}{6}\log_2 \frac{1}{6} = 1.459$$

- 집합 D_1 정보 엔트로피:

$|D_1| = 4$ 이고 A/B/C의 비율은 각각 $\frac{1}{4}$, $\frac{1}{4}$, $\frac{2}{4}$ 이므로,

$$I_{D_1} = -\frac{1}{4}\log_2 \frac{1}{4} - \frac{1}{4}\log_2 \frac{1}{4} - \frac{2}{4}\log_2 \frac{2}{4} = 1.5$$

- 분할 이후의 평균 정보 엔트로피:

$|D| = 10$ 이므로,

$$\frac{6}{10} \times 1.459 + \frac{4}{10} \times 1.5 = 1.475$$

- 정보이득:

$$G(D) = 1.571 - 1.475 = 0.096$$

이 계산 결과는 데이터 집합을 분할하여 0.096의 정보이득을 얻을 수 있다는 것을 보여준다. 이는 분할 전의 정보량 1.571이 분할 후에 1.475로 낮아져 그 차이만큼을 이득으로 얻었다고 해석할 수도 있고, 분할 전 데이터의 복잡도가 1.571이었다가 분할을 통해 복잡도를 1.475로 0.096만큼 낮추었다고 해석할 수도 있다.

6. 의사결정 트리 만들기 - 구현 알고리즘

지금까지 학습한 내용을 이용해 의사결정 트리를 만들어보자. 이를 위해 예제 데이터 집합을 먼저 소개한다. 표 4는 학생 15명의 성적을 평가한 데이터이다. 최종 학점은 P(pass) 또는 F(fail) 중 하나가 부여되고, 이를 위해 출석, 과제, 중간고사, 기말고사 등 네 가지 성적을 활용한다. 따라서 네 가지 성적이 입력 속성이 되고 학점은 출력 레이블이다. 또한 출석과 과제는 A 또는 B로 평가되며 중간고사와 기말고사는 A, B, C 중 하나의 등급으로 평가된다.

[표 4] 예제 데이터

번호	출석 (A/B)	과제 (A/B)	중간고사 (A/B/C)	기말고사 (A/B/C)	학점 (P/F)
1	A	B	B	A	P
2	A	B	C	A	P
3	B	A	A	C	F
4	A	B	A	C	P
5	A	A	A	B	P
6	A	A	B	B	P
7	A	A	A	A	P
8	A	B	B	C	F
9	B	A	C	C	F
10	A	B	C	B	F
11	A	B	A	A	P
12	B	A	B	B	F
13	A	A	A	C	P
14	B	A	C	A	F
15	A	B	B	B	F

6.1 뿌리노드의 선정

뿌리노드는 의사결정 트리의 첫 번째 노드이다. 뿌리노드에 해당하는 속성을 찾기 위해서는 각각의 속성으로 전체 집합을 분할했을 때 얻을 수 있는 정보이득이 가장 큰 속성을 찾으려 한다. 이를 위해 전체 집합을 D 라 하면, 집합 D 는 다음과 같다.

$$D = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15\}$$

집합 D 의 데이터 중 P는 8개, F는 7개이므로 정보 엔트로피는 다음과 같다.

$$I_D = -\frac{8}{15}\log_2\frac{8}{15} - \frac{7}{15}\log_2\frac{7}{15} = 0.997$$

먼저 D 를 출석 속성으로 분할하는 경우를 생각해보자. 출석 속성이 가질 수 있는 값은 A 또는 B이므로 다음과 같이 두 개의 부분 집합으로 나눌 수 있다.

$$D_{\text{출석}=A} = \{1, 2, 4, 5, 6, 7, 8, 10, 11, 13, 15\}$$

$$D_{\text{출석}=B} = \{3, 9, 12, 14\}$$

각 집합의 데이터는 표 5와 6과 같다.

[표 5] 집합 $D_{\text{출석} = A}$

번호	출석 (A/B)	과제 (A/B)	중간고사 (A/B/C)	기말고사 (A/B/C)	학점 (P/F)
1	A	B	B	A	P
2	A	B	C	A	P
4	A	B	A	C	P
5	A	A	A	B	P
6	A	A	B	B	P
7	A	A	A	A	P
8	A	B	B	C	F
10	A	B	C	B	F
11	A	B	A	A	P
13	A	A	A	C	P
15	A	B	B	B	F

[표 6] 집합 $D_{\text{출석} = B}$

번호	출석 (A/B)	과제 (A/B)	중간고사 (A/B/C)	기말고사 (A/B/C)	학점 (P/F)
3	B	A	A	C	F
9	B	A	C	C	F
12	B	A	B	B	F
14	B	A	C	A	F

표 5에서 출석이 A인 집합에는 모두 11개의 데이터가 있으며 각각 P가 8개, F가 3개라는 것을 알 수 있다. 따라서 이 집합의 정보 엔트로피는 다음과 같다.

$$I_{D_{\text{출석} = A}} = -\frac{8}{11}\log_2\frac{8}{11} - \frac{3}{11}\log_2\frac{3}{11} = 0.845$$

같은 방법으로 출석이 B인 집합(표 6)을 구성하는 4개의 데이터 중 P는 0개, F는 4개이므로 정보 엔트로피는 다음과 같이 계산할 수 있다.

$$I_{D_{\text{출석} = B}} = -\frac{0}{4}\log_2\frac{0}{4} - \frac{4}{4}\log_2\frac{4}{4} = 0$$

모든 데이터가 같은 레이블 F를 가지고 있으므로, 이 부분집합의 정보량은 0이 된다. 따라서 출석 속성으로 집합 D 를 분할했을 때 얻을 수 있는 정보이득 $G(D, \text{출석})$ 을 다음과 같이 구할 수 있다.

$$G(D, \text{출석}) = 0.997 - \left(\frac{11}{15} \times 0.845 + \frac{4}{15} \times 0\right) = 0.377 \quad (4)$$

다음으로 과제 속성으로 분할했을 때 얻을 수 있는 정보이득을 계산해보자. 과제 속성 역시 A 또는 B의 값을 가질 수 있으므로, 다음과 같이 두 개의 부분 집합으로 분할할 수 있다.

$$D_{\text{과제} = A} = \{3, 5, 6, 7, 9, 12, 13, 14\}$$

$$D_{\text{과제} = B} = \{1, 2, 4, 8, 10, 11, 15\}$$

과제 속성이 A인 8개의 데이터 중 레이블 P와 F를 갖는 데이터는 각각 4개씩이다. 또한 과제 속성이 B인 데이터는 모두 7개이고 이중 4개는 P, 나머지 3개는 F에 해당한다. 따라서 두 집합의 정보 엔트로피는 각각 다음과 같다.

$$I_{D_{\text{과제}}=A} = -\frac{4}{8}\log_2\frac{4}{8} - \frac{4}{8}\log_2\frac{4}{8} = 1$$

$$I_{D_{\text{과제}}=B} = -\frac{4}{7}\log_2\frac{4}{7} - \frac{3}{7}\log_2\frac{3}{7} = 0.985$$

과제 속성으로 분할했을 때의 정보이득 $G(D, \text{과제})$ 을 다음과 같이 얻을 수 있다.

$$G(D, \text{과제}) = 0.997 - \left(\frac{8}{15} \times 1 + \frac{7}{15} \times 0.985\right) = 0.004 \quad (5)$$

중간고사 속성은 A, B, C 중 하나의 값을 가질 수 있으므로 다음과 같이 세 개의 부분집합으로 분할할 수 있다.

$$D_{\text{중간}=A} = \{3, 4, 5, 7, 11, 13\}$$

$$D_{\text{중간}=B} = \{1, 6, 8, 12, 15\}$$

$$D_{\text{중간}=C} = \{2, 9, 10, 14\}$$

각 부분집합의 정보 엔트로피는 다음과 같이 계산할 수 있다.

$$I_{D_{\text{중간}=A}} = -\frac{5}{6}\log_2\frac{5}{6} - \frac{1}{6}\log_2\frac{1}{6} = 0.650$$

$$I_{D_{\text{중간}=B}} = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} = 0.971$$

$$I_{D_{\text{중간}=C}} = -\frac{1}{4}\log_2\frac{1}{4} - \frac{3}{4}\log_2\frac{3}{4} = 0.811$$

위에서와 같은 방법으로 중간고사 속성에 대한 정보이득 $G(D, \text{중간})$ 은 다음과 같이 구할 수 있다.

$$G(D, \text{중간}) = 0.997 - \left(\frac{6}{15} \times 0.650 + \frac{5}{15} \times 0.971 + \frac{4}{15} \times 0.811\right) = 0.197 \quad (6)$$

마지막 속성인 기말고사 역시 A/B/C 중 하나의 값을 가질 수 있기 때문에, 다음과 같이 세 개의 부분집합으로 분할할 수 있다.

$$D_{\text{기말}=A} = \{1, 2, 7, 11, 14\}$$

$$D_{\text{기말}=B} = \{5, 6, 10, 12, 15\}$$

$$D_{\text{기말}=C} = \{3, 4, 8, 9, 13\}$$

각 집합의 정보 엔트로피는 다음과 같다.

$$I_{D_{\text{기말}=A}} = -\frac{4}{5}\log_2\frac{4}{5} - \frac{1}{5}\log_2\frac{1}{5} = 0.722$$

$$I_{D_{\text{기말}=B}} = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} = 0.971$$

$$I_{D_{\text{기말}=C}} = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} = 0.971$$

특히 기말고사 속성이 C인 집합에 속한 모든 데이터의 라벨이 F이다. 이렇게 같은 종류의 데이터만으로 구성된 집합의 정보량은 0이다. 정보량이 0이라는 것은 복잡도 역시 0이라는 것을 의미한다. 기말고사 속성에 대한 정보이득은 다음과 같다.

$$G(D, \text{기말}) = 0.997 - \left(\frac{5}{15} \times 0.722 + \frac{5}{15} \times 0.971 + \frac{5}{15} \times 0.971\right) = 0.109 \quad (7)$$

이상의 과정을 통해 데이터 집합 D 를 각 입력 속성으로 분할했을 때 얻을 수 있는 정보이득을 계산했다. 계산한 정보이득을 종합해보면 다음과 같다.

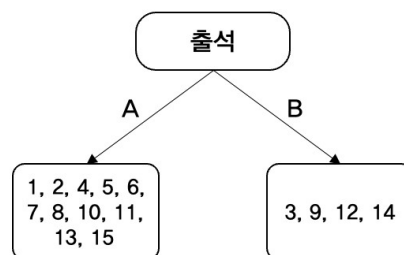
$$G(D, \text{출석}) = 0.377$$

$$G(D, \text{과제}) = 0.004$$

$$G(D, \text{중간}) = 0.197$$

$$G(D, \text{기말}) = 0.109$$

이 결과를 보면 출석으로 분할했을 때 가장 많은 양의 정보를 얻을 수 있는데 반해 과제 속성으로 나누었을 때에는 아주 작은 정보이득을 얻을 수 있다는 것을 알 수 있다. 따라서 의사결정 트리의 첫 번째 노드인 뿌리노드의 속성은 출석 속성이 적절하다. 그림 5는 출석 속성을 뿌리노드에 할당하고 그 속성 값에 따라 데이터 집합을 분할한 것이다.



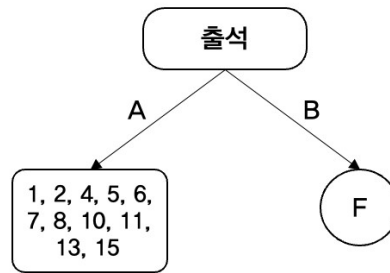
[그림 5] 뿌리노드의 할당과 집합의 분할

다음 작업은 분할된 각 부분집합을 계속 분할하거나 분할을 종료하는 것이다. 각 부분집합을 계속 분할한다는 것은 해당 노드를 내부노드로 만든다는 것을 의미하고 분할을 종료한다는 것은 해당 노드를 단말노드로 만드는 것을 의미한다. 어떤 경우에 분할을 계속하고 어떤 경우에 분할을 종료하는지 알아보자.

6.2 단말노드 만들기 - 분할 종료 조건 1

그림 5에서는 출석이 A인 데이터 집합과 출석이 B인 데이터 집합으로 분할된 두 부분집합을 볼 수 있다. 여기에서 한 가지 특이한 점을 관찰할 수 있는데, 출석이 B인 데이터 3, 9, 12, 14번은 모두 출력 레이블이 F로 동일하다는 것이다. 즉, 집합의 모든 데이터가 동일한 레이블을 갖고 있는 경우이다. 이런 경우, 이 집합을 더 분할하는 것은 불필요하다. 왜냐하면 의사결정 트리의 목표는 연속된 질문을 통해 출력 레이블을 유추하는 것인데, 어떤 질문을 통해 만들어진 데이터 집합이 모두 동일한 레이블을 갖는다는 것은 이미 원하는 해답에 도달했다는 것을 의미한다. 정보량의 관점으로 보면, 모든 데이터가 동일한 레이블을 가지면 정보 엔트로피가 0이므로 이 집합을 분할해서 얻을 수 있는 정보이득 역시 0이다. 따라서 더 이상의 분할은 무의미하다.

이렇게 어떤 집합의 데이터들이 모두 동일한 출력 레이블을 갖는 경우 분할을 종료하고 해당 노드를 단말노드로 지정한다. 또한 데이터들이 갖는 출력 레이블을 단말노드에 할당한다. 그림 6은 출석이 B인 부분집합이 있던 노드를 단말노드로 지정하고, 집합의 데이터들이 가지고 있는 레이블 F를 할당한 결과를 나타낸다. 이 결과에 따르면, 출석이 B인 학생들은 다른 속성과 상관없이 최종 성적 F를 받게 된다.



[그림 6] 단말노드로 지정하고 레이블 F를 할당한 결과

분할 종료 조건 1: 이와 같이 어떤 집합의 모든 데이터가 동일한 출력 레이블을 가지면 분할을 종료한다. 이 조건을 분할 종료 조건이라 한다. 분할 종료 조건은 몇 가지가 있는데, 그 중 처음으로 소개하는 조건이므로 본 교재에서는 다른 조건과 구분하기 위해 편의상 분할 종료 조건 1이라는 명칭을 사용하기로 한다.

6.3 추가 분할 진행 - 내부노드 만들기

분할 종료 조건에 해당하지 않으면 추가 분할을 진행한다. 추가 분할은 뿌리노드 선정과 동일한 방법으로 진행한다. 단, 추가 분할에서는 앞서 선택한 분할 속성은 고려하지 않는다. 즉, 그림 6에서 출석을 이용해 분할된 부분집합에 대해서는 출석 속성을 배제하고 추가 분할을 진행한다. 표 7은 출석 속성이 A인 데이터 집합으로, 추가 분할을 진행하기 위해 이 집합을 D 로 명명하고 남은 세 속성, 즉 과제, 중간고사, 기말고사로 분할했을 때의 정보이득을 비교한다.

[표 7] 출석 속성이 A인 데이터 집합 D

번호	출석 (A/B)	과제 (A/B)	중간고사 (A/B/C)	기말고사 (A/B/C)	학점 (P/F)
1	A	B	B	A	P
2	A	B	C	A	P
4	A	B	A	C	P
5	A	A	A	B	P
6	A	A	B	B	P
7	A	A	A	A	P
8	A	B	B	C	F
10	A	B	C	B	F
11	A	B	A	A	P
13	A	A	A	C	P
15	A	B	B	B	F

먼저 집합 D 의 정보 엔트로피는 다음과 같다.

$$I_D = -\frac{8}{11}\log_2\frac{8}{11} - \frac{3}{11}\log_2\frac{3}{11} = 0.845$$

과제로 분할하는 경우의 정보이득을 계산하는 절차는 다음과 같다.

- 부분집합

$$D_{\text{과제}} = A = \{5, 6, 7, 13\}$$

$$D_{\text{과제}} = B = \{1, 2, 4, 8, 10, 11, 15\}$$

- 정보 엔트로피

$$I_{D_{\text{출석}}=A} = -\frac{4}{4}\log_2 \frac{4}{4} - \frac{0}{4}\log_2 \frac{0}{4} = 0$$

$$I_{D_{\text{출석}}=B} = -\frac{4}{7}\log_2 \frac{4}{7} - \frac{3}{7}\log_2 \frac{3}{7} = 0.985$$

- 정보이득

$$G(D, \text{출석}) = 0.845 - \left(\frac{4}{11} \times 0 + \frac{7}{11} \times 0.985 \right) = 0.218$$

중간고사로 분할하는 경우의 정보이득을 계산하는 절차는 다음과 같다.

- 부분집합

$$D_{\text{중간}} = A = \{4, 5, 7, 11, 13\}$$

$$D_{\text{중간}} = B = \{1, 6, 8, 15\}$$

$$D_{\text{중간}} = C = \{2, 10\}$$

- 정보 엔트로피

$$I_{D_{\text{중간}}=A} = -\frac{5}{5}\log_2 \frac{5}{5} - \frac{0}{5}\log_2 \frac{0}{5} = 0$$

$$I_{D_{\text{중간}}=B} = -\frac{2}{4}\log_2 \frac{2}{4} - \frac{2}{4}\log_2 \frac{2}{4} = 1$$

$$I_{D_{\text{중간}}=C} = -\frac{1}{2}\log_2 \frac{1}{2} - \frac{1}{2}\log_2 \frac{1}{2} = 1$$

- 정보이득

$$G(D, \text{중간}) = 0.845 - \left(\frac{5}{11} \times 0 + \frac{4}{11} \times 1 + \frac{2}{11} \times 1 \right) = 0.3$$

기말고사로 분할하는 경우의 정보이득을 계산하는 절차는 다음과 같다.

- 부분집합

$$D_{\text{기말}} = A = \{1, 2, 7, 11\}$$

$$D_{\text{기말}} = B = \{5, 6, 10, 15\}$$

$$D_{\text{기말}} = C = \{4, 8, 13\}$$

- 정보 엔트로피

$$I_{D_{\text{기말}}=A} = -\frac{4}{4}\log_2 \frac{4}{4} - \frac{0}{4}\log_2 \frac{0}{4} = 0$$

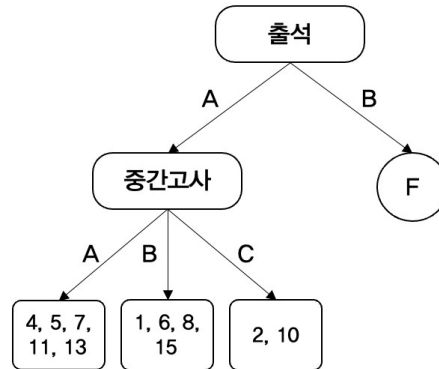
$$I_{D_{\text{기말}}=B} = -\frac{2}{4}\log_2 \frac{2}{4} - \frac{2}{4}\log_2 \frac{2}{4} = 1$$

$$I_{D_{\text{기말}}=C} = -\frac{2}{3}\log_2 \frac{2}{3} - \frac{1}{3}\log_2 \frac{1}{3} = 0.918$$

- 정보이득

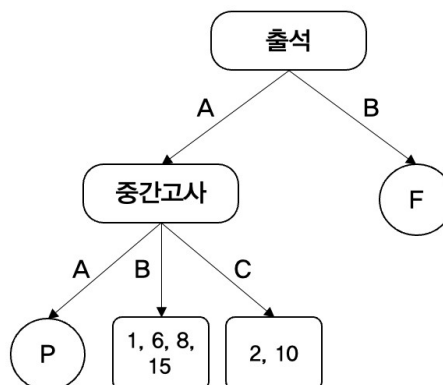
$$G(D, \text{기말}) = 0.845 - \left(\frac{4}{11} \times 0 + \frac{4}{11} \times 1 + \frac{3}{11} \times 0.918 \right) = 0.231$$

계산 결과 중간고사 속성으로 분할하는 경우 정보이득이 가장 높다. 따라서 그림 6에서 출석이 A인 부분집합을 내부노드로 바꾸고 중간고사 속성을 할당한 뒤 중간고사 성적에 따라 추가 분할을 진행한다. 그림 7은 중간고사 속성을 이용해 추가 분할한 결과를 나타낸다.



[그림 7] 중간고사 속성으로 추가 분할한 결과

그림 7에서 중간고사 속성이 A인 부분집합을 살펴보면 데이터 4, 5, 7, 11, 13이 모두 동일한 출력 레이블 P를 가지고 있다. 따라서 위에서 공부한 분할 종료 조건 1에 해당하므로 해당 노드를 단말노드로 지정하고 레이블 P를 할당하면 된다. 그림 8은 그 결과를 보여준다. 지금까지의 분할을 통해 두 개의 단말노드가 생성되었다. 따라서 두 개의 분류 규칙이 만들어진 것이다. 즉, 출석이 A이고 중간고사가 A이면 P로 분류하는 규칙과 출석이 B이면 F로 분류하는 규칙이다. 이 경우를 제외한 나머지 경우에 대한 분류 규칙을 찾기 위해 추가 분할을 진행해보자.



[그림 8] 분할 종료 조건에 따라 단말노드를 지정한 결과

다음으로 중간고사 속성이 B인 집합에 대해서 앞의 과정과 같은 분할을 진행한다. 표 8은 추가 분할을 위해 출석이 A이고 중간고사가 B인 데이터 집합 D를 나타낸다.

[표 8] 출석이 A이고 중간고사 속성이 B인 데이터 집합 D

번호	출석 (A/B)	과제 (A/B)	중간고사 (A/B/C)	기말고사 (A/B/C)	학점 (P/F)
1	A	B	B	A	P
6	A	A	B	B	P
8	A	B	B	C	F
15	A	B	B	B	F

표 8의 집합 D 가 갖는 정보 엔트로피는 다음과 같다.

$$I_D = -\frac{2}{4}\log_2\frac{2}{4} - \frac{2}{4}\log_2\frac{2}{4} = 1$$

과제로 분할하는 경우의 정보이득을 계산하는 절차는 다음과 같다.

- 부분집합

$$D_{\text{과제} = A} = \{6\}$$

$$D_{\text{과제} = B} = \{1, 8, 15\}$$

- 정보 엔트로피

$$I_{D_{\text{과제} = A}} = -\frac{1}{1}\log_2\frac{1}{1} - \frac{0}{1}\log_2\frac{0}{1} = 0$$

$$I_{D_{\text{과제} = B}} = -\frac{1}{3}\log_2\frac{1}{3} - \frac{2}{3}\log_2\frac{2}{3} = 0.918$$

- 정보이득

$$G(D, \text{과제}) = 1 - \left(\frac{1}{4} \times 0 + \frac{3}{4} \times 0.918\right) = 0.311$$

기말고사로 분할하는 경우의 정보이득을 계산하는 절차는 다음과 같다.

- 부분집합

$$D_{\text{기말} = A} = \{1\}$$

$$D_{\text{기말} = B} = \{6, 15\}$$

$$D_{\text{기말} = C} = \{8\}$$

- 정보 엔트로피

$$I_{D_{\text{기말} = A}} = -\frac{1}{1}\log_2\frac{1}{1} - \frac{0}{1}\log_2\frac{0}{1} = 0$$

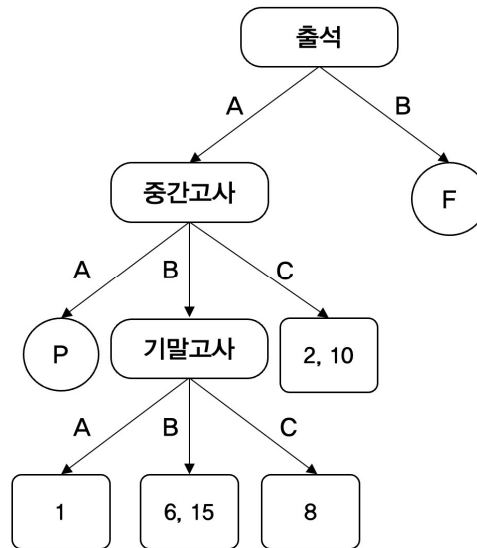
$$I_{D_{\text{기말} = B}} = -\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2} = 1$$

$$I_{D_{\text{기말} = C}} = -\frac{0}{1}\log_2\frac{0}{1} - \frac{1}{1}\log_2\frac{1}{1} = 0$$

- 정보이득

$$G(D, \text{기말}) = 1 - \left(\frac{4}{1} \times 0 + \frac{2}{4} \times 1 + \frac{1}{4} \times 0\right) = 0.5$$

각각의 정보이득을 비교하면 기말고사 속성을 이용해 분할하는 것이 가장 유리하다. 그림 9는 기말고사 속성으로 추가 분할을 진행한 결과를 나타낸다.



[그림 9] 기말고사 속성으로 분할 진행한 결과

다음으로 출석이 A, 중간고사가 C인 집합의 분할을 진행한다. 이를 위해 표 9와 같이 집합 D 를 정의한다. 표에서 남아있는 속성은 과제와 기말고사이므로 각 속성에 대한 정보이득을 계산한다.

[표 9] 출석이 A, 중간고사가 C인 집합 D

번호	출석 (A/B)	과제 (A/B)	중간고사 (A/B/C)	기말고사 (A/B/C)	학점 (P/F)
2	A	B	C	A	P
10	A	B	C	B	F

집합 D 가 갖는 정보 엔트로피는 다음과 같다.

$$I_D = -\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2} = 1$$

과제로 분할하는 경우의 정보이득을 계산하는 절차는 다음과 같다.

- 부분집합

$$D_{\text{과제} = A} = \emptyset$$

$$D_{\text{과제} = B} = \{2, 10\}$$

- 정보 엔트로피

$$I_{D_{\text{과제} = A}} = 0$$

$$I_{D_{\text{과제} = B}} = -\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2} = 1$$

- 정보이득

$$G(D, \text{과제}) = 1 - \left(\frac{0}{2} \times 0 + \frac{2}{2} \times 1\right) = 0$$

기말고사로 분할하는 경우의 정보이득을 계산하는 절차는 다음과 같다.

- 부분집합

$$D_{기말=A} = \{2\}$$

$$D_{기말=B} = \{10\}$$

$$D_{기말=C} = \emptyset$$

- 정보 엔트로피

$$I_{D_{기말=A}} = -\frac{1}{1}\log_2\frac{1}{1} - \frac{0}{1}\log_2\frac{0}{1} = 0$$

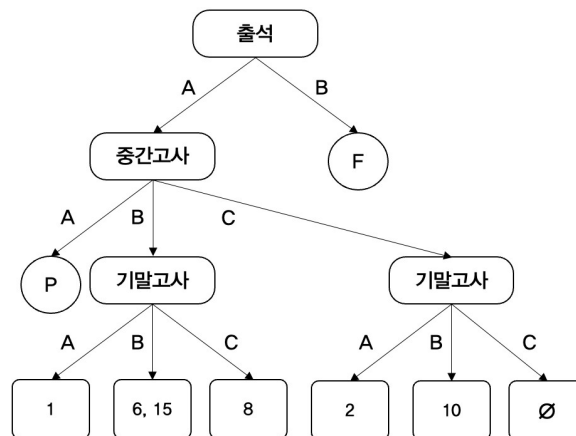
$$I_{D_{기말=B}} = -\frac{0}{1}\log_2\frac{0}{1} - \frac{1}{1}\log_2\frac{1}{1} = 0$$

$$I_{D_{기말=C}} = 0$$

- 정보이득

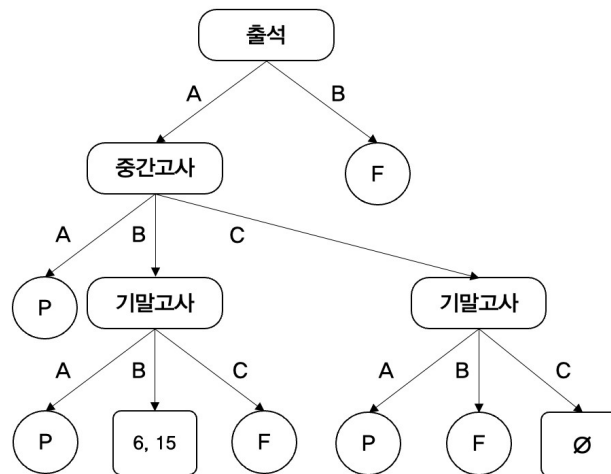
$$G(D, 기말) = 1 - \left(\frac{4}{1} \times 0 + \frac{2}{4} \times 1 + \frac{1}{4} \times 0 \right) = 0.5$$

과제로 분류하는 경우 정보이득을 얻을 수 없지만 기말고사 속성으로 분할하면 0.5의 정보이득을 얻을 수 있다. 따라서 정보이득을 기준으로 분할을 진행한다. 그림 10은 기말고사 속성으로 분할을 진행한 결과이다.



[그림 10] 기말고사 속성으로 추가 분할을 진행한 결과

그림 10의 부분집합들 중에서 네 개의 집합에는 각각 하나씩의 데이터만 들어있다. 데이터가 한 개인 경우 해당 집합의 출력 레이블도 하나뿐이므로 분할 종료 조건 1에 해당하여 자동으로 단말노드가 되고 해당 데이터의 레이블이 할당된다. 그림 11은 분할 종료 조건 1에 해당하는 노드들을 단말노드로 지정한 결과이다.



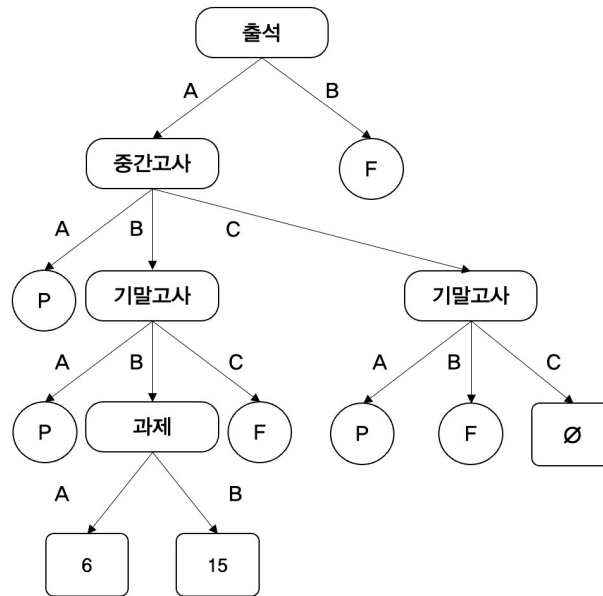
[그림 11] 분할 종료 조건에 따라 단말노드 지정

그림 11에 남아있는 집합은 {6, 15}와 공집합이다. 먼저 집합 {6, 15}를 먼저 분할하고 공집합의 처리에 대해 생각해보자. 집합을 분할하기 위해 표 10과 같이 데이터 집합 D 를 정의한다.

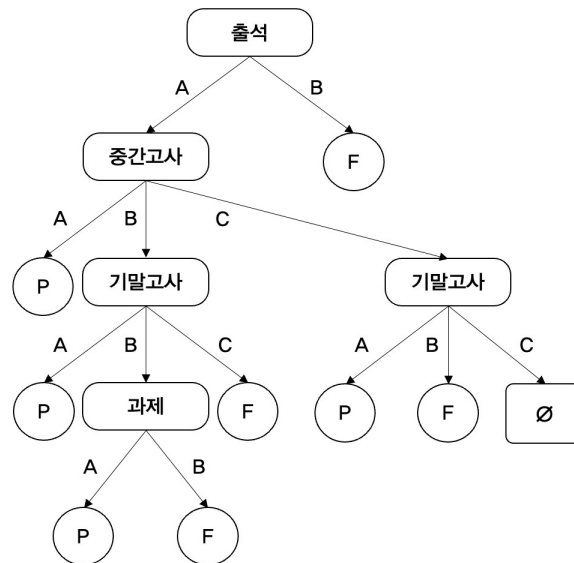
[표 10] 출석 A, 중간고사 B, 기말고사 B인 집합 D

번호	출석 (A/B)	과제 (A/B)	중간고사 (A/B/C)	기말고사 (A/B/C)	학점 (P/F)
6	A	A	B	B	P
15	A	B	B	B	F

이 데이터 집합은 이미 출석, 중간고사, 기말고사의 속성으로 평가가 이루어진 데이터이다. 따라서 마지막 하나의 속성만 남은 상태로 정보이득을 계산할 필요 없이 곧바로 과제 속성으로 분할을 진행한다. 그림 12는 과제 속성으로 분할한 결과이다. 이 분할을 통해 생성된 두 개의 집합 역시 하나의 데이터로 이루어졌기 때문에 분할 종료 조건 1에 해당한다. 따라서 각 집합을 단말노드로 지정하고 각 데이터의 레이블을 할당한다. 그림 13은 분할 종료 조건에 따라 단말노드로 지정한 결과를 보여준다.



[그림 12] 과제 속성으로 분할한 결과



[그림 13] 분할 종료 조건에 따라 단말노드 지정

6.4 공집합 처리 - 분할 종료 조건 2

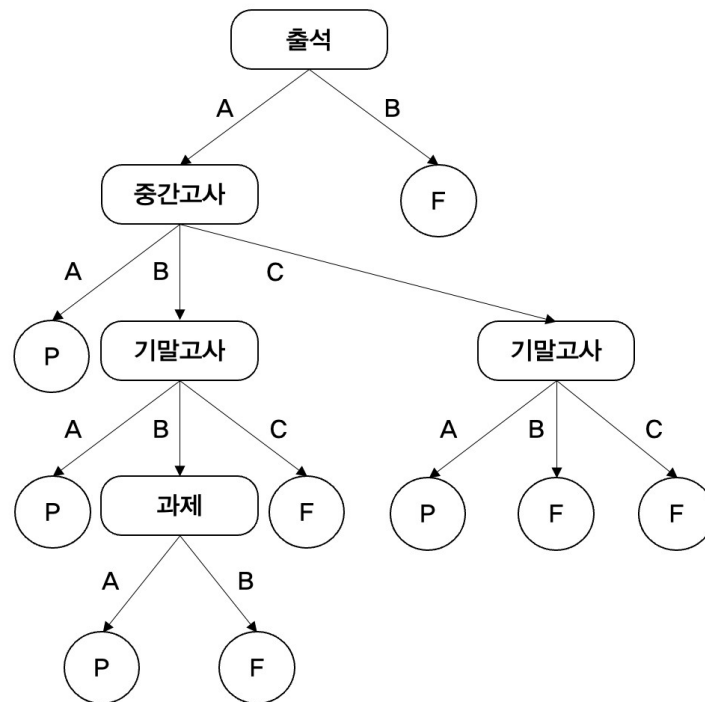
데이터 집합을 연속적으로 분할하다보면 가끔 공집합이 나타나는 경우가 있다. 그림 13에서 출석이 A, 중간고사가 C이고 기말고사가 C인 경우에 해당하는 데이터가 없는 것을 확인할 수 있다. 이런 경우 추가적인 분할은 당연히 불가능하므로 자동적으로 분할이 종료된다. 따라서 해당 노드는 단말노드가 된다. 이때 단말노드의 레이블 할당을 어떻게 할 것인가의 문제가 생길 수 있는데, 이는 설계자가 나름의 규칙을 가지고 정하면 된다. 그림 13의 경우 기말고사 성적이 B인 경우 F로 분류하였으므로 C인 경우에도 F로 분류하는 것이 적절할 것으로 보인다. 이렇게 명확한 판단이 비교적 간단한 경우에는 그 결과를 따르면 되고, 그렇지 않을 경우

에는 무작위적으로 할당하는 것도 가능하다.

분할 종료 조건 2: 위의 예와 같이 분할된 집합이 공집합인 경우 분할을 종료하고 단말노드로 지정한다. 이 조건을 분할 종료 조건 2로 명명한다.

6.5 의사결정 트리의 완성

그림 14는 표 4의 데이터를 이용해 완성한 의사결정 트리이다. 뿌리노드는 출석이고 단말노드는 총 9개이다. 따라서 뿌리노드와 각각의 단말노드를 연결하는 총 9개의 경로가 존재하며 각 경로는 개별적인 분류 규칙이 된다. 표 11은 9개의 분류 규칙을 정리한 것이다.



[그림 14] 완성된 의사결정 트리

[표 11] 그림 14의 분류 규칙

번호	분류 규칙	분류 결과
1	(출석=B)	F
2	(출석=A) & (중간고사=A)	P
3	(출석=A) & (중간고사=B) & (기말고사=A)	P
4	(출석=A) & (중간고사=B) & (기말고사=B) & (과제=A)	P
5	(출석=A) & (중간고사=B) & (기말고사=B) & (과제=B)	F
6	(출석=A) & (중간고사=B) & (기말고사=C)	F
7	(출석=A) & (중간고사=C) & (기말고사=A)	P
8	(출석=A) & (중간고사=C) & (기말고사=B)	F
9	(출석=A) & (중간고사=C) & (기말고사=C)	F

7. 가지치기 (가지 잘라내기) - 트리의 경량화 및 과적합 방지

위에서 상세히 기술한 의사결정 트리 생성 과정을 보면 그림 14의 의사결정 트리가 철저히 표 4의 데이터에 충실하도록 만들어졌다는 것을 알 수 있다. 다시 말해, 그림 14의 트리에 표 4의 데이터를 적용해보면 예측 정확도는 100%이다. 주어진 데이터로 최선을 다해 학습하여 정확도 100%를 달성한 것은 칭찬받을 일이지만, 이런 경우 머신러닝 분야에서는 과적합의 우려도 함께 커진다. 즉, 훈련 데이터에서만 우수한 성능을 보이고 실전 데이터 또는 훈련 과정에서 경험해보지 못한 데이터에 대한 성능, 즉 일반화 성능은 떨어지는 현상이 발생할 수 있다. 이러한 과적합은 머신러닝에서 항상 경계해야하는 위험이다.

또한 과도하게 복잡한 트리 구조는 구현했을 때 연산량의 급격한 증가로 이어질 수 있다. 과적합은 머신러닝 모델의 예측 성능에 대한 문제이지만 연산량은 모델의 구현 가능성에 대한 문제라 할 수 있다.

의사결정 트리에서 과적합이 발생하고 연산량 증가하는 주된 이유는 트리의 구조가 지나치게 복잡하고 세밀하기 때문이다. 따라서 트리를 덜 복잡하게 만드는 것이 이 두 가지 문제의 해법이 될 수 있다. 머신러닝에서 모델의 구조를 단순화하여 과적합과 연산량 증가의 문제를 해결하는 접근법을 모델의 경량화(regularization)라 한다. 의사결정 트리에 적용 가능한 경량화 기법은 가지치기(pruning) 또는 가지 잘라내기이다.

가지치기는 말 그대로 필요에 따라 나무의 가지를 잘라내는 것을 의미한다. 그림 14의 트리 구조를 보면 뿌리노드 또는 내부노드들로부터 가지가 뻗어 전체 구조를 형성한다. 문제는 이 가지들이 너무 많다는 것이고, 이를 개선하기 위해 일부 가지를 잘라내 구조를 단순하게 만드는 과정을 가지치기라 한다.

가지치기는 시점에 따라 사전 가지치기와 사후 가지치기로 나눌 수 있다. 사전 가지치기는 의사결정 트리를 만들어가는 과정에 가지치기를 동시에 진행하는 것이고 사후 가지치기는 의사결정 트리를 먼저 완성한 뒤 가지치기를 수행하는 것을 말한다. 어떤 시점에 가지치기를 수행하건 가지치기는 과적합의 방지, 즉 일반화 성능의 개선이라는 일관된 목표를 갖는다.

7.1 검증 데이터의 준비

가지치기를 위해서는 일반화 성능의 예측이 필요하다. 일반화 성능이란 실제 데이터, 즉 훈련에 사용되지 않은 데이터에 대한 모델의 성능을 의미한다. 그러나 학습의 과정에서 실제 데이터를 확보하는 것은 불가능하므로 주어진 데이터 집합을 훈련 데이터와 검증 데이터로 나누고 검증 데이터를 실제 데이터로 간주하여 일반화 성능을 추정하는 기법을 알고 있을 것이다. 이때 데이터 집합을 훈련 데이터와 검증 데이터로 나누어 일반화 성능을 예측하는 방법으로 홀드아웃 검증, 교차 검증, 부트스트래핑 등이 있다. 각각에 대한 세부적인 내용은 여기에서 언급하지 않는다. 다만, 일반화 성능 예측을 위한 검증 데이터 집합이 준비되었다고 가정하고 설명을 진행한다. 표 12는 가지치기 과정을 설명하기 위해 준비한 검증 데이터 집합이다. 우리가 가지고 있는 표 4의 데이터를 훈련 데이터와 검증 데이터로 나누기에는 너무 부족하여 원활한 설명을 위해 추가로 15개의 데이터를 준비했다. 표 12의 검증 데이터를 그림 14의 의사결정 트리에 적용하면 80%의 정확도를 얻을 수 있다. 참고로 표 12에 중복되는 데이터가 있다. 일반적으로 등급으로 표시된 성적 데이터의 중복은 자연스러운 현상이다. 따라서 데이터 집합의 구성도 실제 상황을 제대로 반영하도록 구성해야 한다.

[표 12] 검증 데이터 집합

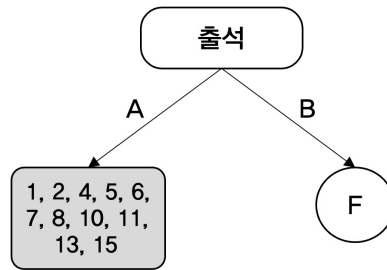
번호	출석 (A/B)	과제 (A/B)	중간고사 (A/B/C)	기말고사 (A/B/C)	학점 (P/F)
1	A	B	B	C	F
2	A	B	A	C	P
3	A	A	B	A	P
4	A	B	A	C	P
5	A	A	A	A	P
6	A	A	C	A	P
7	A	A	A	A	P
8	A	A	B	C	P
9	A	B	A	B	F
10	A	A	B	B	P
11	A	B	C	B	F
12	A	B	B	B	P
13	B	B	C	A	F
14	B	A	B	A	F
15	A	B	C	B	F

7.2 사전 가지치기

사전 가지치기는 트리를 만드는 과정에 가지치기를 함께 수행하는 방식이다. 앞에서 상세히 설명한 바와 같이 트리를 만드는 과정은 집합을 분할해가는 과정이며 분할 종료 조건을 만나기 전에는 분할이 계속된다. 즉, 더 이상 분할할 수 없는 경우(분할 종료 조건)를 제외하고는 무조건 분할하는 것이 기존의 의사결정 트리 생성법이다.

어떤 집합이 분할되면 새로운 부분집합들이 생성되고 그에 따라 가지들이 만들어진다. 사전 가지치기는 집합을 분할했을 때의 일반화 성능과 분할하지 않고 해당 노드를 단말노드로 바꾸었을 때의 일반화 성능을 측정하여 분할이 유리할 경우에만 분할을 진행한다. 즉, 사전 가지치기는 있는 가지를 잘라내는 것이 아니라 새로운 가지의 생성을 막는 방식이다. 명확하게 정리하자면, 사전 가지치기를 한다는 것은 분할을 종료하는 것이고, 반대로 사전 가지치기를 하지 않는다는 것은 분할을 진행한다는 것을 의미한다.

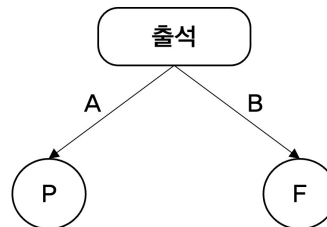
이제 표 4의 훈련 데이터, 표 12의 검증 데이터를 가지고 사전 가지치기의 예를 살펴보자. 그림 6과 같이 뿌리노드로 출석을 지정하여 첫 번째 분할을 진행하였다. 설명의 편의를 위해 그림 6을 그림 15에 다시 표시하였다.



[그림 15] 음영으로 표시된 집합의 분할 여부를 결정해야 함

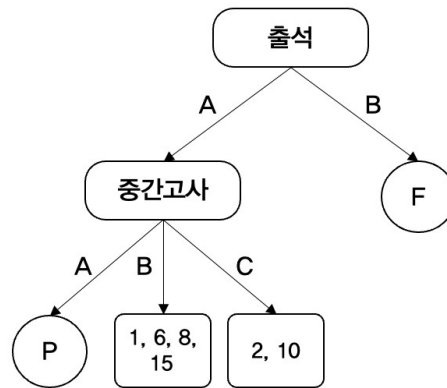
사전 가지치기를 하지 않는다면 그림 15에서 출석이 A인 집합을 일반적인 방법으로 분할하면 된다. 그러나 분할을 진행하기 전에 이 집합을 분할하는 것이 이익인지 확인해야 한다. 이를 위해 집합을 분할하지 않고 단말노드로 지정한 경우와 분할을 진행한 경우의 의사결정 트리를 만들어야 한다.

집합을 분할하지 않고 단말노드로 지정할 경우, 레이블은 데이터들이 가진 레이블 중 다수의 레이블을 할당한다. 그림 15에서 음영으로 표시된 집합에는 11개의 데이터 중 8개의 데이터가 P를 가지고 있으므로 레이블 P를 할당할 수 있다. 그림 16은 해당 집합을 단말노드로 바꾸고 레이블 P를 할당한 결과이다. 즉, 집합의 분할을 억제하여 가지 생성을 막았으므로 사전 가지치기가 적용된 결과라 할 수 있다. 앞서 설명한 것처럼, 그림 16과 같은 형태를 의사결정 그루터기라 한다.



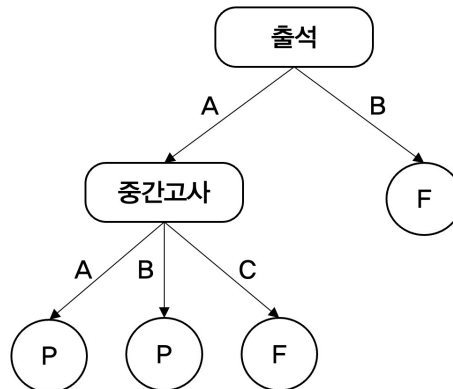
[그림 16] 그림 15에 가지치기를 적용한 의사결정 트리

다음으로 분할을 한 단계 진행해보자. 그림 15에서 출석이 A인 집합을 분할하면 그림 16과 같다. 여기에서 우리에게 중요한 것은 그림 15의 부분집합을 분할했을 때와 하지 않았을 때의 성능 비교이므로, 그림 16의 마지막 두 부분집합을 단말노드로 지정해야 한다. 그러나 두 부분집합 모두 같은 비율의 P와 F 레이블로 구성되어 있다. 이런 경우 공집합의 경우와 같이 설계자의 의도에 따라 적절히 처리하면 된다. 본 예에서는 중간고사가 B인 경우 P, 중간고사가 C인 경우 F를 할당한다.



[그림 17] 그림 15에서 한 단계 분할을 진행한 결과

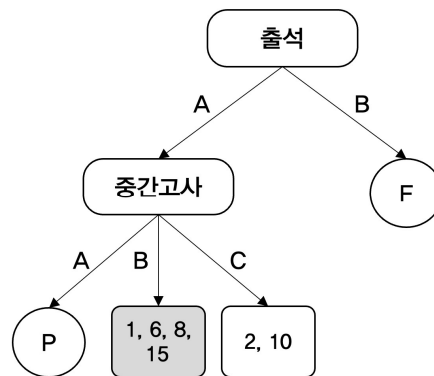
그림 18은 분할을 진행하고 단말노드를 지정한 결과이다.



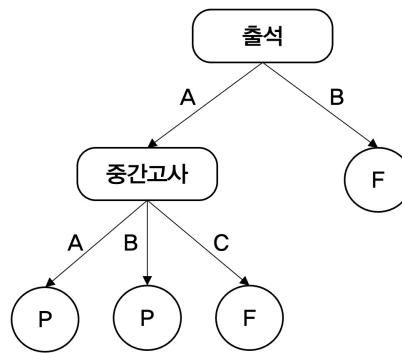
[그림 18] 분할을 진행하고 단말노드를 지정한 결과

이제 그림 16과 그림 18의 성능을 비교한다. 비교 지표는 일반화 성능이며 이는 표 12의 검증 데이터를 이용해 측정한다. 검증 데이터를 그림 16의 트리에 적용하면 정확도 73.3%를 얻을 수 있으며 그림 18의 트리에 적용하면 정확도 80%를 얻을 수 있다. 이는 그림 15의 상태에서 분할을 진행하는 것이 이득이라는 것을 의미한다. 즉, 가지치기를 하지 않는 것이 이익인 상황이다. 따라서 그림 15의 집합을 분할한다.

그림 15에서 분할을 한 단계 진행하면 그림 19와 같다. 그림 19에서 두 개의 부분집합 {1, 6, 8}과 {2, 10}을 확인할 수 있다. 이 두 집합 중 첫 번째 집합을 분할할지 여부를 정하기 위해 두 번째 집합은 단말노드로 두고, 첫 번째 집합을 분할한 경우와 분할하지 않은 경우의 의사결정 트리를 만든다. 그림 20은 그림 19의 집합을 분할하지 않은 경우, 즉 가지치기를 적용한 의사결정 트리이다.

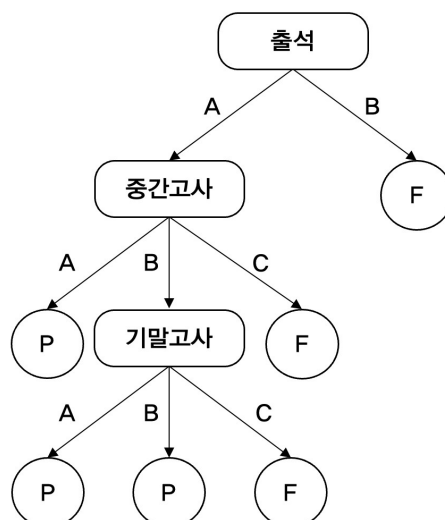


[그림 19] 그림 15에서 한 단계 분할 진행한 의사결정 트리



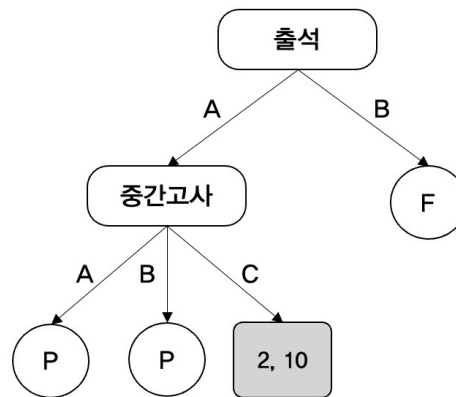
[그림 20] 그림 19에 가지치기를 적용한 의사결정 트리

그림 21은 그림 19에서 음영으로 표시한 집합의 분할을 한 단계 진행한 결과이다. 분할의 결과는 그림 9와 같은데 일반화 성능 측정을 위해 그림 9의 각 집합을 단말노드로 지정하고 레이블을 할당한 결과가 그림 19이다.



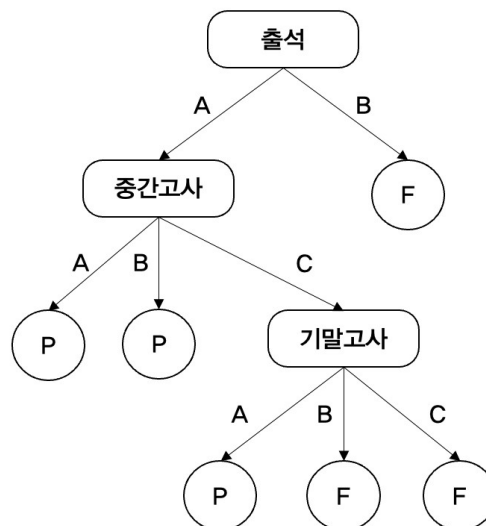
[그림 21] 그림 19에 분할을 진행한 의사결정 트리

이제 표 12의 검증 데이터를 이용해 그림 20과 21의 정확도를 측정한다. 측정 결과 두 경우 모두 80%의 정확도를 보인다. 이렇게 분할을 하거나 하지 않거나 성능의 차이가 없는 경우에는 분할을 진행하지 않는다. 즉, 가지치기를 하는 것이 유리하다. 따라서 그림 19의 음영 표시된 집합은 더 이상 분할을 진행하지 않고 단말노드로 지정한다. 그 결과 그림 22와 같은 트리가 되고, 이제 그림 22에서 음영으로 표시한 집합을 분할할지 아니면 단말노드로 지정하고 분할을 종료할지 결정한다.



[그림 22] 그림 19에 가지치기를 적용한 결과

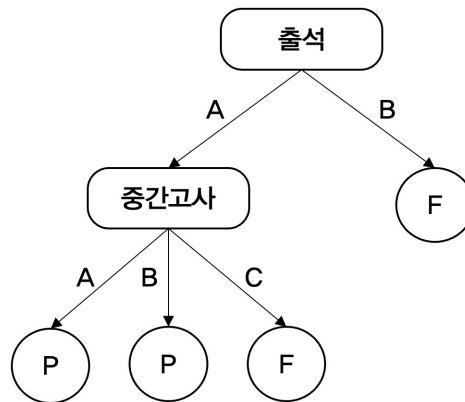
그림 22에서 음영으로 표시한 집합 {2, 10}을 단말노드로 지정하면 그림 20과 같다. 그리고 분할을 진행하면 그림 10을 참고하여 그림 23과 같다.



[그림 23] 그림 22의 집합을 분할한 결과

이제 가지치기 여부를 결정하기 위해 검증 데이터를 그림 20과 그림 23의 의사결정 트리에 넣고 정확도를 측정한다. 측정 결과 그림 20과 23의 두 경우 모두 80%의 정확도를 보였다. 따라서 추가 분할이 도움이 되지 않는 상황이므로 가지치기를 통해 분할을 종료한다. 따라서

최종 의사결정 트리는 그림 24와 같다.



[그림 24] 사전 가지치기를 적용한 최종 의사결정 트리

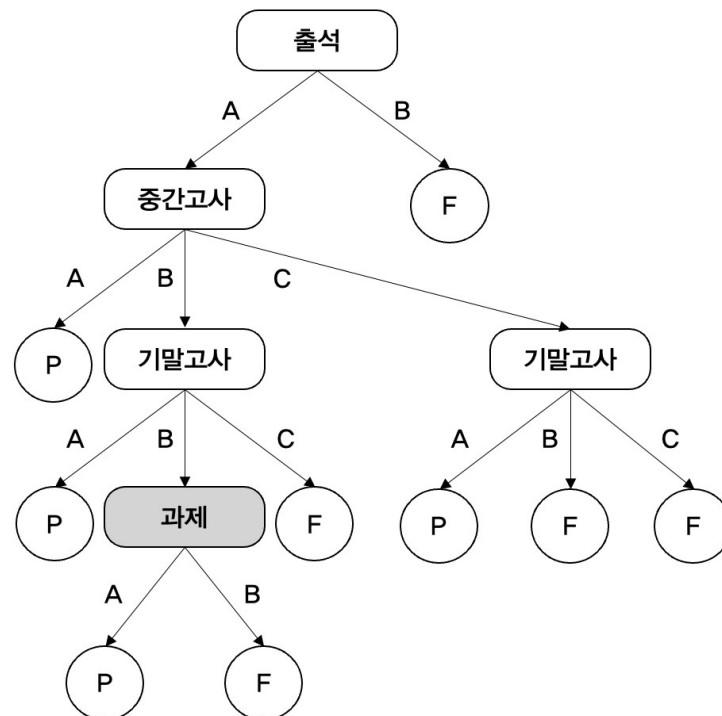
그림 14와 24를 비교해보자. 그림 14는 사전 가지치기를 하지 않은 의사결정 트리이고 24는 사전 가지치기를 적용한 의사결정 트리이다. 사전 가지치기를 통해 트리의 규모가 매우 작아졌음을 한 눈에 알 수 있다. 사전 가지치기는 기본적으로 가지가 뻗어나가는 것을 억제하는 속성을 가지고 있기 때문에 일반적으로 작은 규모의 트리가 생성된다. 이는 사전 가지치기가 트리의 경량화에 매우 효과적임을 의미한다. 한편, 가지 생성을 사전에 억제하는 속성은 과소적합(underfitting)의 위험성을 함께 높이는 결과를 초래할 수 있다.

7.3 사후 가지치기

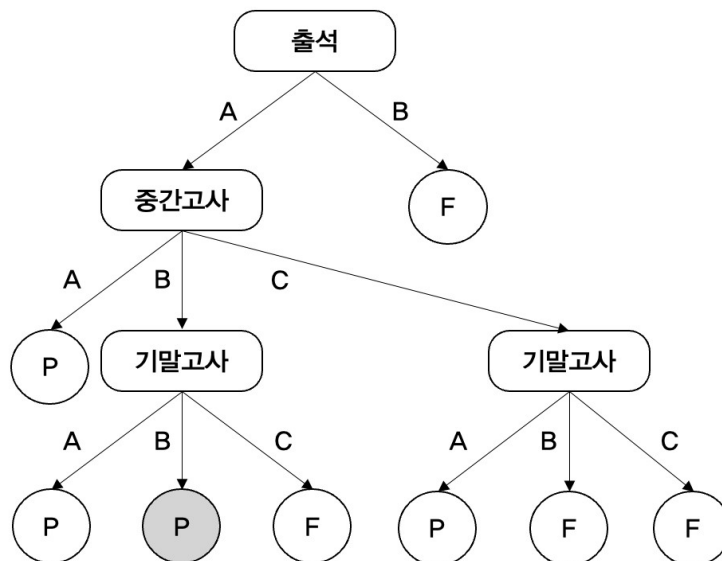
사후 가지치기는 사전 가지치기와 달리, 의사결정 트리를 먼저 완성한 뒤 내부노드의 가지를 잘라냈을 경우와 그렇지 않은 경우의 일반화 성능을 측정하여 이익이 있는 방향으로 가지를 치는 방식을 의미한다. 가지치기의 대상 노드는 뿌리노드로부터 먼 내부노드를 차례로 검토한다.

먼저 그림 14의 완성된 의사결정 트리를 가져와 살펴보자. 이를 그림 25로 다시 표시하였다. 그림 25에서 뿌리노드로부터 가장 먼 내부노드는 음영으로 표시한 과제 노드이다. 과제 노드에서는 두 개의 가지, 즉 A와 B의 값을 갖는 가지가 뻗어 나간다. 사후 가지치기를 위해 가장 먼저 이 노드의 가지를 잘라내는 것이 좋을지를 판단한다. 이 가지들을 잘라낸다는 것은 과제 노드를 단말노드로 바꾼다는 것을 의미한다. 과제 노드를 단말노드로 바꾸기 위해 그림 11과 표 10을 참고하면 그림 26과 같다.

검증 데이터를 이용해 그림 25와 26의 일반화 성능을 측정한 결과 그림 25는 80%, 그림 26은 87%의 정확도를 보였다. 이는 과제 속성으로 분할하는 것보다 P로 결정하고 종료하는 것이 일반화 성능 측면에서는 이익이라는 것을 의미한다. 따라서 그림 25에서 과제 노드의 가지를 잘라내고 과제 노드는 단말노드로 바꾼다.



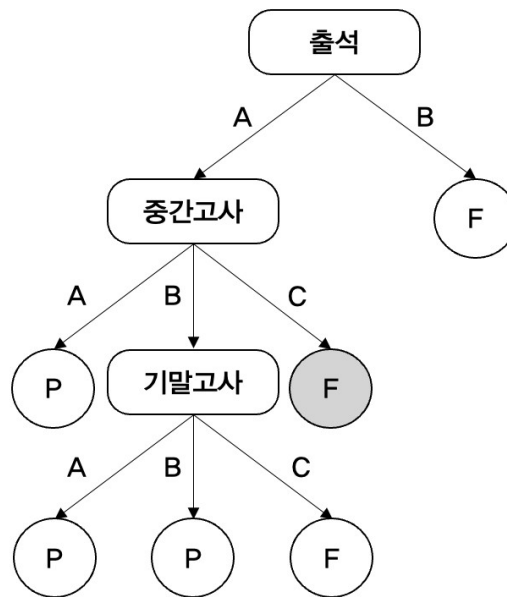
[그림 25] 완성된 의사결정 트리 (가지치기를 하지 않은 상태)



[그림 26] 과제 노드를 가지치기한 트리

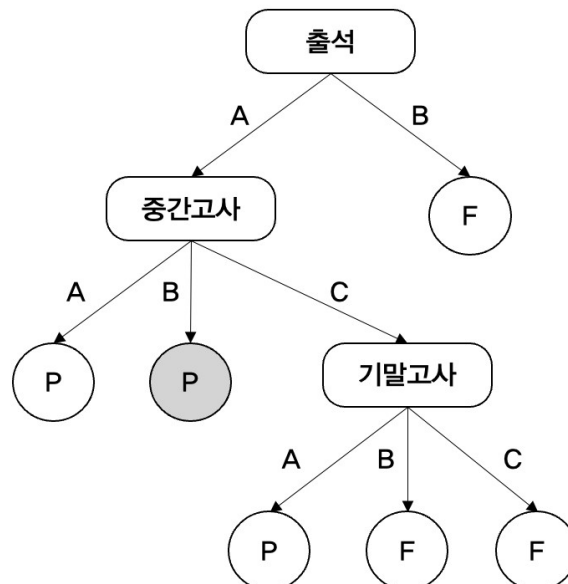
그림 26에서 다음으로 가지치기를 고려할 노드는 출석이 A, 중간고사가 C일 때 나타나는 기말고사 노드이다. 기말고사 노드를 가지치기한 경우 그림 9와 표 9를 참고하면 그림 27과 같이 나타낼 수 있다. 이제 검증 데이터를 이용해 그림 26과 27의 예측 성능을 비교해보자. 그림 26의 경우 87%의 정확도를 보이는 반면 그림 27은 80%의 정확도를 보인다. 이 경우는 가지치기를 하는 것 보다 그림 26과 같이 가지를 그대로 두는 것이 유리한 상황이다. 따라서

가지치기를 수행하지 않는다.



[그림 27] 가지치기를 적용한 결과 (음영으로 표시된 노드)

현재까지 가지치기를 수행한 결과는 그림 26이다. 그림 27은 성능이 좋지 않아 폐기된 모델이다. 다음으로 검토할 노드는 출석이 A, 중간고사가 B인 경우 만나는 기말고사 노드이다. 이 노드에 대해 가지치기를 한다면 그림 28과 같이 된다.

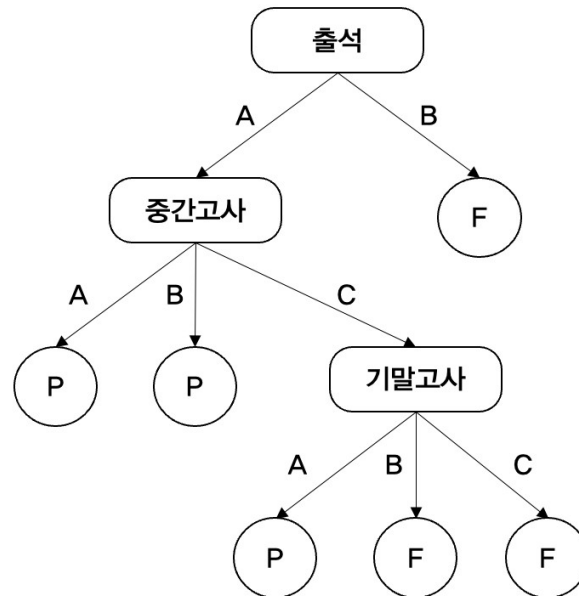


[그림 28] 가지치기를 적용한 결과 (음영으로 표시한 노드)

여기에서 비교 대상은 그림 26과 28이다. 그림 26의 예측 정확도는 87%였다. 같은 검증

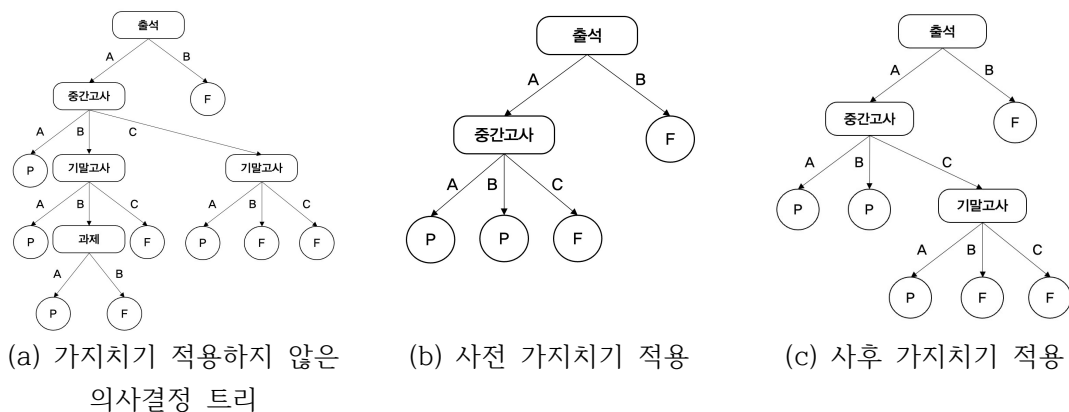
데이터로 그림 28의 예측 정확도를 측정한 결과 동일하게 87%를 얻었다. 같은 정확도일 때에는 간단한 구조를 선택한다. 따라서 가지치기를 적용하여 그림 28을 선택한다.

마지막으로 검토할 노드는 출석이 A일 때 만나는 중간고사 노드인데, 이 노드를 단말노드로 변환하면 그림 16과 같아지고, 그림 16의 정확도는 73.3%라는 것을 이미 측정했다. 따라서 그림 28이 갖는 87%의 정확도보다 훨씬 작은 값이므로 가지치기를 중단하고 그림 29와 같은 최종 의사결정 트리를 얻을 수 있다.



[그림 29] 사후 가지치기를 적용한 최종 의사결정 트리

사후 가지치기는 일단 의사결정 트리를 모두 완성한 뒤, 뿌리에서 먼 내부노드를 차례로 검토하여 일반화 성능이 높아지는 방향으로 가지치기를 진행하는 방식이므로 사전 가지치기에 비해 연산량이 많을 수 있으나 과적합과 과소적합의 위험을 동시에 완화할 수 있는 기법이다. 그림 30은 가지치기의 방식에 따른 의사결정 트리의 변화를 비교한 것이다.



[그림 30] 가지치기 결과 비교

8. 연속 데이터의 전처리(pre-processing)

의사결정 트리는 기본적으로 어떤 속성이 가질 수 있는 값이 이산(discrete)인 데이터의 판단에 적합하다. 어떤 변수의 값이 이산이라는 것은, 그 변수가 가질 수 있는 값의 종류가 유한하다는 것을 의미한다. 반대로 무수히 많은 종류의 값을 가질 수 있는 변수를 연속(continuous) 변수라 한다.

위에서 사용한 표 4 또는 표 12의 데이터를 보면, 모든 입력 속성이 이산변수라는 것을 알 수 있다. 즉, 출석과 과제는 A 또는 B의 두 가지 값만 가질 수 있고 중간고사와 기말고사는 A, B, C 중 하나의 값을 갖는다. 이는 의사결정 트리의 특정 노드에서 뻗어 나가는 가지의 수와 같다.

그런데, 실제로는 이산인 속성 뿐 아니라 연속인 속성도 빈번하게 사용된다. 중간고사 또는 기말고사 속성도 0과 100 사이의 실수값으로 측정되는 경우가 더 많을 것이다. 또한 온도, 습도, 질량, 밀도, 속도 등 많은 데이터에서 빈번하게 사용되는 물리량 역시 임의의 실수값을 갖는 연속 데이터이다. 이러한 연속 데이터를 의사결정 트리에 적용하기 위해서는 연속 변수를 이산 변수로 변환하는 전처리 과정이 필요하다. 예를 들어 중간고사 성적이 60점 이상인 경우 A, 30점 이상 60점 미만인 경우 B, 30점 미만인 경우 C와 같은 방법으로 데이터가 가질 수 있는 값의 구간을 나누고 각 구간을 대표하는 값으로 이산화하는 과정을 통해 데이터를 가공하고 가공된 데이터를 이용해 의사결정 트리를 구성해야 한다.

당연한 말이지만 연속 데이터의 이산화는 의사결정 트리를 만들 때 최대한 유리한 방향으로 진행해야 한다. 예를 들어 0점에서 100점까지 다양한 값을 갖는 중간고사 성적이 있을 때, 몇 점 이상을 A로 하는 것이 가장 좋을지를 생각해보자. 그림 7과 그림 8이 하나의 힌트가 될 수 있다. 그림 7에서 중간고사가 A인 데이터들은 모두 같은 레이블 P를 가지고 있다. 그래서 해당 노드는 손쉽게 P라는 단말노드로 지정이 가능했다. 또한 중간고사가 A인 데이터들이 모두 같은 레이블을 가지고 있었기 때문에 중간고사로 분할했을 때 정보이득도 커진다. 즉, 연속인 중간고사 속성을 이산적인 속성으로 변환할 때, 같은 속성값을 갖는 데이터들이 최대한 같은 레이블을 갖도록 이산화하면 의사결정 트리를 만드는 과정에서 큰 정보이득과 손쉬운 단말노드 할당이 가능해진다는 것을 알 수 있다.

이렇게 같은 종류의 레이블을 갖는 데이터들이 최대한 같은 집합으로 묶는다는 것은 집합의 복잡도(엔트로피)를 최대한 낮춘다는 것이고, 이는 하나의 집합을 여러 집합으로 나눌 때 얻을 수 있는 정보이득이 가장 크게 집합을 나눈다는 것과 같다. 정보이득의 계산은 앞에서 이미 설명하였으므로 한 가지 예를 살펴보고 마무리한다.

표 13은 학생 15명의 기말고사 성적과 학점을 기록한 데이터 집합이다. 지금까지의 예와 달리 기말고사 속성이 가질 수 있는 값은 $[0, 100]$ 범위의 임의의 실수이다. 따라서 기말고사 속성은 연속 변수이다. 목표는 이를 이산화하여 A, B, C 중 하나의 값을 갖도록 처리하는 것이다.

[표 13] 연속 속성 데이터

번호	기말고사	학점	번호	기말고사	학점
1	30.7	P	9	4.3	P
2	44.6	F	10	31.8	F
3	14.2	F	11	2.1	F
4	63.4	P	12	61.8	P
5	25.8	P	13	44.5	P
6	67.3	P	14	3.4	F
7	64.8	P	15	86.9	P
8	96	F			

연속인 기말고사 속성을 이산화한다는 것은 결국, 표 13의 데이터를 기말고사 성적에 따라 A, B, C 세 부분집합으로 분할하는 것을 의미한다. 원칙은 분할했을 때 얻을 수 있는 정보이득을 최대화하는 것이다. 즉, 최대한 같은 종류의 레이블(학점)이 같은 집합에 모이도록 나누는 것이다.

8.1 분할 기준 후보

표 13의 기말고사 값을 세 그룹으로 나누기 위한 분할 기준이 필요하다. 이를 위해 먼저 분할 기준의 후보를 정한다. 이를 위해 데이터를 오름차순으로 정렬하고, 인접한 두 데이터의 평균값을 구한다. 표 14는 기말고사 성적을 기준으로 오름차순으로 정렬한 뒤 인접한 두 성적의 평균을 구해 분할 기준 후보로 정한 것이다. 예를 들어 20점은 14.2점과 25.8점의 평균이며, 만약 이 값을 분할 기준으로 정한다면 기말고사 값은 20점을 기준으로 두 그룹으로 나눌 수 있게 되는 것이다.

[표 14] 분할 기준 후보

번호	기말고사	학점	분할 기준 후보
11	2.1	F	
14	3.4	F	2.75
9	4.3	P	3.85
3	14.2	F	9.25
5	25.8	P	20
1	30.7	P	28.25
10	31.8	F	31.25
13	44.5	P	38.15
2	44.6	F	44.55
12	61.8	P	53.2
4	63.4	P	62.6
7	64.8	P	64.1
6	67.3	P	66.05
15	86.9	P	77.1
8	96	F	91.45

8.2 분할 기준 선택 및 정보이득 계산

데이터의 개수가 15개이므로 분할 기준 후보는 14개이다. 그리고 세 개의 그룹으로 나누어야 하기 때문에 14개 중 두 개의 후보를 선택하여 세 그룹으로 나눈 뒤 정보 이득을 계산한다. 그런데 14개 중에서 두 개를 선택할 수 있는 조합의 수는 총 91개이다. 이는 91번의 분할과 정보이득 계산이 필요하다는 것을 의미한다. 이를 모두 수작업으로 계산하는 것은 거의 불가능하지만, 프로그래밍을 이용해 자동화하면 어렵지 않게 계산할 수 있다.

계산 결과 53.2점과 91.45점을 기준으로 분할하는 것이 최적인 것으로 나타났다. 즉 53.2점보다 낮은 C 그룹, 53.2점과 91.45점 사이의 B 그룹, 91.45점 이상의 A 그룹으로 나누는 것이다. 이렇게 분할하는 경우의 정보이득은 다음과 같다.

$$D_A = \{8\}$$

$$D_B = \{4, 6, 7, 12, 15\}$$

$$D_C = \{1, 2, 3, 5, 9, 10, 11, 13, 14\}$$

각 부분집합의 정보 엔트로피는 다음과 같이 계산할 수 있다.

$$I_{D_A} = -\frac{0}{1}\log_2\frac{0}{1} - \frac{1}{1}\log_2\frac{1}{1} = 0$$

$$I_{D_B} = -\frac{5}{5}\log_2\frac{5}{5} - \frac{0}{5}\log_2\frac{0}{5} = 0$$

$$I_{D_C} = -\frac{4}{9}\log_2\frac{4}{9} - \frac{5}{9}\log_2\frac{5}{9} = 0.991$$

전체 집합의 정보 엔트로피는 0.971이다. 따라서 이 분할로 얻을 수 있는 정보이득은 다음과 같다.

$$G(D) = 0.971 - \left(\frac{1}{15} \times 0 + \frac{5}{15} \times 0 + \frac{9}{15} \times 0.991 \right) = 0.376$$

선택된 분할 기준에 의하면, A그룹과 B그룹이 모두 동일한 레이블의 데이터로 구성된 것을 확인할 수 있다. 즉, 분할을 통해 데이터의 복잡도가 가장 낮아지는 기준을 선택한 것이다. 이는 앞서 설명한 것과 같이 의사결정 트리를 만들었을 때 정보이득을 크게하고 손쉽게 단말노드를 지정할 수 있도록 한다.

8.3 연속 데이터의 이산화

위의 방법으로 분할 기준을 53.2점과 91.45점으로 정하였다. 이를 이용해 중간고사 성적을 이산화하는 전처리를 완료한 데이터 집합은 표 15와 같다.

[표 15] 이산화 완료된 데이터

번호	중간고사(원점수)	중간고사(전처리후)	학점
1	30.7	C	P
2	44.6	C	F
3	14.2	C	F
4	63.4	B	P
5	25.8	C	P
6	67.3	B	P
7	64.8	B	P
8	96	A	F
9	4.3	C	P
10	31.8	C	F
11	2.1	C	F
12	61.8	B	P
13	44.5	C	P
14	3.4	C	F
15	86.9	B	P

이상에서 정보이득을 이용한 연속 변수의 이산화 기법을 소개하였다. 이를 이용해 레이블 정보가 주어진 상황에서 연속적인 입력 속성을 의사결정 트리에 적용하기 위한 최적 이산화가 가능하며 이를 통해 몇 단계로 양자화할지는 설계자의 몫이라 할 수 있다.

6. 클러스터링 (Clustering)

1. 데이터에 내재된 규칙 찾기 - 비지도 학습

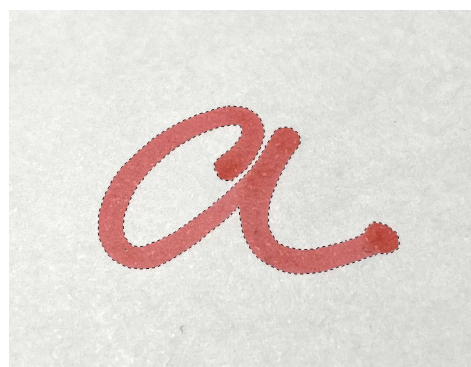
지도학습과 비지도 학습의 가장 큰 차이는 학습 데이터에 레이블 정보가 있는지 없는지 여부이다. 즉, 지도학습을 위한 데이터에는 입력 속성과 함께 레이블 정보가 제공되는 반면 비지도 학습을 위한 데이터는 레이블 정보 없이 입력 속성만으로 구성된다. 이 차이는 지도학습과 비지도 학습이 할 수 있는 일의 차이를 만들어낸다. 지도학습은 데이터의 학습을 통해 입력 속성과 출력 레이블 사이의 함수 관계를 찾는 반면, 비지도 학습은 입력 속성만으로 구성된 데이터에 내재된 어떤 규칙을 찾는다.

데이터에 내재된 규칙은 데이터의 구성 특징이라고 표현할 수 있는데, 데이터의 밀도 또는 구조 등이 데이터에 내재된 규칙이라 할 수 있다. 그 중 대표적인 것은 “비슷한 종류의 데이터가 얼마나 있으며, 어떤 데이터들이 서로 비슷한 종류에 속하는가?”라는 질문에 대한 답이다. 이 답을 찾게 되면, 전체 데이터 집합을 몇 개의 부분집합으로 나눌 수 있다. 이때 같은 부분집합에 속한 데이터들 사이에는 높은 유사성을 갖게 되고 서로 다른 부분집합의 데이터들 사이에는 낮은 유사성을 갖게 된다. 이러한 부분집합을 클러스터(cluster) 또는 군집이라고 부르고, 전체 데이터 집합을 다수의 클러스터로 나누는 과정을 클러스터링(clustering) 또는 군집화라고 한다. 클러스터링은 비지도 학습의 대표적인 응용 기술로 다양한 분야에서 활용되고 있다.

그림 1은 2차원 이미지 데이터를 이용한 클러스터링의 예를 보여준다. 그림 1(a)는 소문자 a의 이미지이다. 이 이미지는 가로 1340 픽셀, 세로 1034 픽셀로 구성되었으며 각 픽셀마다 R/G/B의 값이 할당된다. 따라서 데이터 집합은 세 개의 입력 속성을 갖는 1,385,560개의 데이터로 구성된다. 그림 1(b)는 이 데이터 집합에서 유사도가 높은 데이터를 찾아 클러스터를 구성한 결과를 보여준다. 그림 1의 예는 클러스터링의 대표적인 활용 사례라 할 수 있다.



(a) 원본 이미지



(b) 유사도가 높은 픽셀을 군집화한 결과

[그림 1] 2차원 이미지 데이터를 이용한 군집화의 예

위의 간략한 설명에도 몇 차례 등장했듯이, 클러스터링은 기본적으로 데이터 사이의 유사성을 중요하게 다룬다. 따라서 데이터 사이의 ‘유사성’을 측정하는 것은 클러스터링의 첫 걸음이라 할 수 있다.

2. 데이터 사이의 거리 - 유사성의 척도

머신러닝을 위한 모든 데이터는 적절한 전처리 과정을 통해 수치화된다. 그리고 수치화된 데이터들은 입력속성으로 구성된 속성 공간상의 좌표 또는 벡터로 표현할 수 있다. 이때, 우리는 서로 다른 벡터들 사이의 거리가 가까울수록 두 벡터가 유사할 것이라고 생각할 수 있다. 따라서 속성 공간에서 벡터들 사이의 거리를 유사성으로 척도로 삼는 것은 매우 합리적인 선택이다. 즉, 두 벡터 사이의 거리가 가까울수록 유사성은 높아진다.

n 차원 벡터 공간에서 임의의 벡터 $\mathbf{x} = (x_0, x_1, \dots, x_{(n-1)})$ 의 크기는 다음과 같이 정의된다.

$$\|\mathbf{x}\|_p = (|x_0|^p + |x_1|^p + \dots + |x_{(n-1)}|^p)^{\frac{1}{p}} \quad (1)$$

식 (1)을 p -norm 또는 Lp -norm이라 한다. norm은 ‘놈’이라고 읽으면 된다. 이때 p 는 1 이상의 정수값을 갖는 파라미터이다.

만약 p 가 2라면 식 (1)은 다음과 같다.

$$\|\mathbf{x}\|_2 = \sqrt{|x_0|^2 + |x_1|^2 + \dots + |x_{(n-1)}|^2} \quad (2)$$

이때, 식 (2)는 아주 유명한 거리 측정 공식으로, 원점과 벡터 $\mathbf{x}$ 사이의 **유클리드 거리 (Euclidean distance)**라고 한다. 예를 들어 2차원 평면의 한 벡터 (3, 5)의 유클리드 거리는 다음과 같이 구할 수 있다.

$$\|(3, 5)\|_2 = \sqrt{|3|^2 + |5|^2} = 5.83$$

즉, 피타고라스 정리를 이용해 원점으로부터 좌표 (3, 5)까지의 직선거리를 구하는 것과 같다. 따라서 p -norm에서 p 를 2로 설정하면 유클리드 거리가 되고, 유클리드 거리는 원점으로부터 벡터의 좌표까지의 직선거리를 나타낸다.

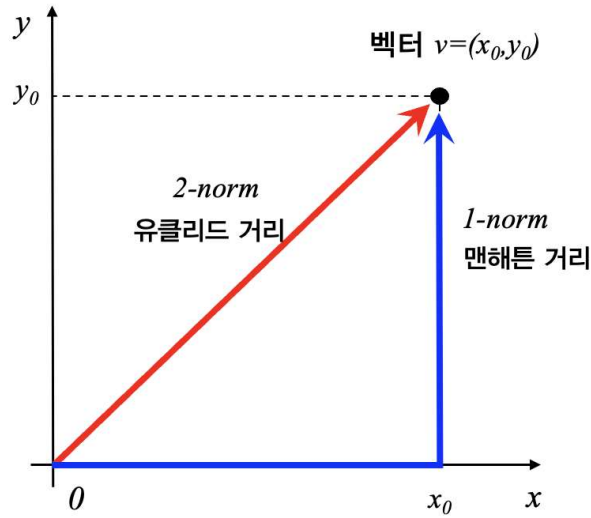
한편 식 (1)에서 p 를 1로 설정한 1-norm을 살펴보면 다음과 같다.

$$\|\mathbf{x}\|_1 = |x_0| + |x_1| + \dots + |x_{(n-1)}| \quad (3)$$

식 (3)은 맨해튼 거리(Manhattan distance) 또는 격자거리(grid distance)라고 하며, 위에서 사용했던 벡터 (3, 5)를 이용해 계산해보면 다음과 같다.

$$\|(3, 5)\|_1 = |3| + |5| = 8$$

이는 원점에서 좌표 (3, 5)까지 수직보행으로 이동했을 때의 거리와 같다. 즉, 원점에서 (3, 5)까지 직선거리로 이동하는 것이 아니라, x축 방향으로 3만큼 이동한 뒤 수직으로 방향을 틀어 5만큼 이동한 거리이다. 이 예를 보면 왜 1-norm이 맨해튼 거리인지 짐작할 수 있다. 즉, 바둑판처럼 생긴 뉴욕의 맨해튼 거리에서 임의의 두 지점이 몇 블록 떨어져있는지 측정하는 방법이 바로 1-norm과 같기 때문이다. 그림 2는 어떤 벡터의 1-norm과 2-norm을 비교한 것이다.



[그림 2] 1-norm과 2-norm의 비교

이제 p -norm을 이용해 임의의 두 벡터 사이의 거리를 정의해보자. n 차원 공간상의 두 벡터 $\mathbf{x}_0$ 와 $\mathbf{x}_1$ 을 가정한다. 이 벡터들의 좌표는 다음과 같이 나타낼 수 있다.

$$\mathbf{x}_0 = (x_{00}, x_{01}, \dots, x_{0(n-1)}), \quad \mathbf{x}_1 = (x_{10}, x_{11}, \dots, x_{1(n-1)})$$

이때 이 두 좌표 사이를 연결하는 벡터는 다음과 같다.

$$\mathbf{d} = \mathbf{x}_0 - \mathbf{x}_1$$

따라서 벡터 $\mathbf{x}_0$ 와 $\mathbf{x}_1$ 사이의 거리 $d_p(\mathbf{x}_0, \mathbf{x}_1)$ 는 p -norm을 이용해 다음과 같이 정의할 수 있다.

$$d_p(\mathbf{x}_0, \mathbf{x}_1) = \|\mathbf{x}_0 - \mathbf{x}_1\|_p \quad (4)$$

이를 상세히 풀어보면 다음과 같다.

$$\begin{aligned} d_p(\mathbf{x}_0, \mathbf{x}_1) &= \left(|x_{00} - x_{10}|^p + |x_{01} - x_{11}|^p + \dots + |x_{0(n-1)} - x_{1(n-1)}|^p \right)^{\frac{1}{p}} \\ &= \left(\sum_{m=0}^{n-1} |x_{0m} - x_{1m}|^p \right)^{\frac{1}{p}} \end{aligned} \quad (5)$$

이때 $p=1$ 이면 두 벡터 사이의 맨해튼 거리, $p=2$ 이면 유클리드 거리를 계산할 수 있다. 식 (5)에서 p 는 1과 2 뿐만 아니라 다양한 양의 정수를 사용할 수 있는데, 주의할 것은 반드시 1 이상의 값을 써야한다는 것이다. 즉, $d_p(\mathbf{x}_0, \mathbf{x}_1)$ 은 $p \geq 1$ 인 경우에만 거리척도로서의 필요조건을 만족시킨다. 어떤 함수 $f(\mathbf{x}_0, \mathbf{x}_1)$ 가 거리척도의 역할을 하기 위해 만족해야 하는 조건은 다음과 같다.

- ① 오직 $\mathbf{x}_0 = \mathbf{x}_1$ 일 경우에만 $f(\mathbf{x}_0, \mathbf{x}_1) = 0$ 이어야 한다.
- ② $f(\mathbf{x}_0, \mathbf{x}_1)$ 와 $f(\mathbf{x}_1, \mathbf{x}_0)$ 은 항상 같아야 한다.
- ③ 임의의 세 벡터에 대해 $f(\mathbf{x}_0, \mathbf{x}_2) \geq f(\mathbf{x}_0, \mathbf{x}_1) + f(\mathbf{x}_1, \mathbf{x}_2)$ 을 만족해야 한다.

즉, 식 (4)에 정의한 $d_p(\mathbf{x}_0, \mathbf{x}_1)$ 은 $p \geq 1$ 인 경우에 위의 세 조건을 항상 만족한다. 따라서 거리척도로 사용할 수 있다.

3. k-평균(k-means) 클러스터링

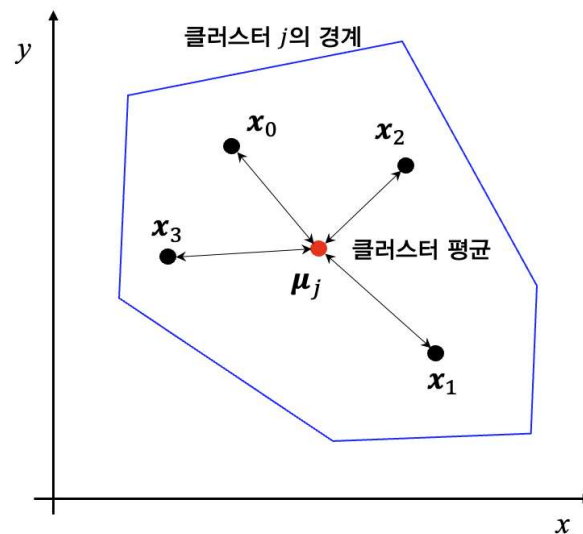
클러스터링 알고리즘 중 가장 단순하면 직관적으로 이해하기 쉬운 알고리즘으로 k-평균 클러스터링 알고리즘이 있다. 이 알고리즘의 k는 클러스터의 개수를 의미하는 시스템 파라미터이다. 즉, 전체 데이터를 교집합이 없는 k개의 클러스터로 분할한다. k-평균 클러스터링 알고리즘의 기원은 신호처리 분야의 벡터 양자화(vector quantization)에 있다.

3.1 클러스터 평균 또는 센트로이드(centroid)

k-평균 클러스터링 알고리즘의 목표는 클러스터에 속한 벡터들과 해당 클러스터 평균과의 거리가 최소가 되도록 k개의 클러스터를 만드는 것이다.

클러스터 평균은 한 클러스터에 포함된 모든 멤버 벡터들의 산술평균에 해당하는 좌표를 의미한다. 이를 센트로이드(centroid)라고 부르기도 한다. 그림 3은 한 클러스터에 속한 네 개의 벡터(검정색 점)와 이들의 평균 좌표인 클러스터 벡터(빨간색 점)의 예를 표시한 것이다. 어떤 방식으로든 클러스터 경계를 정하면 클러스터 멤버 벡터들이 결정되고, 그 멤버들의 평균 좌표인 클러스터 평균을 구할 수 있게 된다. N 개의 벡터 $\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{N-1}$ 이 클러스터 j 의 멤버일 때, 클러스터 평균 μ_j 은 다음과 같다.

$$\mu_j = \frac{1}{N} \sum_{n=0}^{N-1} \mathbf{x}_n \quad (6)$$



[그림 3] 클러스터 평균(빨간색 벡터)과 클러스터 멤버 벡터들(검정색 벡터)

3.2 k-평균 클러스터링 알고리즘의 목표

위에서 k-평균 클러스터링 알고리즘의 목표가 ‘클러스터에 속한 벡터들과 해당 클러스터 평균과의 거리가 최소가 되도록 k개의 클러스터를 만드는 것’이라고 했다. 그러나 엄밀히 정의하자면, k-평균 클러스터링 알고리즘은 벡터들과 클러스터 평균 사이의 거리의 제곱을 최소화하는 클러스터 경계를 찾는다. 즉 전체 데이터 집합이 $D = \{\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{N-1}\}$ 이고 이를 k개의 클러스터 $C = \{C_0, C_1, \dots, C_{k-1}\}$ 로 나누는 경우, 클러스터들의 집합 C 는 다음과 같다.

$$C = \arg \min_C \sum_{j=1}^k \sum_{\mathbf{x} \in C_j} \|\mathbf{x} - \boldsymbol{\mu}_j\|_2^2 \quad (7)$$

식 (7)을 만족하는 클러스터들의 집합 C 는 총 k 개의 클러스터로 구성되며, 이 클러스터들은 각 클러스터의 데이터 벡터와 해당 클러스터 평균과의 거리의 제곱, 즉 2-norm의 제곱을 최소화하는 클러스터들이다. 이제 관건은 주어진 전체 N 개의 데이터를 식 (7)을 만족하는 k 개의 클러스터로 나누는 방법이다.

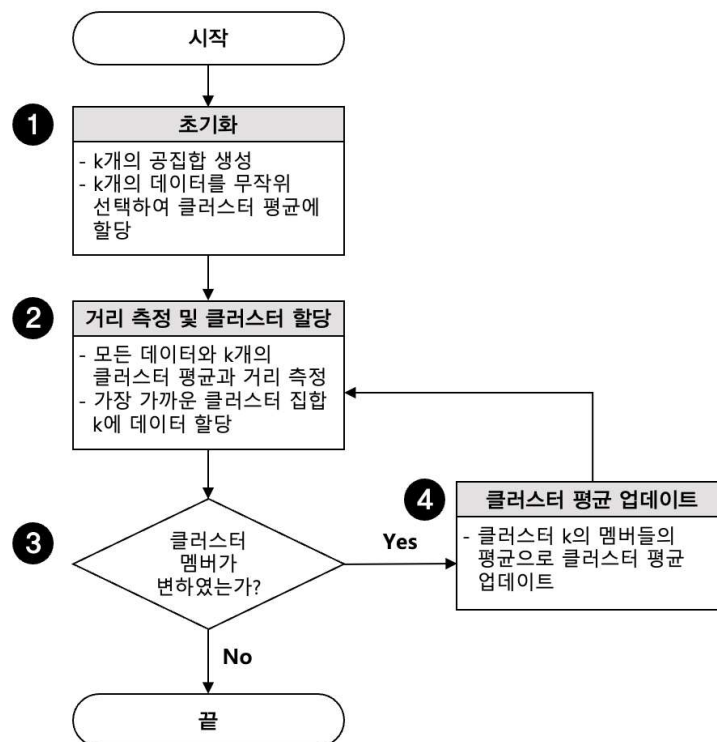
3.3 k-평균 클러스터링 알고리즘

그림 4는 k-평균 클러스터링 알고리즘의 동작을 나타내는 순서도이다. 알고리즘의 입력은 데이터 집합과 클러스터의 개수 k 이고 출력은 k 개의 클러스터, 즉 k 개의 부분집합이다.

① 초기화

알고리즘이 시작되면 먼저 k 개의 공집합을 생성한다. 기호로 표시하면 $C_0, C_1, \dots, C_{k-1}$ 과 같이 표기할 수 있으며, 초기화에 의해 모든 집합이 공집합이다. 클러스터링이 종료되면 이 집합들이 k 개의 클러스터가 된다.

다음으로 전체 데이터 중 k 개의 데이터를 무작위적으로 선택하여 각 클러스터의 평균 역할을 하도록 한다. 초기화 과정에서는 모든 클러스터가 공집합이므로, 평균을 대신할 데이터를 정하는 것이다. 초기에 무작위적으로 정하더라도 알고리즘이 진행되는 과정에서 클러스터 평균의 업데이트를 통해 제대로 된 평균 값으로 수렴하게 된다.



[그림 4] k-평균 클러스터링 알고리즘

② 거리 측정 및 클러스터 할당

이 단계에서는 모든 데이터와 k 개의 클러스터 평균과의 거리를 측정하여 가장 가까운 클러스터로 데이터를 할당한다. 예를 들어 10개의 데이터를 3개의 클러스터로 분할하려고 한다면, 각 데이터에 대해 3개의 클러스터 평균과의 거리를 측정하여 가장 가까운 클러스터에 해당 데이터를 할당한다. 따라서 30번의 거리 측정이 필요하다. 이때 거리척도는 식 5에서 정의한 2-norm, 즉 유클리드 거리를 사용한다.

③ 반복 조건 확인

거리 측정과 클러스터 할당의 과정을 통해 모든 데이터가 k 개의 부분집합으로 분할된다. 이때 각 클러스터의 멤버 데이터가 변화되었는지 확인하여, 변화가 없다면 알고리즘을 종료한다. 만약 클러스터 멤버의 변화가 발생했다면, 추가적인 클러스터링이 필요하므로 알고리즘을 반복 수행한다.

④ 클러스터 평균 업데이트

각 클러스터의 멤버가 변경되었으므로, 평균의 위치도 변경되어야 한다. 따라서 새로 구성된 클러스터 멤버의 평균값을 계산하여 클러스터 평균을 업데이트한다.

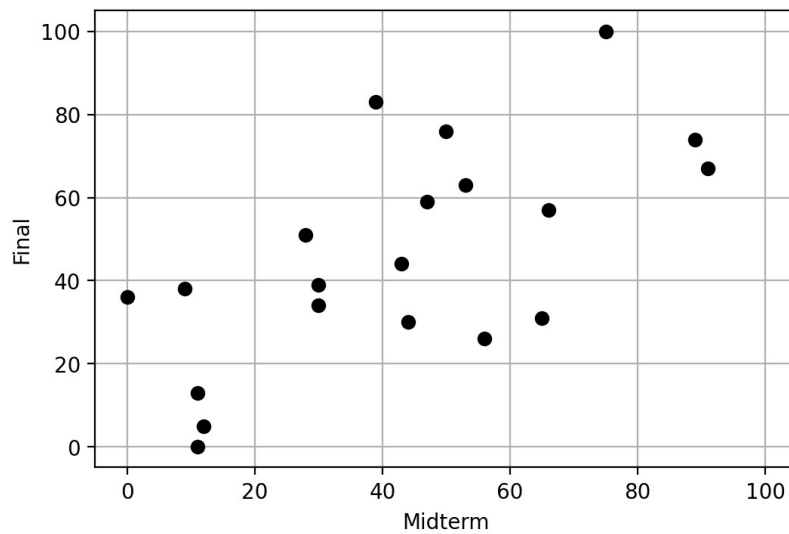
3.4 k-평균 클러스터링 알고리즘 적용의 예

알고리즘의 실제 동작을 확인하기 위해 예제 데이터를 이용해 클러스터링을 진행해보자. 표 1은 클러스터링을 위한 데이터로 20명의 학생에 대한 중간고사와 기말고사 성적이다. 각 성적은 $[0, 100]$ 의 값을 가질 수 있다. 표에서 입력 속성인 중간고사와 기말고사 데이터만 제공되고 레이블 정보는 없는 것을 확인할 수 있다.

[표 1] 클러스터링을 위한 데이터 예

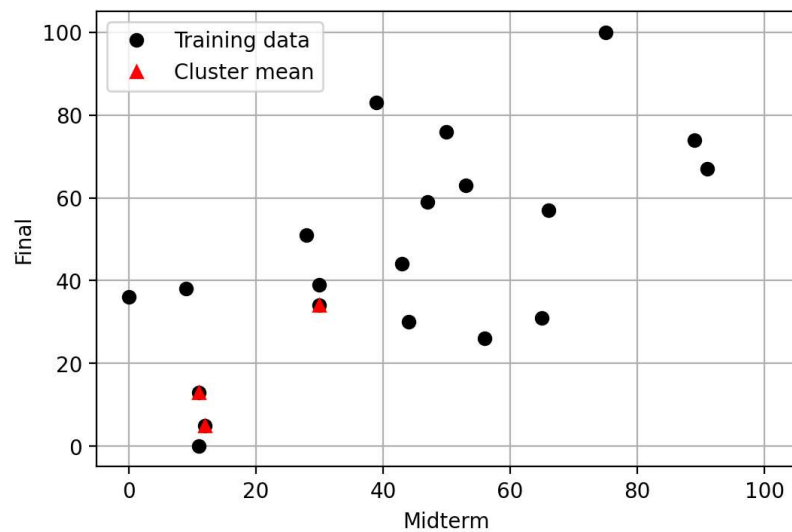
번호	중간고사	기말고사	번호	중간고사	기말고사
1	12	5	11	53	63
2	30	34	12	50	76
3	11	13	13	44	30
4	0	36	14	56	26
5	9	38	15	47	59
6	11	0	16	65	31
7	28	51	17	75	100
8	30	39	18	39	83
9	66	57	19	89	74
10	43	44	20	91	67

그림 5는 표 1의 데이터를 속성공간의 벡터로 표시한 것이다. 입력 속성이 두 개이므로 2차원 속성공간에 각 데이터의 위치를 그림과 같이 표시할 수 있다. 이제 k-평균 클러스터링을 이용해 이 데이터에 내재된 클러스터 구조를 확인해보자.



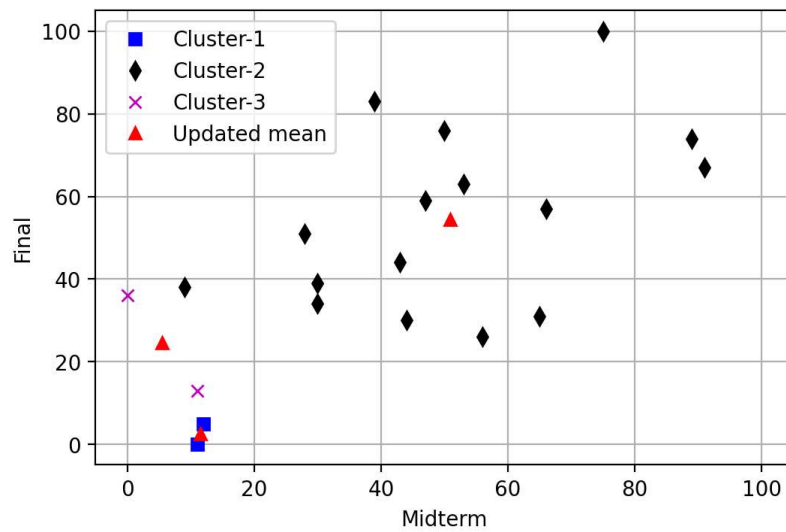
[그림 5] 속성공간과 데이터 벡터

먼저 세 개의 클러스터를 만들기 위해 $k=3$ 으로 설정하고, 무작위적으로 세 개의 데이터를 선택하여 각 클러스터의 평균 역할을 수행하도록 한다. 그림 6의 검정색 점은 훈련 데이터를 나타내고 빨간색 삼각형은 무작위로 선택된 세 개의 데이터이다.



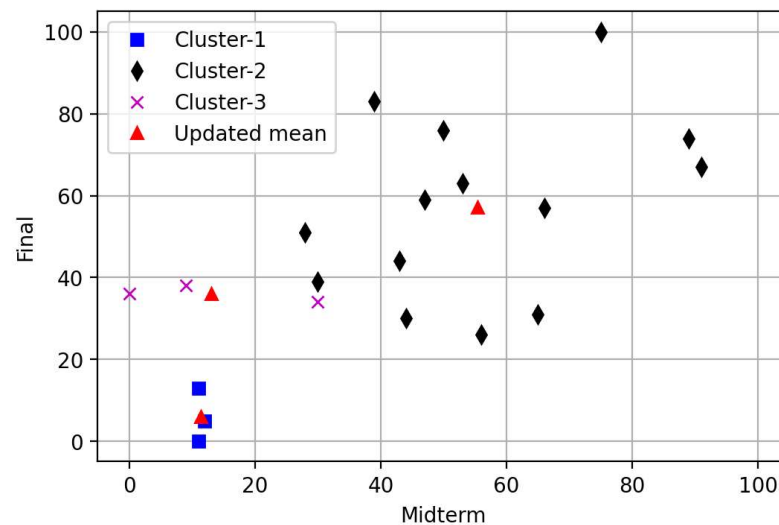
[그림 6] 무작위로 세 개의 데이터를 선택하여 클러스터 평균으로 초기화

다음 단계는 모든 점에 대해 세 개의 클러스터 평균과의 거리를 측정한 뒤 가장 가까운 클러스터로 분류한다. 이는 초기화 이후 첫 번째 클러스터링이므로 각 클러스터의 평균을 업데이트한다. 결과는 그림 7과 같다. 그림 6과 비교하면 클러스터 평균의 위치가 변경된 것을 알 수 있다.



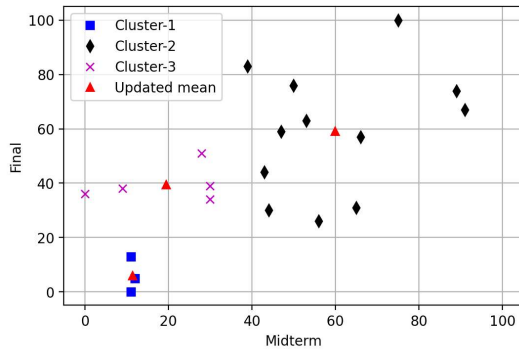
[그림 7] 첫 번째 클러스터링 후 클러스터 평균을 업데이트한 결과

그림 7에서 업데이트된 클러스터 평균을 이용해 다시 모든 데이터와 각 클러스터 평균과의 거리를 측정한 뒤 두 번째 클러스터링을 진행한다. 그림 8은 그림 7에서 업데이트된 클러스터 평균을 이용해 클러스터링을 반복한 결과와 이 결과를 바탕으로 각 클러스터의 평균을 업데이트한 것이다.

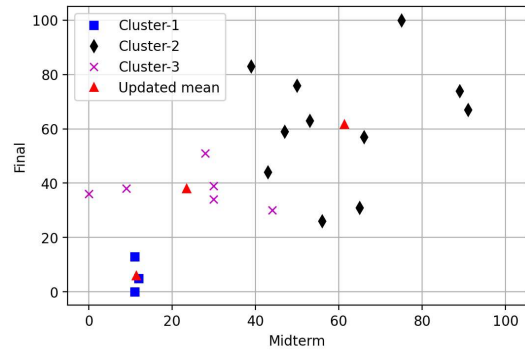


[그림 8] 두 번째 클러스터링 후 클러스터 평균을 업데이트한 결과

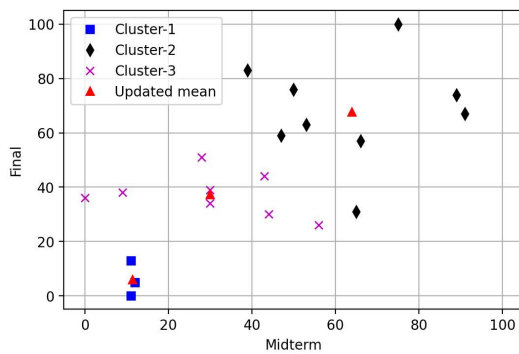
이후의 과정은 지금까지의 과정을 반복하는 것이다. 반복은 클러스터 멤버의 변화가 발생하지 않을 때까지 진행한다. 그림 9는 세 번째 반복부터 열 번째 반복까지의 결과를 순차적으로 보여준다. 반복을 거듭할수록 클러스터 평균의 위치가 변경되고 그에 따라 클러스터 구성이 달라지는 것을 확인할 수 있다. 또한 아홉 번째 반복 후 결과와 여덟 번째 반복 후 결과를 비교해보면 클러스터 구성이 변하지 않았음을 알 수 있다. 클러스터 멤버가 그대로 유지되면 클러스터 평균의 위치도 변하지 않게 되므로 더 이상의 반복은 의미가 없다. 열 번째 반복 결과와 비교해보면 이를 쉽게 확인할 수 있다. 따라서 아홉 번째 반복 후 알고리즘을 종료하는 것이 적절하다.



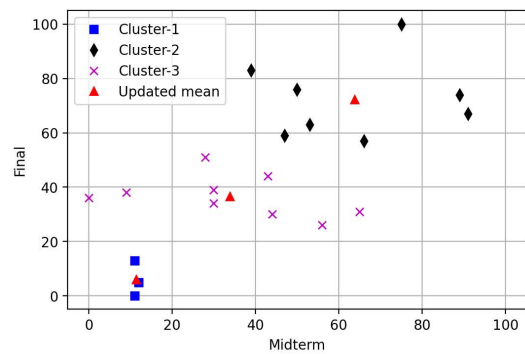
(a) 세 번째 반복 결과



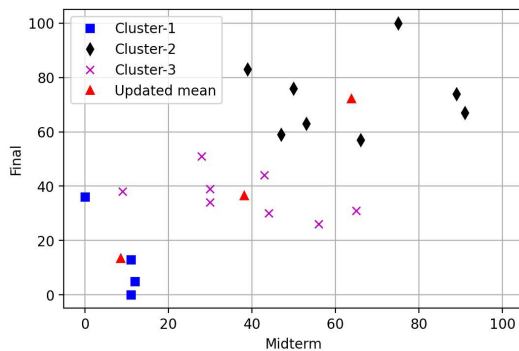
(b) 네 번째 반복 결과



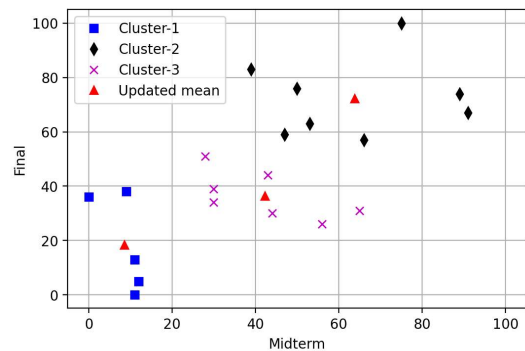
(c) 다섯 번째 반복 결과



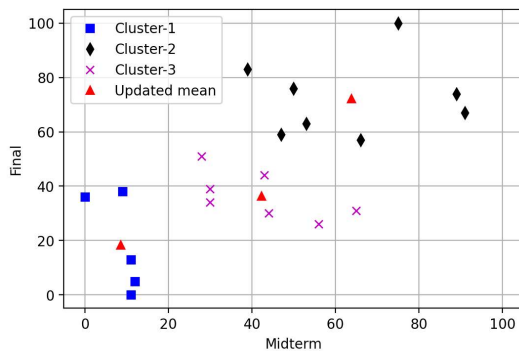
(d) 여섯 번째 반복 결과



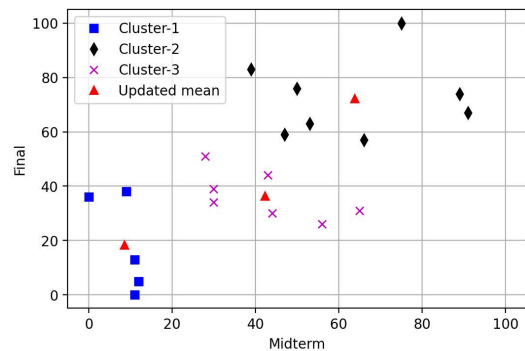
(e) 일곱 번째 반복 결과



(f) 여덟 번째 반복 결과



(g) 아홉 번째 반복 결과



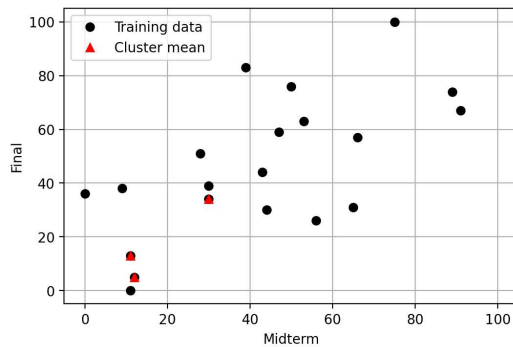
(h) 열 번째 반복 결과

[그림 9] 반복 횟수에 따른 클러스터의 변화

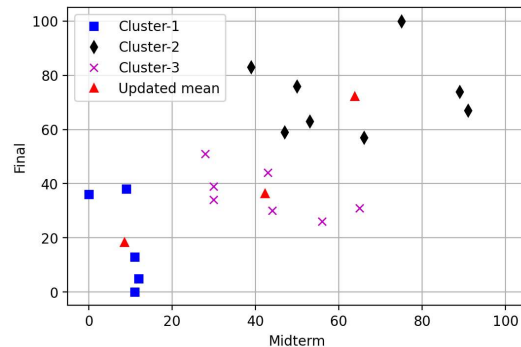
3.5 k-평균 클러스터링 알고리즘은 항상 최적의 클러스터를 보장하는가?

k-평균 클러스터링 알고리즘에 대한 설명을 마치기 전에 한 가지 중요한 점을 언급하고자 한다. k-평균 클러스터링의 궁극적인 목적은 식 (7)을 만족하는 클러스터를 찾는 것이다. 그렇다면, 앞에서 공부한 알고리즘(그림 4)은 식 (7)을 만족하는 클러스터를 만들어줄까? 결론은 항상 그렇지 않는다는 것이다. 즉, 그림 3의 알고리즘은 식 (7)을 만족하는 최적해를 찾을 수도 있고 그렇지 않을 수도 있다는 것이다. 다만 그림 8의 예에서 보듯이 반복을 거듭하다보면 더 이상 변화가 발생하지 않게 되는데, 이는 식 (7)을 만족하는 최적해라기 보다는 극소값, 즉 로컬 미니멈에 수렴한 것으로 보는 것이 적절하다.

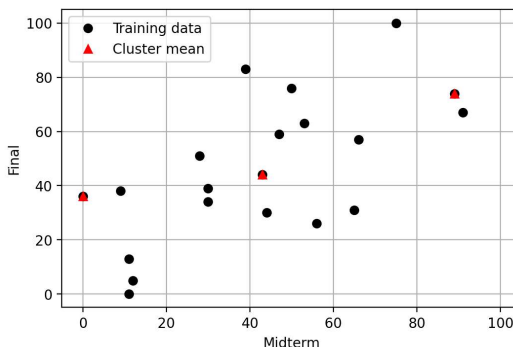
이를 실험적으로 확인해볼 수 있다. 알고리즘의 초기화 과정에서 데이터 중 k개를 무작위적으로 선택하여 클러스터 평균으로 지정하는데, 어떤 데이터를 선택하느냐에 따라 클러스터링 결과가 달라질 수 있다. 그림 9는 서로 다른 초기화에 따른 클러스터링 결과의 차이를 보여주는 예이다. 그림 10의 (a)는 앞의 예에서 사용한 초기 클러스터 평균이고 여기에서 반복을 거듭해 완성한 클러스터 결과는 (b)와 같다. 반면 (c)는 (a)와 다른 데이터를 선택해 초기 클러스터 평균으로 지정한 것이고 이에 대한 클러스터링 결과는 (d)와 같다. (b)와 (d)가 다르다는 것은 출발점에 따라 다른 결과로 수렴한다는 것을 의미하고, 이는 k-평균 알고리즘이 최적 클러스터링 결과를 보장하지 않는다는 것을 의미한다. 따라서 실전에서 활용하기 위해서는 다양한 초기 클러스터 평균에 대한 반복적인 수행과 선택을 통해 적절한 수준의 성능을 확보하는 것이 중요하다.



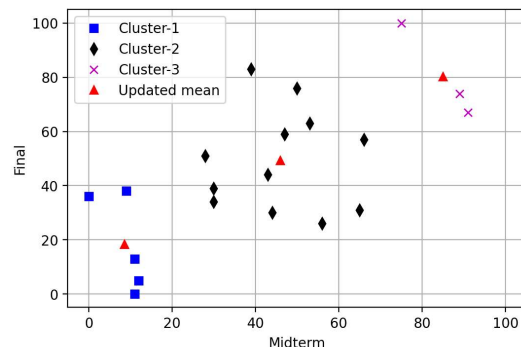
(a) 무작위로 선택한 클러스터 평균



(b) (a)에서 출발한 클러스터링 결과



(c) 무작위로 선택한 클러스터 평균



(d) (c)에서 출발한 클러스터링 결과

[그림 10] 서로 다른 초기화에 따른 클러스터링 결과의 차이

4. DBSCAN (Density-based spatial clustering of applications with noise)

k-평균 클러스터링 기법은 클러스터의 중심이라 할 수 있는 클러스터 평균을 설정하고 이것으로부터의 거리가 가까운 데이터들을 하나의 클러스터로 묶는 방식이었다. 따라서 거리 기반의 클러스터링 방식이라 할 수 있다. 이와 달리 데이터가 밀집된 정도, 즉 밀도 기반의 클러스터링 기법도 널리 사용되고 있는데 대표적인 밀도기반 클러스터링 기법인 DBSCAN을 소개한다.

4.1 시스템 파라미터 및 기본 정의

기본적으로 DBSCAN은 데이터를 속성 공간의 점(포인트, point)으로 표현하고 일정한 영역에 얼마나 많은 점들이 분포하는지를 분석한다. 또한 k-평균 클러스터링과 달리 클러스터의 개수를 지정하지 않고, 데이터의 밀도 분포에 따라 자연스럽게 클러스터의 개수가 결정된다.

DBSCAN의 동작을 위해 두 가지 시스템 파라미터가 필요한데, 각각 ϵ 과 minPts이다. ϵ 은 어떤 점으로부터의 거리를 나타내며 minPts는 특정 영역에 포함된 점들의 최소 개수를 나타낸다. 이 값들이 알고리즘에서 어떻게 활용되는지는 추후에 설명한다. 다만, 시스템 파라미터이므로 알고리즘 시작 전에 결정되어야 하는 값이라는 점을 기억하자.

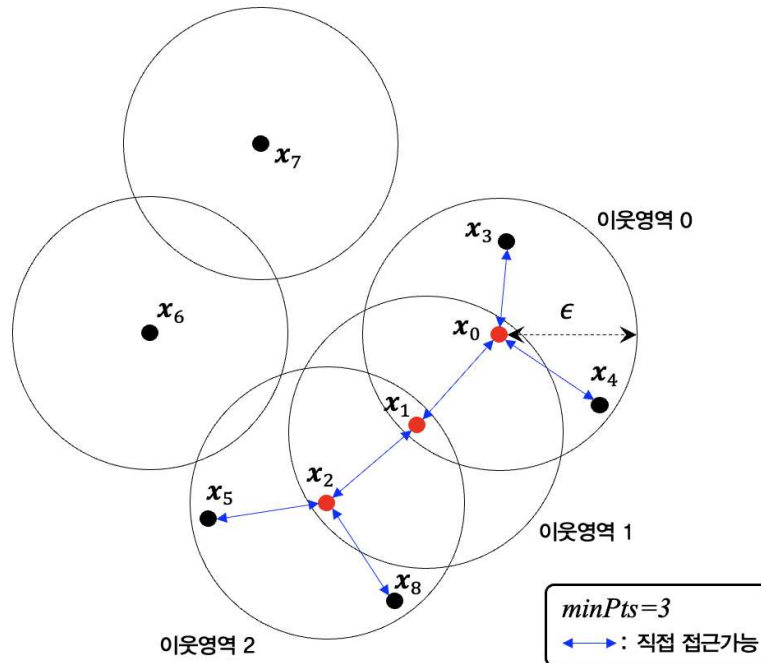
다음으로 DBSCAN 기법은 모든 점들에 대해 다음과 같은 성질을 정의한다.

- ① **코어 포인트(core point)**: 어떤 점 x_0 로부터 반경 ϵ 이내의 영역에 minPts 이상의 점들이 있으면, 점 x_0 를 코어 포인트라 한다.
- ② **직접접근 가능(directly reachable)**: 점 x_1 이 코어 포인트 x_0 로부터 ϵ 이내의 거리에 위치한다면, 점 x_1 은 점 x_0 로부터 직접접근 가능하다.
- ③ **접근가능(reachable)**: 코어 포인트 x_0 로부터 몇 단계의 직접 접근가능한 코어 포인트들을 통해 점 x_1 에 도달할 수 있다면, 점 x_1 은 점 x_0 로부터 접근가능하다. 이때 점 x_0 와 중간점들은 모두 코어 포인트여야 하며, 점 x_1 은 코어 포인트가 아니어도 된다.
- ④ **밀도연결(density-connected)**: 위에서 정의한 접근가능은 출발점이 코어 포인트인 경우에만 정의된다. 따라서 출발점이 코어 포인트가 아니라면 접근가능을 정의하기 어렵다. 접근가능을 확장하여, 코어 포인트인지 여부에 상관없이 어떤 두 점이 몇 단계의 직접 접근가능한 점들을 통해 연결되는 경우, 이 두 점이 서로 밀도연결되었다고 정의한다.
- ⑤ **아웃라이어(outlier) 또는 노이즈(noise)**: 접근가능한 점이 전혀 없는 점을 아웃라이어 또는 노이즈라 한다.

그림 11은 minPts=3인 경우, 각 점의 역할을 도식화한 것이다. 그림에서 9개의 벡터, 즉 $x_0, x_1, \dots, x_8$ 의 위치를 확인할 수 있다. 점 x_0 를 중심으로 반경 ϵ 이내에 minPts 이상인 네 개의 점(x_0, x_1, x_3, x_4)이 있으므로 점 x_0 는 코어 포인트가 된다. 이렇게 코어 포인트를 중심으로 반경이 ϵ 인 영역을 이웃영역(neighbor area)이라고 부른다. 그림에서 x_0 를 코어 포인트로 하는 이웃영역을 이웃영역 0이라고 표시하였다. 같은 이웃영역에 속한 점들은 코어 포인트로부터 직접 접근가능한 점들이다. 따라서 점 x_1, x_3, x_4 는 x_0 로부터 직접 접근가능하다.

또한 그림에서 점 x_1 으로부터 반경 ϵ 이내에 점 x_0 와 x_2 가 있으므로 minPts를 만족하여 x_1 도 이웃영역 1의 코어 포인트가 된다. 이때, 이웃영역 1에 속한 점 x_2 는 이웃영역 0에 속

하지는 않지만 x_0 와 직접 접근가능한 x_1 과 직접 접근가능하고 x_0 와 x_1 이 모두 코어 포인트
이므로 점 x_2 는 점 x_0 로부터 접근가능하다. 즉, $x_0 \rightarrow x_1 \rightarrow x_2$ 로 접근가능하다. 또한 점
 x_3 과 x_8 은 코어 포인트는 아니지만, $x_0 \rightarrow x_1 \rightarrow x_2$ 을 통해 연결되므로 서로 밀도연결된다.



[그림 11] minPts=3일 경우의 예, 원의 반지름은 ϵ 임.

또한 점 x_2 역시, 반경 ϵ 이내에 네 개의 점이 있으므로 새로운 이웃영역의 코어 포인트가 된다. 그리고 이 이웃영역에 속한 점 x_5 와 x_8 은 점 x_2 와 직접 접근가능하고 x_0 , x_1 으로부터는 접근가능하다. 또한 그림에는 어떤 이웃영역에도 속하지 않는 두 점 x_6 , x_7 이 있는데, 이들은 자신을 중심으로 반경 ϵ 이내에 아무 점도 없어 이웃영역을 구성하지 못한 점들이다. 이런 점들을 노이즈 또는 아웃라이어라고 한다.

4.2 DBSCAN 알고리즘

DBSCAN에 의해서 같은 클러스터로 분류된 모든 점들은 서로 밀도연결되어야 한다. 예를 들어 그림 11에서 점 x_0 , x_1 , x_2 , x_3 , x_4 , x_5 , x_8 은 모두 밀도연결된 점들이다. 즉, 이웃영역 0, 1, 2에 속한 어떤 두 점을 선택하더라도 두 점 사이를 연결하는 경로가 존재한다. 이를 코어 포인트와 이웃영역의 관점으로 생각해보면, 어떤 두 이웃영역의 코어 포인트가 서로의 영역에 포함되면 두 이웃영역의 모든 점들은 밀도연결된다. 그림 11에서 이웃영역 0과 1을 살펴보면 이웃영역 0의 코어 포인트가 이웃영역 1에 포함되고 반대로 이웃영역 1의 코어 포인트가 이웃영역 0에 포함된다. 이렇게 이웃영역 0과 1이 서로 상대 영역에 포함되므로 이 두 영역에 포함된 모든 점은 밀도연결된다. 따라서 이웃영역 0과 1은 하나의 클러스터가 되는 것이다.

DBSCAN을 이용해 클러스터를 구성하는 구체적인 절차는 다음과 같다.

① 초기화

초기화에서는 시스템 파라미터인 ϵ 과 이웃영역 설정을 위한 최소 점의 개수 minPts를 설정한다.

② 코어 포인트 탐색

데이터 집합의 모든 점에 대해, ϵ 보다 가까운 점의 개수가 minPts 이상인 점을 코어 포인트로 지정한다. 코어 포인트가 되지 못한 나머지 점들은 비코어 포인트(non-core point)가 된다. 이때 거리척도는 유클리드 거리 척도를 사용한다.

③ 클러스터 구성

접근가능한 코어 포인트들을 같은 클러스터에 포함시킨다. 모든 코어 포인트는 반드시 하나의 클러스터에 포함되어야 한다. 모든 코어 포인트가 특정 클러스터에 포함되면 클러스터 구성을 종료한다. 접근가능하지 않은 코어 포인트들은 서로 다른 클러스터의 멤버가 된다. 따라서 접근가능한 코어 포인트 그룹의 개수에 따라 클러스터의 개수가 결정된다.

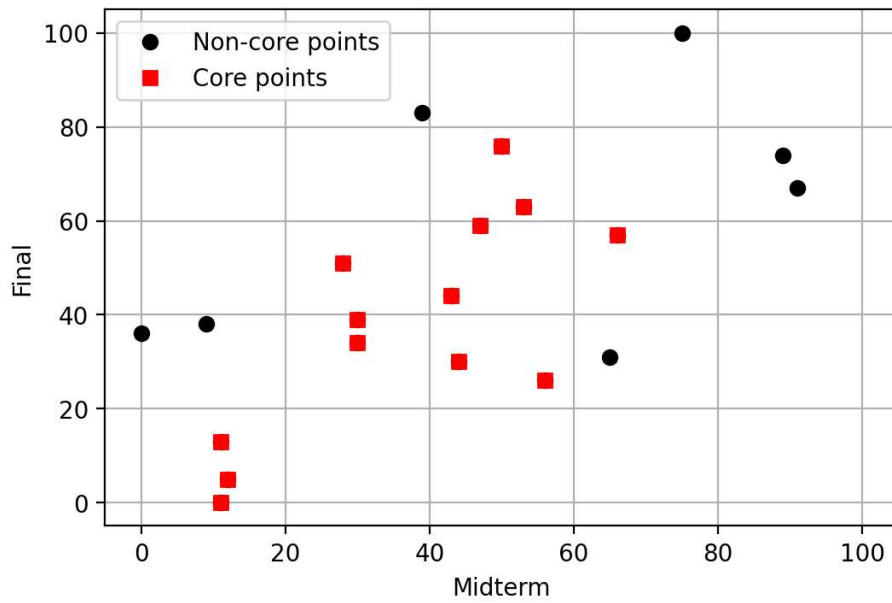
④ 비코어 포인트들의 클러스터 편입

어떤 비코어 포인트로부터 가장 가까운 코어 포인트까지의 거리가 ϵ 보다 작다면, 해당 비코어 포인트를 코어 포인트의 클러스터 멤버로 편입시킨다. 동일한 작업을 모든 비코어 포인트에 대해 수행한다. 특정 비코어 포인트로부터 ϵ 보다 가까운 코어 포인트가 하나도 없다면, 이 비코어 포인트는 어떤 클러스터에도 포함되지 않는 노이즈 또는 아웃라이어가 된다.

4.3 DBSCAN을 이용한 클러스터링의 예

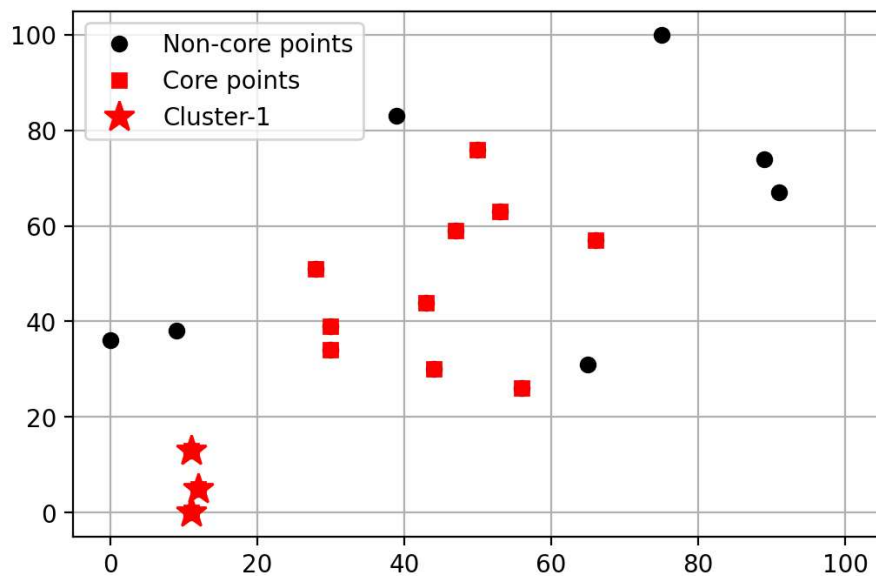
이제 표 1에서 제시한 데이터를 이용해 DBSCAN 클러스터링을 해보자. 먼저 시스템 파라미터를 $\epsilon=20$, minPts=3으로 설정하였다. 즉, 이웃영역의 반경은 20이고, 이 영역 내에 3개 이상의 데이터가 존재하면 중심점은 코어 포인트가 된다.

그림 12는 표 1의 데이터에 대해 코어 포인트를 탐색한 결과이다. 그림에서 빨간색 사각형의 점들이 코어 포인트의 위치를 나타내고, 검정색 점들은 비코어 포인트들의 위치들이다.



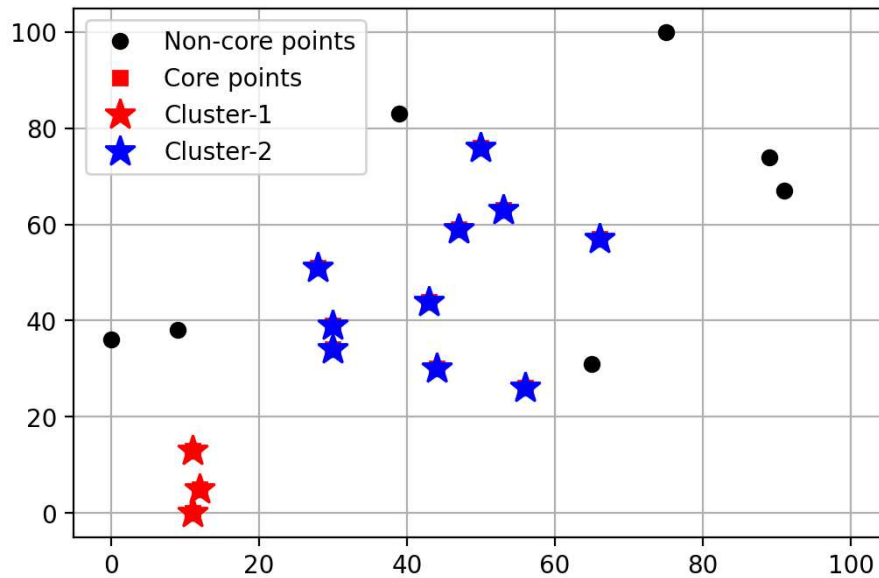
[그림 12] 코어 포인트 탐색 결과 ($\epsilon=20$, minPts=3)

그림 13은 각 코어 포인트로부터 직접접근 가능한 점들을 모아 첫 번째 클러스터를 구성한 결과를 보여준다. 첫 번째 클러스터는 빨간색 별표로 표시된 좌측 하단의 세 점으로, 이 점들은 서로 직접접근 가능한 코어 포인트들이다.



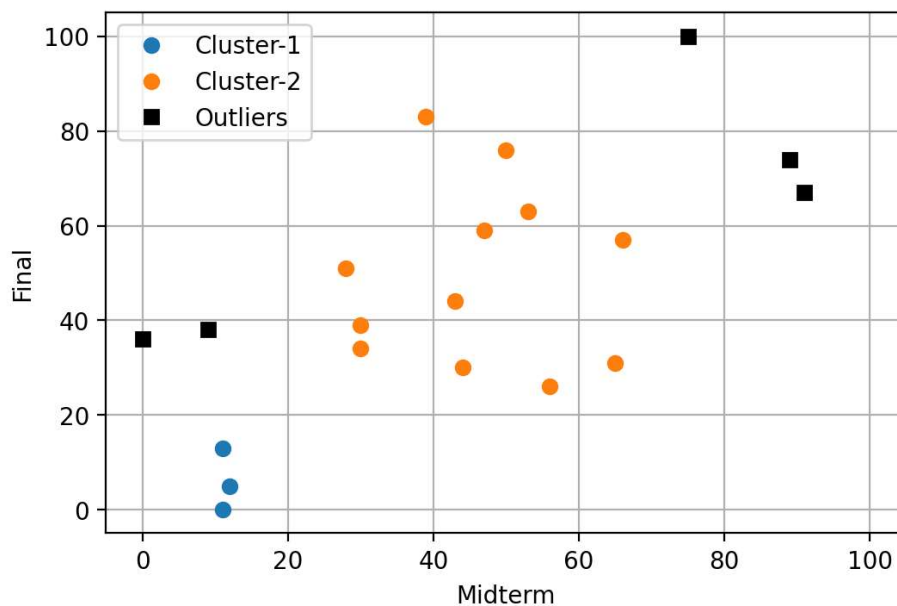
[그림 13] 좌측 하단이 세 점으로 첫 번째 클러스터를 구성한 결과

그림 14는 코어 포인트를 이용해 두 번째 클러스터를 구성한 결과이다. 그림에서 파란색 별표로 표시된 점들이 서로 직접접근 가능한 코어 포인트들이다. 그림에서 모든 코어 포인트들이 클러스터에 포함되었으므로 두 개의 클러스터를 구성하는 것으로 클러스터 구성을 종료한다.



[그림 14] 코어 포인트로 두 번째 클러스터를 구성한 결과

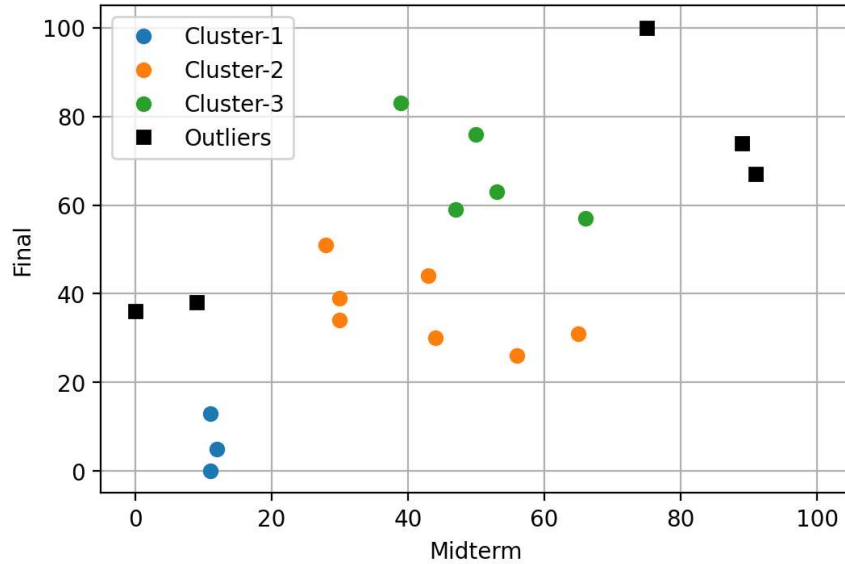
다음으로 그림 14의 비코어 포인트에 대해 가장 가까운 코어 포인트와의 거리를 측정하여 그 거리가 ϵ 이내이면 해당 비코어 포인트를 가까운 코어 포인트의 클러스터에 편입시킨다. 측정 결과 그림 14에서 파란색 별 클러스터 근처의 두 점이 직접접근 가능한 것으로 판정되어, 이 점들을 파란색 별 클러스터에 편입시켰다. 반면 나머지 점들은 가장 가까운 코어 포인트가 반경 ϵ 바깥에 위치하여 어떤 클러스터에도 편입되지 못했다. 그림 15는 이 절차를 거쳐 최종 완성된 클러스터를 나타낸다. 20개의 데이터를 두 개의 클러스터로 구분하였고, 다섯 개의 데이터는 어떤 클러스터에도 포함되지 않는 노이즈 또는 아웃라이어로 결정되었다.



[그림 15] 최종 클러스터링 결과

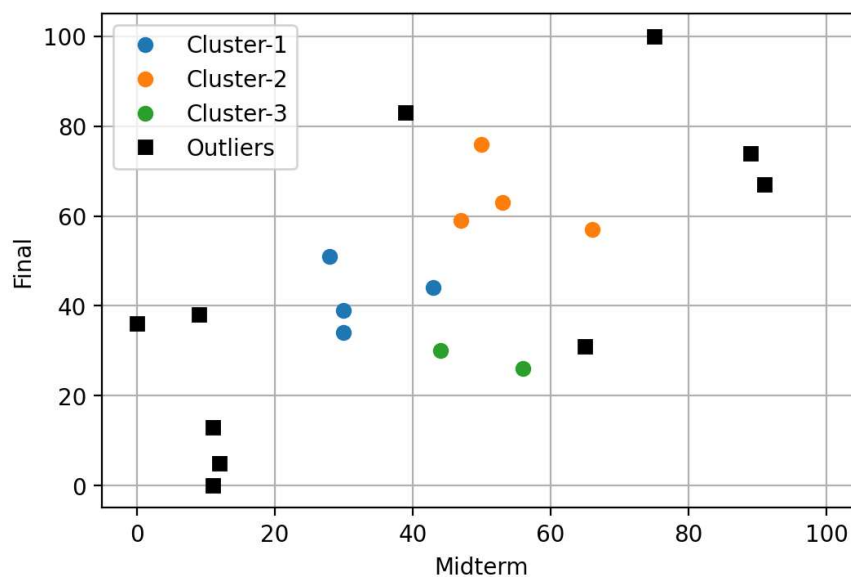
4.4 시스템 파라미터에 대한 민감도

DBSCAN의 결과는 시스템 파라미터인 ϵ 과 minPts에 많은 영향을 받는다. 그림 16은 표 1의 데이터를 $\epsilon=15$, minPts=3의 파라미터로 클러스터링 한 결과이다. 그림 15와 비교하면 ϵ 의 변화로 인해 클러스터의 수가 증가한 것을 알 수 있다. 당연한 결론이지만, ϵ 이 작아질수록 클러스터의 크기가 감소하는 대신 클러스터의 개수가 증가한다.



[그림 16] $\epsilon=15$, minPts=3일 때의 클러스터링 결과

또한 그림 17은 $\epsilon=15$, minPts=4인 경우의 결과를 나타낸다. 즉, 그림 16에 비해 minPts 값이 1 증가한 것이다. minPts가 증가하면 이웃영역 형성이 까다로워지므로 코어 포인트로 지정되는 점의 개수가 줄어든다. 이는 노이즈 또는 아웃라이어의 개수가 증가하는 결과로 이어진다. 그림 17을 16과 비교하면 아웃라이어의 개수가 크게 증가한 것을 확인할 수 있다.



[그림 17] $\epsilon=15$, minPts=4일 때의 클러스터링 결과

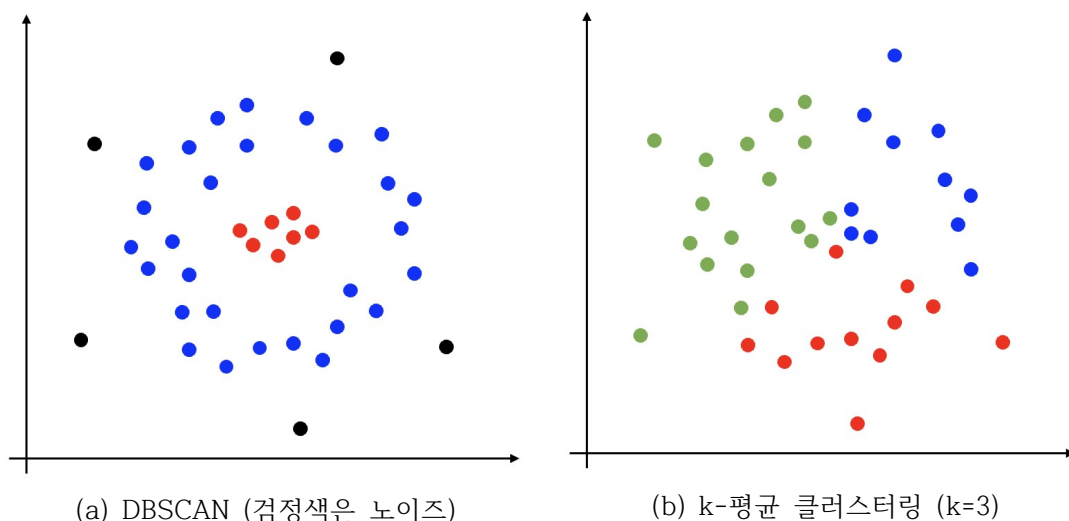
이상의 예를 통해 관찰한 것처럼, DBSCAN의 결과는 시스템 파라미터의 값에 따라 달라진다. 어떤 값이 최적 파라미터인지는 알려진바 없으며 데이터에 따라, 응용 분야에 따라 경험적으로 값을 설정해야 한다. 이는 DBSCAN 알고리즘의 유연성을 높이는데 기여하는 반면, 알고리즘의 활용을 어렵게 하는 부분이기도 하다.

4.5 k-평균 클러스터링과의 비교

DBSCAN과 같은 클러스터링 기법을 밀도기반 클러스터링 기법이라 하고 k-평균 클러스터링과 같은 기법을 프로토타입 기반 클러스터링이라 한다. 여기에서 프로토타입이란 어떤 클러스터의 중심점을 의미한다. 본 자료에서 소개한 두 종류의 클러스터링 기법 외에도 다양한 클러스터링 기법이 있으며 여기에서 소개하지 않은 대표적인 기법으로 계층적 클러스터링 기법이 있다.

DBSCAN과 같은 밀도기반 클러스터링 기법은 사람들이 직관적으로 데이터의 구조를 파악하는 동작과 닮아있다. 즉, 사람들은 어떤 속성 공간에 분포하는 데이터를 볼 때 시각적으로 데이터가 많이 몰려있는 영역을 동일한 클러스터로 인지하는 경향이 있는데, 이는 DBSCAN의 결과와 매우 유사하다.

k-평균 클러스터링 알고리즘은 외부적으로 클러스터의 개수 k 를 지정하는 반면 DBSCAN은 데이터의 밀도에 따라 자동적으로 클러스터가 형성된다. 두 알고리즘의 가장 큰 차이는, DBSCAN 알고리즘은 비선형적 구조의 데이터에서도 클러스터링 성능이 우수하다는 점이다. 그림 18은 이 차이를 나타낸다. 그림 18의 (a)는 DBSCAN의 결과이고 (b)는 같은 데이터에 대한 k-평균 클러스터링의 결과이다. 데이터의 구조를 보면 외부의 고리와 내부의 원을 중심으로 데이터가 밀집되어 있으므로, 그림 (a)와 같이 클러스터를 구성하는 것이 직관적이나 k-평균 클러스터링 알고리즘은 (b)와 같이 원 구조를 3등분하여 클러스터를 구성한다. 이는 DBSCAN은 데이터의 밀집도에 따라 클러스터를 인식하는 반면, k-평균 클러스터링은 각 클러스터의 중심점, 즉 프로토타입과의 거리를 이용해 클러스터를 형성하기 때문이다.



[그림 18] DBSCAN과 k-평균 클러스터링의 비교

그림 18의 예에서 보듯이, 비선형적 분포를 갖는 데이터의 경우 DBSCAN이 k-평균 클러스터

링에 비해 적절한 클러스터를 구성한다는 것을 알 수 있다.

또한 알고리즘의 연산 복잡도 측면에서 DBSCAN 알고리즘은 k -평균 클러스터링에 비해 연산 복잡도가 높다. 따라서 데이터 규모가 커질수록, 데이터의 차원이 높아질수록 k -평균 클러스터링 기법이 더 효율적으로 동작한다는 것이 알려져 있다.



한국공학대학교
TECH UNIVERSITY OF KOREA